



BACHELOR OF COMPUTER APPLICATIONS SEMESTER 4

**DCA2203
SYSTEM SOFTWARE**

Unit 1

Introduction to Software Processor

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
1.1	Learning Objectives	-	-	
2	System Software and Machine Architecture	-	1	4-10
3	Architecture of Intel 8086	1, 2, 3, 1, 2, 3, 4	2	11-21
4	Data and Instruction Formats	-	-	21-24
5	Addressing modes	-	3	24-26
6	Instruction sets	-	-	27-35
7	I/O and Programming	-	4	36-40
8	Summary	-	-	41
9	Glossary	-	-	42
10	Terminal Questions	-	-	42-43
10.1	Answers	-	-	
11	Suggested books	-	-	43

1. INTRODUCTION

The Unit introduces the concepts of design and implementation of system software. Software is a set of instructions or programs written to carry out the certain tasks on digital computers. It is classified into *system software* and *application software*.

In this unit, we are going to discuss system software and machine architecture. In the next session, we will discuss the Architecture of Intel 8086. Then we will discuss data and instruction format and addressing modes. In the last sessions, we will discuss Instruction sets and I/O, and programming.

1.1 Learning Objectives

After studying this unit, learners should be able to:

- ❖ *Describe system software and machine architecture*
- ❖ *Explain the Architecture of Intel 8086.*
- ❖ *Define data and instruction formats*
- ❖ *Describe various addressing modes.*
- ❖ *Explain instruction sets and I/O Programming.*

2. SYSTEM SOFTWARE AND MACHINE ARCHITECTURE

One characteristic in which most system software differs from application software is machine dependency.

System software – support operation and use of the computer.

Application software – solution to a problem.

System software

System software consists of a variety of programs that support the operation of a computer. This software makes it possible for the user to focus on an application or other problem to be solved, without needing to know the details of how the machine works internally. Examples are operating systems, compilers, assemblers, macro processors, loader or linker, debugger, text editors, database management systems, and, software engineering tools.

Application software

Application software focuses on an application or problem to be solved. System software consists of a variety of programs that support the operation of a computer.

Assembler

An assembler translates mnemonic instructions into machine code. The design of an assembler is directly affected by the instruction formats, addressing modes, etc. Similarly, Compilers must generate machine language code, considering hardware features like the quantity and type of registers and the available machine instructions. Operating systems are directly concerned with the management of all of the resources of a computing system.

There are aspects of system software that do not directly depend upon the type of computing system, general design and logic of an assembler, compiler and code optimization techniques which are independent of target machines.

Likewise, the process of linking together independently assembled subprograms does not usually depend on the computer being used.

The Simplified Instructional Computer (SIC)

A simplified Instructional Computer (SIC) is a hypothetical computer that includes the hardware features most often found on real machines. There are two versions of SIC, they

are, standard model (SIC) and the extension version (SIC/XE) (extra equipment or extra expensive).

SIC Machine Architecture

The SIC Machine Architecture contains the Memory and Registers, Data Formats, Instruction Formats, Addressing Modes, Instruction Set, Input, and Output

Memory

The computer's RAM has 215 bytes, that is 32,768 bytes, It uses Little Endian format to store the numbers, 3 consecutive bytes form a word, and each location in memory contains 8-bit bytes.

Registers

There are five registers, every 24 bits in length. Their mnemonic, number, and use are given below.

Mnemonic Number Use:

A	0	Accumulator: used for arithmetic operations
X	1	Index register: used for addressing
L	2	Linkage register: JSUB
PC	8	Program counter
SW	9	Status word, including CC

Data Formats

Integers are stored as 24-bit binary number format, 2's complement representation is used for negative values, and characters are stored using their 8-bit ASCII codes, No floating-point hardware on the standard version of SIC.

Instruction Formats

Opcode(8) x Address (15)

All machine instructions on the standard version of SIC have the 24-bit format as shown above.

Addressing Modes

Addressing modes are the ways in which architectures specify the address of an object they want to access.

There are two addressing modes available, which are as shown below.

Mode	Indication	Target address calculation
1)	Direct $x = 0$	TA = address
2)	Indexed $x = 1$	TA = address + (x)

Parentheses are used to indicate the contents of a register or a memory location.

Instruction Set

SIC is used to load and store instructions (LDA, LDX, STA, STX, etc.). Integer arithmetic operations: (ADD, SUB, MUL, DIV, etc.). All arithmetic operation involves register A and a word in memory, with the result being left in the register. Two instructions are provided for subroutine linkage. COMP Compares the value in register A with a word in memory; this instruction sets a condition code CC to indicate the result. There are conditional jump instructions: (JLT, JEQ, JGT), these instructions test the setting of CC and jump accordingly. JSUB jumps to the subroutine by placing the return address in register L, and RSUB returns by jumping to the address contained in register L.

Input and Output

Input and Output are performed by transferring 1 byte at a time to or from the rightmost 8bits of register 'A' (accumulator). The Test Device (TD) instruction tests whether the addressed device is ready to send or receive a byte of data. Read Data (RD), and Write Data (WD) are used for reading or writing the data.

Data movement and Storage Definition

LDA, STA, LDL, STL, LDX, STX (A - Accumulator, L – Linkage Register, X – Index Register), all uses the 3-byte word. LDCH, STCH associated with characters uses 1-byte. There are no memory-memory move instructions Storage definitions are

WORD – ONE-WORD CONSTANT
 RESW – ONE-WORD VARIABLE
 BYTE – ONE-BYTE CONSTANT

RESB – ONE-BYTE VARIABLE

SIC/XE Machine Architecture

Memory

The maximum memory available on a SIC/XE system is 1 Megabyte (2²⁰ bytes)

Registers

Additional B, S, T, and F registers are provided by SIC/XE, in addition to the registers of SIC

Mnemonic	Number	Special use
B	3	Base register
S	4	General working register
T	5	General working register
F	6	Floating-point accumulator (48 bits)

Instruction Formats

The new set of instruction formats for SIC/XE machine architecture is as follows.

Format 1 (1 byte):

op {8}

Format 2 (2 bytes):

op {8} r1 {4} r2 {4}

Format 3 (3 bytes):

op {8} n i x b p e displacement {12}

Format 4 (4 bytes):

op {8} n i x b p e address {20}

Formats 3 & 4 introduce addressing mode flag bits:

(Note I will use TA for "target address" & disp for "displacement")

flags n & i:

- n=0 & i=1 *immediate addressing* - TA is used as an operand value (no memory reference)

- $n=1$ & $i=0$ *indirect addressing* - word at TA (in memory) is fetched & used as an address to fetch the operand from
- $n=0$ & $i=0$ *simple addressing* TA is the location of the operand
- $n=1$ & $i=1$ *simple addressing* same as $n=0$ & $i=0$

flag x:

$x=1$ *indexed addressing* add contents of X register to TA calculation

flag b & p (Format 3 only):

- $b=0$ & $p=0$ *direct addressing* displacement/address field contains TA (note Format 4 always uses direct addressing)
- $b=0$ & $p=1$ *PC relative addressing* - $TA = (PC) + \text{disp}$ $-2048 \leq \text{disp} \leq 2047$ *
- $b=1$ & $p=0$ *Base relative addressing* - $TA = (B) + \text{disp}$ $(0 \leq \text{disp} \leq 4095)$ **

* note - in PC relative, disp is interpreted as a 12-bit signed integer in 2's complement

** note - in Base relative, disp is interpreted as a 12-bit unsigned integer

flag e:

$e=0$ use Format

3 $e=1$ use Format 4

Addressing modes & Flag Bits

Five possible addressing modes with the combinations are as follows.

Direct (x, b , and p all set to 0): operand address goes as it is. n and i are both set to the same value, either 0 or 1. Although typically that value is 1, if set to 0 for format 3 can assume that the rest of the flags (x, b, p , and e) are used as a part of the address of the operand, to make the format compatible to the SIC format

Relative (either b or p equal to 1 and the other one to 0): the address of the operand should be added to the current value stored at the B register (if $b = 1$) or to the value stored at the PC register (if $p = 1$)

Immediate ($i = 1, n = 0$): The operand value is already enclosed on the instruction (i.e. lies on the last 12/20 bits of the instruction)

Indirect (i = 0, n = 1): The operand value points to an address that holds the address for the operand value.

Indexed (x = 1): value to be added to the value stored at the register x to obtain the real address of the operand. This can be combined with any of the previous modes except immediate.

Instruction Set

SIC/XE provides the instructions that are available in the standard version. In addition to that the Instructions are used to load and store the new registers such as LDB, STB and so on. Floating-point arithmetic operations such asADDF, SUBF, MULF, DIVFand use Register move instruction such as RMO, Register-to-register arithmetic operations, ADDR, SUBR, MULR, DIVR and, Supervisor call instruction: SVC.

Input and Output

The I/O Channels are used to perform input and output while the CPU is executing other instructions. Allows overlap of computing and I/O, resulting in more efficient system operation. The instructions SIO, TIO, and HIO are used to start the test and halt the operation of I/O channels.

Comparison of CISC and RISC Architectures:

The architectures of CISC and RISC machines are introduced in the section that follows. Traditional machines are CISC machines. We also have more current RISC machines in addition to these.

CISC MACHINES:

Traditional (CISC) machines are just computers with a complex instruction set have a relatively large and complex instruction set, different instruction formats, different lengths, different addressing modes, and the implementation of hardware for these computers is complex.

Example: VAX and Intel x86 processors

RISC Machines

RISC means Reduced Instruction Set Computers. These machines are intended to simplify the design of processors. They have greater reliability, faster execution, and less expensive processors and also they have standard and fixed instruction lengths. The number of machine instructions, instruction formats, and addressing modes is relatively small. Example: Ultra SPARC Architecture and Cray T3E Architecture

SELF ASSESSMENT QUESTIONS - 1

1. _____ translates mnemonic instructions into machine code.
2. Maximum memory available on a SIC/XE system is_____.
3. In_____addressing Mode, operand address goes as it is.

3. ARCHITECTURE OF INTEL 8086

The **8086** is a 16-bit microprocessor chip designed by Intel between early 1976 and mid-1978. 8086 has 16-bit ALU; this means 16-bit numbers are directly processed by 8086. It contains a 16-bit data bus, so it can read data or write data to memory or I/O ports either 16 bits or 8 bits at a time. It has 20 address lines, so it can address up to 2^{20} i.e. $1048576 = 1\text{M}$ bytes of memory (words i.e. 16-bit numbers are stored in consecutive memory locations). Due to the 1M bytes memory size multiprogramming is made feasible as well as several multiprogramming features have been incorporated into the 8086 design.

Features of 8086

- 8086 is a 16bit processor. It's ALU, internal registers work with 16bit binary words.
- 8086 has a 16bit data bus. It can read or write data to a memory/port either 16bits or 8bits at a time.
- 8086 has a 20bit address bus which means, it can address up to $2^{20} = 1\text{MB}$ memory location.
- The frequency range of 8086 is 6-10 MHz.
- 8086 processors employ parallel processing.
- It can support up to 64K I/O ports.
- It requires up to +5Volts power supply.
- The permissible range of the memory address is from 00000H to FFFFFH.
- 8086 operates in two modes, that are minimum mode and maximum mode.
- Memory is byte-addressable, a separate address is there for every byte.
- It can prefetch up to 6 bytes of instructions and queue them so that execution speed is increased.

Architecture of Microprocessor 8086

Microprocessor 8086 is a 40-pin 16-bit Intel IC made by using HMOS technology. It consists of 2900 transistors. The term 16-bit means that the arithmetic logic unit, its internal registers, and instructions are designed to work with 16-bit binary words. As stated earlier it has a 16-bit data bus and 20-bit address bus. The pipelined architecture of microprocessor 8086 is shown in figure 1.1.

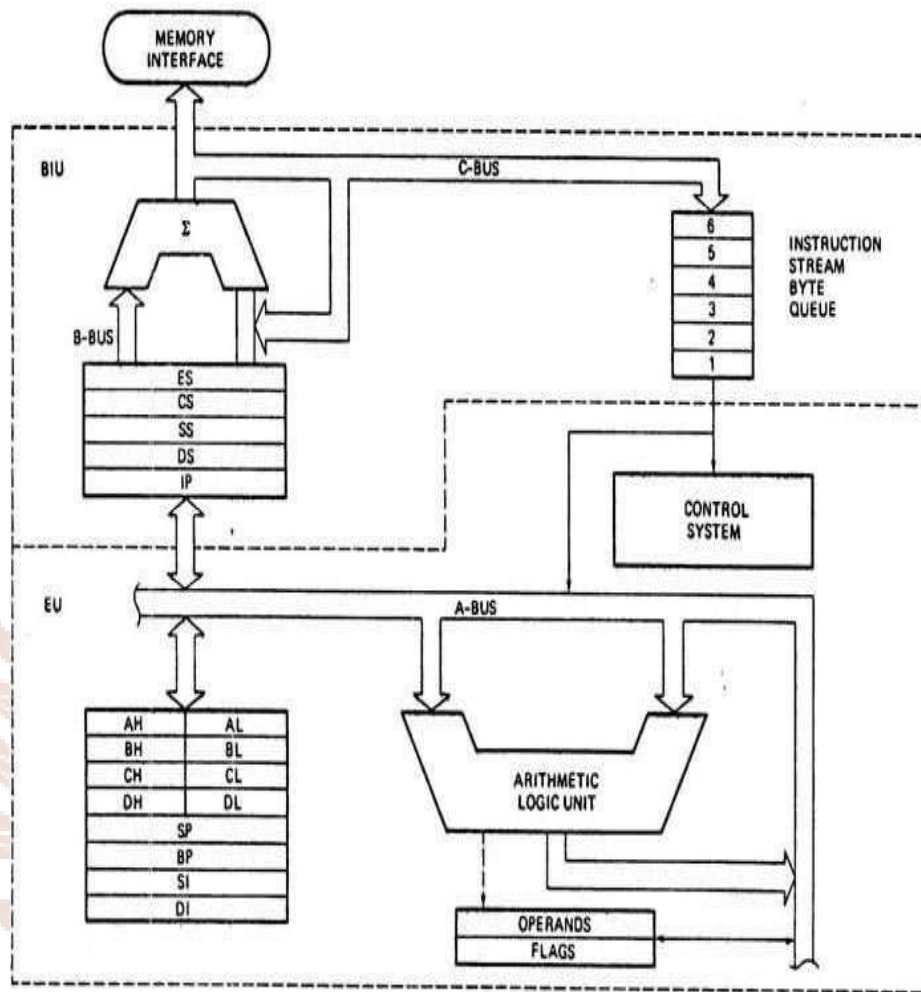


Fig 1.1: Block Diagram of Microprocessor 8086

The architecture is divided into two functional units as you can see in the above figure:

1. Bus Interface Unit(BIU)
2. Execution Unit(EU)

Bus Interface Unit (BIU): The BIU handles all the data and addresses on the buses for the execution unit (EU). It performs all bus operations such as instruction fetching, reading, and writing operands for memory and calculating the addresses of the memory operands. The instruction bytes are transferred to the instruction queue. The execution unit uses the instruction queue to execute the instructions. Both BIU and EU work asynchronously to execute instructions by using Pipelining mechanism which means overlapping of instruction fetch and execute mechanism. The pipelining results in efficient use of the system bus and

increases system performance. Here will study pipelining mechanisms in detail in the coming section.

The various parts of BIU are as follows:

1. Instruction Queue

The prefetched instruction bytes are stored in a first in first out group of registers called an **instruction queue** for the Execution Unit (EU). When the EU is ready for its next instruction, it simply reads the instruction from this instruction queue. This is much faster than sending out an address to the system memory and sending back the next instruction byte. Fetching the next instruction while the current instruction executes is called **pipelining**. In 8086 pipelining is used to speed up the execution of the program as the instruction fetch and execution steps are overlapped. The 8086, a 6-byte instruction queue is presented at the Bus Interface Unit (BIU). It is used to prefetch and store a maximum of 6 bytes of instruction code from the memory.

2. Segment Registers

The BIU contains four 16-bit segment registers. These are:

- Extra segment (ES) register
- Code segment (CS) registers
- Data segment (DS) registers
- Stack segment (SS) registers

The upper bits of the starting address of each of the segments are held by these segment registers. The part of the segment starting address stored in the segment register is known as the segment base. The description of segment registers is as follows:-

- i) *Code Segment Register*: It is a 16-bit register used for accessing instructions referenced by instruction pointer or we can say that it is used to address the memory location in the code segment, where the executable program is stored.
- ii) *Stack Segment Register*: It is also a 16-bit register and contains the data used when the subprogram is executing. It contains the data pointed by Stack Pointer (SP) and base pointer(BP).

- iii) *Data Segment*: It is a 16-bit register containing the data used by the program. The data used by pointed by general-purpose registers (AX, BX, CX, DX) and Index registers (SI, DI) is located in the data segment.
- iv) *Extra Segment*: It is a 16-bit register used to hold extra destination data during some string manipulation operations. It contains the data pointed by the DI register.
- v) *Instruction Pointer*: It holds the 16-bit address and holds the address of the next instruction to be executed.

Execution Unit (EU)

It decodes instructions fetched by BIU, generates control signals, and executes the instruction. The main parts of the control unit are the control system, arithmetic, and logic unit instruction decoder unit. The Control systems perform various internal operations. Arithmetic and the logical unit perform different arithmetic operations like increment, decrement operations, etc., and logical operations like AND, OR, NOT, etc. Decoder translates instruction fetched from memory to perform the necessary operation.

Flag Register

It shows the condition or changes produced by the execution of an instruction and these are modified by the CPU automatically after some mathematical or logical operation is performed. Nine flags are in microprocessor 8086 and are shown in figure 1.2.

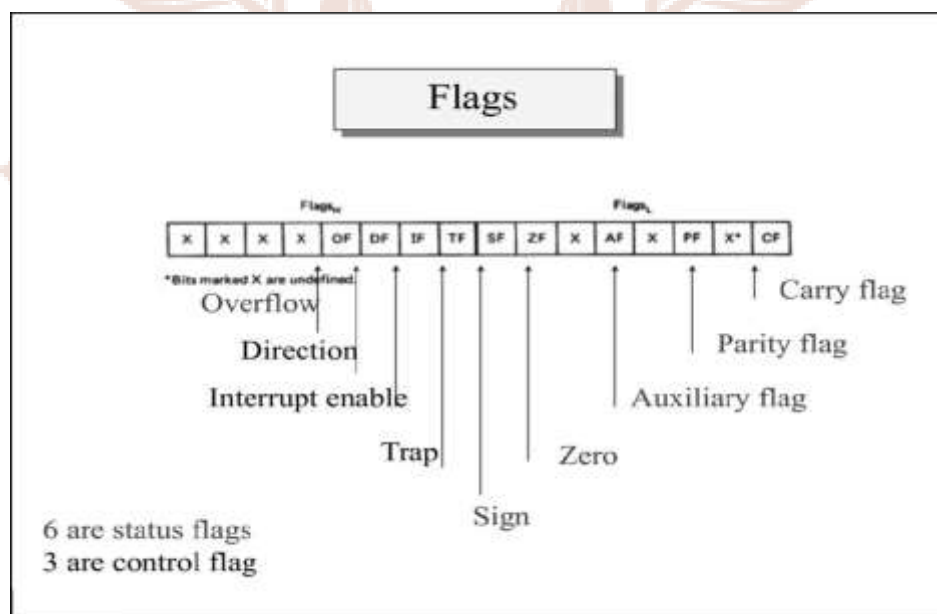


Fig 1.2: Flag register

The flags are categorized into two categories:

1. Conditional Flags
2. Control Flags

Conditional Flags: The result of the last arithmetic and logical instruction is represented by these flags.

The description of all conditional flags is as follows:

- Carry flag: If by any arithmetic operation a carry is produced at MSB (most significant bit position) then it is set to 1.
- Auxiliary flag: If an operation performed in ALU generates a carry/borrow from lower nibble (i.e. D0 – D3) to upper nibble (i.e. D4 – D7), the AF flag is set i.e. carry given by D3 bit to D4 is AF flag. This is not a general-purpose flag; it is used internally by the processor to perform Binary to BCD conversion.
- Parity flag: This flag is used to show the parity bits. If an even number of 1 bit are there then it is set to 1 else set to zero.
- Zero flag: If the result of the arithmetic or logical operation is 0 then it is set to 1 else set to 0.
- Sign flag: If the result of the operation is negative then this flag is set to one and when the result is positive it is set to zero. These flags take the value of MSB as in sign-magnitude representation MSB indicates a sign of the number.
- Overflow flag: When the signed numbers are added or subtracted and there is a signed overflow then this flag is set else reset. If the overflow flag is set then this indicates that the result has exceeded the capacity of the machine.

Control Flags: These flags are not automatically set or reset these flags are needed to be set or reset by the user to control certain operations.

Description of these flags is as follows:

- Trap Flag: When this flag is set then the user can execute one instruction at a time and able to debug.
- Interrupt flag: If this flag is set then interrupt (interruption of the program) is enabled if reset then the interrupt is disabled.

- **Direction Flag (DF):** Accessing of string bytes is from higher to lower memory address if this flag is set and if reset then accessing of string bytes is from lower to higher memory address.

General Purpose Registers

In microprocessor 8086, there are eight general purpose registers, which are AL, BL, CL, DL, AH, BH, CH and DH.. All these registers can store up to 8 bits (1 byte of data). Some pairs of registers are used to store 16 bits of data and these pairs are AL-AH, BL-BH, CL-CH, and DL-DH these pairs are called AX, BX, CX, and DX respectively.

AX (AL-AH) register: This register pair is used for 16-bit operation also known as Accumulator and is used to store operands for arithmetic operations.

BX (BL-BH) register: This register pair is used as a base register that is it is used to store the starting address of a data segment.

CX (CL-CH): It is known as counter register and is used in loop instructions. **DX (DL-DH):** It is used to store the I/O port addresses for I/O instructions.

Stack Pointer Register

It contains the offset of 16 bits from the start of the segment to the top of the stack. The top of the stack indicates the location where the word is most recently stored.

Other Pointer and Index Registers

Some other registers are also there in the execution unit. These are SI (Source Index), DI (Destination Index), and BP (Base Pointer) and are used to store the 16-bit data.

Pin diagram and description of 8086 Microprocessor

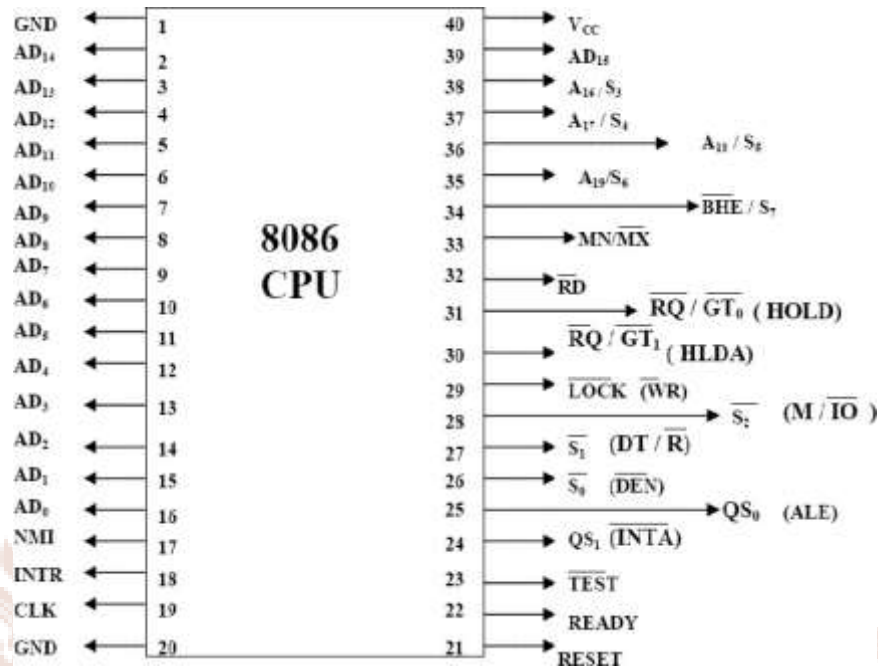


Fig 1.3: Pin Diagram of 8086

The Microprocessor 8086 is a 16-bit CPU available at different clock rates and packaged in a 40-pin Cerdip (CERamic Dual In-line Package) or plastic package. It operates in either single processor mode or multiprocessor mode so that it can achieve high performance. 8086 signal can be divided into three categories and these are as follows:-

- The signals have common functions in both modes.
- The signals that have a specific function are in minimum mode.
- And the signals that have specific functions in maximum mode.

Pin description of 8086 common to both the modes:

1. **AD₀-AD₁₅:** These are the time-multiplexed addresses and data lines. During T1 state address is there on the line and during T2, T3, T4, AND T5 state address data are there.
2. **A₁₉/S₆, A₁₈/S₅, A₁₇/S₄, A₁₆/S₃:** These lines are upper address lines and are multiplexed with status signals (S₃-S₆). These lines act as address lines during T states and show status for the remaining T states. These lines remain low during I/O operations. S₄ and S₃ indicate which segment is being used by the processor. and S₆ is always has a low.

Table 1.1: Encoding of S4 and S3

S4	S3	Indications
0	0	alternate data
0	1	Stack
1	0	code or none
1	1	Data

3. BHE/S7: Bus High Enable/Status

BHE stands for bus high enable signal shows the data transfer over the higher-order (D15-D8) data bus. When BHE is low then data is transferred over D15-D18.

Table 1.2: Encoding of BHE and A0

BHE	A0	Indication
0	0	Complete word (16 bit is accessed)
0	1	Upper byte is accessed
1	0	Lower byte is accessed
1	1	None

4. RD: Read

It is an active low output signal. When this signal is low, it indicates the processor is performing a read (memory or I/O read) operation. It is activated during T2, T3, and TW of any read machine cycle and in other T states, it is high.

5. **READY:** This is an active-high input signal and is used for slower peripherals. When a peripheral device is ready to receive or transmit the data then it sends the READY signal to the processor.
6. **INTR:** Interrupt Request: This is an active high-level triggered input signal. This signal is sampled during the last clock cycle of every instruction it is sampled to determine the availability of the request and if there is any request then the processor enters the interrupt acknowledgment cycle.
7. **TEST:** It is an active low input signal. Execution will continue if TEST=0 else the processor will be in an idle state.

8. **NMI: Non-Maskable Interrupt** It is an edge-triggered input and cannot be masked. And if there is a transition from low to high it indicates an interrupt response at the end of the current instruction.
9. **RESET:** It is an input signal. This signal results in the termination of current activities of the system and results in restarting of the system.
10. **CLK: Clock Input**
The clock input provides the basic timing for processor operation and bus control activity.
11. **Vcc:** +5 volts power supply is connected to this pin.
12. **GND:** It is the GROUND pin.
13. **MN/MX:** It indicates in which mode the processor will operate that is minimum mode or maximum mode.

Pin Description of 8086 in minimum mode:

1. **M/IO (Status line):** This is an output signal which shows whether the processor is accessing memory or I/O.
2. **WR (Write):** It is an active low output signal, if high indicates that the write operation is being performed either from I/O or from memory.
3. **INTA:** It is an active low output signal. If this signal is low then this indicates that it means the acceptance of the interrupt signal by the processor.
4. **ALE:** It is an active high output signal and shows whether the address present on data or address bus is valid or not.
5. **DT/R:** it is the data transmit and receive signal and shows the direction of the data flow through transceivers. If it is high then data will be transmitted else not.
6. **DEN:** It is data enable signal and indicates the presence of valid data over address/data lines.
7. **HOLD and HLDA:** It means to hold and hold acknowledge. When the HOLD is high then it means that the bus is requested by some other external device.

When the HOLD signal is received by the processor it releases the bus and issues an HLDA signal as an acknowledgment.

Pin description of 8086 in maximum mode:

1. **S2, S1, S0:** To the bus controller 8288 these signals are applied to generate different control signals and in the maximum mode of the INTEL. In the T1, T2, and T3 states these signals are active and passive during T3 and T4 states.

Table 1.3: S2,S1,S0 encoding

S2	S1	S0	Operation
0	0	0	Interrupt acknowledges
0	0	1	Read I/O port
0	1	0	Write I/O port
0	1	1	Halt
1	0	0	Code Access
1	0	1	Read Memory
1	1	0	Write Memory
1	1	1	Passive

2. **RQ/GT₀, RQ/GT₁:** It is an I/O request grant. Other local buses use these signals to force the processor to release the local bus at the end of the current bus cycle of the processor. With RQ/GT₀ every pin is bidirectional and has higher priority over EQ/GT₁.
3. **LOCK:** It is an active low output signal. This signal does not allow other devices to take control over the system bus when LOCK=0. The LOCK instruction activates this signal and it is active till the completion of the next instruction.
4. **Qs1, Qs0:** It means queue status and gives the status of the code-prefetch queue. This signals give the status of 8086 queues to the maths coprocessor as these signals are interfaced with the signals of the math coprocessor.

Table 1.4: Encoding of Qs1 and Qs2

Qs1	Qs0	Encoding
0	0	No operation
0	1	First byte of opcode from queue
1	0	Empty the queue
1	1	Subsequent byte from the queue

SELF ASSESSMENT QUESTIONS – 2

4. 8086 CPU has _____ bit data bus and _____ bit address bus.
5. 8086 microprocessor consists of _____ transistors.
6. The prefetched instruction bytes are stored in a first in first out group of registers called an _____
7. _____ register shows changes produced by execution of an instruction.
8. _____ are time multiplexed address and data lines.

4. DATA AND INSTRUCTION FORMATS

The complete collections of instructions that are understood by a CPU are called instruction set. 8086 is two addresses, registered to memory architecture. Operation and operands are the elements of instruction.

- **Operation:** It represents what is being done.
- **Operands:** It represents how data to use in the operation.

For example,

$$C = A + B.$$

- Operations: addition (+) and assignment (=).
- Operands: A, B & C.

Source operands provide values to use (i.e., Inputs, here A & B) and Destination operands receive results (i.e., output, here C). Consider the Symbolic representation,

E.g.: ADD A, B

Here, the Addition operation is performing, and the operands are A, B, and Opcode 'ADD'. This can be interpreted as $A = A + B$. Here, 'A' can be a register or memory address and 'B' can be a register, memory reference or a constant. But 'A' and 'B' cannot be memory addresses. There are some one address instructions, which leave the destination and first source implicit. In machine code each instruction has a unique bit pattern, for human consumption, a symbolic representation is used (for example, ADD, SUB, LOAD, etc.).

Instruction Types:

Three main classifications of Instruction types are the following:

- Data Transfer.
- Data Manipulation.
- Control Flow.

Data Transfer:

These types of instructions transfer data among variables (registers, memory, and I/O ports). They do not alter FLAGS. Transfer of 8 and 16-bit data is done using the MOV instruction. Either the source or destination must be a register. The other operand can come from another register, from memory, from immediate data (i.e., a value included in the instruction), or from a memory location "Pointed at" by the register BX. For example, If COUNT is the label of a memory location, then the following are possible assembly-language instructions:

- a) Register: move contents of BX to AX
MOV AX, BX
- b) Direct: move the contents of AX to memory
MOV COUNT, AX
- c) Immediate: load CX with the value 0F0H
MOV CX, 0F0H
- d) Memory: load CX with the value at; address 0F0H
MOV CX, [0F0H]
- e) register indirect: move contents of AL; to a memory location in BX
MOV [BX], AL

Data Manipulation:

These are instructions that modify variable values and are executed within the ALU data path. They modify the flags. Following are examples of data manipulation operations,

1) Data Manipulation: ADD

ADD dest, src

Here, 'dest' is both a source and destination operand (semantics: $\text{dest} = \text{dest} + \text{src}$). This operation will modify flags as shown below,

ZF: =1, if result=0

SF: =1, if MS bit of result = 1 (sign = negative)

CF: =1, if carry out of MS bit

OF: =1, if result overflowed signed capacity.

2) Data Manipulation: SUB and CMP

SUB dest, src (i.e., the operation is $\text{dest} = \text{dest} - \text{src}$).

-(Zero Flag)ZF: =1, if result=0

SF: =1, if MS bit of result = 1 (sign = negative)

CF: =1, if borrow into MS bit

OF: =1, if result overflowed signed capacity

CMP dest, src (compare)

This modifies flags only to reflect $\text{dest} - \text{src}$.

3) Data Manipulation: Logical Operations

Syntax: BOOLEAN_MNEMONIC dest, src.

Semantics: $\text{dest} = \text{dest} \text{ BOOLEAN_MNEMONIC } \text{src}$.

Example:

- 1) AND AL, 80h /* Doing AND operation with 80H and Contents of AL */
- 2) OR AL, BH /*Doing OR operation with the Contents of AL and BH registers */
- 3) XOR AX, BX /*Doing XOR operation with the Contents of AX and BX registers */
- 4) XOR AH, 0FFh/* Doing XOR operation with 0FFH and the Contents of AH register */

Control Flow:

These instructions determine the next instruction to execute. This allows non-sequential execution. Unconditional & Conditional are two types of switching execution to a different instruction sequence.

Syntax: MNEMONIC address

Example:

- 1) JMP 00FF // Jump to the address 00FF.
- 2) CALL 00FF // calls a subroutine and saves the return address on the stack

- 3) RET // returns from the subroutine to the main program
- 4) LOOP 00FF // returns from the subroutine to the main program
- 5) JC 00FF // jump if CF = 1
- 6) JNC 00FF // jump if CF = 0
- 7) JZ 00FF // jump if ZF = 1
- 8) JNZ 00FF // jump if ZF = 0

5. ADDRESSING MODES

Addressing mode indicates a way of locating data or operands. Depending upon the data types used in the instruction and the memory addressing modes, any instruction may belong to one or more addressing modes or some instruction may not belong to any of the addressing modes. Thus the addressing modes describe the types of operands and the way they are accessed for executing an instruction.

The addressing modes for sequential control transfer instructions are explained as follows:

1. **Immediate:** In this type of addressing, immediate data is a part of instruction, and appears in the form of successive bytes or bytes.

Example: MOV AX, 0005H

In the above example, 0005H is the immediate data. The immediate data may be 8-bit or 16-bit in size.

2. **Direct:** In the direct addressing mode, a 16-bit memory address (offset) is directly specified in the instruction as a part of it.

Example: MOV AX, [5000H]

Here, data resides in a memory location in the data segment, whose effective address may be computed using 5000H as the offset address and the content of DS as the segment address. The effective address, here, is $10H \cdot DS + 5000H$.

3. **Register:** In register addressing mode, the data is stored in a register and it is referred to using the specific register. All the registers, except IP, may be used in this mode.

Example: MOV BX, AX.

- 4. Register Indirect:** Sometimes, the address of the memory location, which contains data or operand, is determined in an indirect way, using the offset registers. This mode of addressing is known as the register indirect mode. In this addressing mode, the offset address of data is in either BX or SI, or DI registers. The default segment is either DS or ES. The data is supposed to be available at the address pointed to by the content of any of the above registers in the default data segment.

Example: MOV AX, [BX]

Here, data is present in a memory location in DS whose offset address is in BX. The effective address of the data is given as $10H \cdot DS + [BX]$.

- 5. Indexed:** In this addressing mode, the offset of the operand is stored in one of the index registers. DS and ES are the default segments for index register SI and DI respectively. This mode is a special case of the above-discussed register indirect addressing mode.

Example: MOV AX, [SI]

Here, data is available at an offset address stored in SI in DS. The effective address, in this case, is computed as $10H \cdot DS + [SI]$.

- 6. Register Relative:** In this addressing mode, the data is available at an effective address formed by adding an 8-bit or 16-bit displacement with the content of any one of the registers BX, BP, SI, and DI in the default (either DS or ES) segment. The example is given before explains this mode.

Example: MOV Ax, 50H [BX]

Here, the effective address is given as $10H \cdot DS + 50H + [BX]$.

- 7. Based Indexed:** The effective address of data is formed, in this addressing mode, by adding the content of a base register (any one of BX or BP) to the content of an index register (any one of SI or DI). The default segment register may be ES or DS.

Example: MOV AX, [BX] [SI]

Here, BX is the base register and SI is the index register. The effective address is computed as $10H \cdot DS + [BX] + [SI]$.

- 8. Relative Based Indexed:** The effective address is formed by adding an 8-bit or 16-bit displacement with the sum of contents of any one of the base registers (BX or BP) and any one of the index registers, in a default segment.

Example: MOV AX, 50H [BX] [SI]

Here, 50H is an immediate displacement, BX is a base register and SI is an index register. The effective address of data is computed as $160H \cdot DS + [BX] + [SI] + 50H$. For the **control transfer instructions**, the addressing modes depend upon whether the destination location is within the same segment or a different one. It also depends upon the method of passing the destination address to the processor. Basically, there are two addressing modes for the control transfer instructions, viz. inter-segment, and intra-segment addressing modes.

If the location to which the control is to be transferred lies in a different segment other than the current one, the mode is called inter-segment mode. If the destination location lies in the same segment, the mode is called intra-segment.

SELF ASSESSMENT QUESTIONS – 3

9. _____ Instructions transfer data among variables.
10. 8086 instructions may be categorized as _____ and _____.
11. In _____ addressing mode, the data is stored in a register and it is referred using the particular register.
12. In _____ addressing modes, immediate data is a part of instruction, and appears in the form of successive byte or bytes.

6. INSTRUCTION SETS

An instruction is a binary pattern designed inside a microprocessor to perform a specific function. The entire group of instructions that a microprocessor supports is called **Instruction Set**. 8086 has more than **20,000** instructions

Classification of Instruction Set:

- Data Transfer Instructions
- Arithmetic Instructions
- Bit Manipulation Instructions
- Program Execution Transfer Instructions
- String Instructions
- Processor Control Instructions

Data Transfer Instruction:

These instructions are used to transfer data from source to destination. The operand can be a constant, memory location, register, or I/O port address.

- **MOV Des, Src:**

SRC operand can be a register, memory location, or immediate operand. DES can be a register or memory operand. Both Src and Des cannot be memory locations at the same time.

E.g.:

MOV CX, 037A H

MOV AL, BL

MOV BX, [0301 H]

- **PUSH Operand:**

It pushes the operand to the top of the stack.

E.g.: PUSH BX

- **POP Des:**

It pops the operand from the top of the stack to Des. Des can be a general-purpose register, segment register (except CS), or memory location.

E.g.: POP AX

- **XCHG Des, Src:**

This instruction exchanges Src with Des. It cannot exchange two memory locations directly.
E.g.: XCHG DX, AX

- **IN Accumulator, Port Address:**

It transfers the operand from the specified port to the accumulator register. E.g.: IN AX, 0028 H

- **OUT Port Address, Accumulator:**

It transfers the operand from the accumulator to the specified port. E.g.: OUT 0028 H, AX

- **LEA Register, Src:**

It loads a 16-bit register with the offset address of the data specified by the Src.

E.g.: LEA BX, [DI]

This instruction loads the contents of DI (offset) into the BX register

- **LDS Des, Src:**

It loads a 32-bit pointer from the memory source to the destination register and DS.

The offset is placed in the destination register and the segment is placed in DS. To use this instruction the word at the lower memory address must contain the offset and the word at the higher address must contain the segment.

E.g.: LDS BX, [0301 H]

- **LES Des, Src:**

It loads a 32-bit pointer from the memory source to the destination register and ES.

The offset is placed in the destination register and the segment is placed in ES.

This instruction is very similar to LDS except that it initializes ES instead of DS.

E.g.: LES BX, [0301 H]

- **LAHF:**

It copies the lower byte of the flag register to AH.

- **SAHF:**

It copies the contents of AH to the lower byte of the flag register.

- **PUSHF:**

Pushes flag register to the top of the stack.

- **POPF:**

Pops the stack top to flag register.

Arithmetic Instructions:

- **ADD Des, Src:**

It adds a byte to a byte or a word to word.

It effects AF, CF, OF, PF, SF, and ZF flags.

E.g.:

ADD AL, 74H

ADD DX, AX

ADD AX, [BX]

- **ADC Des, Src:**

It adds the two operands with CF. It effects AF, CF, OF, PF, SF, and ZF flags. E.g.:

ADC AL, 74H

ADC DX, AX

ADC AX, [BX]

- **SUB Des, Src:**

It subtracts a byte from another byte or a word from a word. It effects AF, CF, OF, PF, SF, and ZF flags. For subtraction, CF acts as a borrow flag.

E.g.:

SUB AL, 74H

SUB DX, AX

SUB AX, [BX]

- **SBB Des, Src:**

It subtracts the two operands and the borrows from the result. It effects AF, CF, OF, PF, SF, and ZF flags.

E.g.:

SBB AL, 74H

SBB DX, AX

SBB AX, [BX]

- **INC Src:**

It will increments the byte or word by one. The operand can be a register or memory location. It effects AF, OF, PF, SF, and ZF flags. CF is not affected.

E.g.: INC AX

- **DEC Src:**

It will decrements the byte or word by one. The operand can be a register or memory location. It effects AF, OF, PF, SF, and ZF flags. CF is not affected.

E.g.: DEC AX

- **AAA (ASCII Adjust after Addition):**

The data entered from the terminal is in ASCII format. In ASCII, 0 – 9 are represented by 30H – 39H. This instruction allows us to add the ASCII codes. This instruction does not have an operand.

- **Other ASCII Instructions:**

AAS (ASCII Adjust after Subtraction)

AAM (ASCII Adjust after Multiplication)

AAD (ASCII Adjust Before Division)

- **DAA (Decimal Adjust after Addition)**

It is used to make sure that the result of adding two BCD numbers is adjusted to be a correct BCD number. It only works on the AL register.

- **DAS (Decimal Adjust after Subtraction)**

It is used to make sure that the result of subtracting two BCD numbers is adjusted to be a correct BCD number. It only works on the AL register.

- **NEG Src:**

It creates a 2's complement of a given number. That means it changes the sign of a number.

- **CMP Des, Src:**

It compares two specified bytes or words. The Src and Des can be a constant, register, or memory location. Both operands cannot be in a memory location at the same time. The comparison is done simply by internally subtracting the source from the destination. The value of source and destination does not change, but the flags are modified to indicate the result.

- **MUL Src:**

It is an unsigned multiplication instruction. It multiplies two bytes to produce a word or two words to produce a double word.

$AX = AL * Src$

$DX: AX = AX * Src$

This instruction assumes one of the operands in AL or AX. Src can be a register or memory location.

- **IMUL Src:**

It is a signed multiplication instruction.

- **DIV Src:**

It is an unsigned division instruction. It divides word by byte or double word by word. The operand is stored in AX, the divisor is Src and the result is stored as:

AH = remainder AL = quotient

- **IDIV Src:**

It is a signed division instruction.

- **CBW (Convert Byte to Word):**

This instruction converts byte in AL to word in AX. The conversion is done by extending the sign bit of AL throughout AH.

- **CWD (Convert Word to Double Word):**

This instruction converts words in AX to double words in

DX: AX. The conversion is done by extending the sign bit of AX throughout DX.

Bit Manipulation Instructions:

These instructions are used at the bit level. These instructions can be used for:

Testing a zero bit

Set or reset a bit

Shift bits across registers.

- **NOT Src:**

It complements each bit of Src to produce 1's complement of the specified operand. The operand can be a register or memory location.

- **AND Des, Src:**

It performs AND operation of Des and Src. Src can be the immediate number, register, or memory location. Des can be a register or memory location. Both operands cannot be memory locations at the same time. CF and OF become zero after the operation. PF, SF, and ZF are updated

- **OR Des, Src:**

It performs OR operation of Des and Src. Src can be the immediate number, register, or memory location. Des can be a register or memory location. Both operands cannot be memory locations at the same time. CF and OF become zero after the operation. PF, SF, and ZF are updated.

- **XOR Des, Src:**

It performs the XOR operation of Des and Src. Src can be the immediate number, register, or memory location. Des can be a register or memory location. Both operands cannot be memory locations at the same time. CF and OF become zero after the operation. PF, SF, and ZF are updated.

- **SHL Des, Count:**

It shifts bits of byte or word left, by count. It puts zero(s) in LSBs. MSB is shifted into the carry flag. If the number of bits desired to be shifted is 1, then the immediate number 1 can be written in Count. However, if the number of bits to be shifted is more than 1, then the count is put in the CL register.

- **SHR Des, Count:**

It shifts bits of a byte or word right, by count. It puts zero(s) in MSBs. LSB is shifted into the carry flag. If the number of bits desired to be shifted is 1, then the immediate number 1 can be written in Count. However, if the number of bits to be shifted is more than 1, then the count is put in the CL register.

- **ROL Des, Count:**

It rotates bits of byte or word left, by count. MSB is transferred to LSB and also to CF. If the number of bits desired to be shifted is 1, then the immediate number 1 can be written in Count. However, if the number of bits to be shifted is more than 1, then the count is put in the CL register.

- **ROR Des, Count:**

It rotates bits of a byte or word right, by count. LSB is transferred to MSB and also to CF. If the number of bits desired to be shifted is 1, then the immediate number 1 can be written in Count. However, if the number of bits to be shifted is more than 1, then the count is put in the CL register.

Program execution Transfer Instructions:

These instructions cause a changes in the sequence of the execution of the instruction. This change can be through a condition or sometimes unconditional. The conditions are represented by flags.

- **CALL Des:**

This instruction is used to call a subroutine or function or procedure. The address of the next instruction after CALL is saved onto the stack.

- **RET**

It returns the control from the procedure to the calling program. Every CALL instruction should have a RET.

- **JMP Des:**

This instruction is used for the unconditional jump option from one place to another.

- **Jxx Des (Conditional Jump):**

All the conditional jumps follow some conditional statements or any instruction that affects the flag.

Mnemonic Meaning Jump Condition		
JA	Jump if Above	CF = 0 and ZF = 0
JAЕ	Jump if Above or Equal	CF = 0
JB	Jump if Below	CF = 1
JBE	Jump if Below or Equal	CF = 1 or ZF = 1
JC	Jump if Carry	CF = 1
JE	Jump if Equal	ZF = 1
JNC	Jump if Not Carry	CF = 0
JNE	Jump if Not Equal	ZF = 0
JNZ	Jump if Not Zero	ZF = 0
JPE	Jump if Parity Even	PF = 1
JPO	Jump if Parity Odd	PF = 0
JZ	Jump if Zero	ZF = 1

- **Loop Des:**

This is a looping instruction. The number of times looping is required is placed in the CX register. With each iteration, the contents of CX are decremented. ZF is checked whether to loop again or not.

String Instructions:

String in assembly language is just sequentially stored bytes or words. There is a very strong set of string instructions in 8086. By using these string instructions, the size of the program is considerably reduced.

- **CMPS Des, Src:**

It compares the string bytes or words.

- **SCAS String:**

It scans a string. It compares the String with a byte in AL or with the word in AX.

- **MOVS / MOVSB / MOVSW:**

It causes the moving of a byte or word from one string to another. In this instruction, the source string is in Data Segment and the destination string is in Extra Segment. SI and DI store the offset values for the source and destination index.

- **REP (Repeat):**

This is an instruction prefix. It causes the repetition of the instruction until CX becomes zero. E.g.: REP MOVSB STR1, STR2

It copies byte by byte contents. REP repeats the operation MOVSB until CX becomes zero.

Processor Control Instructions:

These instructions control the processor itself. 8086 allows controlling certain control flags that cause the processing in a certain direction, and processor synchronization if more than one microprocessor is attached.

- **STC:**

It sets the carry flag to 1.

- **CLC:**

It clears the carry flag to 0.

- **CMC:**

It complements the carry flag.

- **STD:**

It sets the direction flag to 1. If it is set, string bytes are accessed from a higher memory address to a lower memory address.

- **CLD:**

It clears the direction flag to 0. If it is reset, the string bytes are accessed from the lower memory address to the higher memory address.

7. I/O AND PROGRAMMING

The 8086 microprocessor is one of the families of 8086, 80286, 80386, 80486, Pentium, Pentium I, II, and III Also referred to as the X86 family. Learning any imperative programming language involves mastering a some of common concepts:

Variables

Declaration/definition

Assignment

assigning values to variables

Input/Output:

Displaying messages, Displaying variable values

Control flow:

If – then Loops

Subprograms:

Definition and Usage Programming in assembly language involves mastering the same concepts and a few other issues.

Variables

For the moment we will skip details of variable declaration and simply use the 8086 registers as the variables in our programs. Registers have predefined names and do not need to be declared.

The 8086 has 14 registers. Each of these is a 16-bit register. Initially, we will use four of them – the so-called general-purpose registers:

ax, bx, cx, dx

These four 16-bit registers can also be treated as eight 8-bit registers:

ah, al, bh, bl, ch, cl, dh, dl.

Assignment

Consider the assignment : $x = 42$; $y = 24$; $z = x + y$; In assembly language we carry out the same operation but we use an instruction to denote the assignment operator (“=”). The above assignments would be carried out in 8086 assembly language as follows

```
mov x, 42
mov y, 24
add z, x
add z, y
```

The mov Instruction carries out the assignment. It allows us to place a number in a register or in a memory location (a variable) i.e. it assigns a value to a register or variable.

The 8086 provides a variety of arithmetic instructions. For the moment, we only consider a few of them. To carry out arithmetic such as addition or subtraction, you use the appropriate instruction. In assembly language, you can only carry out a single arithmetic operation at a time. This means that if you wish to evaluate an expression such as:

$$z = x + y + w - v$$

You will have to use 3 assembly language instructions – one for each arithmetic operation. These instructions combine assignment with the arithmetic operation.

Example:

```
mov ax, 5      ; load 5 into ax
add ax, 3      ; add 3 to the contents of ax,
                  ; ax now contains 8
inc ax         ; add 1 to ax
                  ; ax now contains 9
dec ax         ; subtract 1 from ax
                  ; ax now contains 8
sub ax, 6      ; subtract 4 from ax
                  ; ax now contains 2.
```

The add instruction adds the source operand to the destination operand, leaving the result in the destination operand.

The destination operand is always the first operand in 8086 assembly language.

The inc instruction takes one operand and adds 1 to it. It is provided because of the frequency of adding 1 to an operand in programming. The dec instruction like inc takes one operand

and subtracts 1 from it. This is also a frequent operation in programming. The subinstruction subtracts the source operand from the destination operand leaving the result in the destination operand.

Input/output

Each microprocessor provides instructions for I/O with the devices that are attached to it, e.g. the keyboard and screen. The 8086 provides the instructions in for input and out for output. These instructions are quite complicated to use, so usually use the operating system to do I/O for us instead. In assembly language, we must have a mechanism to call the operating system to carry out I/O. In addition, must be able to tell the operating system what kind of I/O operation wish to carry out, e.g. to read a character from the keyboard, to display a character or string on the screen, or to do disk I/O. In 8086 assembly language, we do not call operating system subprograms by name; instead, we use a software interrupt mechanism. An interrupt signals the processor to suspend its current activity (i.e. running your program) and to pass control to an interrupt service program (i.e. part of the operating system).

A software interrupt is one generated by a program (as opposed to one generated by hardware). The 8086 **int** instruction generates a software interrupt. It uses a single operand which is a number indicating which MS-DOS subprogram is to be invoked. For I/O and some other operations, the number used is **21h**. Thus, the instruction **int 21h** transfers control to the operating system, to a subprogram that handles I/O operations. This subprogram handles a variety of I/O operations by calling appropriate subprograms. This means that you must also specify which I/O operation (e.g. read a character, display a character) you wish to carry out. This is done by placing a specific number in a register.

The ah register is used to pass this information.

For example, the subprogram to display a character is the subprogram number **2h**.

This number must be stored in the ah register. We are now in a position to describe character output.

When the I/O operation is finished, the interrupt service program terminates, and our program will be resumed at the instruction following **int**.

Character Output

The task here is to display a single character on the screen. There are three elements involved in carrying out this operation using the int instruction:

- 1) We specify the character to be displayed. This is done by storing the character's ASCII code in a specific 8086 register. In this case, we use the **dl** register, i.e. we use dl to pass a parameter to the output subprogram.
- 2) We specify which of MS-DOS's I/O subprograms we wish to use. The subprogram to display a character is subprogram number **2h**. This number is stored in the ah register.
- 3) We request MS-DOS to carry out the I/O operation using the int instruction. This means that we **interrupt** our program and transfer control to the MS-DOS subprogram that we have specified using the ah register.

Example:

Write a code fragment to display the character 'a' on the screen:

```
mov dl, 'a' ; dl = 'a'
mov ah, 2h ; character output subprogram
int 21h ; call MS-DOS output character
```

As you can see, this simple task is quite complicated in assembly language.

Character Input

The task here is to read a single character from the keyboard. There are also three elements involved in performing character input:

1. As for character output, we specify which of MS-DOS's I/O subprograms we wish to use, i.e. the character input from the keyboard subprogram. This is MS-DOS subprogram number **1h**. This number must be stored in the ah register.
2. We call MS-DOS to carry out the I/O operation using the int instruction for character output.
3. The MS-DOS subprogram uses the al register to store the character it reads from the keyboard.

Example:

Write a code fragment to read a character from the keyboard:

```
mov ah, 1h ; keyboard input subprogram  
int 21h; character input; character is stored in al
```

The following example combines the two previous ones, by reading a character from the keyboard and displaying it.

SELF ASSESSMENT QUESTIONS – 4

13. The interrupt generated by a program is called_____.
14. In 8086_____instruction generates a software interrupt.

8. SUMMARY

Let us recapitulate the important concepts discussed in this unit:

- System software is computer software designed to operate and control the computer hardware and to provide a platform for running application software.
- Software development tools like a compiler, debugger, and loader are examples of system software and software that allows users to do things like to create text documents, playing games, listening to music, or surfing the web is called application software.
- 8086 has 16-bit ALU; this means 16-bit numbers are directly processed by 8086. It has a 16-bit data bus, so it can read data or write data to memory or I/O ports either 16 bits or 8 bits at a time. It has 20 address lines, so it can address up to 220.
- The 8086 is a 16-bit microprocessor chip designed by Intel between early 1976 and mid-1978.
- The entire group of instructions that a microprocessor supports is called Instruction Set. 8086 has more than 20,000 instructions.
- The 8086 microprocessor is one of the families of 8086, 80286, 80386, 80486, Pentium, Pentium I, II, and III Also referred to as the X86 family.

9. GLOSSARY

Address: A number, character, or group of characters that identifies a given device or a storage location that may contain a piece of data or a program step.

Architecture: The organizational structure of a system or component. See: component, module, subprogram, routine

ASCII: American Standard Code for Information Interchange.

Data: Representations of facts, concepts, or instructions in a manner suitable for communication, interpretation, or processing by humans or by automated means.

Instruction: A program statement that causes a computer to perform a particular operation or set of operations.

Software: Software is a set of instructions or programs written to carry out a certain tasks on digital computers.

10. TERMINAL QUESTIONS

Short Answer Questions

1. Describe the internal architecture of Intel 8086.
2. Describe the Pin diagram of 8086.
3. Explain three different instruction types.
4. List different addressing modes and explain each.
5. Explain different instruction sets in detail.

10.1 ANSWERS

A. Self-Assessment Questions

1. Assembler
2. 1 megabyte
3. Direct
4. 16-bit data, 20-bit address
5. 5. 2900
6. Instruction queue

7. Flag
8. AD0-AD15
9. Data Transfer
10. Sequential control flows instructions and control transfer instructions.
11. Register
12. Immediate
13. Software Interrupt
14. int

B. Short Answer Questions

1. 8086 architecture is divided into Execution unit (EU) and Bus Interface unit (BIU).
(Refer section 3 for detail)
2. Refer section 3 Pin diagram of 8086 for detail
3. Instruction types are classified into three. (Refer section 4 for detail)
4. Refer section 5 addressing modes for detail.
5. Refer section 6 Instruction sets for detail.

11. SUGGESTED BOOKS

- Dhamdhare (2002). Systems programming and operating systems Tata McGraw-Hill.
- M. Joseph (2007). **System software**, Firewall Media.



**BACHELOR OF COMPUTER
APPLICATIONS
SEMESTER 4**

**DCA2203
SYSTEM SOFTWARE**

Unit 2

Language Processor

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
1.1	Learning Objectives	-	-	
2	Language Processing activities	1, 2, 3, 4	1	4-8
3	Fundamentals of language processing	5, 6, 7, 8, 9, 10	2	9-16
4	Fundamentals of language specification	11	3	17-25
5	Language processor development tools	12, 13	4	26-28
6	Summary	-	-	29
7	Glossary	-	-	30
8	Terminal Questions	-	-	30-31
8.1	Answers	-	-	
9	Suggested books	-	-	31

1. INTRODUCTION

This chapter introduces the concepts underlying the design and implementation of language processors. Mainly, computer programming languages are classified into three categories, they are machine language, assembly language, and high-level language. Machine language is a machine-readable language statements that is a pattern of bits (binary representation) and which can be directly executable by the CPU (Central Processing Unit). Assembly language uses mnemonic codes to represent the low-level machine operations. These instructions should be converted into machine instructions for further processing. For this, one of the language processors called language translator is used. Assembler is the language translator which converts assembly language to machine language.

High-level language (HLL) uses keywords similar to English and is easier to write. Compilers and interpreters are the language translators which convert high-level language into machine language.

In this chapter, we will study different language processing activities in detail. Also, this chapter describes the fundamentals of language processing and language specification. This chapter also explains different language processor development tools like Lex, Yacc, and others.

1.1 Learning Objectives

After studying this unit, learners should be able to:

- ❖ *Define Language Processor*
- ❖ *Describe different Language Processing Activities*
- ❖ *Explain Language specification*
- ❖ *Explain different Language Processor Development tools*

2. LANGUAGE PROCESSING ACTIVITIES

The need for language processing arises from the distinction between how software behaves and how it is used in a computer system. The developer expresses the ideas related to the application domain of the software. To implement these ideas, their description should be interpreted as related to the execution domain of the computer system. Semantics represents the rules of the meaning of a domain; the Semantic gap represents the difference between the semantics of two domains. Software implementation using a PL (Programming Language) introduces a new domain, the PL domain. The semantic gap between the application domain and execution domain is bridged by the software engineering steps. The first step bridges the gap between the PL and execution domains, while the second step bridges the gap between the PL and execution domains. We refer to the gap between the application and PL domains as the specification and design gap or simply the *specification gap*, and the gap between the PL and execution domains as the *execution gap*. The specification gap is bridged by the software development team, while the execution gap is bridged by the designer of the programming language processor, using a translator or an interpreter.

Semantic gaps have many consequences like large development time, large development efforts, and poor quality of software. These issues are tackled using programming languages (PL) as problem-oriented languages (are used with specific applications to bridge the gap between the application domain and execution domain) and procedural-oriented languages (used with most of the applications to provide general purpose facilities to bridge the gap between application domain and execution domain).

Steps for using a PL can be grouped into two:

- 1) Specification, design, and coding steps
- 2) PL implementation Steps

A **language processor** is Software that bridges a specification or execution gap. The activity performed by the language processor is called Language processing. The program that inputs to the language processor are called the source program and its output is the target program. The languages in which these programs are written are called source language

and destination language respectively. A language processor abandons the generation of the target program if it detects errors in the source program.

Some examples of language processors are,

- 1) A *language translator* bridges the execution gap to the machine language of a computer system. (E.g: Assembler, compiler, interpreter, etc.)
- 2) The *detranslator* bridges the same execution gap as a language translator but in the reverse direction.
- 3) *Preprocessor* is another language processor which bridges the execution gap.
- 4) The *language migrator* bridges the specification gap between two programming languages.

Language processing activities are those that bridge the specification gap and those that bridge the execution gap. These can be divided into two.

- 1) Program generation activities.
- 2) Program execution activities.

Program generation activity aims at the automatic generation of a program.

A program execution activity organizes the execution of a program written in a programming language in a computer system.

Program Generation:

A program generator is a software system that accepts the specifications of a program to be generated and executes the program in target programming languages. The program generator is the middleware between the program generator and the application. The specification gap is the gap between the application domain and the program generating domain. Figure 2.1 shows the program generation activities.

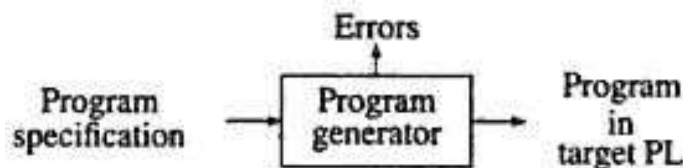


Fig 2.1: Program generation

Reduction in specification gap will increase the reliability of the program. The generator is Bridging the gap to the PL (Programming Language) domain. By testing the correctness of the specification input, we can test an application generated by using a generator. The compiler or interpreter for the PL bridges the execution gap between the target PL domain and the execution domain.

Program Execution:

Translation and interpretation are the two popular models for program execution.

Program Translation:-

This model bridges the execution gap by translating a program written in a PL, which is treated as the source program (SP), into an equivalent program in the machine or assembly language of the computer system which treated as the target program (TP), (see fig 2.2).

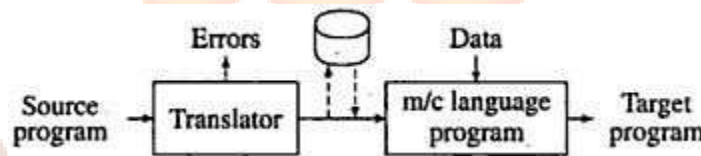


Fig 2.2: Program translation Model

Characteristics of the program translation model are,

1. A program must be translated before it can be executed.
2. The translated program may be saved in a file. The saved program may be executed repeatedly.
3. A program must be retranslated following modifications.

Program interpretation:

The interpreter reads the source program and stores it in memory. During interpretation, it reads a statement, determines its meaning, and performs actions that implement it. This includes computational and input-output actions. Figure 2.3(a) shows the schematics of program interpretation. Fig 2.3(b) shows the schematic of the execution of a machine language program by the CPU of a computer system. CPU uses *Instruction Pointer (IP)* to note the address of the next instruction to be executed. The instruction execution cycle consists of the following steps:

1. Fetch the instruction.
2. Decode the instruction to find out the operation to be performed, and also its operands.

3. Execute the instruction.

At the end of one cycle, the program counter will be updated by the next instruction and the cycle will be repeated.

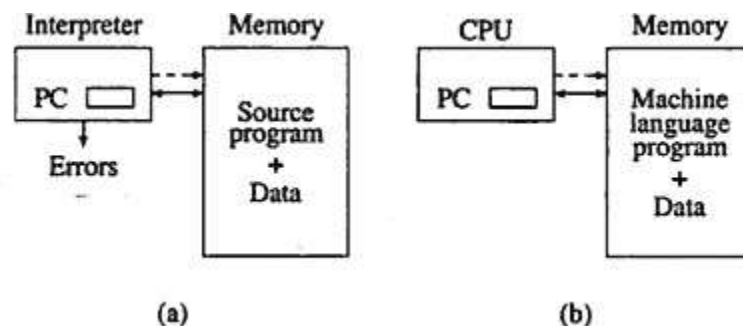


Fig 2.3: Schematics of (a) interpretation, (b) Program execution

The program interpretation cycle consists of the following steps:

1. Fetch the statement.
2. Analyze the statement and find out its meaning, the computation to be performed, and its operands.
3. Execute the meaning of the statement.

In this case, the source program is retained in the source form itself, i.e. no target program form exists, and a statement is analyzed during its interpretation.

Comparison:

In the use of the program translation model, a fixed cost is incurred. If the source program is modified, the translation cost must be incurred again irrespective of the size of the modification. Execution of the target program is efficient since the target program is in the machine language. The use of the interpretation model does not incur translation overheads. This is advantageous if the program is modified between executions, as in program testing and debugging. Interpretation is slower than the execution of a machine language program. Figure 2.4 shows a flow diagram of the Activities of a language processor.

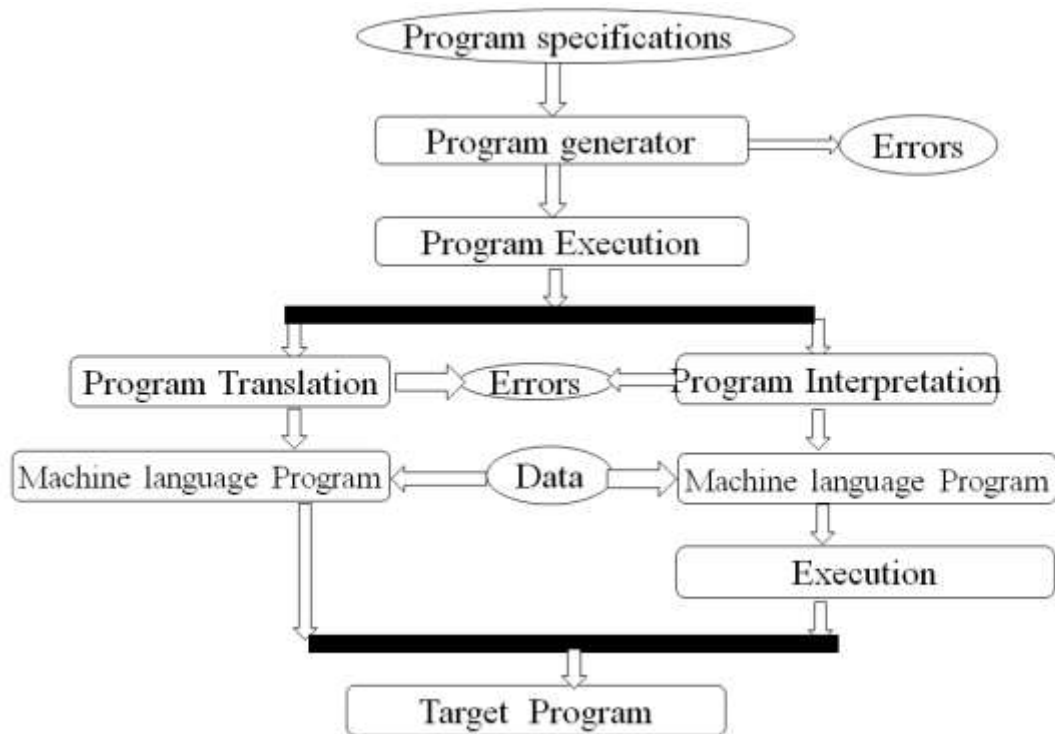


Fig 2.4: Activities of a language processor

SELF ASSESSMENT QUESTIONS - 1

1. The software which bridges a specification or execution gap is called _____.
2. A _____ bridges the execution gap to the machine language of a computer system.
3. CPU uses _____ to note the address of the next instruction to be executed.
4. _____ and _____ are the two popular models for program execution.

3. FUNDAMENTALS OF LANGUAGE PROCESSING

Analysis of Source Program (SP) + Synthesis of Target Program (TP) is called language processing.

A specification of the source language forms the basis of source program analysis. The specification consists of three components:

- 1) Lexical rules govern the formation of valid tokens in the source language.
- 2) Syntax rules govern the formation of valid statements in the source language.
- 3) Semantic rules associate meaning with valid statements of the language.

To determine relevant information concerning a statement in the source program, the analysis phase uses each component of the source language specification. Thus, analysis of the source statement consists of lexical, syntax, and semantics analysis. The synthesis phase is concerned with the construction of a target language statement that has the same meaning as a source statement. This consists of 2 main activities:

- Creation of data structures in the target program
- Generation of target code

These activities are called memory allocation and code generation respectively.

Phases and passes of a language processor:

A language processor consists of two distinct phases. They are the analysis phase and synthesis phase. Figure 2.5 shows the schematics of the language processor.

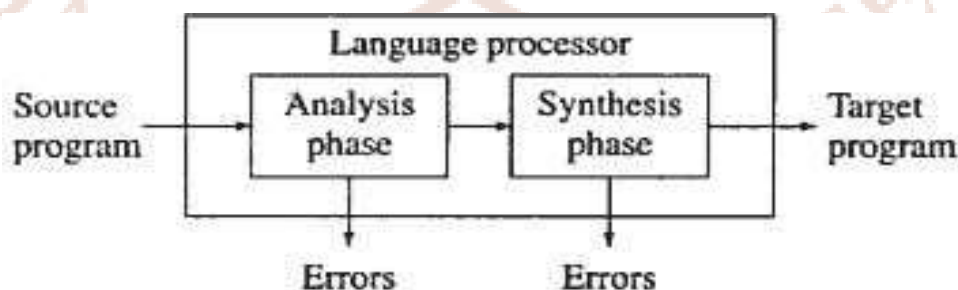


Fig 2.5: Phases of a language processor

This figure shows the impression that: language processing can be performed on a statement by statement basis. That is, analysis of the source statement can be immediately followed by synthesis of equivalent target statements. This may not be feasible due to

- Forward references.
- Issues concerning memory requirements and organization of a language processor.

Forward Reference:

A forward reference of a program entity is a reference to the entity which precedes its definition in the program.

A language processor does not possess all relevant information concerning the referenced entity while processing a statement containing a forward reference. This creates problems during the synthesis phase. This problem can be solved by postponing the generation of the target code until more information concerning the entity becomes available. This also reduces the memory requirements of the language processor and simplifies its organization.

Consider the example:

```
If (12>10) then
goto A;
.....
.....
```

A: Display '12 is greater'

Here, the statement 'goto A' constitutes a forward reference because the declaration of label 'A' occurs later in the program.

Language Processor Pass:

A language processor pass is the processing of every statement in a source program, or its equivalent representation, to perform a language processing function or a set of language processing functions.

Intermediate Representation (IR) of programs:

An intermediate representation (IR) is a representation of a source program that reflects the effect of some, but not all, analysis and synthesis tasks performed during language processing. Figure 2.6 depicts the schematic of a two-pass language processor.

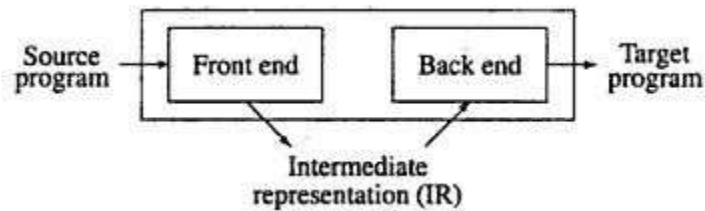


Fig 2.6: Two-pass schematic for language processing

The first pass performs an analysis of the source program and reflects its results in the intermediate representation. To perform synthesis of the target program; the second pass reads and analyses the IR, instead of the source program. This avoids repeated processing of the source program.

The first pass is concerned exclusively with source language issues. Hence it is called the front end of the language processor. The second pass is concerned with program synthesis for a specific target language. Hence it is called the back end of the language processor.

Desirable properties of an IR are:

- Ease of use: IR should be easy to construct and analyze.
- Processing Efficiency: efficient algorithms must exist for constructing and analyzing the IR.
- Memory efficiency: IR must be compact.

The front end of a language processor analyses the source program and constructs an IR. All actions performed by the front end, except lexical and syntax analysis, are called semantic actions. They are the following:

- 1) Checking semantic validity of constructs in SP
- 2) Determining the meaning of SP
- 3) Constructing an IR.

A Toy Compiler:

In the following section, we can see the front end and back end of a toy compiler.

The Front End:

The front end performs lexical, syntax, and semantic analysis of the source program. Each kind of analysis involves the following functions:

1. Determine the validity of a source statement from the viewpoint of analysis.
2. Determine the 'content' of a source statement.
3. Construct a suitable representation of the source statement for use by subsequent analysis functions, or by the synthesis phase of the language processor.

Each analysis represents the 'content' of a source statement in the form of (1) tables of information and (2) description of a source statement. The tables and descriptions at the end of semantic analysis form the IR of the front end.

Output of the front end:

The Intermediate Representation (IR) produced by the front end consists of two components.

- 1) Tables of information.
- 2) An intermediate code(IC) is a description of the source program.

Tables:

Tables contain the information obtained during different analyses of SP (Source Program). The most important table which contains information concerning all identifiers used in the SP is the Symbol table. The symbol table is built during lexical analysis. The semantic analysis adds information concerning symbols while processing declaration statements. It may also add new names designating temporary results.

Intermediate Code(IC):

The Intermediate Code(IC) is a sequence of IC units. These IC units represent the meaning of one action in SP. IC units may contain references to the information in various tables.

For example, the following figure 2.7 shows the IR produced by the analysis phase for the program.

```
i: integer; a, b:
real;
a: = b+;
```

In the following example, the symbol table contains information concerning the identifiers and their types. This information is determined during lexical and semantic analysis, respectively. In IC, the specification (Id, #1) refers to the id occupying the first entry in the table. Note that i^* and temp are temporary names added during the semantic analysis of the assignment statement.

Symbol table

	<i>symbol</i>	<i>type</i>	<i>length</i>	<i>address</i>
1	i	int		
2	a	real		
3	b	real		
4	i^*	real		
5	temp	real		

Intermediate code

1. Convert (Id, #1) to real, giving (Id, #4)
2. Add (Id, #4) to (Id, #3), giving (Id, #5)
3. Store (Id, #5) in (Id, #2)

Fig 2.7: IR example**Lexical Analysis (Scanning):**

Lexical analysis identifies the lexical units in a source statement. The units are then entered into various tables after being divided into various lexical classes, such as ids, constants, reserved ids, etc. This classification may be based on the nature of a string or on the specification of the source language. Lexical analysis builds a descriptor called a token, for each lexical unit. A token contains two fields; they are class code and number in class. Class code identifies the class to which a lexical unit belongs. Number in class is the entry number of the lexical unit in the relevant table.

Syntax Analysis (Parsing):

Syntax analysis processes the string of tokens which are built by lexical analysis to determine the statement class, e.g. Assignment statement, if statement, etc. it then builds an IC which

represents the structure of the statement. The IC is passed to semantic analysis to determine the meaning of the statement.

Semantic Analysis:

Semantic analysis of declaration statements differs from the semantic analysis of imperative statements. Semantic analysis of declaration statements results in the addition of information in the symbol table (e.g. type, length, and dimensionality of variables). Semantic analysis of imperative statements identifies the sequence of actions necessary to implement the meaning of a source statement. In both cases, the structure of a source statement guides the application of the semantic rules. When semantic analysis determines the meaning of a subtree in the IC (Intermediate Code), it adds information to a table or adds an action to the sequence of actions. It then modifies the IC to enable further semantic analysis. The analysis ends when the tree has been completely processed. The updated tables and sequence of actions constitute the IR produced by the analysis phase.

The Figure 2.8 front end of the compiler shows the schematic of the front end where arrows indicate the flow of data.

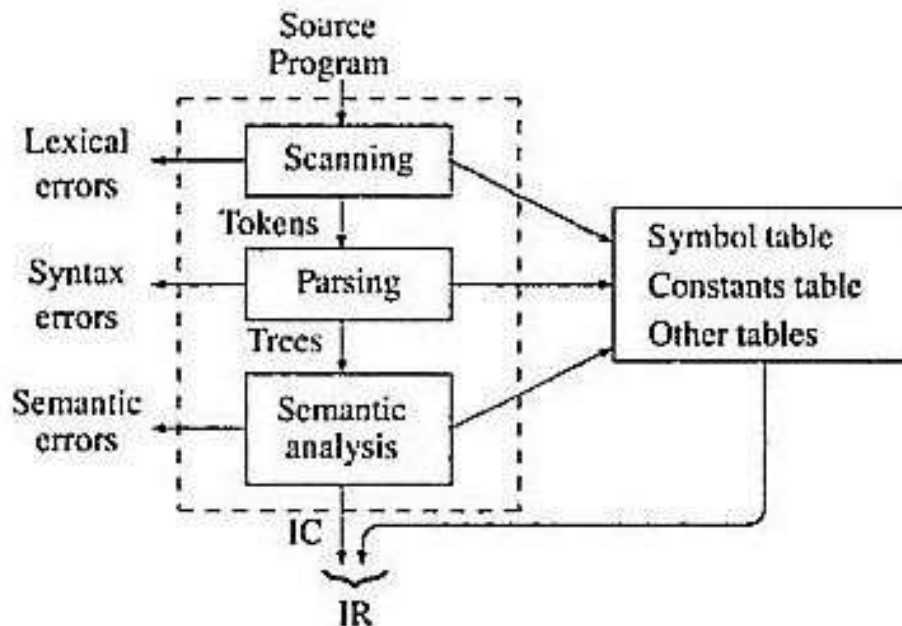


Fig 2.8: Front end of the toy compiler

The Back End:

The back end performs memory allocation and code generation.

Memory allocation:

Memory allocation is a simple task given the presence of the symbol table. The memory requirement of an identifier is computed from its type, length, and dimensionality, and memory is allocated to it. The address of the memory area is entered in the symbol table.

Consider the statement $a = b + i$; after memory allocation, the symbol table looks as shown in figure 2.9.

	<i>symbol</i>	<i>type</i>	<i>length</i>	<i>address</i>
1	i	int		2000
2	a	real		2001
3	b	real		2002

Fig 2.9: Symbol table after memory allocation

Code generation:

Code generation uses knowledge of the target architecture, knowledge of instructions, and addressing modes in the target computer, to select appropriate instructions. The important issues in code generation are

- 1) Determine the places where the intermediate results should be kept, i.e. whether they should be kept in memory locations or held in machine registers. This is a preparatory step for code generation.
- 2) Determine which instructions should be used for type conversion operations.
- 3) Determine which addressing modes should be used for accessing variables.

Figure 2.10 back end of the toy compiler shows a schematic of the back end.

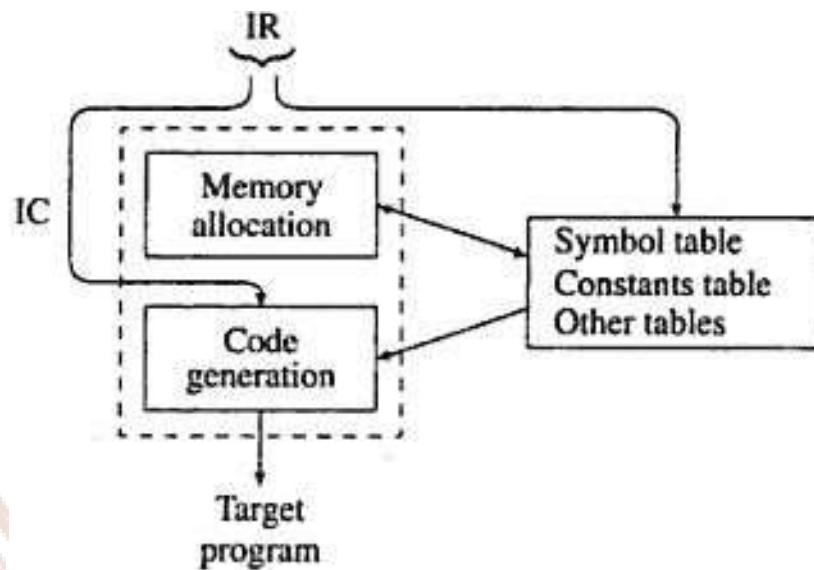


Fig 2.10: Back end of the toy compiler

SELF ASSESSMENT QUESTIONS - 2

5. Analysis of Source Program (SP) + Synthesis of Target Program (TP) is called_____.
6. A_____of a program entity is a reference to the entity which precedes its definition in the program.
7. The most important table which contains information concerning all identifiers used in the Source Program is _____
8. The_____performs memory allocation and code generation

4. FUNDAMENTALS OF LANGUAGE SPECIFICATION

In this section, we will discuss important lexical, syntax, and semantic features of a programming language.

Programming Language Grammars:

The lexical and syntactic features of a programming language are specified by its grammar. A Language L can be a collection of valid sentences. Each Sentence can be looked upon as a sequence of words and each word as a sequence of letters or graphic symbols acceptable in L. A language specified in this manner is known as a *Formal Language*. A Formal Language grammar is a set of rules which precisely specify the sentences of L. Thenatural languages are not formal languages due to their rich vocabulary. However, PLs are formal languages.

Terminal Symbols, Alphabet, and Strings:

The *alphabet* of L, denoted by the Greek symbol Σ , is the collection of symbols in its character set. We will use lower case letters a, b, c, etc. to denote symbols Σ . A symbol in the alphabet is known as a terminal symbol (T) of L. The alphabet can be represented using the mathematical notation of a set, e.g.

$$\Sigma \equiv \{a, b \dots z, 0, 1 \dots 9\}$$

Here, the symbols $\{, ', \text{and} \}$ are part of the notation. We call them meta symbols to differentiate them from terminal symbols. We assume that Meta symbols are distinct from terminal symbols. If this is not the case, i.e. if a terminal symbol and Meta symbol are identical, we enclose the terminal symbol in quotes to differentiate it from the meta-symbol. For example, the set of punctuation symbols of English can be defined as

$$\{ :, ;, ', \dots \}, \text{ Where } ' \text{ denotes the terminal symbol 'comma'.$$

A string is a finite sequence of symbols. We will represent string by Greek symbols α, β, γ , etc. Thus $\alpha = axy$ is a string over Σ . The length of the string is number of symbols in it. The absence of any symbol is also a string, the null string ϵ . The concatenation operation combines two strings into a single string. It is used to build larger strings from existing strings. For example, if $\alpha = ab$, $\beta = axy$, then the concatenation of α and β , is represented as $\alpha \cdot \beta$ or simply $\alpha\beta$, gives the string $abaxy$. The null string also participated in the concatenation, thus $a \cdot \epsilon = \epsilon \cdot a = a$.

Nonterminal Symbols:

A Nonterminal symbol (NT) is the name of a syntax category of a language, e.g. noun, verb, etc. An NT is written as a single capital letter, or as a name enclosed between < ... >, e.g. A or <Noun>. During grammatical analysis, a nonterminal symbol represents an instance of the category. Thus, < Noun> represents a noun.

Productions:

A production, also called a rewriting rule, is a rule of grammar. Production has the form,

A nonterminal symbol ::= String of Ts and NTs.

and defines the fact that the NT on the LHS of the production can be rewritten as the string of (Terminal Symbols) Ts and NTs appearing on the RHS. When an NT can be written as one of many different strings, the symbol '|' (standing for 'or') is used to separate the strings on the RHS, e.g.

< Article > ::= a | an | the

The string on the RHS of a production can be a concatenation of component strings, e.g. the production

< Noun Phrase > ::= <Article > < Noun >

expresses the fact that the noun phrase consists of an article followed by a noun.

Each grammar G defines a language LG. G contains an NT called the distinguished symbol or the start NT of G. unless otherwise specified, we use the symbol S as the distinguished symbol of G. A valid string α of LG is obtained by using the following procedure

1. Let $\alpha = 'S'$.
2. While α is not a string of terminal symbols
 - (a) Select an NT appearing in α , say X.
 - (b) Replace X with a string appearing on the RHS of production of X.

Derivation, reduction, and parse trees:

A grammar G is used for two purposes, to generate valid strings of LG and to recognize valid strings of LG. The derivation operation helps to generate valid strings while the reduction

operation helps to recognize valid strings. A parse tree is used to depict the syntactic structure of a valid string as it emerges during a sequence of derivations or reductions.

Derivation:

Let production P1 of grammar G be of the form

$$P1:A ::= \alpha$$

And let p be a string such that $p \equiv \gamma A \delta$, then the replacement of A by α in string p constitutes a derivation according to production P1. We use the notation $N \rightarrow \eta$ to denote the direct derivation of η from N and $N \rightarrow^* \eta$ to denote the transitive derivation of η from N , respectively. δ is a valid string according to G only if $S \rightarrow^* \delta$, where S is the distinguished symbol of G .

Reduction:

Let Production P1 of grammar G be of the form

$$P1 : A ::= \alpha$$

and let σ be a string such that $\sigma = \gamma \alpha \delta$, then the replacement of α by A in string σ constitutes a reduction according to production P1. We use the notations $\eta \rightarrow N$ and $\eta \rightarrow^* N$ to depict direct and transitive reduction, respectively. Thus, $\alpha \rightarrow A$ only if $A ::= \alpha$ is a production of G and $\alpha \rightarrow^* A$ if $\alpha \rightarrow \dots \rightarrow A$. we define the validity of some string δ according to grammar G as follows: δ is a valid string of LG if $\delta \rightarrow^* S$, where S is a distinguished symbol of G .

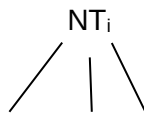
Parse Trees:

A sequence of derivations or reductions reveals the syntactic structure of a string with respect to G . We represent the syntactic structure in the form of a parse tree. Derivation according to the production $A ::= \alpha$ gives rise to the following elemental parse tree:



(Sequence of Ts and NTs constituting α)

A subsequent step in the derivation replaces an NT in α , say NT_i , with a string. We can build another elemental parse tree to depict this derivation,



We can combine the two trees by replacing the node of NT_i in the first tree with this tree. In essence, the parse tree has grown in the downward direction due to a derivation. We can obtain a parse tree from a sequence of reductions by performing the converse actions. Such a tree would grow in the upward direction.

Recursive Specification:

Consider a Grammar say, Grammar 2.1,

Grammar 2.1

```

<exp> ::= <exp>+<term> | <term>
<term> ::= <term>*<factor>|<factor>
<factor> ::= <factor><primary>| <primary>
<primary> ::= <id>|<constant>|(<exp>)
<id> ::= <letter>|<id><letter>|<id><digit>
<const> ::= [+|-]<digit>|<const><digit>
<letter> ::= a|b|c|...|z
<digit> ::= 0|1|2|3|4|5|6|7|8|9
  
```

This is a complete grammar for an arithmetic expression containing the operators $\dagger$ (exponentiation), $*$ and $+$. This grammar uses the notation known as the Backus NaurForm (BNF). Apart from the familiar elements $::=$, $|$ and $\langle \dots \rangle$, a new element here is $[\dots]$, which is used to enclose an optional specification. Thus, the rules for $\langle id \rangle$ and $\langle const \rangle$ in the above grammar are equivalent to the rules

```

<id> ::= <letter>|<id><letter>|<id><digit>
<const> ::= <digit>+<digit>|-<digit>|<const><digit>
  
```

This grammar uses recursive specification, whereby the NT being defined in a production itself occurs in a RHS string of the production, e.g. $X ::= \dots X \dots$

The RHS alternative employing recursion is called a recursive rule. Recursive rules simplify the specification of recurring constructs.

Classification of Grammars:

Grammars are classified based on the nature of the productions used in them. Each grammar class has its own characteristics and limitations.

Type-0 Grammars:

These grammars are known as phrase structure grammars, contain productions of the form

$$\alpha ::= \beta$$

where both α and β can be strings of Ts and NTs. Such productions permit arbitrary substitution of strings during derivation or reduction; hence they are not relevant to the specification of programming languages.

Type - 1 Grammars:

These grammars are known as context-sensitive grammars because their productions specify that derivation or reduction of strings can take place only in specific contexts. A Type-1 production has the form

$$\alpha A \beta ::= \alpha \pi \beta$$

Thus, a string π in a sentential form can be replaced by 'A' only when it is enclosed by the strings α and β . These grammars are also not particularly relevant for PL specification since recognition of PL constructs is not context-sensitive in nature.

Type - 2 Grammars:

These grammars impose no context requirements on derivations or reductions. A typical Type-2 production is of the form

$$A ::= \pi$$

which can be applied independently of its context. These grammars are therefore known as context-free grammars (CFG). CFGs are ideally suited for programming language specification.

Type - 3 Grammars:

Type-3 grammars are characterized by productions of the form

$$A ::= tB \mid t \text{ or}$$
$$A ::= Bt \mid t$$

Note that these productions also satisfy the requirements of Type-2 grammars. The specific form of the RHS alternatives- namely a single T or a string containing a single T and a single NT- gives some practical advantages in scanning. However, the nature of the productions restricts the expressive power of these grammars, e.g. nesting of constructs or matching of parentheses cannot be specified using such productions. Hence, the use of Type-3 productions is restricted to the specification of lexical units, e.g. identifiers, constants, labels, etc. The Productions for <constant> and <identifier> in the grammar (2.1) are in fact Type-3 in nature. This can be seen clearly when we rewrite the production for < id > in the form $Bt \mid t$, viz.

$$\langle id \rangle ::= l \mid \langle id \rangle l \mid \langle id \rangle d$$

where l and d stand for a letter and digit respectively.

Type-3 grammars are also known as linear grammars or regular grammars. These are further categorized into left-linear and right-linear grammars depending on whether the NT in the RHS alternative appears at the extreme left or extreme right.

Operator grammars:

An operator grammar is a grammar none of whose productions contain two or more consecutive NTs in any RHS alternative.

Thus, nonterminals occurring in an RHS string are separated by one or more terminal symbols. All terminal symbols occurring in the RHS strings are called Operators of the grammar.

Ambiguity in Grammatic specification:

Ambiguity implies the possibility of different interpretations of a source string. In natural languages, ambiguity may concern the meaning or syntax category of a word, or the syntactic structure of a construct. For example, a word can have multiple meanings or can be both noun and verb (e.g. the word 'base') and a sentence can have multiple syntactic structures (e.g. 'police ordered to stop speeding on roads').

Formal language grammars avoid ambiguity at the level of a lexical unit or a syntax category. This is achieved by the simple rule that identical strings cannot appear on the RHS of more than one product in the grammar. The existence of ambiguity at the level of the syntactic structure of a string would mean that more than one parse tree can be built for the string. In turn, this would mean that the string can have more than one meaning associated with it.

Eliminating ambiguity:

An ambiguous grammar should be rewritten to eliminate ambiguity. In figure

2.10 the first tree does not reflect the conventional meaning associated with $a+b*c$, while the second tree does. Hence the grammar must be rewritten such that the reduction of '*' precedes the reduction of '+' in $a+b*c$. the normal method of achieving this is to use a hierarchy of NTs in the grammar and to associate the reduction or derivation of an operator with an appropriate NT.

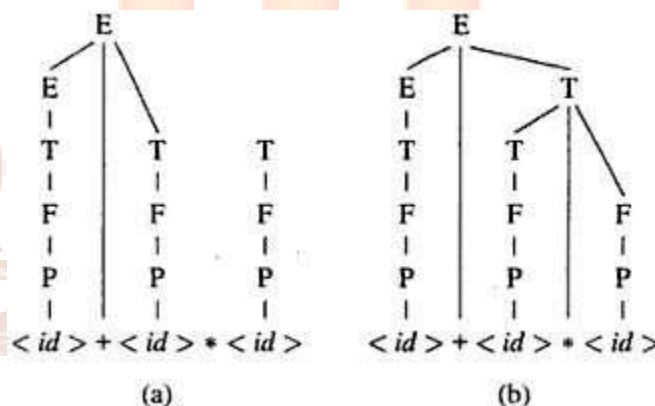


Fig 2.11: Ensuring a unique parse tree for an expression

Figure 2.11 illustrates the reduction of $a+b*c$ according to grammar 2.1. Part (a) depicts an attempt to reduce $a+b$ to $\langle \text{exp} \rangle$. This attempt fails because the resulting string $\langle \text{exp} \rangle * \langle \text{id} \rangle$ cannot be reduced to $\langle \text{exp} \rangle$. part(b) depicts the correct reduction of $a+b*c$ in which $b*c$ is first reduced to $\langle \text{exp} \rangle$. this sequence of reductions can be explained as follows:

Grammar 2.1 associates the recognition of '*' with the reduction of a string to a $\langle \text{term} \rangle$, which alone can take part in a reduction involving '+'. Consequently, in $a+b*c$, '*' must be necessarily reduced before '+'. This yields the conventional meaning of the string. Other NTs, viz. $\langle \text{factor} \rangle$ and $\langle \text{primary} \rangle$, similarly take care of the operator '+' and the parentheses '(...)'. Hence there is no ambiguity in grammar.

Binding and Binding Times:

Each program entity pei in program P has a set of attributes $A_i \equiv \{a_j\}$ associated with it. If pei is an identifier, it has an attribute *kind* whose value indicates whether it is a variable, a procedure, or a reserved identifier (i.e. a Keyword). A variable has attributes like *type*, *dimensionality*, *scope*, *memory address*, etc. Note that the attribute of one program entity may be another program entity. For example, *type* is an attribute of a variable. It is also a program entity with its own attributes, e.g. *size* (i.e. number of memory bytes). The values of the attributes of the type *typ* should be determined sometime before a language processor processes a declaration statement using that type.

Var: *type*;

Binding (Definition): A binding is the association of an attribute of a program entity with a value.

Binding time is the time at which binding is performed. Thus the *type* attribute of variable *var* is bound to *typ* when its declaration is processed. The *size* attribute of *typ* is bound to a value sometime prior to this binding. We are interested in the following binding times:

1. Language Definition time of L
2. Language implementation time of L
3. Compilation time of P
4. Execution init time of *proc*
5. Execution time of *proc*.

Where L is a programming language, P is a program written in L and *proc* is a procedure in P . Note that language implementation time is the time when a language translator is designed. The preceding list of binding times is not exhaustive; other binding times can be defined, viz. binding at the linking time of P . The language definition of L specifies binding times for the attributes of various entities of a program written in L .

Importance of binding times:

The way a language processor can handle using an entity depends on the binding time of an attribute of a programme entity. A compiler can generate code specifically tailored to a binding performed during or before compilation time. However, a compiler cannot generate

such code for bindings performed during or before compilation time. However, a compiler cannot generate such code for bindings performed later than compilation time. This affects the execution efficiency of the target program.

Static and Dynamic bindings:

Static binding is a binding *performed before* the execution of a program begins.

Dynamic binding is a binding *performed after* the execution of a program has begun.

Static binding led to a more efficient execution of a program than dynamic bindings.

SELF ASSESSMENT QUESTIONS - 3

9. The lexical and syntactic features of a programming language are specified by its_____.
10. A rule of the grammar also called a rewriting rule is_____.
11. A grammar in which no productions contain two or more consecutive NTs in any RHS alternative is called _____
12. Association of an attribute of a program entity with a value is called _____.
13. A binding *performed before* the execution of a program begins is called _____.

5. LANGUAGE PROCESSOR DEVELOPMENT TOOLS

The analysis phase of a language processor has a standard form irrespective of its purpose; the source text is subjected to lexical, syntax, and semantic analysis, and the results of the analysis are represented in an IR. Thus writing of language processors is a well-understood and repetitive process that ideally suits the program generation approach to software development. This has led to the development of a set of language processor development tools (LPDTs) focusing on the generation of the analysis phase of language processors.

Figure 2.12 shows the schematic of an LPDT which generates the analysis phase of a language processor whose source language is L.

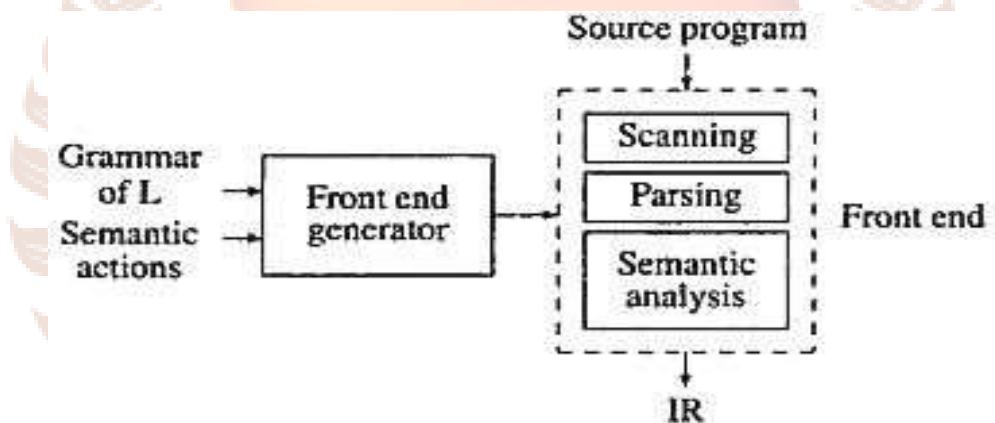


Fig 2.12: A language processor development tool (LPDT)

The LPDT requires the following two inputs:

1. Specification of a grammar of language L
2. Specification of semantic actions to be performed in the analysis phase

It generates programs that perform lexical, syntax, and semantic analysis of the source program and constructs the IR. These programs collectively form the analysis phase of the language processor.

We briefly discuss two LPDTs widely used in practice. These are the lexical analyzer generator LEX, and the parser generator YACC. The input to these tools is a specification of the lexical and syntactic constructs of L, and the semantic actions to be performed on recognizing the constructs. The specification consists of a set of translation rules of the form

<String specification>{ < semantic action>}

Where <semantic action> consists of C code. This code is executed when a string matching <string specification> is encountered in the input. LEX and YACC generate C programs that contain the code for scanning and parsing, respectively, and the semantic actions contained in the specification. A YACC generated parser can use a LEX generated scanner as a routine if the scanner and parser use the same conventions concerning the representation of tokens. Fig 2.12 shows a schematic for developing the analysis phase of a compiler for language L using LEX and YACC.

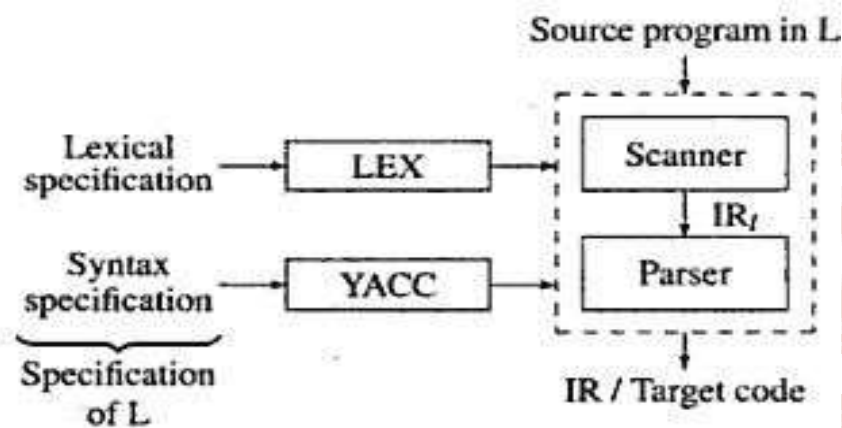


Fig 2.13: Using LEX and YACC

The analysis phase processes the source program to build an intermediate representation (IR). A single pass compiler can be built using LEX and YACC if the semantic actions are aimed at generating target code instead of IR. Note that the scanner also generates an intermediate representation of a source program for use by the parser. We call it IR_f in figure 2.13 to differentiate it from the IR of the analysis phase.

LEX:

LEX accepts an input specification that consists of two components. The first component is a specification of strings representing the lexical units in L, e.g. id's and constants. This specification is in the form of regular expressions. The second component is a specification of semantic actions aimed at building an IR. The IR consists of a set of tables of lexical units and a sequence of tokens for the lexical units occurring in a source statement. Accordingly, the semantic actions make new entries in the tables and build tokens for the lexical units.

YACC:

Each string specification in the input to YACC resembles a grammar production. The parser generated by YACC performs reductions according to this grammar. The actions associated with a string specification are executed when a reduction is made according to the specification. An attribute is associated with every nonterminal symbol. The value of this attribute can be manipulated during parsing. The attribute can be given any user-designed structure. A symbol '\$n' in the action part of a translation rule refers to the attribute of the n^{th} symbol in the Right Hand Side (RHS) of the string specification. '\$\$' represents the attribute of the Left Hand Side (LHS) symbol of the string specification.

SELF ASSESSMENT QUESTIONS – 4

14. Language processor development tools (LPDTs) focuses on generation of the _____ phase of language processors.
15. Two LPDTs widely used in practice are _____ and _____.

6. SUMMARY

Let us recapitulate the important concepts discussed in this unit:

- Computer programming languages are classified into three categories, they are, machine language, assembly language, and high-level language.
- A language processor is Software that bridges a specification or execution gap.
- A language translator is a program that takes input written in a language and produces output written in other programming languages.
- Assembler, compiler, interpreter are examples of language processor.
- Language processing contains different phases, lexical analysis, syntax analysis, and semantic analysis.
- Lexical analysis performs scanning and syntax analysis performs parsing of a given sentence.
- A forward reference of a program entity is a reference to the entity which precedes its definition in the program.
- Syntax analysis processes the string of tokens built by lexical analysis to determine the statement class.
- A binding is the association of an attribute of a program entity with a value.
- The popular language processor development tools are Lex and Yacc.

7. GLOSSARY

Assembly language: A low-level programming language, that corresponds closely to the instruction set of a given computer, allows symbolic naming of operations and addresses and usually results in a one-to-one translation of program instructions [mnemonics] into machine instructions.

Compilation: Translating a program expressed in a problem-oriented language or a procedure-oriented language into object code. In contrast with assembling, interpret

Error: A fault in a program that causes the program to perform in an unintended or unanticipated manner

Syntax: The structural or grammatical rules that define how symbols in a language are to be combined to form words, phrases, expressions, and other allowable constructs.

Translation: Converting from one language form to another. See: assembling, compilation, interpret.

8. TERMINAL QUESTIONS

Short Answer Questions

1. What is language processing? Explain its activities in detail.
2. Define Grammar. Explain different types of Grammars.
3. Write a short note on Lex and Yacc.
4. What is parsing? Why is it required?
5. What is binding? When does it take place?

8.1 Answers

A. Self-Assessment Questions

1. Language Processor
2. Language Translator
3. Instruction Pointer
4. Translation and Interpretation
5. Language Processing
6. Forward reference

7. Symbol table
8. Back end
9. Grammar
10. Production
11. Operator grammar
12. Binding
13. Static binding
14. Analysis
15. LEX and YACC

Short Answer Questions

1. Language processing activities are those that bridge the specification gap and those that bridge the execution gap. (Refer section 2 for detail)
2. The lexical and syntactic features of a programming language are specified by its grammar. (Refer section 4 for detail)
3. LEX and YACC are two language processor development tools. (Refer section 5 for detail)
4. Refer section 3, Syntax analysis (parsing) for detail.
5. A Binding is the association of an attribute of a program entity with a value. Refer section 4, Binding and Binding times for detail.

9. SUGGESTED BOOKS

- Dhamdhere (2002). Systems programming and operating systems Tata McGraw-Hill.
- M. Joseph (2007). System software, Firewall Media.



**BACHELOR OF COMPUTER
APPLICATIONS
SEMESTER 4**

DCA2203

SYSTEM SOFTWARE

Unit 3

Assemblers

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
1.1	Learning Objectives	-	-	
2	Introduction to Assembler	1	-	4 -5
3	Assembler Directives	-	-	6 - 8
4	Forward Reference	-	1	9 - 10
5	Types of Assemblers	-	-	11 - 12
6	Data Structures of Assembler	2, 3, 4	2	13 - 16
7	Assembler Design – One Pass Assembler	-	-	17 - 18
8	Two Pass Assembler	-	3	19 - 20
9	Summary	-	-	21
10	Glossary	-	-	21
11	Terminal Questions	-	-	22
12	Answers	-	-	22 -23
13	Suggested Books and E-References	-	-	23

1. INTRODUCTION

An assembler is a program (system software) that accepts an assembly language program as input and produces its equivalent machine language program as output along with information for the loader. The input to the assembler program is called the source program and the output is called the object program.

This Unit discusses the functions of assemblers in detail. In this unit, we will see the design of an assembler, assembler directives, Data structures used for the construction of the assembler, and Types of assemblers. This unit also discusses single pass and two pass assembler designs.

1.1 Learning Objectives

After studying this unit, you should be able to:

- ❖ *Define an Assembler*
- ❖ *Describe forward references*
- ❖ *Describe Assembler directives*
- ❖ *Explain about Types of Assemblers*
- ❖ *Explain different Data structures of Assembler*
- ❖ *Explain about one pass and two pass Assembler design*

2. INTRODUCTION TO ASSEMBLER

Assembler is a program, which translates Assembly Language Program (ALP) into machine language program (object program). It places the object program in the secondary memory.

An **assembly language** is a machine-dependent, low-level programming language which is specific to a certain computer system (or a family of computer systems). Compared to the machine language of a computer system, it provides three basic features which simplify programming:

1. **Mnemonic operation codes:** The use of mnemonic operation codes (also called mnemonic opcodes) for machine instructions eliminates the need to memorize numeric operation codes. It also enables the assembler to provide helpful diagnostics, for example, the indication of a misspelling of operation codes.
2. **Symbolic operands:** Symbolic names can be associated with data or instructions. These symbolic names can be used as operands in the assembly statement. The assembler performs memory bindings to these names; the programmer need not know any details of the memory bindings performed by the assembler.
3. **Data declarations:** Data can be declared in a variety of notations, including decimal notation. This avoids manual conversion of constants into their internal machine representation, for example, conversion of -5 into (11111010)₂ or 10.5 into (41A80000)₁₆.

An assembler translates a file of assembly language statements into a file of binary machine instructions. The translation process has two major parts. The first step is to find memory locations with labels so that the relationship between symbolic names and addresses is known when instructions are translated. The second step is to translate each assembly statement by combining the numeric equivalents of opcodes, register Specifies, and labels into a legal instruction. The assembler produces an output file, called an object file, which contains the machine instructions, data, and book keeping (reference) information.

The functions of the assemblers can be understood from Fig. 3.1. The assembler receives the assembly language program as input and converts it into the corresponding object program.

It also provides information to the loader. The loader places this object code into the main memory during execution.

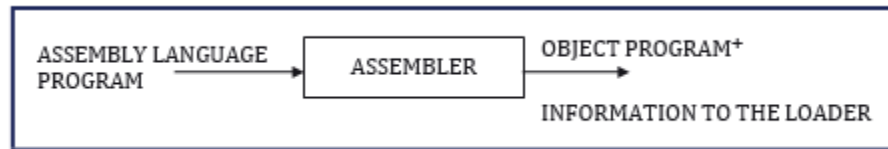


Fig 1: Assembler

The assembler does receive an assembly language program as its input. The output should be an object program. This will be placed generally in the secondary memory. This will be again loaded placed in the main memory for execution of the program by the loader. So the assembler has to also generate necessary information for the loader.

Advantages of coding in assembly language are

- Provides more control over handling particular hardware components
- May generate smaller, more compact executable modules
- Often results in faster execution

Disadvantages

- Not portable
- More complex
- Requires understanding of hardware details (interfaces)

An assembler does the following:

1. Generate machine instructions
 - evaluate the mnemonics to produce their machine code
 - evaluate the symbols, literals, and addresses to produce their equivalent machine addresses
 - convert the data constants into their machine representations
2. Process pseudo operations.

3. ASSEMBLER DIRECTIVES

These are the instructions present in the source program used to instruct the assembler to perform certain actions during the translation of a program. These are not translated into machine language instructions. The assembler directives are also called '*pseudo instructions*'. These are not the instructions for the processors, but for assemblers. The assembler directives discussed are from MASM assembler for the Intel family of architectures. These assembler directives are used in Intel 8086 macro assembler, Borland Turbo Assembler, and IBM macro assembler (MASM).

E.g.: ORG 8000H

-----END

An *assembler directive* is a message to the assembler that tells the assembler something it needs to know in order to carry out the assembly process; for example, an assemble directive tells the assembler where a program is to be located in memory. We are going to use the following directives:

<label>	EQU	<value>	//Equate
	ORG	<value>	//Origin
<label>	DC	<value>	//Define constant
<label>	DS	<value>	//Define storage
	END	<value>	//End of assembly language program and "starting address" for execution

In each case, the term <label> indicates a user-defined label (i.e., symbolic name) that must start in column 1 of the program, and <value> indicates a value that must be supplied by the programmer (this may be a number or a symbolic name that has a value).

EQU

The **EQU** assembler directive simply equates a symbolic name to a numeric value. Consider:

Sunday	EQU	1
Monday	EQU	2

The assembler substitutes the equated value for the symbolic name; for example, if you write the instruction **ADD.B #Sunday, D2**, the assembler treats it as if it were **ADD.B #1, D2**.

You could also write

Sunday	EQU	1
Monday	EQU	Sunday + 1

In this case, the assembler evaluates “Sunday + 1” as 1 + 1 and assigns the value 2 to the symbolic name “Monday”.

Do not think that the EQU directive creates variables or constants. It doesn’t and it does not affect the code generated by the program. This directive simply allows you to make a name equivalent to its value (i.e., it’s a form of shorthand).

Origin

The *origin directive* tells the assembler where to load instructions and data into memory. The first 1024 bytes of memory are reserved for exception vectors. So, Your programs will start at location 1024; that is, you should begin your program with **ORG 1024** or **ORG \$400** (remember that $1024 = 400_{16}$).

Define Constant

The *defined constant* assembler directive allows you to put a data value in *memory at the time that the program is first loaded*. The **DC** directive takes the suffix. **B**, **W**, or **.L**. You can put several values on one line (each value is separated by a comma). The optional *label field* is given the address of the first location in memory allocated to the DC function. Consider the example:

	ORG	\$2000	Locate data here
Val1	DC.B	20,34	Store 20 and 34 in consecutive bytes

Define Storage

The *defined storage* directive is used to reserve one or more memory locations. This directive is similar to the Pascal **type** declaration. Consider:

Result	DS.B 1	Save a byte for Result
Table	DS.W 10	Save 10 words (20 bytes) for Table
Point	DS.L 1	Save 1 long word (4 bytes) for Point



4. FORWARD REFERENCE

It is a reference of a label (variable name), which is defined later in the program. Now consider the simple program in 8086 given in example 3.1.:

Example 3.1

CODE SEGMENT

ASSUME CS: CODE JMP **XXYY**

NOP NOP

XXYY: MOV AX, 0000H

NOP

CODE ENDS

END

This is a simple program written for Intel 8086 processor. This is a delay program. In the above program, there is a JMP statement to statement labeled 'XXYY'. In this case, there is a reference to a label named 'XXYY', which is defined later in the program. This is called forward reference. This is also called forward jump. This causes some problems in the translation of the assembly program into the object program. There can be a backward reference or backward jump possible in the program. In this case, reference to a label or symbol, which is defined already in the program.

The forward reference causes problems during assembly. During the assembly process, the assembler reads the instruction from the source program and translates them one after another. The assembler integrates the opcode and address of each instruction. In the case of an instruction having forward reference, the address will not be readily available. The address is actually defined later, after the particular instruction. See the instruction 'JMP **XXYY**' in example 3.1. The label is defined later in the program. This causes a problem in the assembly process. The assembly cannot be done till the address is known. This is called the *forward reference problem*. This problem is handled by some suitable mechanisms in the assemblers.

Self-Assessment Questions -1

1. A System Software that accepts assembly language program as input and produces its equivalent machine language program as output is called _____.
2. The System Software which places the object code into the main memory during execution is called _____.
3. The instructions present in the source program used to instruct the assembler to perform certain actions during the translation of a program are called _____.
4. The assembler directive simply equates a symbolic name to a numeric value is called _____.
5. A reference of a label, which is defined later in the program is called _____.



5. TYPES OF ASSEMBLERS

The assemblers can be of any one of the following types:

- 1) Single pass assemblers
- 2) Two pass assemblers and
- 3) Multi pass assemblers

Pass is a terminology used in system software. Each reading of a program can be called a **pass**. The assembly process can be done within a pass, if so, such assemblers are single pass assemblers. If the assembly is done in two passes, then those assemblers are called two pass assemblers. The translation of the assembly language program can be done in several passes. Such assemblers are called multi pass assemblers.

Single Pass Assemblers

In single pass assemblers, the entire translation of the assembly language program into an object program is done in only one pass. The source program is read only once. These are also called 'one pass assembler'. These assemblers suffer the problem of Forward references. Handling the forward reference in a single pass assembler is difficult.

The object code can be produced in the single pass assemblers in two different ways. In a first way, the object code is directly loaded into the main memory for execution. Here, no loader is required. This type of loading scheme is called the 'compile and loading scheme'. In a second way, the object program will be loaded into the main memory for execution later as the necessity arises. Here, a separate loader program is necessary.

The forward reference problem is also handled to some extent in single pass assemblers. The assembler scans the source assembly language program, instruction by instruction and translates them into machine codes. If there is an instruction, which has a forward reference, the assembler leaves the address of that particular symbol (operand) and proceeds further. This symbol is entered in the symbol table and marked as undefined. There is another list of instructions that uses this symbol, which is also maintained. In that list, this particular instruction is also included. When the symbol definition is encountered the assembler inserts the address in the symbol table. The assembler also inserts the address to the

instructions, which use this symbol in the list of Forward references. This is how the forward reference problem is handled in the single pass assembler in a single pass. The forward reference problem is conveniently handled in the two pass assemblers. If undefined symbols are encountered after reaching the end of the statement, then the assembler generates the error messages. After translating all the instructions, the object program is used with and without the use of loaders. The control is transferred to the start of the object program for execution.

An assembler, which goes through an assembly language program only once, is known as One-pass assembler. It suffers from a forward reference problem. This is faster because they scan the program only once. This does not have many features supported by the two pass assemblers.

Two Pass Assemblers

The two pass assemblers are widely used and the translation process is done in two passes. The two pass assemblers resolve the problem of Forward references conveniently. An assembler, which goes through an assembly language program twice, is called a two ***pass assembler***. During the first pass, it collects all labels. During the second pass, it produces the machine code for each instruction and assigns the address to each of them. It assigns addresses to labels by counting their position from the starting address. It provides more features than the single pass assembler. It is widely used.

Multi Pass Assemblers

If the assembly process is done more than two passes then those assemblers are called ***multi pass assemblers***. One reading of a program (source program or any program in concern) can be called a pass. In assemblers, the passes are necessary to solve the problem of Forward references. In order to process the entire symbol definitions, several passes are made over the source program.

6. DATA STRUCTURES OF ASSEMBLER

This section discusses the data structures used for the construction of an assembler. The data structures used in the assembler are given below:

- 1) Mnemonics Table (MT)
- 2) Symbol Table (ST)
- 3) Pseudo Instruction Table (PIT)
- 4) Location Counter (LC)
- 5) Literal Table (LT)
- 1) Mnemonics Table (OPTAB)**

Mnemonics Table also called 'OPTAB' is a data structure used for maintaining the details (attributes) about the instructions of an instruction set of any machine. It will have mnemonics of the instructions, corresponding hexadecimal opcode, and the length of the instruction. The structure of the mnemonic table is shown in below figure 3.2

Mnemonics	Opcode	Length
SUB B	97	1
ADD B	80	1
...	...	...
HLT	76	1

Fig 2: Mnemonic Table Organization

This table can also be called a *machine code table* or *Machine opcode table* or *machine table*.

This is a static table (no update) used for maintaining mnemonic, machine code (instruction format, length), etc. Contents of this table include:

- Mnemonic Codes of all instructions:
- Machine language opcode

- Other Information (for architectures with more than one length of instruction format) like Length of instruction and Instruction formats

Operations in pass 1:

- Validate opcodes
- Compute instruction length (in SIC/XE) In pass 2
- Translate opcodes to machine language

2) Symbol Table (SYMTAB)

A symbol table also called SYMTAB is also a data structure used for maintaining the details (attributes) of the symbols present in the assembly language program. The symbols may be labels like identifier names. The attributes, which can be maintained, are name and address. Other attributes like scope and value are also maintained if necessary. The structure of the symbol table is given in the following figure 3.3

Symbol name	Address
A	8080
B	8090
...	...
C	8100

Fig 3: Symbol table organization

The suggested data structure for both symbol table and Mnemonic table can be Hash Table, because of the efficiency in Insertion and retrieval.

Properties

This table is used for storing label name, value, flag, (type, length), etc.

- Dynamic table (insert, delete, search)
- Hash table, non-random keys, a hashing function
- Contents include
 - Name of symbol

- Address (value)
 - Error flags (from pass1)
 - Other information: attributes of data or instruction labeled
- Operations in pass 1
 - Enter labels as they are encountered in the source program
 - Enter addresses from LOCCTR
- Operations in pass 2
 - Look up operand labels for addresses
- Also organized as a hash table
 - Entries only (no deletions)
 - Non-random keys

3) Pseudo Instruction Table (PIT)

Sometimes the assemblers maintain a separate data structure called Pseudo Instruction Table (PIT). This table maintains the details of the Pseudo instructions or assembler directives like ORG, END, ASSUME, and others.

4) Location Counter (LOCCTR)

Location Counter also called 'LOCCTR' is another data structure used to keep the address of the next memory word. LC needs to know the length of different instructions (for increment-ing), hence MT (Mnemonic Table) can be included to have the length of the instruction. The process of maintaining the address of the next memory word can be called 'LC processing'. The location counter is also called an instruction counter. It is a variable, which keeps track of the execution-time address of the instruction being assembled.

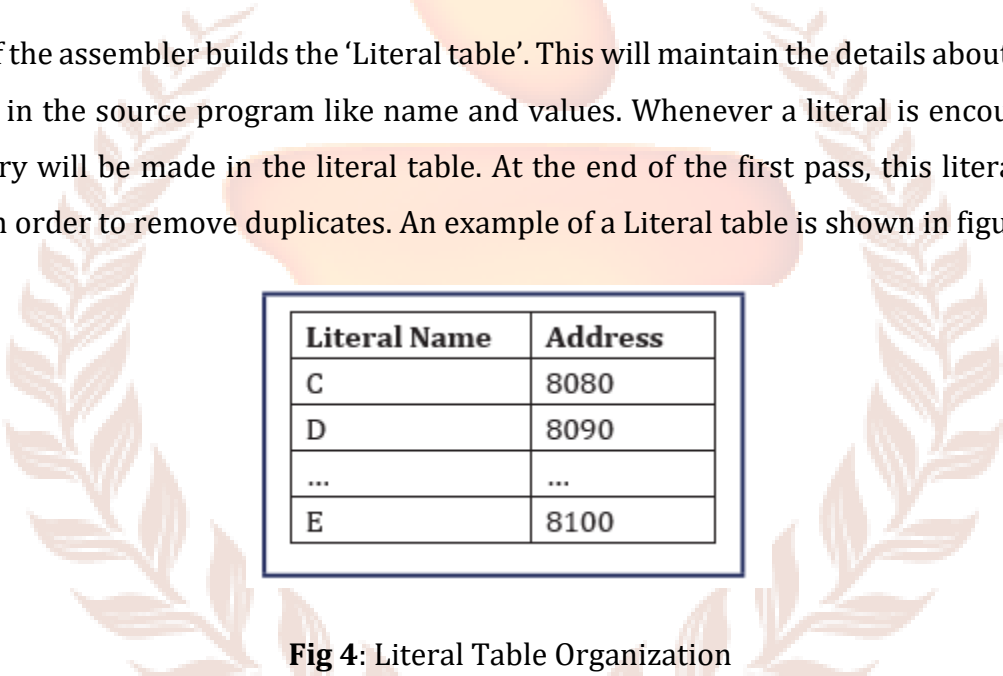
Properties

- i) Counted in bytes.
- ii) Initialized to address specified in START directive.
- iii) Length of each assembled instruction is added to LOCCTR.
- iv) LOCCTR points to starting address of each statement in the program.

Literal Table (LT)

Constants for which assembler automatically reserves memory is called Literals. A literal is an operand that states the required value directly in an instruction. It is an alternative to defining a constant elsewhere in the program and using a label to refer to it. The literals improve the readability of a program by making the value of the constant apparent in the source statement.

Pass 1 of the assembler builds the 'Literal table'. This will maintain the details about the literals used in the source program like name and values. Whenever a literal is encountered, a new entry will be made in the literal table. At the end of the first pass, this literal table is sorted in order to remove duplicates. An example of a Literal table is shown in figure 3.4.



Literal Name	Address
C	8080
D	8090
...	...
E	8100

Fig 4: Literal Table Organization

Self-Assessment Questions -2

- Each reading of a program can be called a _____.
- A data structure used for maintaining the details (attributes) about the instructions of instruction set of any machine is called ____.
- The table maintains the details of the Pseudo instructions or assembler directive is called ____.

7. ASSEMBLER DESIGN – ONE PASS ASSEMBLER

A one-pass assembler scans the program just once. One-pass assemblers are used when,

1. It is necessary or desirable to avoid a second pass over the source program.
2. The external storage for the intermediate file between two passes is slow or inconvenient to use.

Here, the Main problem is forward references to both data and instructions. One simple way to eliminate this problem is, it requires that all areas be defined before they are referenced. But, forward references to labels on instructions cannot be eliminated as easily. The one-pass assembler must make some special provision for handling forward references. There are two types of one-pass assemblers. One type of one-pass assembler produces object code directly in memory for immediate execution. Here, No object program is written out and no loader is needed. The other type of one-pass assembler produces the usual kind of object program for later execution.

The assembler that does not write the object program out and does not need a loader is called a **load-and-go** assembler. It avoids the overhead of writing the object program out and reading it back in. It is useful in a system that is oriented toward program development and testing. A load-and-go assembler can be a one pass assembler or a two-pass assembler.

Handling of Forward references in one-pass load-and-go assembler

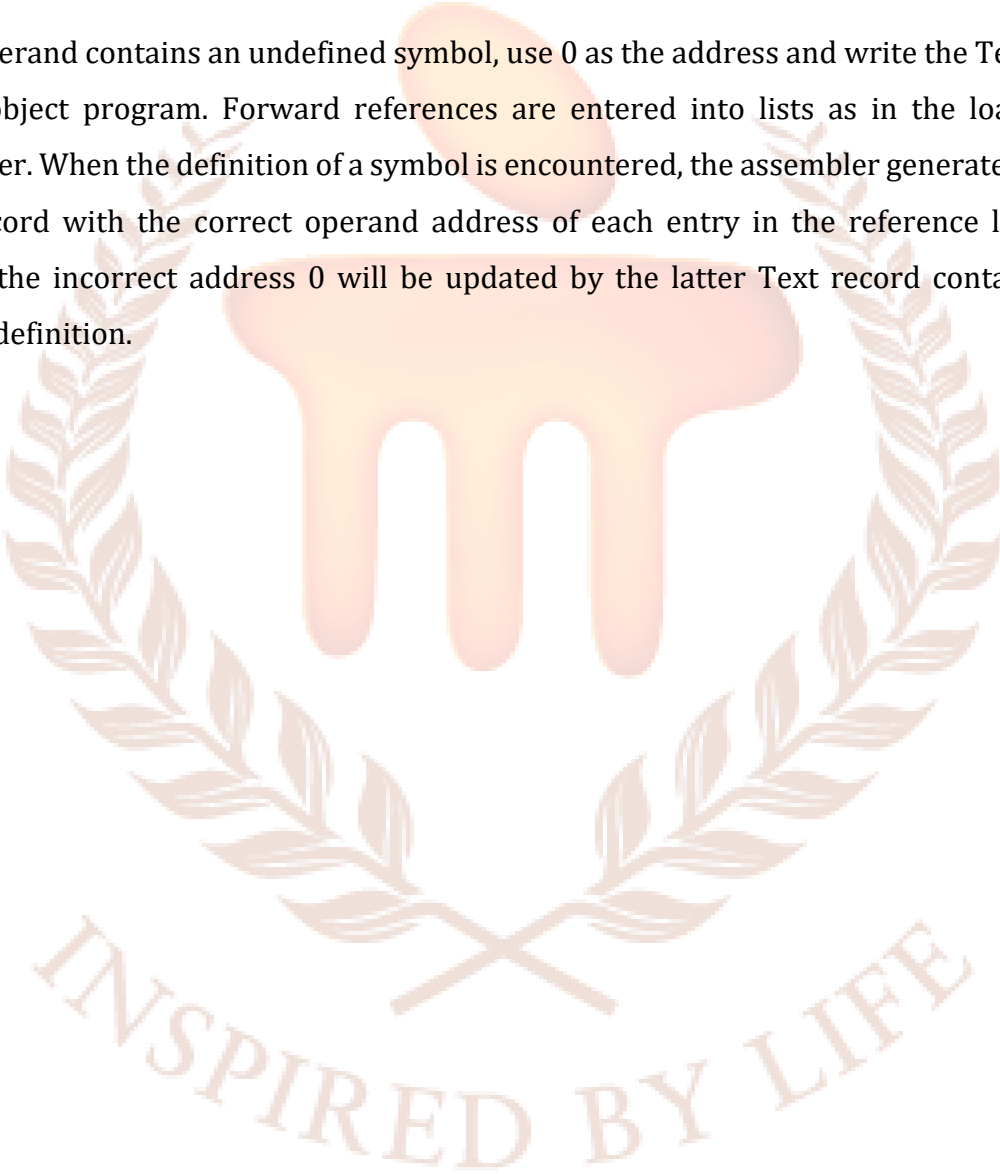
The assembler generates object code instructions as it scans the source program. If an instruction operand is a symbol that has not yet been defined, then,

- The symbol is entered into the symbol table with a flag indicating that the symbol is undefined;
- The operand address is omitted when the instruction is assembled;
- The operand address is added to a list of Forward references associated with the symbol table entry.

When the definition for a symbol is encountered, the forward reference list for that symbol is scanned, and the proper address is inserted into any instructions previously generated. For a load-and-go assembler, the actual address must be known at assembly time.

One Pass Assembler that Produces Object Program

If the operand contains an undefined symbol, use 0 as the address and write the Text record to the object program. Forward references are entered into lists as in the load-and-go assembler. When the definition of a symbol is encountered, the assembler generates another Text record with the correct operand address of each entry in the reference list. When loaded, the incorrect address 0 will be updated by the latter Text record containing the symbol definition.



8. TWO PASS ASSEMBLER

The Design of Two pass assemblers is discussed in this section. The design of two pass assembler involves the design of Pass 1 and Pass 2.

Pass 1 of the Two Pass Assembler

It scans the assembly language program completely and analyses the same. This phase can be also called as *Analysis Phase*. The assembler analyses the assembly language program considering the following factors:

1. Symbols and labels.
2. Mnemonics and
3. Declarations.

The assembler uses the location counter and Mnemonics table for the pass1. It generates an intermediate representation of the source code (IR) and symbol table. The symbol table will maintain the details of the Symbols.

Functions of Pass 1 of the Two Pass Assembler

1. Tracks the location counter
2. Generate the symbol table by determining the symbols, labels, and their addresses
3. Generates the Literal Table (LT)
4. Finds the length of assembly level instructions from the mnemonics table.
5. Processes the assembler directives
6. Finally generates the intermediate representation of the Source Program.

Pass 2 of the Two Pass Assembler

This phase only generates the object code of the source program. It uses the intermediate representation from the pass 1 and symbol table. It actually synthesizes the outputs of the Pass 1 and opcodes to generate the object code. This pass can also be called as *Synthesis Phase*. It just integrates the Opcode and addresses. It uses the symbol table, Literal table, mnemonic Table, and Intermediate representation of the ALP from Pass1. The function of the pass2 is to generate the assembly listing. It should also generate some information

needed by the loader and linkers for linking up procedures assembled at different times into a single executable file and loading into the main memory.

Functions of Pass 2 of the Two Pass Assembler

1. Finds out the machine codes (Opcodes) for the assembly level instructions from the mnemonics table.
2. Finds out the addresses of the symbols from the symbol table and substitutes them in the assembly sequence.
3. Processes the assembler directives
4. Generate the object code sequence
5. Generates necessary information to the loader

Self-Assessment Questions -3

9. The assembler that does not write object program out and does not need a loader is called_____.
10. The phase which scans the assembly language program completely and analyses the same can be also called as_____.
11. The assembler uses the_____, _____for the pass1.

9. SUMMARY

Let us recapitulate the important concepts discussed in this unit:

- An assembler is a program (system software) that accepts assembly language program as input and produces its equivalent machine language program as output along with information for the loader.
- An assembly language is a machine-dependent, low-level programming language which is specific to a certain computer system.
- An *assembler directive* is a message to the assembler that tells the assembler something it needs to know in order to carry out the assembly process.
- Each reading of a program can be called a pass. The assembly process can be done within a pass, if so, such assemblers are single pass assemblers. If the assembly is done in two passes, then those assemblers are called two pass assemblers.
- Constants for which assembler automatically reserves memory is called Literals.

10. GLOSSARY

Assembler: A computer program that translates programs [source code files] written in assembly language into their machine language equivalents [object code files].

Forward Reference: Is a label of the variable name

Literals: Constants for which assembler automatically reserves memory

Location Counter: 'LOCCTR' is the data structure used to keep the address of the next memory word.

Opcode: Another name for Mnemonic operation codes

Pseudo instructions: Instructions in the source program are used to instruct the assembler to perform certain actions during the translation of a program.

Single Pass Assemblers: Translation of assembly language program into object program is done in only one pass. The source program is read only once. Also called as 'one pass assembler'

11. TERMINAL QUESTIONS

SHORT ANSWER QUESTIONS

Q1. Explain briefly about Assembler directives. Q2. Describe Forward Reference.

Q3. Explain about different types of Assemblers.

Q4. Explain about different data structures of Assembler.

Q5. Describe the design of One Pass and Two Pass Assembler.

12. ANSWERS

SELF ASSESSMENT QUESTIONS

1. Assembler
2. Loader
3. Assembler directives
4. EQU
5. Forward Reference
6. Pass
7. Mnemonic Table
8. Pseudo Instruction Table
9. Load and Go Assembler
10. Analysis phase
11. Location counter, mnemonics table

TERMINAL QUESTIONS

SHORT ANSWER QUESTIONS

Answer 1: These are the instructions present in the source program used to instruct the assembler to perform certain actions during the translation of a program. These are not translated into machine language instructions. (Refer section 3 for detail)

Answer 2: Forward Reference is a reference of a label (variable name), which is defined later in the program. (Refer section 4)

Answer 3: There are three types of Assemblers, single pass, two pass and multi pass. (Refer section 5)

Answer 4: Different data structures are used in the passes of the Assembler. (Refer section 6)

Answer 5: A one pass assembler scans the program just once. (Refer section 7) and the design of two pass assemblers involves the design of Pass 1 and Pass 2. (Refer section 8)

13. SUGGESTED BOOKS AND E-REFERENCES

- Dhamdhare (2002). Systems programming and operating systems Tata McGraw-Hill.
- M. Joseph (2007). System software, Firewall Media.



BACHELOR OF COMPUTER APPLICATIONS SEMESTER 4

DCA2203

SYSTEM SOFTWARE

Unit 4

Macros and Macro Processor

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Learning Objectives	-	-	
2	Macro Definition and Call	1	1	4 - 6
3	Macro Expansion	2, 3	-	7 - 11
4	Nested Macro Calls	-	2	12 - 13
5	Advanced Macro Facility	-	-	14 - 19
	5.1 Conditional Expansion	-	-	
	5.2 Other Facilities for Expansion Time Loops	-	-	
	5.3 Semantic Expansion	-	-	
6	Design of Macro Processor	4	3	20 - 22
	6.1 Data Structures Used in Macro Processor Design	-	-	
7	Summary	-	-	23
8	Glossary	-	-	23
9	Terminal Questions	-	-	24
10	Answers	-	-	24 - 25
11	Suggested Books and E-References	-	-	25

1. INTRODUCTION

We have studied various language processors in the previous units. In this unit discusses about macros and macro processors in detail. Before discussing macros and macro processors, some introduction about preprocessors and subroutines are given, which will help in understanding the concepts of macros and macro processors.

In this unit, we are discussing Macro definition and call, Macro expansion, and Nested macro calls. We are also discussing advanced macro facilities. In the last section of this unit, we are discussing the design of a Macro preprocessor.

1.1 Learning Objectives

After studying this unit, you should be able to:

- ❖ *Define a macro and macro processor*
- ❖ *Describe macro definition and call*
- ❖ *Explain macro expansion*
- ❖ *Describe the advanced macro facility*
- ❖ *Explain the design of the macro processor*

2. MACRO DEFINITION AND CALL

In computer science, a preprocessor is a program that processes its input data to produce output that is used as input to another program. The output is said to be a preprocessed form of the input data, which is often used by some subsequent programs like compilers. The amount and kind of processing done depend on the nature of the preprocessor; some preprocessors are only capable of performing relatively simple textual substitutions and macro expansions, while others have the power of full-fledged programming languages.

A common example from computer programming is the processing performed on source code before the next step of compilation. In some computer languages, there is a phase of translation known as preprocessing.

A general purpose macro processor is a macro processor that is not tied to, or integrated with, a particular language or piece of software.

In its simplest form, a macro processor is a preprocessor, which handles the macros. Whenever there is a macro call the macro processor substitutes the definition in the place of the macro call. This is macro expansion. Macros are used to provide a program generation facility through macro expansion.

Definition: A Macro is a unit of specification of program generation through expansion.

A *macro* consists of a name, a set of formal parameters, and a body of code. A macro name is an abbreviation, which stands for some related lines of code. Macros are useful for the following purposes:

- To simplify and reduce the amount of repetitive coding
- To reduce errors caused by repetitive coding
- To make an assembly program more readable.

The use of a macro name with a set of actual parameters is replaced by some code generated by the program body. This is called macro expansion.

Macros allow a programmer to define pseudo operations, typically operations that are generally desirable, are not implemented as part of the processor instruction, and can be implemented as a sequence of instructions. Each use of a macro generates new program instructions; the macro has the effect of automating the writing of the program.

A *macro definition* is enclosed between a *macro header* statement and a *macro* end statement. Macro definitions are typically located at the start of a program. A macro definition consists of

1. A *macro prototype* statement
2. One or more model statements
3. *Macro preprocessor* statements.

The macro prototype statement declares the name of a macro and the names and kinds of its parameters. A model statement is a statement from which an assembly language statement may be generated during macro expansion. A preprocessor statement is used to perform auxiliary functions during macro expansion.

The macro prototype statement has the following syntax:

Macro_name<formal parameters spec>

Where <macro name> appears in the mnemonic field of an assembly statement and <formal parameter spec> is of the form

&<parameter name> [<parameter kind>]

Macros are similar to functions in that they can take arguments and in that, they are called to lengthier sets of instructions. Unlike functions, macros are replaced by the actual commands they represent when the program is prepared for execution. Function instructions are copied into a program only once.

Macro Call

A macro is called by writing the macro name in the mnemonic field of an assembly statement. The macro call has the syntax

<macro name> [<actual parameter spec>[...]]

where an actual parameter typically resembles an operand specification in an assembly language statement.

Consider the example,

Example 4.1: Following figure 4.1 shows the definition of macro INCR. MACRO and MEND are the macro header and macro end statement, respectively. The prototype statement indicates that three parameters called MEM_VAL, INCR_VAL, and REG exist for the macro. Since parameter kind is not specified for any of the parameters. They are all of the default kind 'positional parameter'. Statements with the operation codes MOVER, ADD and MOVEM are model statements. No preprocessor statements are used in this macro. Figure 4.1 shows a macro definition.

MACRO	
INCR	&MEM_VAL, &INCR_VAL, ®
MOVER	®, &MEM_VAL
ADD	®, &INCR_VAL
MOVEM	®, &MEM_VAL
MEND	

Fig 1: A macro definition.

Self-Assessment Questions - 1

1. A unit of specification of program generation through expansion is called _____.
2. The use of macro name with set of actual parameters is replaced by some code generated by its body is called _____.
3. A *macro definition* is enclosed between a _____ statement and a _____ statement.

3. MACRO EXPANSION

The use of a macro name with a set of actual parameters is replaced by some code generated from its body. This is called Macro expansion. Two kinds of expansion are:

1. Lexical Expansion

Lexical expansion implies the replacement of a character string with another character string during program generation. Lexical expansion is typically employed to replace occurrences of formal parameters with corresponding actual parameters.

2. Semantic Expansion

Semantic expansion implies the generation of instructions tailored to the requirements of a specific usage.- for example, the generation of type-specific instructions for the manipulation of byte and word operands. Semantic expansion is characterized by the fact that different uses of a macro can lead to codes that differ in the number, sequence, and opcodes of instructions.

A macro call leads to macro expansion. During macro expansion, the macro call statement is replaced by a sequence of assembly statements. To differentiate between the original statements of a program and the statements resulting from macro expansion, each expanded statement is marked with a '+' preceding its label field.

Two key notions concerning macro expansion are:

1. Expansion time control flow: This determines the order in which model statements are visited during macro expansion.
2. Lexical substitution: Lexical substitution is used to generate an assembly statement from a model statement.

Flow of control during expansion:

The default flow of control during macro expansion is sequential. Thus, in the absence of preprocessor statements, the model statements of a macro are visited sequentially starting with the statement following the macro prototype statement and ending with the statement preceding the MEND statement. A preprocessor statement can alter the flow of control during expansion such that some model statements are either never visited during expansion, or are repeatedly visited during expansion. The former results in *conditional expansion* and the latter in *expansion time loops*. The flow of control during macro expansion is implemented using a *macro expansion counter* (MEC).

Algorithm for Outline of Macro expansion is as follows:

1. MEC:= statement number of the first statement following the prototype statement;

2. While the statement pointed out by MEC is not a MEND statement
 - a) If a model statement then
 - (i) Expand the statement.
 - (ii) MEC:= MEC+1;
 - a) Else (i.e. a preprocessor statement)
 - (i) MEC:=new value specified in the statement;
3. Exit from macro expansion.

MEC is set to point at the statement following the prototype statement. It is incremented by 1 after expanding a model statement. Execution of a preprocessor statement can set MEC to a new value to implement conditional expansion or expansion time loops.

Lexical Substitution

A model statement consists of 3 types of strings

- 1) An ordinary string, which stands for itself.
- 2) The name of a formal parameter is preceded by the character '&'.
- 3) The name of a preprocessor variable, which is also preceded by the character '&'.

During lexical expansion, Strings of type 1 are retained without substitution. Strings of types 2 and 3 are replaced by the 'values' of the formal parameters or preprocessor variables. The value of a formal parameter is the corresponding actual parameter string. The rules for determining the value of a formal parameter depend on the kind of parameter.

Positional Parameters

A positional formal parameter is written as &<parameter name>, e.g. &SAMPLE where SAMPLE is the name of a parameter. In other words, <parameter kind> of formal parameter specification is omitted. The actual parameter specification in a call on a macro using positional parameters is simply an <ordinary string>.

The value of a positional formal parameter XYZ is determined by the rule of positional association as follows:

1. Find the ordinal position of XYZ in the list of formal parameters in the macro proto-type statement.
2. Find the actual parameter specification occupying the same ordinal position in the list of actual parameters in the macro call statement.

Example 4.2: Consider a call

INCR A, B, AREG

on macro INCR, Following the rule of positional association, values of the formal parameters are:

Formal Parameter value

MEM_VAL A

INCR_VAL B

REG AREG

Lexical expansion of the model statements now leads to the code

```
+   MOVER   AREG, A
+   ADD     AREG, B
+   MOVEM   AREG, A
```

Keyword Parameters

For keyword parameters, <parameter name> is an ordinary string, and <parameter kind> is the string '='. The <actual parameter spec> is written as <formal parameter name>=<ordinary string>. The value of a formal parameter XYZ is determined by the rule of keyword association as follows:

1. Find the actual parameter specification which has the form XYZ = <ordinary string>.
2. Let <ordinary string> in the specification be the string ABC. Then the value of formal parameter XYZ is ABC.

Note that the ordinal position of the specification XYZ= ABC in the list of actual parameters is immaterial. This is very useful in situations where long lists of parameters have to be used.

Example 4.3: Figure 4.2 shows the Macro INCR of Figure 4.1. rewritten as Macro INCR_M using keyword parameters. The following macro calls are now equivalent.

```

INCR_M      MEM_VAL = A, INCR_VAL=B, REG=AREG
.....
INCR_M      INCR_VAL=B, REG=AREG, MEM_VAL= A.
MACRO
INCR_M      &MEM_VAL=, &INCR_VAL=, &REG=
MOVER      &REG, &MEM_VAL
ADD        &REG, &INCR_VAL
MOVEM      &REG, &MEM_VAL
MEND

```

Fig 2: A macro definition using keyword parameters.

Default Specification of parameters

A default is a standard assumption in the absence of an explicit specification by the programmer. Default specification of parameters is useful in situations where a parameter has the same value in most calls. When the desired value is different from the default value, the desired value can be specified explicitly in a macro call. This specification overrides the default value of the parameter for the duration of the call.

Default specification of keyword parameters can be incorporated by extending syntax, the syntax for formal parameter specification, as follows:

&<parameter name>[<parameter kind>[<default value>]]

Example 4.4: Register AREG is used for all arithmetic in a program. Hence most calls on macro INCR_M contain the specification ®=AREG. The macro can be redefined to use a default specification for the parameter REG shown in Figure 4.3. Consider the following calls.

```

INCR_D      MEM_VAL=A, INCR_VAL=B
INCR_D      INCR_VAL=B, MEM_VAL=A
INCR_D INCR_ VAL=B, MEM_VAL=A, REG=BREG

```

The first two calls are equivalent to the calls in example 4.3. The third call overrides the default value for REG with the value BREG. BREG will be used to perform the arithmetic in its expanded code.

MACRO	
INCR_D	&MEM_VAL=,&INCR_VAL=, ®=AREG
MOVER	®, &MEM_VAL
ADD	®, &INCR_VAL
MOVEM	®, &MEM_VAL
MEND	

Fig 3: A macro definition with default parameter

Macros with mixed parameter lists

A macro may be defined to use both positional and keyword parameters. In such a case, all positional parameters must precede all keyword parameters. For example, in the macro call

SUMUP A, B, G=20, H=X

A, B are positional parameters while G, H are keyword parameters. Correspondence between actual and formal parameters is established by applying the rules governing positional and keyword parameters separately.

Other uses of parameters

The model statements of Examples 4.1-4.4 have used formal parameters only in operand fields. However, the use of parameters is not restricted to these fields. Formal parameters can also appear in the label and opcode fields of model statements.

Example 4.5

```

MACRO
CALC                &X, &Y, &OP=MULT, &LAB=
&LAB MOVER           AREG, &X
      &OP            AREG, &Y
MOVEM              AREG, &X
MEND

```

Expansion of the call CALC A, B, LAB=LOOP leads to the following code:

```

+ LOOP MOVER        REG, A
+        MULT        REG, B
+        MOVEM       REG, A

```

4. NESTED MACRO CALLS

A model statement in a macro may constitute a call on another macro. Such calls are known as ***nested macro calls***. Calling macro is called outer macro and called macro is called Inner macro.

Macro bodies may also contain macro calls, and so may the bodies of those called macros and so forth. If a macro call is seen throughout the expansion of a macro, the assembler starts immediately with the expansion of the called macro. For this, its expanded body lines are simply inserted into the expanded macro body of the calling macro, until the called macro is completely expanded. Then the expansion of the calling macro is continued with the body line following the nested macro call.

Expansion of nested macro calls follows the ***last-in-first-out (LIFO)*** rule. Thus, in a structure of nested macro calls, expansion of the latest macro call (i.e. the innermost macro call in the structure) is completed first.

Example 4.6

```

INSIDE MACRO
SUBB A, R3
ENDM
OUTSIDE MACRO
MOV A, #42
INSIDE
MOV R7, A
ENDM

```

In the body of the macro OUTSIDE, the macro INSIDE is called. If OUTSIDE is called, then one gets something like the following expansion:

Line I	Addr	Code	Source
15+ 1	0000	74 2A	MOV A, #42
17+ 2	0002	9B	SUBB A, R3
18+ 1	0003	FF	MOV R7, A

If macros are calling themselves, (nested/recursive macro calls), In this case, there must be some stop criterion, to prevent the macro of calling itself over and over until the assembler is running out of memory. Here again, conditional assembly is the solution.

Nested Macro Definitions

A macro body may also contain further macro definitions. However, these nested macro definitions aren't valid until the enclosing macro has been expanded! That means, the enclosing macro must have been called before the nested macros can be called.

Example 4.7

A macro, which can be used to define macros with arbitrary names, may look as follows:

```
DEFINE MACROMACNAME  
MACNAME MACRO  
DB 'I am the macro &MACNAME.'  
ENDM  
ENDM
```

In order not to overload the example with "knowhow", the nested macro only introduces itself kindly with a suitable character string in ROM. The call

```
    DEFINE Macc  
Would define the macro  
Macc MACRO  
    DB 'I am the macro Macc.'  
    ENDM  
and the call  
    DEFINE Macc2  
would define the following macro:  
Macc2 MACRO  
    DB 'I am the macro Macc2.'  
    ENDM
```

Self-Assessment Questions - 2

4. A preprocessor statement can alter the flow of control during expansion such that some model statements are never visited during expansion is called_____.
5. The flow of control during macro expansion is implemented using _____.
6. Expansion of nested macro calls follows _____ rule.

5. ADVANCED MACRO FACILITY

Advanced macro facilities are aimed at supporting semantic expansion. These facilities can be grouped into

1. Facilities for alternation of the flow of control during expansion.
2. Expansion time variables.
3. Attributes of Parameters.

Alternation of the flow of control during expansion

Two features are provided to facilitate alternation of the flow of control during expansion:

- 1) Expansion time sequence symbols.
- 2) Expansion time statements AIF, AGO, and ANOP. A sequencing symbol (SS) has the syntax

. <ordinary string >

As **SS** is defined by putting it in the label field of a statement in the macro body. It is used as an operand in an AIF or AGO statement to designate the destination of an expansion time control transfer. It never appears in the expanded form of a model statement.

An AIF statement has the syntax

AIF(<expression>)<sequencing symbol>

Where <expression> is a relational expression involving ordinary strings, formal parameters and their attributes, and expansion time variables. If the relational expression evaluates to true, expansion time control is transferred to the statement containing <sequencing symbol> in its label field. An AGO statement has the syntax

AGO<sequencing symbol>

And unconditionally transfers expansion time control to the statement containing <sequencing symbol> in its label field. An ANOP statement is written as

<sequencing symbol> ANOP

And simply has the effect of defining the sequencing symbol.

Expansion Time Variables

Expansion time variables (EV's) are variables that can only be used during the expansion of macro calls. A local EV is created for use only during a particular macro call. A global EV exists across all macro calls situated in a program and can be used in any macro which has a declaration for it. Local and global EV's are created through declaration statements with the following syntax:

LCL <EV specification>[,<EV specification>....]

GBL <EV specification>[,<EV specification>....]

And <EV specification> has the syntax &<EV name>, where <EV name> is an ordinary string.

Values of EV's can be manipulated through the preprocessor statement SET. A SET statement is written as

<EV specification> SET <SET-expression>

Where <EV specification> appears in the label field and SET in the mnemonic field. A SET statement assigns the value of <SET-expression> to the EV specified in <EV specification>. The value of an EV can be used in any field of a model statement, and in the expression of an AIF statement.

Example 4.8

```

MACRO
CONSTANTS
LCL      &A
&A      SET      1
        DB      &A
&A      SET      &A+1
        DB      &A
MEND

```

A call on Macro CONSTANTS is expanded as follows: The local EV A is created. The first SET statement assigns the value '1' to it. The first DB statement thus declares a byte constant '1'. The second SET statement assigns the value '2' to A and the second DB statement declares a constant '2'.

Attributes of formal parameters

An attribute is written using the syntax

<attribute name>'<formal parameter spec>

and represents information about the value of the formal parameter, i.e. about the corresponding actual parameter. The type, length, and size attributes have the names T, L, and S.

Example 4.9

```

MACRO
DCL_CONST
&A
AIF          (L'&A EQ 1) .NEXT
.....
NEXT .....
.....
MEND

```

Here, expansion time control is transferred to the statement having .NEXT in its label field only if the actual parameter corresponding to the formal parameter A has the length of '1'.

5.1 Conditional Expansion

While writing a general purpose macro it is important to ensure the execution efficiency of its generated code. Conditional expansion helps in generating assembly code specifically suited to the parameters in a macro call. This is achieved by ensuring that a model statement is visited only under specific conditions during the expansion of a macro. The AIF and AGO statements are used for this purpose.

Example 4.10: It is required to develop a Macro EVAL such that a call

```
EVAL      A, B, C
```

generates efficient code to evaluate A-B+C in AREG. When the first two parameters of a call are identical, EVAL should generate a single MOVER instruction to load the 3rd parameter into AREG. This is achieved as follows:

```

MACRO
EVAL      &X, &Y, &Z
AIF      (&Y EQ &X) .ONLY
MOVER    AREG, &X
SUB      AREG, &Y
ADD      AREG, &Z
AGO      .OVER
.ONLY    MOVER    AREG, &Z
.OVER    MEND

```

Since the value of a formal parameter is simply the corresponding actual parameter, the AIF statement effectively compares the names of the first two actual parameters. If the names are the same, expansion time control is transferred to the model statement `MOVER AREG, &Z`. If not, the `MOVE-SUB-ADD` sequence is generated and expansion time control is transferred to the statement `OVER MEND` which terminates the expansion. Thus efficient code is generated under all conditions.

Expansion time loops

It is often necessary to generate many similar statements during the expansion of a macro. This can be achieved by writing similar model statements in the macro.

Example 4.11

```
MACRO
CLEAR      &A
MOVER      AREG, ='0'
MOVEM      AREG, &A
MOVEM      AREG, &A+1
MOVEM      AREG, &A+2
MEND
```

When called `CLEAR B`, the `MOVER` statement puts the value '0' in `AREG`, while the three `MOVEM` statements store this value in 3 consecutive bytes with the addresses `B`, `B+1`, and `B+2`.

Alternatively, the same effect can be achieved by writing an expansion time loop that visits a model statement, or a set of model statements, repeatedly during macro expansion. Expansion time loops can be written using expansion time variables (EV's) and expansion time control transfer statements `AIF` and `AGO`.

Example 4.12

```
MACRO
CLEAR      &X,&N
LCL        &M
&M         SET      0
MOVER      AREG, ='0'
.MORE      MOVEM     AREG, &X+&M
&M         SET      &M+1
AIF        (&M NE N) .MORE
MEND
```

Consider expansion of the macro call

```
CLEARB,    3
```

The LCL statement declares M to be a local EV. At the start of the expansion of the call, M is initialized to zero. The expansion of model statement MOVEM AREG, &X+&M thus leads to the generation of the statement MOVEM AREG, B. The value of M is incremented by 1 and the model statement MOVEM... is expanded repeatedly until its value equals the value of N, which is 3 in this case. Thus the macro call leads to the generation of the statements

```
+   MOVER      AREG, =0'
+   MOVEM      AREG, B
+   MOVEM      AREG, B+1
+   MOVEM      AREG, B+2
```

Comparison with execution time loops

Most expansion time loops can be replaced by execution time loops. For example, instead of generating many MOVEM statements as in the above example. 4.12 to clear the memory area starting on B, it is possible to write an execution time loop that moves 0 into B, B+1, and B+2. An execution time loop leads to more compact assembly programs. However, such programs would execute slower than programs containing expansion time loops. Thus a macro can be used to trade program size for execution efficiency.

5.2 Other Facilities For Expansion Time Loops

Many assemblers provide other facilities for conditional expansion, an ELSE clause in AIF being an obvious example. The assemblers for M 68000 and Intel 8088 processors provide explicit expansion time looping constructs. We discuss two such facilities here.

The REPT statement

```
REPT <expression>
```

<expression> should evaluate to a numerical value during macro expansion. The statements between REPT and an ENDM statement would be processed for expansion <expression> number of times. Example 4.13 illustrates the use of this facility to declare 10 constants with the values 1,2, .. 10.

Example 4.13

```
MACRO
CONST 10
LCL    &M
```

&M	SET	1
	REPT	10
	DC	'&M'
&M	SETA	&M+1
	ENDM	
	MEND	

The IRP statement

IRP <formal parameter>, <argument-list>

The formal parameter mentioned in the statement takes successive values from the argument list. For each value, the statements between the IRP and ENDM statements are expanded once.

Example 4.14

```
MACRO
CONSTS      &M, &N, &Z
IRP         &Z, &M, 7, &N
DC          '&Z'
ENDM
MEND
```

A macro call CONSTS 4, 10 leads to a declaration of 3 constants with the values 4, 7, and 10.

5.3 Semantic Expansion

Semantic expansion is the generation of instructions tailored to the requirements of specific usage. It can be achieved by a combination of advanced macro facilities like AIF, AGO statements, and expansion time variables. The CLEAR macro of Example. 4.12 is an instance of semantic expansion. Here, the number of MOVEM AREG, ..statements generated by a call on CLEAR is determined by the value of the second parameter of CLEAR. Macro EVAL of example 4.10 is another instance of conditional expansion wherein one of two alternative code sequences is generated depending on the peculiarities of actual parameters of a macro call. Example 4.15 illustrates semantic expansion using the type attribute.

Example 4.15

```
MACRO
CREATE_CONST  &X,&Y
AIF          (T'&X EQ B).BYTE
```

```

&Y    DW      25
      AGO      .OVER
.BYTE ANOP
&Y    DB      25
.OVER MEND

```

This macro creates a constant '25' with the name given by the 2nd parameter. The type of the constant matches the type of the first parameter.

6. DESIGN OF MACRO PROCESSOR

In this section, the design concepts of macro processors are discussed. The source program is read and is then transformed into another program from which all macro definitions have been removed. And also all macro calls have also been replaced with the corresponding macro bodies. The output of the macro processor will be an assembly language program containing no macros. Like in assemblers, the macro processor can also be designed in two passes. In the first pass, all the macro definitions are saved and expanded. The macro body, that is, the text is processed in the second pass. Figure 4.4 shows A macro processor design.

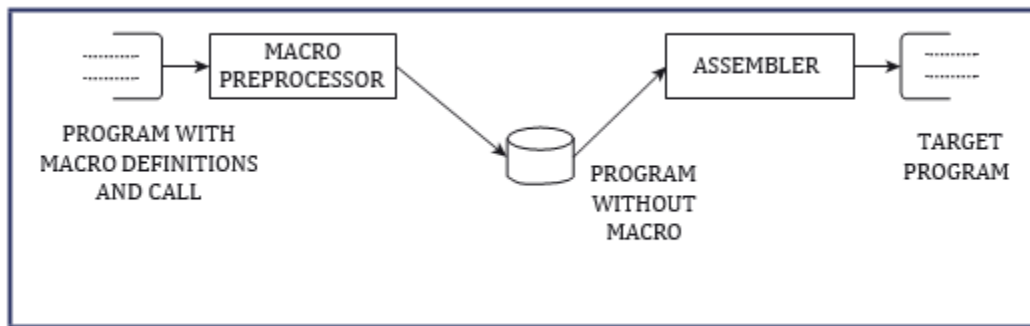


Fig 4: A macro processor design

The macro processor treats it, as though it is the original program. The macro processor is system software. It is actually a program, which is a part of the assembler program. The macro processor is designed in passes. In the design of a macro processor, there are two important functions to be considered. They are given below:

1. Saving the macro definitions
2. Expanding the macro calls.

These functions are done in two passes. Here pass refers to reading the source program once. The functions of each pass are given below:

Functions of Pass 1

1. Reads the source program.

2. If there is a macro call, store the definition of macro in the macro definition Table (MDT).
3. Stores the macro name in the Macro Name Table (MNT).
4. Prepares the List of formal Arguments (AL) in the macro.
5. Repeats the similar processes for all the other macros.

Functions of Pass 2

1. Reads the source program.
2. If there is a macro call, expand the macro with the help of the Macro Name Table.
3. Replaces the formal parameters in the Argument List with the actual parameters.
4. Repeats the similar processes for all the other macros.

These are the functions done by the macro processor in two different passes. The next section discusses the data structures used in the two passes.

6.1 Data Structures Used In Macro Processor Design

There are some important data structures used in the design of a macro processor. They are,

1. Macro Name Table (MNT)
2. Argument List (AL)
3. Macro Definition Table (MDT)

Macro Name Table (MNT)

It is a data structure, which maintains the names and addresses (pointers) where the macro definition is available. This address is helpful to retrieve the macro definition during the macro call.

Argument List (AL)

It also maintains the details of the formal parameters. A list of formal parameters is also maintained. The formal parameters are indicated by some special symbols.

MOV EBX,&R

MOV EAX,&S

Here 'R', 'S' are formal parameters, and a special symbol '&' is used to precede these parameters so as to indicate that these are formal parameters.

Macro Definition Table

It is also a data structure that keeps the macro body i.e., text. The macro body can be retrieved and used during the macro expansion by the macro processor. Whenever a macro definition is encountered, the body is read and stored in the macro definition table.

The functions can be rewritten using the above-mentioned data structures as given below:

Functions of Pass 1 of the Macro processor

- Create the Macro Name Table (MNT), Macro Definition Table (MDT) during pass1.
- Create a list of formal parameters (AL) in Macro Name Table.

Functions of Pass 2 of the Macro processor

- During any macro call, reads the macro body from Macro Definition Table (MDT) with the help of the Macro Name Table (MNT)
- Extracts the formal parameters from the macro body and replaces them with the actual parameters provided in the macro call.

Self-Assessment Questions - 3

7. Variables which can only be used during the expansion of macro calls are called _____.
8. A model statement is visited only under specific conditions during the expansion of a macro is ensured by _____ and _____ statements.
9. The generation of instructions tailored to the requirements of a specific usage is called _____.
10. A data structure, which maintains the names and addresses (pointers) where the macro definition is available is called _____.

7. SUMMARY

Let us recapitulate the important concepts discussed in this unit:

- Macros are used to provide a program generation facility through macro expansion.
- A macro consists of a name, a set of formal parameters, and a body of code.
- Macro definitions are located at the start of a program.
- A macro call leads to macro expansion. During Macro expansion, the Macro call statement is replaced by a sequence of assembly statements.
- A statement in macro results in the call to another macro, then it is called nested macro calls.
- Facilities for alternation of the flow of control during expansion, Expansion time variables, and Attributes of Parameters are the advanced macro facilities.

8. GLOSSARY

Lexical Expansion: Lexical expansion implies the replacement of a character string with another character string during program generation

Macro: A predefined sequence of computer instructions that are inserted into a program, usually during assembly or compilation, at each place that its corresponding macroinstruction appears in the program.

Macro Expansion Counter: Execution of a preprocessor statement can increase the counter by one to implement conditional expansion or expansion time loops.

Macroinstruction: A source code instruction that is replaced by a predefined sequence of source instructions, usually in the same language as the rest of the program and usually during assembly or compilation.

Nested macro: A statement in macro results in the call to another macro

9. TERMINAL QUESTIONS

SHORT ANSWER QUESTIONS

- Q1. Explain about Macro definition and call.
- Q2. Discuss Macro expansion.
- Q3. Write short notes about Nested macro calls.
- Q4. Explain in detail about the Advanced Macro facility.
- Q5. Explain various Data Structures used in Macro Processor Design.

10. ANSWERS

SELF ASSESSMENT QUESTIONS

- 1. Macro.
- 2. Macro expansion.
- 3. Macro header and Macro end.
- 4. Conditional expansion.
- 5. Macro expansion Counter.
- 6. Last-in-first-out (LIFO).
- 7. Expansion time variables (EV's).
- 8. AIF, AGO statements.
- 9. Semantic Expansion.
- 10. Macro Name Table.

TERMINAL QUESTIONS

SHORT ANSWER QUESTIONS

Answer 1: A Macro is a unit of specification of program generation through expansion. Refer section 2 for detail.

Answer 2: The use of a macro name with a set of actual parameters is replaced by some code generated from its body. This is called Macro expansion. Refer section 3 for detail.

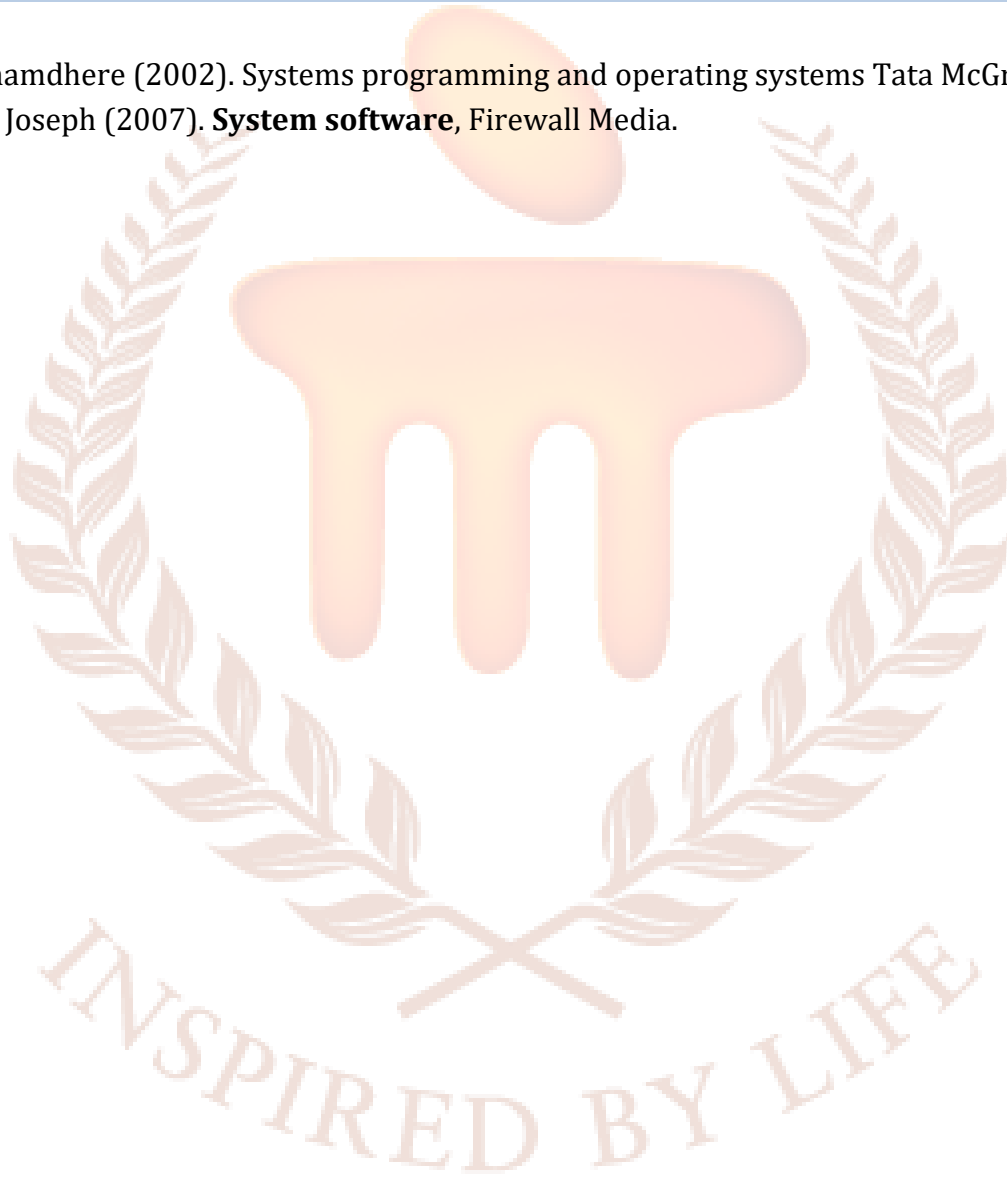
Answer 3: A model statement in a macro may constitute a call on another macro. Such calls are known as nested macro calls. Refer section 4 for detail.

Answer 4: Advanced macro facilities are aimed at supporting semantic expansion. Refer section 5

Answer 5: There are some important data structures used in the design of a macro processor. They are macro name table(MNT), Argument List(AL) and Macro Definition Table(MDT). Refer subsection 6.1

11. SUGGESTED BOOKS AND E- REFERENCES

- Dhamdhare (2002). Systems programming and operating systems Tata McGraw-Hill.
- M. Joseph (2007). **System software**, Firewall Media.





BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4

DCA2203
SYSTEM SOFTWARE

Unit 5

Loaders

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
1.1	Learning Objectives	-	-	
2	Basic functions of loader	-	-	4
3	Design of an Absolute Loader	1, 2, 3	-	5 – 6
4	A Simple Bootstrap Loader	4	1	7 – 8
5	Machine dependent loader feature	5, 6, 7, 8, 9, 10, 11, 12	-	9 – 23
5.1	Relocation	-	-	
5.2	Program Linking	-	-	
5.3	Algorithm and Data Structures for a Linking Loader	-	-	
6	Machine Independent loader features	-	2	24 – 25
6.1	Automatic Library Search	-	-	
6.2	Loader Options	-	-	
7	Loader design options	13, 14, 15, 16,	3	26 - 32
7.1	Linkage Editors	-	-	
7.2	Dynamic Linking	-	-	
7.3	Bootstrap Loaders	-	-	
8	Summary	-	-	33
9	Glossary	-	-	34
10	Terminal Questions	-	-	34
11	Answers	-	-	35
12	Suggested Books and E-References			36

1. INTRODUCTION

A loader is the part of an operating system responsible for loading programs. It is one of the essential stages in starting a program, as it places programs into memory and prepares them for execution. Loading a program involves reading the contents of the executable file, the file containing the program text, into memory and then carrying out other required preparatory tasks to prepare the executable for running. Once loading is complete, the operating system starts the program by passing control to the loaded program code.

In this chapter, we'll talk about a loader's fundamental duties before moving on to its machine-dependent and machine-independent characteristics. In the last section of this unit, we are discussing the loader design options.

1.1 Learning Objectives:

After studying this unit, learneres should be able to:

- ❖ *Define a Loader.*
- ❖ *Describe the basic functions of the loader.*
- ❖ *Explain the machine dependent loader feature.*
- ❖ *Describe different loader design options.*

2. BASIC FUNCTIONS OF LOADER

The first step in developing any machine code manipulation program is understanding the source object file format – what type of information is stored, how it is stored, and how it can be accessed. Normally we are not concerned with any of these issues since the operating system (OS) automatically handles all file access operations and provides users with an interface to these functions. When a program needs to be executed, the OS first extracts all relevant file information (usually from the file header), and carries out any necessary actions before putting it into memory; in other words, the OS decodes the object file into an understandable form in memory. This behavior is often described as the loading of a program.

An operating system utility is one that copies programs from a storage device to the main memory, where they can be executed. In addition to copying a program into the main memory, the loader can replace virtual addresses with physical addresses. Most loaders are transparent; that is, we cannot directly execute them, but the operating system uses them when necessary.

A binary object file is either an executable file that runs on a particular machine or contains object code that needs to be linked. The object code (or executable code) is generated by compiler or an assembler.

Three steps are required to run an object program.. They are Relocation, Linking, Loading, and allocation. Relocation modifies the object programs so that they can be loaded at an address different from the location originally specified. Linking combines two or more separate object programs and supplies the information needed to allow references between them. Loading and allocation allocate memory location and bring the object program into the memory for execution.

Fundamental functions of a loader:

1. Bringing an object program into memory.
2. Starting its execution.

3. DESIGN OF AN ABSOLUTE LOADER

For a simple absolute loader, all functions are accomplished in a single pass as follows:

- 1) *The Header record* of object programs is checked to verify that the correct program has been presented for loading.
- 2) As each *Text record* is read, the object code it contains is moved to the indicated address in memory.
- 3) When the *End record* is encountered, the loader jumps to the specified address to begin the execution of the loaded program.

An example object program is shown in Figure 5.1(a) and figure 5.1(b) shows how this object program is stored in memory

```

H COPY  00100000107A
T0010001E1410334820390010362810303010154820613C100300102A0C103900102D
T00101E150C10364820610810334C0000454F46000003000000
T0020391E041030001030E0205D30203FD8205D2810303020575490392C205E38203F
T0020571C1010364C0000F1001000041030E02079302064509039DC20792C1036
T002073073820644C000005
E001000
  
```

Figure 5.1 (a): An example object program

- It is very important to realize that in Fig 5.1 (a), each printed character represents one byte of the object program record.
- In Fig 5.1(b), on the other hand, each printed character represents one hexadecimal digit in memory (a half-byte).
- Therefore, to save space and execution time of loaders, most machines store object programs in a binary form, with each byte of object code stored as a single byte in the object program.

Memory address	Contents			
0000	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
0010	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
⋮	⋮	⋮	⋮	⋮
0FF0	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
1000	14103348	20390010	36281030	30101548
1010	20613C10	0300102A	0C103900	102D0C10
1020	36482061	0810334C	0000454F	46000003
1030	000000xx	xxxxxxxx	xxxxxxxx	xxxxxxxx
⋮	⋮	⋮	⋮	⋮
2030	xxxxxxxx	xxxxxxxx	xx041030	001030E0
2040	205D3020	3FD8205D	28103030	20575490
2050	392C205E	38203F10	10364C00	00F10010
2060	00041030	E0207930	20645090	39DC2079
2070	2C103638	20644C00	0005xxxx	xxxxxxxx
2080	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
⋮	⋮	⋮	⋮	⋮

(b) Program loaded in memory

- In this type of representation, a byte may contain any binary value.

Figure 5.1(b): Program loaded in memory

Algorithm for Absolute Loader:

Figure 5.2 shows the algorithm for the absolute loader.

Algorithm for an absolute loader

```

begin
  read Header record
  verify program name and length
  read first Text record
  while record type is ≠ 'E' do
    begin
      { if object code is in character form, convert into
        internal representation }
      move object code to specified location in memory
      read next object program record
    end
    jump to address specified in End record
  end
end

```

Fig. 5.2: Algorithm for Absolute loader

4. A SIMPLE BOOTSTRAP LOADER

When a computer is first turned on or restarted, a special type of absolute loader, called a **bootstrap loader**, is executed. This bootstrap loads the first program to be run by the computer – usually an operating system.

Working on a simple Bootstrap loader

- The bootstrap begins at address 0 in the memory of the machine.
- It loads the operating system at address 80.
- Each byte of object code to be loaded is represented on device F1 as two hexadecimal digits just as it is in a SIC text record (refer to fig. 5.3) object program.
- The object code from device F1 is always loaded into consecutive bytes of memory, starting at address 80. The main loop of the bootstrap keeps the address of the next memory location to be loaded in register X.
- After all of the object code from device F1 has been loaded, the bootstrap jumps to address 80, which begins the execution of the loaded program.
- Much of the work of the bootstrap loader is performed by the subroutine GETC (refer to fig. 5.3).
- GETC (refer to fig. 5.3) is used to read and convert a pair of characters from device F1 representing 1 byte of object code to be loaded. For example, two bytes = C “D8”
'4438'H converting to one byte 'D8'H.
- The resulting byte is stored at the address currently in register X, using STCH (refer to fig. 5.3) instruction that refers to location 0 using indexed addressing.
- The TIXR (refer to fig.5.3) instruction is then used to add 1 to the value in X.

GETC	TD	INPUT	TEST INPUT DEVICE
	JEQ	GETC	LOOP UNTIL READY
	RD	INPUT	READ CHARACTER
	COMP	#4	IF CHARACTER IS HEX 04 (END OF FILE),
	JEQ	80	JUMP TO START OF PROGRAM JUST LOADED
	COMP	#48	COMPARE TO HEX 30 (CHARACTER '0')
	JLT	GETC	SKIP CHARACTERS LESS THAN '0'
	SUB	#48	SUBTRACT HEX 30 FROM ASCII CODE
	COMP	#10	IF RESULT IS LESS THAN 10, CONVERSION IS
	JLT	RETURN	COMPLETE. OTHERWISE, SUBTRACT 7 MORE
	SUB	#7	(FOR HEX DIGITS 'A' THROUGH 'F')
RETURN	RSUB		RETURN TO CALLER
INPUT	BYTE	X'F1'	CODE FOR INPUT DEVICE
	END	LOOP	

Source code for the bootstrap loader

Figure 5.3: Source code for Bootstrap loader

Self-Assessment Questions - 1

1. The process of bringing the object program into the memory for execution is called _____.
2. When a computer is first turned on or restarted, a special type of absolute loader is executed, which is called _____.
3. The first program loaded by Bootstrap to be run by _____.

5. MACHINE-DEPENDENT LOADER FEATURE

The absolute loader has several potential disadvantages. One of the most obvious is the need for the programmer to specify the actual address at which it will be loaded into memory. On a simple computer with small memory, the actual address at which the program will be loaded can be specified easily. On a larger and more advanced machine, we often like to run several independent programs together, sharing memory between them. We have yet to determine in advance where a program will be loaded. Hence, we write relocatable programs instead of absolute ones. Writing absolute programs also makes it difficult to use subroutine libraries efficiently. This could not be done effectively if all subroutines had pre-assigned absolute addresses.

The need for *program relocation* indirectly impacts the change to larger and more powerful computers. The way relocation is implemented in a loader is also dependent upon machine characteristics. Loaders that allow for program relocation are called relocating loaders or relative loaders.

5.1 Relocation

Loaders that allow for program relocation are called *relocating loaders* or *relative loaders*. The need for program relocation results from the change to larger and more powerful computers.

Schemes for Relocation:

1) Modification Record

A modification record describes each part of the object code that must be changed when the program is relocated. In the object program, there is one Modification record for each value that must be changed during relocation. Each modification record specifies the starting address and length of the field whose value is to be altered.

A Modification describes each part of the object code that must be changed when the program is relocated.

Consider the program in Figure 5.4.

Line	Loc	Source statement			Object code
5	0000	COPY	START	0	
10	0000	FIRST	STL	RETADR	17202D
12	0003		LDB	#LENGTH	69202D
13			BASE	LENGTH	
15	0006	CLOOP	+JSUB	RDREC	4B101036
20	000A		LDA	LENGTH	032026
25	000D		COMP	#0	290000
30	0010		JEQ	ENDFIL	332007
35	0013		+JSUB	WRREC	4B10105D
40	0017		J	CLOOP	3F2FEC
45	001A	ENDFIL	LDA	EOF	032010
50	001D		STA	BUFFER	0F2016
55	0020		LDA	#3	010003
60	0023		STA	LENGTH	0F200D
65	0026		+JSUB	WRREC	4B10105D
70	002A		J	@RETADR	3E2003
80	002D	EOF	BYTE	C' EOF'	454F46
95	0030	RETADR	RESW	1	
100	0033	LENGTH	RESW	1	
105	0036	BUFFER	RESB	4096	

```

115      .      READ RECORD INTO BUFFER
120      .
125 1036  RDREC  CLEAR  X      B410
130 1038      CLEAR  A      B400
132 103A      CLEAR  S      B440
133 103C      +LDT  #4096    75101000
135 1040  RLOOP  TD      INPUT  E32019
140 1043      JEQ      RLOOP  332FFA
145 1046      RD      INPUT  DB2013
150 1049      COMPR  A,S      A004
155 104B      JEQ      EXIT   332008
160 104E      STCH   BUFFER,X  57C003
165 1051      TIXR   T        B850
170 1053      JLT      RLOOP  3B2FEA
175 1056  EXIT   STX      LENGTH 134000
180 1059      RSUB                     4F0000
185 105C  INPUT  BYTE  X'F1'      F1
195      .
200      .      WRITE RECORD FROM BUFFER
205      .
210 105D  WRREC  CLEAR  X      B410
212 105F      LDT      LENGTH  774000
215 1062  WLOOP  TD      OUTPUT  E32011
220 1065      JEQ      WLOOP  332FFA
225 1068      LDCH   BUFFER,X  53C003
230 106B      WD      OUTPUT  DF2008
235 106E      TIXR   T        B850
      ... (omitted)

```

Figure 5.4: First method- Example program.

- Most of the instructions in this program use relative or immediate addressing.
- The only portions of the assembled program that contain actual addresses are the extended format instructions on lines 15, 35, and 65. Thus these are the only items whose values are affected by relocation.

Object program

Figure 5.5 shows the object program.

```

HCOPY 000000001077
T0000001D17202D69202D4B1010360320262900003320074B10105D3F2FEC032010
T00001D130F20160100030F200D4B10105D3E2003454F46
T0010361DB410B400B44075101000E32019332FFADB2013A00433200857C003B850
T0010531D3B2FEA1340004F0000F1B410774000E32011332FFA53C003DF2008B850
T001070073B2FEF4F000005
M00000705+COPY
M00001405+COPY
M00002705+COPY
E000000

```

Figure 5.5: Object Program

- Each Modification record specifies the starting address and length of the field whose value is to be altered.
- It then describes the modification to be performed.
- In this example, all modifications add the value of the symbol COPY, which represents the program's starting address.

Figure 5.6: Consider a Relocatable program for a Standard SIC machine.

Line	Loc	Source statement			Object code
5	0000	COPY	START	0	
10	0000	FIRST	STL	RETADR	140033
15	0003	CLOOP	JSUB	RDREC	481039
20	0006		LDA	LENGTH	000036
25	0009		COMP	ZERO	280030
30	000C		JEQ	ENDFIL	300015
35	000F		JSUB	WRREC	481061
40	0012		J	CLOOP	3C0003
45	0015	ENDFIL	LDA	EOF	00002A
50	0018		STA	BUFFER	0C0039
55	001B		LDA	THREE	00002D
60	001E		STA	LENGTH	0C0036
65	0021		JSUB	WRREC	481061
70	0024		LDL	RETADR	080033
75	0027		RSUB		4C0000
80	002A	EOF	BYTE	C' EOF'	454F46
85	002D	THREE	WORD	3	000003
90	0030	ZERO	WORD	0	000000
95	0033	RETADR	RESW	1	
100	0036	LENGTH	RESW	1	
105	0039	BUFFER	RESB	4096	

Fig. 5.6: Relocatable program for a Standard SIC machine

- The Modification record is not well suited for all machine architectures. Consider, for example, the program in Fig. 5.6. This is a relocatable program written for the standard version of SIC.
- The important difference between this example and the one in Fig. 5.4 is that the standard SIC machine does not use relative addressing.
- In this program, the addresses in all the instructions except RSUB must be modified when the program is relocated. This would require 31 Modification records, which results in an object program more than twice as large as the one shown in Fig. 5.4.

2) Relocation Bit

Each word of object code has a relocation bit that determines whether or not the word should be changed when the programme is relocated. If the relocation bit corresponding to a word of object code is set to 1, the program's starting address will be added to this word when the program is relocated. The bits corresponding to unused words are set to 0, which signifies that no changes are required.

Fig. 5.7 shows an Object program with relocation by a bit mask

- There are no Modification records.
- The Text records are the same as before except for a *relocation bit* associated with each word of object code.
- Since all SIC instructions occupy one word, there is one relocation bit for each possible instruction.

```

HCOPY 00000000107A
T0000001E FFC1400334810390000362800303000154810613C000300002A0C003900002D
T000001E15E000C00364810610800334C0000454F46000003000000
T0010391E FFC040030000030E0105D30103FD8105D2800303010575480392C105E38103F
T0010570A8001000364C0000F1001000 F1 is one-byte
T00106119FE0040030E01079301064508039DC10792C00363810644C000005
E000000

```

Figure 5.7: Object program with relocation by a bit mask

- The relocation bits are framed into a **bit mask by** following the length indicator in each Text record. This mask is represented (in character form) as three hexadecimal digits in Fig 5.7.
- If the relocation bit corresponding to a word of object code is set to 1, the program's starting address will be added to this word when the program is relocated. A bit value of 0 indicates that no modification is necessary.
- If a Text record contains fewer than 12 words of object code, the bits corresponding to unused words are set to 0.
- For example, the bit masks FFC (representing the bit string 11111111100) in the first Text record specifies that all 10 words of object code are to be modified during relocation.

Example: Note that the LDX instruction on line 210 (Fig 5.6) begins a new Text record. If it were placed in the preceding Text record, it would not be properly aligned to correspond to a relocation bit because of the 1-byte data value generated from line 185.

3) Hardware Relocation

Some computers provide a hardware relocation capability that eliminates some of the need for the loader to perform program relocation.

5.2 Program Linking

A single control section makes up each of the three (separately constructed) programmes in figure 5.8.

Program 1 (Program A):

Loc		Source statement	Object code
0000	PROGA	START 0 EXTDEF LISTA, ENDA EXTREF LISTB, ENDB, LISTC, ENDC	
0020	REF1	LDA LISTA	03201D
0023	REF2	+LDT LISTB+4	77100004
0027	REF3	LDX #ENDA-LISTA	050014
0040	LISTA	EQU *	
0054	ENDA	EQU *	
0054	REF4	WORD ENDA-LISTA+LISTC	000014
0057	REF5	WORD ENDC-LISTC-10	FFFFFFF6
005A	REF6	WORD ENDC-LISTC+LISTA-1	00003F
005D	REF7	WORD ENDA-LISTA- (ENDB-LISTB)	000014
0060	REF8	WORD LISTB-LISTA	FFFFC0
		END REF1	

Program 2 (Program B):

Loc		Source statement	Object code
0000	PROGB	START 0 EXTDEF LISTB, ENDB EXTREF LISTA, ENDA, LISTC, ENDC	
0036	REF1	+LDA LISTA	03100000
003A	REF2	LDT LISTB+4	772027
003D	REF3	+LDX #ENDA-LISTA	05100000
0060	LISTB	EQU *	
0070	ENDB	EQU *	
0070	REF4	WORD ENDA-LISTA+LISTC	000000
0073	REF5	WORD ENDC-LISTC-10	FFFFFFF6
0076	REF6	WORD ENDC-LISTC+LISTA-1	FFFFFFF7
0079	REF7	WORD ENDA-LISTA- (ENDB-LISTB)	FFFFFFF0
007C	REF8	WORD LISTB-LISTA	000060
		END	

Program 3 (Program C):

Loc		Source statement	Object code
0000	PROGC	START 0	
		EXTDEF LISTC, ENDC	
		EXTREF LISTA, ENDA, LISTB, ENDB	
		.	
		.	
0018	REF1	+LDA LISTA	03100000
001C	REF2	+LDT LISTB+4	77100004
0020	REF3	+LDX #END- LISTA	05100000
		.	
		.	
0030	LISTC	EQU *	
		.	
		.	
0042	ENDC	EQU *	
0042	REF4	WORD ENDA-LISTA+LISTC	000030
0045	REF5	WORD ENDC-LISTC-10	000008
0048	REF6	WORD ENDC-LISTC+LISTA-1	000011
004B	REF7	WORD ENDA-LISTA- (ENDB-LISTB)	000000
004E	REF8	WORD LISTB-LISTA	000000
		END	

Figure 5.8: Program Linking Example

Consider first the reference marked REF1.

For the first program (Program A),

- REF1 is simply a reference to a label within the program.
- It is assembled in the usual way as a PC relative instruction.
- No modification for relocation or linking is necessary.

In Program B, the same operand refers to an external symbol.

- The assembler uses an extended-format instruction with an address field set to 00000.
- The object program for Program B contains a Modification record instructing the loader to add *the value of the symbol LISTA to this address field* when the program is linked.

For Program C, REF1 is handled in exactly the same way.

Fig 5.9 shows corresponding object programs,

Program A:

```

HPROGA 00000000000063
DLISTA 000040^END^A 000054
RLISTB ^ENDB ^LISTC ^ENDC
:
T0000200A03201D77100004050014
:
T0000540F000014FFFFFF600003F000014FFFC0
M00002405+LISTB
M00005406+LISTC
M00005706+ENDC
M00005706-LISTC
M00005A06+ENDC
M00005A06-LISTC
M00005A06+PROGA
M00005D06-ENDB
M00005D06+LISTB
M00006006+LISTB
M00006006-PROGA
E000020

```

Program B:

```

HPROGB 0000000000007F
DLISTB 000060^ENDB ^000070
RLISTA ^ENDA ^LISTC ^ENDC
:
T0000360B0310000077202705100000
:
T0000700F0000000FFFFFF6FFFFFFF0000060
M00003705+LISTA
M00003E05+ENDA
M00003E05-LISTA
M00007006+ENDA
M00007006-LISTA
M00007006+LISTC
M00007306+ENDC
M00007306-LISTC
M00007606+ENDC
M00007606-LISTC
M00007606+LISTA
M00007906+ENDA
M00007906-LISTA
M00007C06+PROGB
M00007C06-LISTA
E

```

Program C:

```

HPRGCG 000000000051
DLISTC 000030ENDC 000042
RLISTA ENDA LISTB ENDB
:
T0000180C031000007710000405100000
:
T0000420F000030000008000011000000000000
M00001905+LISTA
M00001D05+LISTB
M00002105+ENDA
M00002105-LISTA
M00004206+ENDA
M00004206-LISTA
M00004206+PRGCG
M00004806+LISTA
M00004B06+ENDA
M00004B06-LISTA
M00004B06-ENDB
M00004B06+LISTB
M00004E06+LISTB
M00004E06-LISTA
E

```

Figure 5.9: Object Programs

- The reference marked REF2 is processed in a similar manner.
- REF3 is an immediate operand whose value is to be the difference between ENDA and LISTA (that is, the length of the list in bytes).
- In Program A, the assembler has all the information necessary to compute this value. During the assembly of Program B (and Program C), the values of the labels are unknown.
- In these programs, the expression must be assembled as an external reference (*with two Modification records*) even though the result will be an absolute value independent of the locations at which the programs are loaded.

Consider REF4.

- The assembler for Program A can evaluate all of the expressions in REF4 except for the value of LISTC. This results in an initial value of '000014'H and one Modification record.

- The same expression in Program B contains no terms that can be evaluated by the assembler. The object code, therefore, contains an initial value of 000000 and *three* Modification records.
- For Program C, the assembler can supply the value of LISTC relative to the beginning of the program (but not the actual address, which is not known until the program is loaded).

all the same.

Memory address	Contents			
0000	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
...	...	...	...	...
3FF0	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
4000				
4010	03201D77	1040C705	0014	
4020				
4030				
4040				
4050		00412600	00080040	51000004
4060	000083			
4070				
4080				
4090			031040	40772027
40A0	05100014			
40B0				
40C0				
40D0		00	41260000	00004051
40E0	00B3			00000400
40F0				
4100	40C70510	0014		0310
4110				40407710
4120				
4130	000083			
4140	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
...	...	...	...	...

← PROGA
← PROGB
← PROGC

- The initial value of this data word contains the relative address of LISTC ('000030'H). Modification records instruct the loader to add the beginning address of the program (i.e., the value of Program C), to add the value of ENDA, and subtract the value of LISTA.

Figure 5.10: The three programs as they might appear in memory after loading and linking.

In Figure 5.10, Program A has been loaded starting at address 4000, with Program B and Program C immediately following.

For example, the value for reference REF4 in Program A is located at address 4054 (that is, the beginning address of Program A plus 0054).

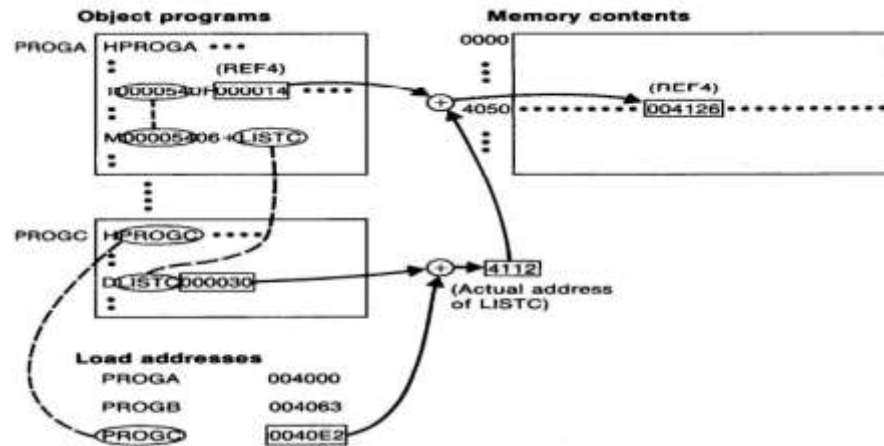


Figure 5.11: Relocation and linking operations performed on REF4 in PROGA

Consider the above figure 5.11,

The initial value (from the Text record) is 000014. To this is added the address assigned to LISTC, which is 4112 (the beginning address of Program C plus 30).

5.3 Algorithm And Data Structures For A Linking Loader

The algorithm for a *linking* loader is considerably more complicated than the *absolute loader* algorithm.

- A linking loader usually makes two passes over its input, just as an assembler does. In terms of general function, the two passes of a linking loader are quite similar to the two passes of an assembler:
- . In Pass 1, addresses are assigned to all external symbols for further process
- Pass 2 performs the actual loading, relocation, and linking functionalities.

The main data structure needed for our linking loader is an *external symbol table (ESTAB)*.

(1) This table, which is analogous to SYMTAB in our assembler algorithm, is used to store the name and address of each external symbol in the set of control sections being loaded.

(2) A *hashed organization* is typically used for this table.

Two other important variables such as PROGADDR (program load address)

and CSADDR (control section address) are implicated in the process.

- (1) PROGADDR is *the beginning address in memory* where the linked program is to be loaded. Its value is supplied to the loader by the OS.
- (2) CSADDR contains *the starting* address assigned to the control section currently being scanned by the loader. This value is added to all relative addresses within the control section to convert them to actual addresses.

Pass 1

During Pass 1, the loader is concerned only with Header and Define record types in the control sections.

```

begin
get PROGADDR from operating system
set CSADDR to PROGADDR {for first control section}
while not end of input do
  begin
    read next input record {Header record for control section}
    set CSLTH to control section length
    search ESTAB for control section name
    if found then
      set error flag {duplicate external symbol}
    else
      enter control section name into ESTAB with value CSADDR
    while record type ≠ 'E' do
      begin
        read next input record
        if record type = 'D' then
          for each symbol in the record do
            begin
              search ESTAB for symbol name
              if found then
                set error flag {duplicate external symbol}
              else
                enter symbol into ESTAB with value
                  (CSADDR + indicated address)
            end {for}
          end {while ≠ 'E'}
          add CSLTH to CSADDR {starting address for next control section}
        end {while not EOF}
      end {Pass 1}
    end
  end
end

```

Algorithm for Pass 1 of a Linking loader:

- 1) The beginning load address for the linked program (PROGADDR) is obtained from the OS. This becomes the starting address (CSADDR) for the first control section in the input sequence.
- 2) The control section name from the Header record is entered into ESTAB, with the value given by CSADDR. All **external symbols** appearing in the Define record for the control section are also entered into ESTAB. Their addresses are obtained by adding the value specified in the Define record to CSADDR.

- 3) When the End record is read, the control section length CSLTH (which was saved from the End record) is added to CSADDR. This calculation gives the starting address for the next control section in sequence.
 - At the end of Pass 1, ESTAB contains all external symbols defined in the set of control sections together with the address assigned to each.
 - Many loaders include as an option the ability to print a **load map** that shows these symbols and their addresses.

Pass 2

Pass 2 performs the actual *loading, relocation, and linking* of the program.

Algorithm for Pass 2 of a Linking loader

- 1) As each Text record is read, the object code is moved to the specified address (plus the current value of CSADDR).
- 2) When a Modification record is encountered, the symbol whose value is to be used for modification is looked up in ESTAB.
- 3) This value is then added to or subtracted from the indicated location in memory.
- 4) The last step performed by the loader is usually the transferring of control to the loaded program to begin execution.

The End record for each control section may contain the address of the first instruction in that control section to be executed. Our loader takes this as the transfer point to begin execution. If more than one control section specifies a transfer address, the loader arbitrarily uses the last one encountered. If no control section contains a transfer address, the loader uses the beginning of the linked program (i.e., PROGADDR) as the transfer point. Normally, a transfer address would be placed in the End record for the main program, but not for a subroutine.


```

begin
  set CSADDR to PROGADDR
  set EXECADDR to PROGADDR
  while not end of input do
    begin
      read next input record (Header record)
      set CSLTH to control section length
      while record type ≠ 'E' do
        begin
          read next input record
          if record type = 'T' then
            begin
              (if object code is in character form, convert
               into internal representation)
              move object code from record to location
                (CSADDR + specified address)
            end (if 'T')
          else if record type = 'M' then
            begin
              search ESTAB for modifying symbol name
              if found then
                add or subtract symbol value at location
                  (CSADDR + specified address)
              else
                set error flag (undefined external symbol)
              end (if 'M')
            end (while ≠ 'E')
          if an address is specified (in End record) then
            set EXECADDR to (CSADDR + specified address)
            add CSLTH to CSADDR // the next control section
          end (while not EOF)
          jump to location given by EXECADDR (to start execution of loaded program)
        end (Pass 2)
      end
    end
  end

```

This algorithm can be made more efficient. Assign a reference number, which is used (instead of the symbol name) in Modification records, to each external symbol referred to in a control section. Suppose we always assign the reference number 01 to the control section name.

6. MACHINE INDEPENDENT LOADER FEATURES

Machine independent loader features are:

- Loading and linking are often thought of as OS service functions. Therefore, most loaders include fewer different features than are found in a typical assembler.
- They include the use of an automatic library search process for handling external references and some common options that can be selected at the time of loading and linking.

6.1 Automatic Library Search

Many linking loaders can automatically incorporate routines from a subprogram library into the program being loaded. Linking loaders that support *automatic library* search must keep track of external symbols that are referred to, but not defined, in the primary input to the loader.

At the end of Pass 1, the symbols in ESTAB that remain undefined represent unresolved external references. The loader searches the library or libraries specified for routines that contain the definitions of these symbols and processes the subroutines found by this search exactly as if they had been part of the primary input stream. The subroutines fetched from a library in this way may themselves contain external references. It is therefore necessary to repeat the library search process until all references are resolved. If unresolved external references remain after the library search is completed, these must be treated as errors.

6.2 Loader Options

Many loaders allow the user to specify options that modify the standard processing

- **Typical loader option 1:** Allows the selection of alternative sources of input.

Example: INCLUDE program-name (library-name) might direct the loader to read the designated object program from a library and treat it as if it were part of the primary loader input.

- **Loader option 2:** Allows the user to delete external symbols or entire control sections.

Example: DELETE csect-name might instruct the loader to delete the named control section(s) from the set of programs being loaded.

CHANGE name1, name2 might cause the external symbol name1 to be changed to name2 wherever it appears in the object programs.

- **Loader option 3:** Involves the automatic inclusion of library routines to satisfy external references.

Example: LIBRARY MYLIB

Such user-specified libraries are normally searched before the standard system libraries. This allows the user to use special versions of the standard routines.

NOCALL STDDEV, PLOT, CORREL

To instruct the loader that these external references are to remain unresolved. This avoids the overhead of loading and linking the unneeded routines, and saves the memory space that would otherwise be required.

Self-Assessment Questions - 2

4. Loaders that allow for program relocation are called _____.
5. Each part of the object code that must be changed when the program is relocated is described in _____.
6. In the case of Relocation bits, the bits corresponding to unused words are set to _____.
7. Capability that eliminates some of the need for the loader to perform program relocation is called _____.

7. LOADER DESIGN OPTIONS

There are some common alternatives for organizing the loading functions, including relocation and linking. Linking Loaders – Perform all linking and relocation at load time. The Other Alternatives are Linkage editors, which perform linking prior to load time, and, dynamic linking, in which the linking function is performed at execution time

Linking Loaders

The below figure 5.12 shows the processing of an object program using Linking Loader. The source program is first assembled or compiled, producing an object program. A linking loader performs all linking and loading operations and loads the program into memory for execution.

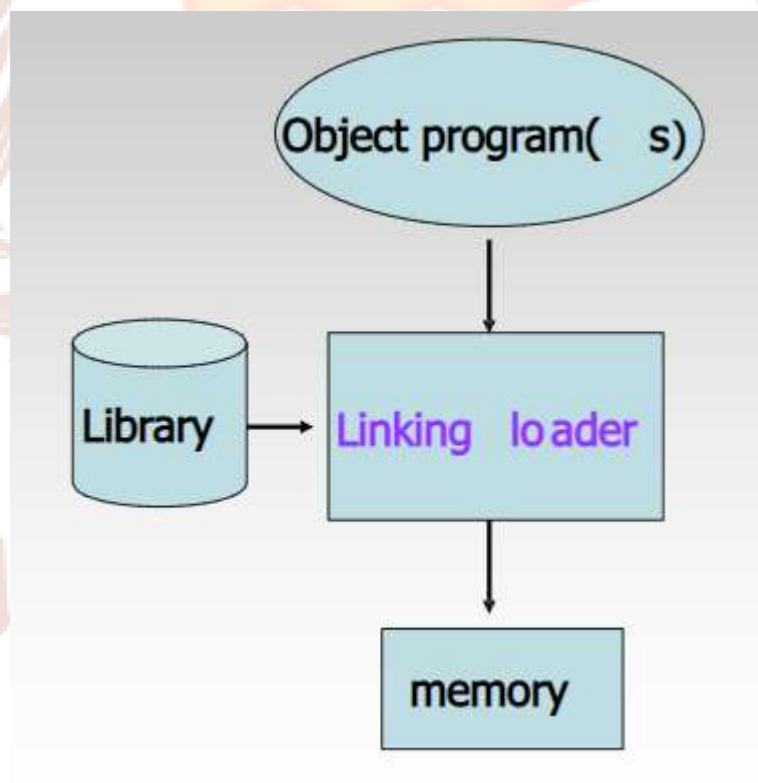


Figure 5.12: Processing of an object program using Linking Loader

Linking loaders perform all linking and relocation at load time. There are two alternatives:

1. **Linkage editors**, which perform linking prior to load time.
2. **Dynamic linking**, in which the linking function is performed at execution time.

Precondition: The source program is first assembled or compiled, producing an object program.

- A **linking loader** performs all linking and relocation operations, including automatic library search if specified, and loads the linked program directly into memory for execution.
- A **linkage editor** produces a linked version of the program (load module or executable image), which is written to a file or library for later execution.

7.1 Linkage Editors

The figure 5.13 shows the processing of an object program using the Linkage editor.

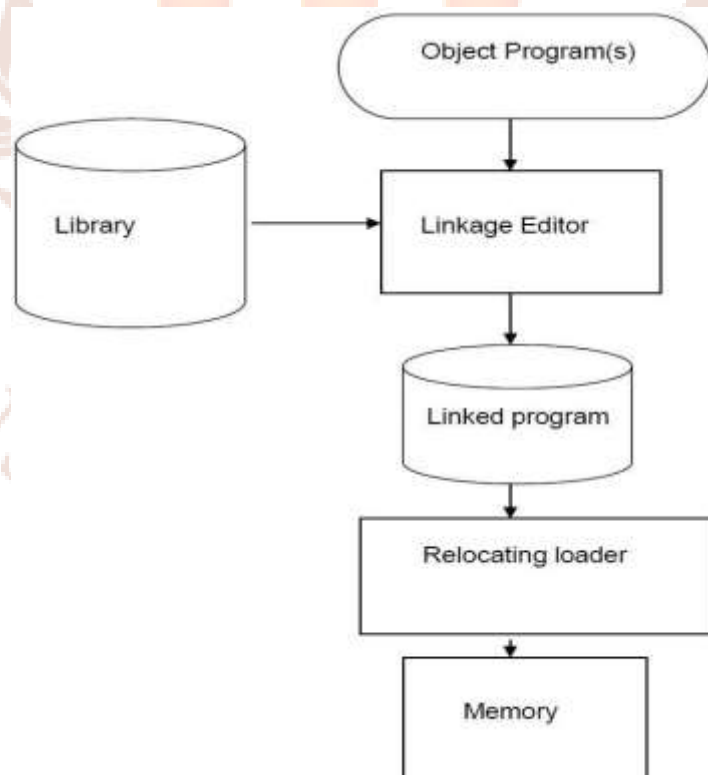


Figure 5.13: Processing of an object program using Linkage editor

A linkage editor produces a linked version of the program – often called a *load module* or an *executable image* – which is written to a file or library for later execution. The linked program produced is generally in a form that is suitable for processing by a relocating loader.

The Linkage editor has some helpful features, such as the ability to generate an absolute object programme if the starting address is already known.. New versions of the library can be included without changing the source program. Linkage editors can also be used to build packages of subroutines or other control sections that are generally used together. Linkage editors often allow the user to specify that external references are not to be resolved by automatic library search – linking will be done later by linking loader – linkage editor + linking loader – savings in space.

The linkage editor performs relocation of all control sections relative to the start of the linked program. Thus, all items that need to be modified at load time have values that are relative to the start of the linked program.

This means that the loading can be accomplished in one pass with no external symbol table required. If a program is to be executed many times without being reassembled, the use of a linkage editor substantially reduces the overhead required.

Linkage editors are capable of much more than just getting an object programme ready for execution.. Linkage editors can also be used to build packages of subroutines or other control sections that are generally used together. This can be useful when dealing with subroutine libraries that support high-level programming languages.

Linkage editors often include a variety of other options and commands. Compared to linking loaders, linkage editors in general tend to offer more flexibility and control.

7.2 Dynamic Linking

Dynamic linking is a method that delays the linking operations until execution. That is a subroutine is loaded and linked to the rest of the program when it is first called – usually called dynamic linking, dynamic loading, or load on call. The advantages of dynamic linking are, that it allows several executing programs to share one copy of a subroutine or library.

In an object-oriented system, dynamic linking makes it possible for one object to be shared by several programs. Dynamic linking provides the ability to load the routines only when they are needed. The actual loading and linking can be accomplished using an operating system service request. Figure 5.14 (a, b, c, d, e) shows the operations of Dynamic Linking.

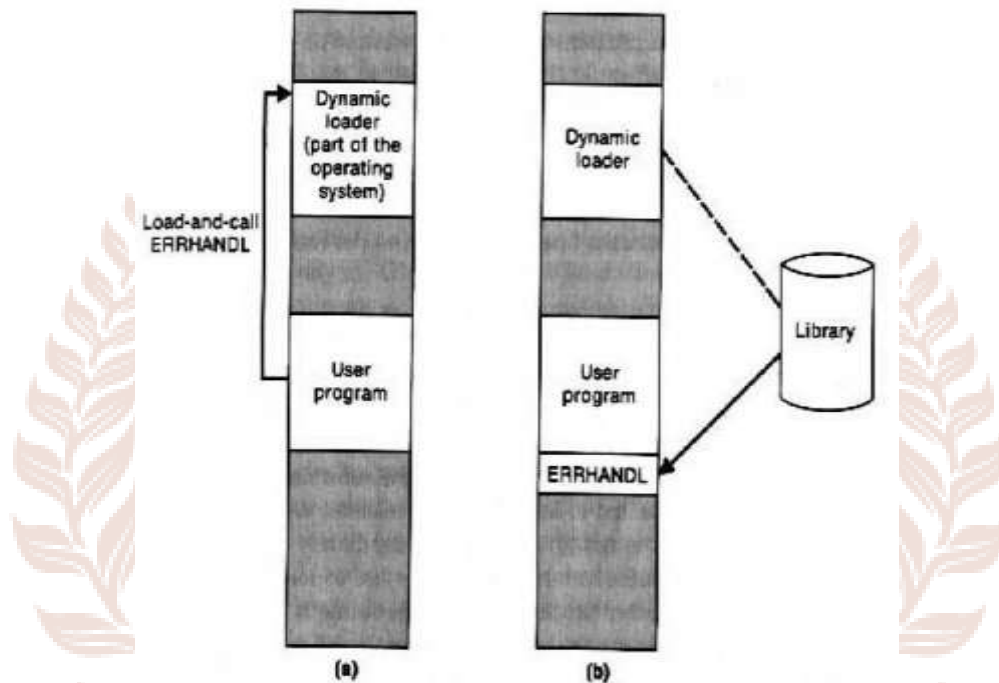


Figure 5.14 (a and b): Dynamic Linking.

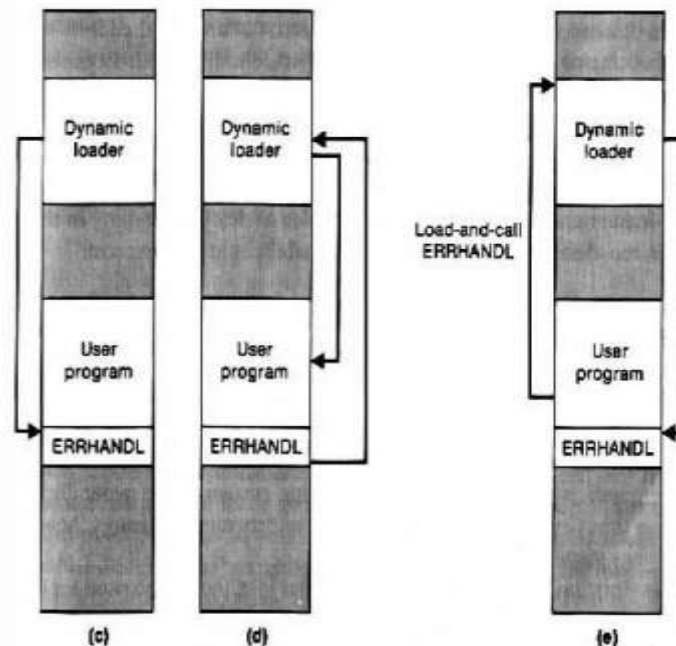


Figure 5.14 (c, d, e): Dynamic Linking.

Fig (a): Instead of executing a JSUB instruction referring to an external symbol, the program makes a load-and-call service request to OS. The parameter of this request is the symbolic name of the routine to be called.

Fig (b): OS checks its internal tables to see if the procedure has previously been loaded. If necessary, the routine is loaded from the specified user or system libraries.

Fig (c): Control is then passed from OS to the routine being called.

Fig (d): When the called subroutine completes its processing, it returns to its caller (i.e., OS). OS then returns control to the program that issued the request.

Fig (e): If a subroutine is still in memory, a second call to it may not require another load operation. Control may simply be passed from the dynamic loader to the called routine.

Linkage editors perform linking operations before the program is loaded for execution. Linking loaders perform these same operations at load time.

Dynamic linking, dynamic loading, or load on call postpones the linking function until execution time: a subroutine is loaded and linked to the rest of the program when it is first called.

Dynamic linking is often used to allow several executing programs to share one copy of a subroutine or library. With a program that allows its user to interactively call any of the subroutines of a large mathematical and statistical library, all of the library subroutines could potentially be needed, but only a few will actually be used in any one execution.

Dynamic linking can avoid the necessity of loading the entire library for each execution except those necessary subroutines

7.3 Bootstrap Loaders

If the question is how the loader is itself loaded into the memory? then the answer is when the computer is started – with no program in memory, a program present in ROM (absolute address) can be made executed – may be OS itself or A Bootstrap loader, which in turn loads OS and prepares it for execution. The first record (or records) is generally referred to as a bootstrap loader – makes the OS to be loaded. All object programmes that are to be loaded into a system that is empty and idle have one of these loaders added to the beginning of them. on some computers, an absolute loader is permanently resident in ROM. When the power is on, the machine begins to execute this ROM program. But it is inconvenient to change a ROM program if modifications in the absolute loader are required. An intermediate solution is to have a built-in hardware function that reads a fixed length record from some device into memory at a fixed location and then jump to execute it. This record contains machine instructions that load the absolute program that follows.

If the loading process requires more instructions, the first record causes the reading of the others, and these in turn can cause the reading of still more records. Hence the term is called Bootstrap. The first record (or records) is generally referred to as a bootstrap loader.

Self-Assessment Questions - 3

8. Some common alternatives for organizing the loading functions are_____.
9. Loaders which perform all linking and relocation at load time are called_____.
10. _____perform linking prior to load time.
11. Linking in which the linking function is performed at execution time is called _____.
12. A loader which loads OS and prepares it for execution is called_____.



8. SUMMARY

Let us recapitulate the important concepts discussed in this unit:

- A loader is the part of an operating system that is responsible for loading programs. It is one of the essential stages in the process of starting a program, as it places programs into memory and prepares them for execution.
- Loading a program involves reading the contents of an executable file, and then carrying out other required preparatory tasks to prepare the executable for running.
- Loaders that allow for program relocation are called *relocating loaders* or *relative loaders*.
- A modification record is used to describe each part of the object code that must be changed when the program is relocated.
- A relocation bit associated with each word of object code is used to indicate whether or not this word should be changed when the program is relocated.
- There are some common alternatives for organizing the loading functions, including relocation and linking.
- Linking Loaders perform all linking and relocation at load time.
- Linkage editors perform linking prior to load time and, dynamic linking, in which the linking function is performed at execution time.

9. GLOSSARY

Bootstrap: A short computer program that is permanently resident or easily loaded into a computer and whose execution brings a larger program, such as an operating system or its loader, into memory.

Loader: A program that copies other [object] programs from auxiliary [external] memory to main [internal] memory prior to its execution.

Modification Record: This is to describe which part of the object code that be changed when the program is relocated.

Operating System: The program that runs on the hardware creating information objects such as files, processes, accounts, and access control of data.

Relative loaders: Loaders that allow for program relocation are called relocating loaders

10. TERMINAL QUESTIONS

Short Answer Questions

1. Compare the working of an absolute loader and a simple Bootstrap loader.
2. Explain in detail about Machine dependent loader features.
3. List and Describe different schemes of Relocation.
4. Describe briefly machine independent loader features.
5. Explain different loader design options.

11. ANSWERS

A. Self-Assessment Questions

1. Loading
2. Bootstrap Loader
3. An Operating System
4. Relocating Loader
5. Modification record
6. Zero
7. Hardware relocation
8. Relocation and linking
9. Linking loaders
10. Linkage editor
11. Dynamic linking
12. Bootstrap loader.

Short Answer Questions

1. For a simple absolute loader, all functions are accomplished in a single pass; Bootstrap loader loads the first program to be run by the computer – usually an operating system. Refer section 3 and 4 for more detail.
2. Main Machine dependent features are Relocation and Program linking. Refer section 5 for detail.
3. Different schemes of relocation are Modification record, relocation bit and hardware relocation. Refer section 5.1 for detail
4. Loading and linking are often thought of as OS service functions. Therefore, most loaders include fewer different features than are found in a typical assembler. Refer section 6 for detail
5. Different loader design options are linkage editor, dynamic linking and linking loader. Refer section 7 for detail

12. SUGGESTED BOOKS

- Dhamdhere (2002). Systems programming and operating systems Tata McGraw-Hill.
- M.Joseph (2007). System software, Firewall Media.





BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4

DCA2203
SYSTEM SOFTWARE

Unit 6

Linker

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
1.1	Learning Objectives	-	-	
2	Introduction to Linker	1	1	4 - 6
3	Relocation and Linking Concepts		2	7 - 11
3.1	Program Relocation	-	-	
3.2	Linking	-	-	
3.3	Object Module	-	-	
4	Design of a Linker	2	3	12 - 16
4.1	Relocation and Linking Requirements in segmented Addressing	-	-	
4.2	Relocation Algorithm	-	-	
4.3	Linking Requirements	-	-	
5	Self-Relocating Programs		-	17
6	Linking for Overlays	3	4	18 - 20
7	Summary			21
8	Glossary	-	-	21
9	Terminal Questions	-	-	22
10	Answers	-	-	22 - 23
11	Suggested Books and E-References	-	-	24

1. INTRODUCTION

In the previous unit, we studied Loaders and loader features. In this unit, we are going to discuss Linkers. Execution of a program written in a language, say L involves four steps. They are,

- 1) Translation of the program,
- 2) linking of the program with other programs needed for its execution,
- 3) Relocation of the program to execute from the specific memory area allocated to it,
- 4) loading of the program in memory for execution.

In these four steps, step 1 is performed by a language translator, step 2 and 3 are performed by a *linker*, and step 4 is performed by a loader.

In this unit, we are discussing Relocation and Linking concepts, the Design of a linker, and the self-relocating program linking for overlays.

1.1 Learning Objectives

After studying this unit, you should be able to:

- ❖ *Define a Linker*
- ❖ *Describe relocation and linking concepts*
- ❖ *Explain the design of a linker*
- ❖ *Describe self-relocating program*
- ❖ *Explain linking for overlays*

2. INTRODUCTION TO LINKER

A linker or link editor is a program that takes one or more objects generated by compilers and assembles them into a single executable program.

Link editors are commonly known as linkers. The compiler automatically invokes the linker as the last step in compiling a program. The linker inserts code (or maps in shared libraries) to resolve program library references and/or combines object modules into an executable image suitable for loading into memory.

Definition

Linker is a program that combines one or more files containing object code from separately compiled program modules into a single file containing loadable or executable code. . This process involves resolving references between the modules and fixing the relocation information used by the operating system kernel when loading the file into memory to run it.

The objects are program modules containing machine code and information for the linker. This information comes mainly in the form of symbol definitions, which come in two varieties:

- Defined or exported symbols are functions or variables that are present in the module represented by the object, and which should be available for use by other modules.
- Undefined or imported symbols are functions or variables that are called or referenced by this object, but not internally defined.

In short, the linker's job is to resolve references to undefined symbols by finding out which other object defines a symbol in question and replacing placeholders with the symbol's address.

There are four different steps for the execution of a program, such as Translation, linking, relocation, and loading. Figure 6.1 shows the schematic which shows these four program execution steps.

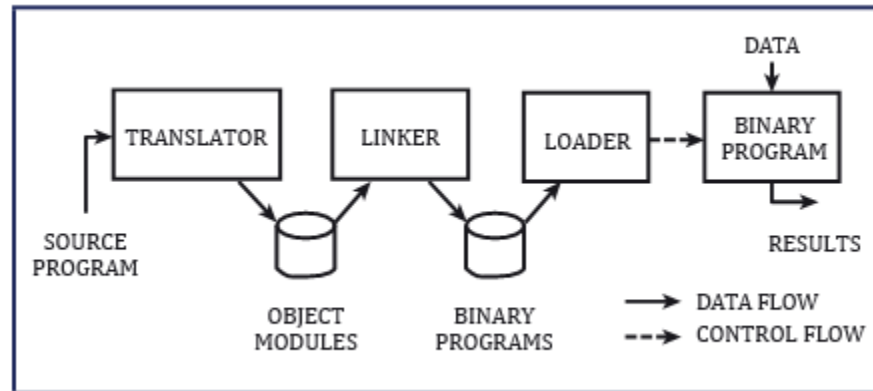


Fig 1: A schematic of program execution

The translator outputs a program form called *object module* for the program. The linker processes a set of object modules to produce a ready-to-execute program form, which we will call a *binary program*. The loader loads this program into the memory for the purpose of execution. As shown in the schematic, the object modules and ready-to-execute program forms can be stored in the form of files for repeated use.

Linkers can take objects from a collection called a library. Some linkers do not include the whole library in the output; they only include its symbols that are referenced from other object files or libraries. Libraries for diverse purposes exist, and one or more system libraries are usually linked in by default.

The linker also takes care of arranging the objects in a program's address space. This may involve relocating code that assumes a specific base address to another base. Since a compiler seldom knows where an object will reside, it often assumes a fixed base location (for example, zero). Relocating machine code may involve re-targeting of absolute jumps, loads, and stores.

The executable output by the linker may need a relocation pass when it is finally loaded into memory just before execution. On hardware offering virtual memory, this is usually omitted, though – every program is put into its own address space, so there is no conflict even if all programs load at the same base address.

Self-Assessment Questions - 1

1. A program that takes one or more objects generated by compilers and assembles them into a single executable program is called _____.
2. A Program module containing machine code and information for the linker is called _____.
3. The functions or variables that are present in the module represented by the object, and which should be available for use by other modules are called _____.
4. The functions or variables that are called or referenced by this object, but not internally defined are called _____.
5. The linker processes a set of object modules to produce a ready to execute program form called _____.



3. RELOCATION AND LINKING CONCEPTS

Different relocation and linking concepts are discussed in this section.

3.1 Program Relocation

Let AA be a set of absolute addresses-instruction or data addresses used in the instruction of a program say P. $AA \neq \Sigma$ implies that program P assumes its instruction and data to occupy memory words with specific addresses. Such a program called an address-sensitive program contains one or more of the following:

1. An address sensitive instruction: an instruction that uses an address $ai \in AA$
2. An address constant: a data word that contains an address $ai \in AA$.

An address-sensitive program P can execute correctly only if the start address of the memory area allocated to it is the same as its translated origin. To execute correctly from any other memory area, the address used in each address-sensitive instruction of P must be 'corrected'.

Definition: Program relocation is the process of modifying the addresses used in the address-sensitive instructions of a program such that the program can execute correctly from the designated area of memory.

If linked origin $\neq$ translated origin, relocation must be performed by the linker. If load origin $\neq$ linked origin, relocation must be performed by the loader. In general, a linker always performs relocation, whereas some loaders do not.

For simplicity, it has been assumed that loaders do not perform relocation-that is, load origin = linked origin. Such loaders are called absolute loaders. Hence the terms 'load origin' and 'linked origin' are used interchangeably. However, it would be more precise to use the term 'linked origin'.

Performing relocation

Let the translated and linked origins of a program P be t_origin_p , and L_origin_p respectively. Consider a symbol $symbol$ in P. Let its translation time address be t_{symbol} and link time address is l_{symbol} . The relocation factor of P is defined as

$$relocation_factor_p = L_origin_p - t_origin_p \quad (6.1)$$

Note that $relocation_factor_p$ can be positive, negative or zero.

Consider a statement that uses $symbol$ as an operand. The translator puts the address t_{symbol} in the instruction generated for it. Now,

$$t_{symbol} = t_origin_p + d_{symbol}$$

Where d_{symbol} is the offset of $symbol$ in P. Hence

$$l_{symbol} = L_origin_p + d_{symbol}$$

using (6.1),

$$l_{symbol} = t_origin_p + relocation_factor_p + d_{symbol}$$

$$= t_origin_p + d_{symbol} + relocation_factor_p$$

$$= t_{symbol} + relocation_factor_p \quad (6.2)$$

Let IRR_p designate the set of instructions requiring relocation in program P (IRR_p). From (6.2), relocation of program P can be performed by computing the relocation factor for P and adding it to the translation time address in every instruction $I \in IRR_p$.

3.2 Linking

Definition

Linking is the process of binding an external reference to the correct link time address.

Consider an application program AP consisting of a set of program units $SP = \{P_i\}$. A program unit P_i interacts with another program unit P_j by using addresses of P_j 's instructions and data in its own instructions. To realize such interactions, P_j and P_i must contain public definitions and external references as defined in the following:

- *Public definition:* A symbol *pub_symb* defined in a program unit which may be referenced in other program units
- *External reference:* A reference to a symbol *ext_symb* that is not defined in the program unit containing the reference.

The handling of public definitions and external references is described in the following.

EXTRN and ENTRY statements

The ENTRY statement lists the public definitions of a program unit, i.e. it lists those symbols defined in the program unit which may be referenced in other program units. The EXTRN statement lists the symbols to which external references are made in the program unit.

Resolving external references

Before the application program AP can be executed, it is necessary that for each P_i in SP, every external reference in P_i should be bound to the correct link time address.

An external reference is said to be unresolved until linking is performed for it. It is said to be resolved when its linking is completed.

Binary Programs

A binary program is a machine language program comprising a set of program units SP (Source Program) such that all P_i belongs to SP.

1. P_i has been relocated to the memory area starting at its link origin, and
2. Linking has been performed for each external reference in P_i .

To form a binary program from a set of object modules, the programmer invokes the linker using the command

Linker <link origin>, <object module names>

[, <execution start address>]

Where <link origin> specifies the memory address to be given to the first word of the binary program. <execution start address> is usually a pair (program unit name, offset in program unit). The linker converts this into the linked start address. This is stored along with the binary program for use when the program is to be executed. If the specification of <execution start address> is omitted the execution start address is assumed to be the same as the linked origin.

Note that a linker converts the object modules in the set of program units SP into a binary program. Since we have assumed link address = load address, the loader simply loads the binary program into the appropriate area of memory for the purpose of execution.

3.3 Object Module

The object module of a program contains all information necessary to relocate and link the program with other programs. The object module of a program P consists of 4 components.

1. *Header*: The header contains translated origin, size and execution start address of P.
2. *Program*: This component contains the machine language program corresponding to P.
3. *Relocation table*: (RELOCTAB) this table describes IRRp. Each RELOCTAB entry contains a single field:

Translated address: Translated address of an address-sensitive instruction.

4. *Linking table (LINKTAB)*: This table contains information concerning the public definitions and external references in P.

Each LINKTAB entry contains three fields:

Symbol: Symbolic name

Type: PD/EXT indicating whether public definition or external reference

Translated address: For a public definition, this is the address of the first memory word allocated to the symbol. For an external reference, it is the address of the memory word which is required to contain the address of the symbol.

Self-Assessment Questions - 2

6. The process of modifying the addresses used in the address-sensitive instructions of a program such that the program can execute correctly from the designated area of memory is called _____.
7. A symbol `pub_symb` defined in a program unit which may be referenced in other program units is called _____.
8. A reference to a symbol `ext_symb` which is not defined in the program unit containing the reference is called _____.
9. The process of binding an external reference to the correct link time address is called _____.

4. DESIGN OF A LINKER

In this section, we are discussing the design of a linker.

4.1 Relocation And Linking Requirements In Segmented Addressing

The relocation requirements of a program are influenced by the addressing structure of the computer system on which it is to execute. The use of the segmented addressing structure reduces the relocation requirements of the program.

Example

Consider the program written in the assembly language of intel 8088 shown in table 6.1. The ASSUME statement declares the segment registers CS and DS to be available for memory addressing. Hence all memory addressing is performed by using suitable displacements from their contents. Translation time address of A is 0196. In statement 16, a reference to A is assembled as a displacement of 196 from the contents of the CS register. This avoids the use of an absolute address; hence the instruction is not address sensitive. Now no relocation is needed if segment SAMPLE is to be loaded with address 2000 by a calling program (or by the OS). The effective operand address would be calculated as $\langle CS \rangle + 0196$, which is the correct address 2196. A similar situation exists with the reference to B in statement 17. The reference to B is assembled as a displacement of 0002 from the contents of the DS register. Since the DS register would be loaded with the execution time address of DATA_HERE, the reference to B would be automatically relocated to the correct address.

Table 1: An 8088 assembly program for linking

<u>Sr. No</u>		<u>statement</u>	<u>offset</u>
0001	DATA_HERE	SEGMENT	
0002	ABC	DW 25	0000
0003	B	DW ?	0002
.		.	
.		.	
0012	SAMPLE	SEGMENT	
0013		ASSUME CS: SAMPLE, DS: DATA_HERE	
0014		MOV AX, DATA_HERE	0000
0015		MOV DS, AX	0003
0016		JMP A	0005
0017		MOV AL, B	0008
.		.	
.		.	
0027	A	MOV AX, BX	0196
.		.	
.		.	
0043	SAMPLE	ENDS	
0044		END	

Though the use of a segment register reduces the relocation requirements, it does not completely eliminate the need for relocation. Consider statement 14 in table 6.1.

MOV AX, DATA_HERE

This loads the segment base of DATA_HERE into the AX register preparatory to its transfer into the DS register. Since the assembler knows DATA_HERE to be a segment, it makes provision to load the higher order 16 bits of the address of DATA_HERE into the AX register. However it does not know the link time address of DATA_HERE, hence it assembles the MOV instruction in the immediate operand format and puts zeroes in the operand field. It also makes an entry

for this instruction in RELOCTAB so that the linker would put the appropriate address in the operand field. Inter-segment calls and jumps are handled in a similar way.

Relocation is somewhat more involved in the case of intra-segment jumps assembled in the FAR format. For example, consider the following program:

```
FAR_LABEL EQU THIS FAR; FAR_LABEL is a FAR label
```

```
JMP     FAR_LABEL; A FAR jump
```

Here the displacement and the segment base of FAR_LABEL are to be put in the JMP instruction itself. The assembler puts the displacement of FAR_LABEL in the first two operand bytes of the instruction and makes a RELOCTAB entry for the third and fourth operand bytes which are to hold the segment base address. A segment like

```
ADDR_A DW     OFFSET A
```

(Which is an 'address constant') does not need any relocation since the assembler can itself put the required offset in the bytes. In summary, the only RELOCATAB entries that must exist for a program using segmented memory addressing are the bytes that contain a segment base address.

For linking, however, both segment base address and offset of the external symbol must be computed by the linker. Hence there is no reduction in the linking requirements.

4.2 Relocation Algorithm

Let us go through the algorithm of program relocation.

Algorithm 6.1 (Program relocation)

1. Program_linked_origin := <link origin> from linker command;
2. For each module
 - a. t_origin := translated origin of the object module; OM_size := size of the object module;
 - b. relocation_factor := program_linked_origin - t_origin;
 - c. Read the machine language program in work_area.

- d. Read RELOCTAB of the object module.
- e. For each entry in RELOCTAB
 - i. $\text{Translated_addr} := \text{address in the RELOCTAB entry.}$
 - ii. $\text{Address_in_work_area} := \text{address of work_area} + \text{translated_address} - \text{t_origin};$
 - iii. Add relocation_factor to the operand address in the word with the address $\text{address_in_work_area}$.
- f. $\text{Program_linked_origin} := \text{program_linked_origin} + \text{OM_size};$

The computation of the work area address of the word requiring relocation (step 2(e) (ii)). Step 2(f) increments $\text{program_linked_origin}$ so that the next object module would be granted the next available load address.

4.3 Linking Requirements

Features of a programming language influence the linking requirement of programs. Procedure references do not require linking, they can be handled through relocation. References to built-in functions, however, require linking.

A reference to an external symbol says alpha can be resolved only if alpha is declared as a public definition in some object module. This observation forms the basis of program linking. The linker processes all object modules being linked and builds a table of all public definitions and their load time addresses. Linking for alpha is simply a matter of searching for alpha in this table and copying its linked address into the word containing the external reference.

A name table (NTAB) is defined for use in program linking. Each entry of the table contains the following fields:

Symbol: Symbolic names of an external reference or an object module.

Linked_address: For a public definition, this field contains the linked address of the symbol. An object module contains the linked origin of the object module.

Most information in NTAB is derived from LINKTAB (Linking Table) entries with type = PD.

Algorithm 6.2 (Program Linking)

1. Program_linked_origin := <link origin> from linker command.
2. For each object module
 - a. t_origin := translated origin of the object module; OM_size := size of the object module;
 - b. relocation_factor := program_linked_origin - t_origin;
 - c. Read the machine language program in work_area.
 - d. Read LINKTAB of the object module.
 - e. For each LINKTAB entry with type=PD name := symbol;
 - f. linked_address := translated_address + relocation_factor; Enter(name, linked_address) in NTAB;
 - g. Enter (object module name, program_linked_origin) in NTAB.
 - h. Program_linked_origin := Program_linked_origin + OM_size;
3. For each object module
 - a. t_origin := translated origin of the object module; program_linked_origin := load_address from NTAB;
 - b. For each LINKTAB entry with type = EXT
 - i. address_in_work_area := address of work_area + program_linked_origin - <link origin> + translated address - t_origin;
 - ii. Search symbol in NTAB and copy its linked address. Add the linked address to the operand address in the word with the address address_in_work_area.

Self-Assessment Questions - 3

10. The relocation requirements of a program are influenced by _____ of the computer system on which it is to execute.
11. For linking, both _____ and _____ of the external symbol must be computed by the linker.

5. SELF RELOCATING PROGRAMS

The manner in which a program can be modified, or can modify itself, to execute from a given load origin can be used to classify programs into the following:

1. Non-relocatable programs,
2. Relocatable programs,
3. Self-relocatable programs.

A non-relocatable program is a program that cannot be executed in any memory area other than the area starting on its translated origin. Non-relocatability is the result of the address sensitivity of a program and the lack of information concerning the address-sensitive instructions in the program. The difference between a relocatable program and a non-relocatable program is the availability of information concerning the address-sensitive instructions in it. A relocatable program can be processed to relocate it to a desired area of memory. Representative examples of non-relocatable and relocatable programs are a hand-coded machine language program and an object module, respectively.

A self-relocating program is a program that can perform the relocation of its own address-sensitive instructions. It contains the following two provisions for this purpose:

1. A table of information concerning the address-sensitive instructions exists as a part of the program.
2. Code to perform the relocation of address-sensitive instructions also exists as a part of the program. This is called the relocating logic.

The start address of the relocating logic is specified as the execution start address of the program. Thus the relocating logic gains control when the program is loaded in memory for execution. It uses the load address and the information concerning address-sensitive instructions to perform its own relocation. Execution control is now transferred to the relocated program.

A self-relocating program can execute in any area of the memory. This is very important in time-sharing operating systems where the load address of a program is likely to be different for different executions.

6. LINKING FOR OVERLAYS

Overlay-DeFinition: An overlay is a part of a program (or software package) that has the same load origin as some other part(s) of the program.

Overlays are used to reduce the main memory requirement of a program.

Overlay structured programs

We refer to a program containing overlays as an overlay structured program. Such a program consists of

1. A permanently resident portion called the root
2. A set of overlays.

Execution of an overlay structured program proceeds as follows: To start with, the root is loaded in memory and given control for the purpose of execution. Other overlays are loaded as and when needed. Note that the loading of an overlay overwrites a previously loaded overlay with the same load origin. This reduces the memory requirement of a program. It also makes it possible to execute programs whose size exceeds the amount of memory that can be allocated to them.

The overlay structure of a program is designed by identifying mutually exclusive modules, that is, modules that do not call each other. Such modules do not need to reside simultaneously in memory. Hence they are located in different overlays with the same load origin.

Example: Consider a program with 6 sections named init, read, trans_a, trans_b, trans_c, and print. Init perform some initialization and passes control to read. read reads one set of data and invokes one of trans_a, trans_b, trans_c depending on the values of the data. Print is called to print the results.

Trans_a, trans_b, and trans_c are mutually exclusive. Hence they can be made into separate overlays. Read and print are put at the root of the program since they are needed for each set of data. For simplicity, we put init also in the root, though it could be made into an overlay by itself.

Consider Figure 6.2, which shows the proposed structure of the program. The overlay structured program can execute in 40K bytes though it has a total size of 65K bytes. It is possible to overlay parts of trans_a against each other by analyzing its logic. This will further reduce the memory requirements of the program.

The overlay structure of an object program is specified in the linker command.

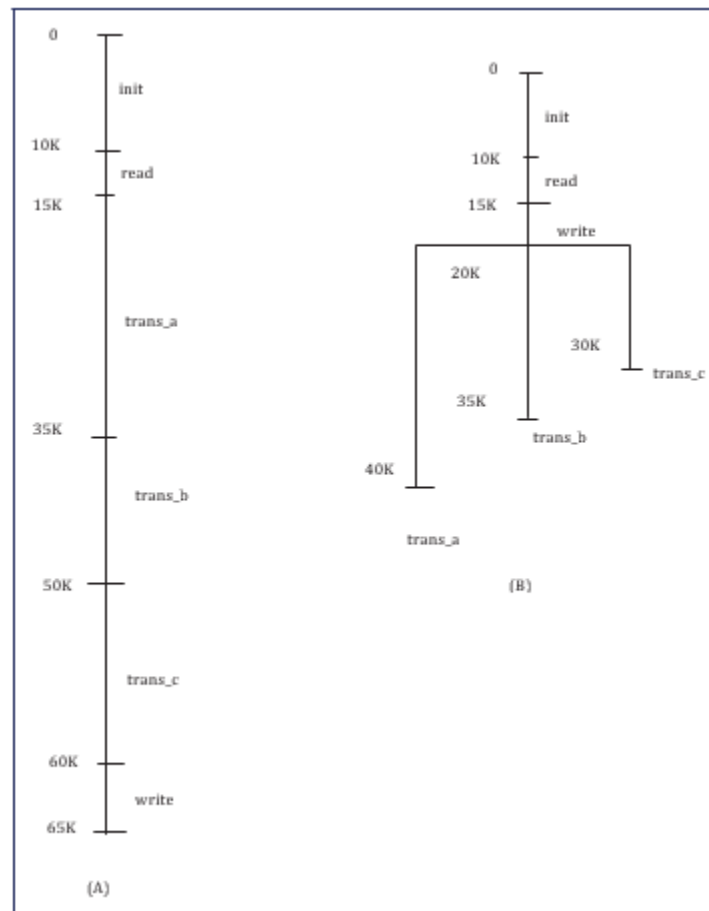


Fig 2: An overlay tree

Execution of an overlay structured program

For linking and execution of an overlay structured program, the linker produces a single executable file at the output, which contains two provisions to support overlays. First, an overlay manager module is included in the executable file. This module is responsible for loading the overlays when needed. Second, all calls that cross overlay boundaries are replaced by an interrupt producing instruction. To start with, the overlay manager receives

control and loads the root. A procedure call that crosses overlay boundaries leads to an interrupt. This interrupt is processed by the overlay manager and the appropriate overlay is loaded into memory. When each overlay is structured into a separate binary program, a call that crosses overlay boundaries leads to an interrupt which is attended by the OS (Operating System) kernel. Control is now transferred to the OS loader to load the appropriate binary program.

Usage of Overlays

Constructing an overlay program involves manually dividing a program into self-contained object code blocks called overlays laid out in a tree structure. Sibling segments, those at the same depth level, and sharing the same memory is called overlay region or destination region. An overlay manager, either part of the operating system or part of the overlay program, loads the required overlay from external memory into its destination region when it is needed. Often linkers provide support for overlays.

Self-Assessment Questions - 4

12. Part of a program that has the same load origin as some other part of the program is called _____.
13. Code to perform the relocation of address sensitive instructions exists as a part of the program is called _____.
14. Program which can be processed to relocate it to a desired area of memory is called _____.
15. Program which can perform the relocation of its own address sensitive instructions is called _____.

7. SUMMARY

Let us recapitulate the important concepts discussed in this unit:

- Linker is a program that combines one or more files containing object code from separately compiled program modules into a single file containing loadable or executable code.
- This process involves resolving references between the modules and fixing the relocation information used by the operating system kernel when loading the file into memory to run it.
- Program relocation is the process of modifying the addresses used in the address sensitive instructions of a program such that the program can execute correctly from the designated area of memory.
- A self-relocating program is a program that can perform the relocation of its address-sensitive instructions.
- Overlays are used to reduce the main memory requirement of a program.

8. GLOSSARY

Binary program: Linker processes a set of object modules to produce a ready-to-execute program form.

Linking: This is the process of binding an external reference to the correct link time address.

Link editing: Makes a single program from several files of relocatable machine code.

Object module: The translator outputs a program form called an object module

Source program: A computer program that must be compiled, assembled, or otherwise translated to be executed by a computer. In contrast with the object program

Register: A small, high-speed memory circuit within a microprocessor that holds addresses and values of internal operations

9. TERMINAL QUESTIONS

SHORT ANSWER QUESTIONS

- Q1.** Describe briefly about linker and linking of a program.
- Q2.** Explain program relocation concepts.
- Q3.** Explain the design of a linker.
- Q4.** Describe briefly self-relocating programs.
- Q5.** What are Overlays? Explain the process of linking for Overlays.

10. ANSWERS

SELF ASSESSMENT QUESTIONS

1. Linker or Link editor
2. Linker Objects
3. Defined or exported symbols
4. Undefined or imported symbols
5. Binary program
6. Program relocation
7. Public definition
8. External reference
9. Linking
10. Addressing structure
11. Segment base address and offset
12. Overlays
13. Relocating logic
14. Relocatable programs
15. Self-relocating program.

TERMINAL QUESTIONS

SHORT ANSWER QUESTIONS

Answer 1: Linker is a program that combines one or more files containing object code from separately compiled program modules into a single file containing loadable or executable code. (Refer to section 2 for detail).

Answer 2: Program relocation is the process of modifying the addresses used in the address-sensitive instructions of a program such that the program can execute correctly from the designated area of memory. (Refer section 3 for detail).

Answer 3: The relocation requirements of a program are influenced by the addressing structure of the computer system on which it is to execute. The use of the segmented addressing structure reduces the relocation requirements of the program. (Refer section 6.4 for detail).

Answer 4: A self-relocating program is a program that can perform the relocation of its own address-sensitive instructions. (Refer section 5 for detail)

Answer 5: An overlay is a part of a program that has the same load origin as some other part of the program. Overlays are used to reduce the main memory requirement of a program. (Refer section 6 for detail)

11. SUGGESTED BOOKS AND E-REFERENCES

- Dhamdhere (2002). Systems programming and operating systems Tata McGraw-Hill.
- M. Joseph (2007). **System software**, Firewall Media.





BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4

DCA2203
SYSTEM SOFTWARE

Unit 7

Interpreter

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
1.1	Learning Objectives	-	-	
2	Introduction to Interpreter	1	-	4
3	Overview of compiler	2	-	5
4	Phases of Compiler	3	-	6 - 8
5	Lexical Analysis	-	-	9
6	Syntax Analysis	-	-	10
7	Intermediate Code Generation	-	-	11
8	Code Optimization	-	-	12
9	Code Generation	-	1	13 - 14
10	Difference between compiler and Interpreter	4, 5	-	15 - 16
11	Scanning	6, 7	2	17 - 18
12	Symbol table	-	-	19 - 20
13	Parsing expression and assignment	8	3	21 - 23
14	Control statements	9	-	24 - 27
15	Simple interpreter design	-	4	28 - 29
16	Summary	-	-	30
17	Glossary	-	-	31
18	Terminal Questions	-	-	31
19	Answers	-	-	32 - 33
20	Suggested Books and E-References	-	-	33

1. INTRODUCTION

A program that executes instructions written in a high-level or assembly language by translating that source code to object code is called Language Translator. There are two ways to run programs written in a high-level language. The most common is to compile the program; the other method is to pass the program through an interpreter.

In this unit, we are discussing Interpreter and compiler and their differences. Also, we will discuss scanning, symbol table, parsing expressions and assignments, and control statements. In this unit's last section, we discuss a simple Interpreter Design.

1.1 Learning Objectives:

After Studying this unit, Learners are able to

- ❖ *Define an Interpreter and a Compiler.*
- ❖ *Differentiate between compiler and Interpreter.*
- ❖ *Describe about Scanning and Parsing expressions.*
- ❖ *Explain about a symbol table*
- ❖ *Explain about control statements.*
- ❖ *Describe a simple Interpreter design.*

2. INTRODUCTION TO INTERPRETER

An Interpreter is a language translator who converts a high-level language to a low-level language (machine language). It is similar to a compiler, but during compilation, the whole source program (a program written in high-level language) is converted into the object program (a program written in low-level language), but an interpreter is a line-by-line translation.



Figure 7.1: Interpreter

There are lots of definitions available for the interpreter. These definitions will give an idea about interpreters and their functions. Interpreters take the HLL (High-Level Language) program as input, examine each instruction, and execute the equivalence machine language program. This process is called interpretation high-level as shown in figure 7.1. An interpreter occupies *less memory space; it is cheaper and suitable for smaller systems. It is slower than the compiler.* Interpreters have a quick turn-around time, which is useful for interactive systems. There is no edit, compile, or test cycle. They are portable.

Strategies of an Interpreter

It can work in three ways:

- Directly run the source code to generate the result.
- Execute the intermediate code created by converting the source code.
- Generating precompiled code using an internal compiler. After that, run this precompiled code.

Advantages

- Carries out line by line. Debugging is, therefore simple.
- Because there is no intermediate code, memory is used effectively.

Disadvantages:

- More time is spent on execution.

3. OVERVIEW OF COMPILER

The compiler is a translator. It translates the program written in a high-level language into an object language, a low-level language (machine language). Executing a program written in a high-level programming language is a two-step process, as illustrated in Figure 7.2. The source program must first be compiled and translated into the object program. Then the resulting object program is loaded into memory and executed.



Figure 7.2: Compilation and Execution

With machine language the communication is direct with a computer in terms of bits, registers, and very primitive machine operations. Since a machine language program is nothing more than a sequence of 0's and 1's, programming a complex algorithm in such a language is tedious, and there is a chance of making mistakes. Perhaps the most serious disadvantage of machine language coding is that all operations and operands must be specified in numeric code. Not only is a machine language program cryptic, but it also may be impossible to modify conveniently.

So, to overcome these problems, there has been an evolution of new programming languages. A high-level programming language allows a programmer to express algorithms in a more natural notation and avoids many of the details of how a specific computer functions.

A high-level programming language simplifies the programming task but also introduces some problems. The most obvious is that we need a program to translate the high-level language into a language the machine can understand.

4. PHASES OF COMPILER

A compiler takes a source program as input and produces an equivalent sequence of machine instructions as output. This process is so complex that it is not reasonable to consider the compilation process as occurring in one step, either from a logical point of view or from an implementation point of view. For this reason, it is customary to partition the compilation process into a series of sub-processes called phases, as shown in Figure 7.3.

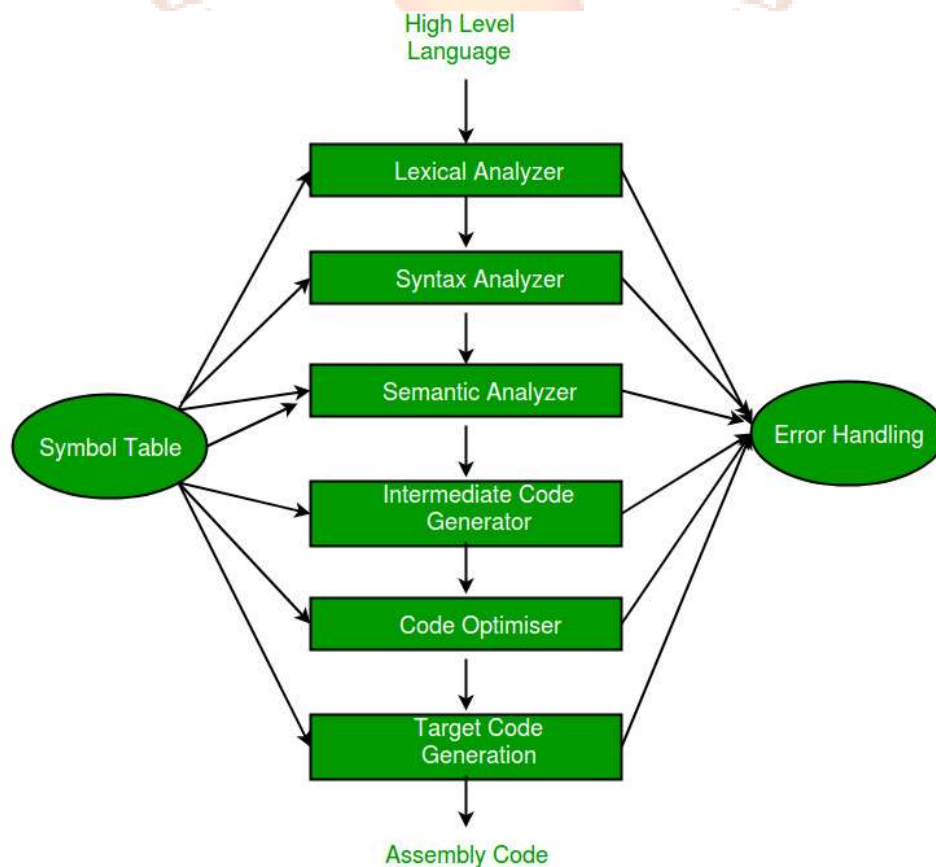


Figure 7.3: Phases of a compiler

A phase is a logically cohesive operation that takes as input one representation of the source program and produces as output another representation.

The first phase, called the *lexical analyzer* or *scanner*, separates characters of the source language into groups that logically belong together; these groups are called tokens. The usual tokens are keywords, such as DO or IF; identifiers, such as X or NUM; operator symbols, such as < = or +, and punctuation symbols, such as parentheses or commas. The output of the

lexical analyzer is a stream of tokens, which passes to the next phase, the syntax analyzer, or parser. The tokens in this stream can be represented by codes that may regard as integers. Thus, DO might be represented by 1, + by 2, and "identifier" by 3. In the case of a token like 'identifier', a second quantity, telling which of those identifiers used by the program is represented by this instance of token "identifier," is passed along with the integer code for "identifier."

The *syntax analyzer* groups tokens together into syntactic structures. For example, the three tokens representing $A + B$ might be grouped into a syntactic structure called an expression. Expressions might further be combined to form statements. Often the syntactic structure can be regarded as a tree whose leaves are the tokens. The interior nodes of the tree represent strings of tokens that logically belong together.

The *intermediate code generator* uses the structure produced by the syntax analyzer to create a stream of simple instructions. Many styles of intermediate code are possible. One common style uses instructions with one operator and several operands. These instructions can be viewed as simple macros like the macro ADD2. The primary difference between intermediate and assembly codes is that the intermediate code need not specify the registers to be used for each operation.

Code Optimization is an optional phase designed to improve the intermediate code so that the ultimate object program runs faster and takes less space. Its output is another intermediate code program that does the same job as the original but perhaps in a way that saves time and space.

The final phase, *code generation*, produces the object code by deciding on the memory locations for data, selecting the code to access each datum, and selecting the registers in which each computation is to be done. Designing a code generator that produces truly efficient object programs is one of the most difficult parts of compiler design, practically and theoretically.

Table-Management, or bookkeeping, a portion of the compiler keeps track of the names used by the program and records essential information about each, such as its type (integer, real, etc.). The data structure used to record this information is called a Symbol table.

The Error Handler is invoked when a flaw in the source program is detected. It must warn the programmer by issuing a diagnostic and adjusting the information being passed from phase to phase so that each phase can proceed. Compilation should be completed on flawed programs, at least through the syntax-analysis phase, so that as many errors as possible can be detected in one compilation. The table management and error-handling routines interact with all compiler phases. .



5. LEXICAL ANALYSIS

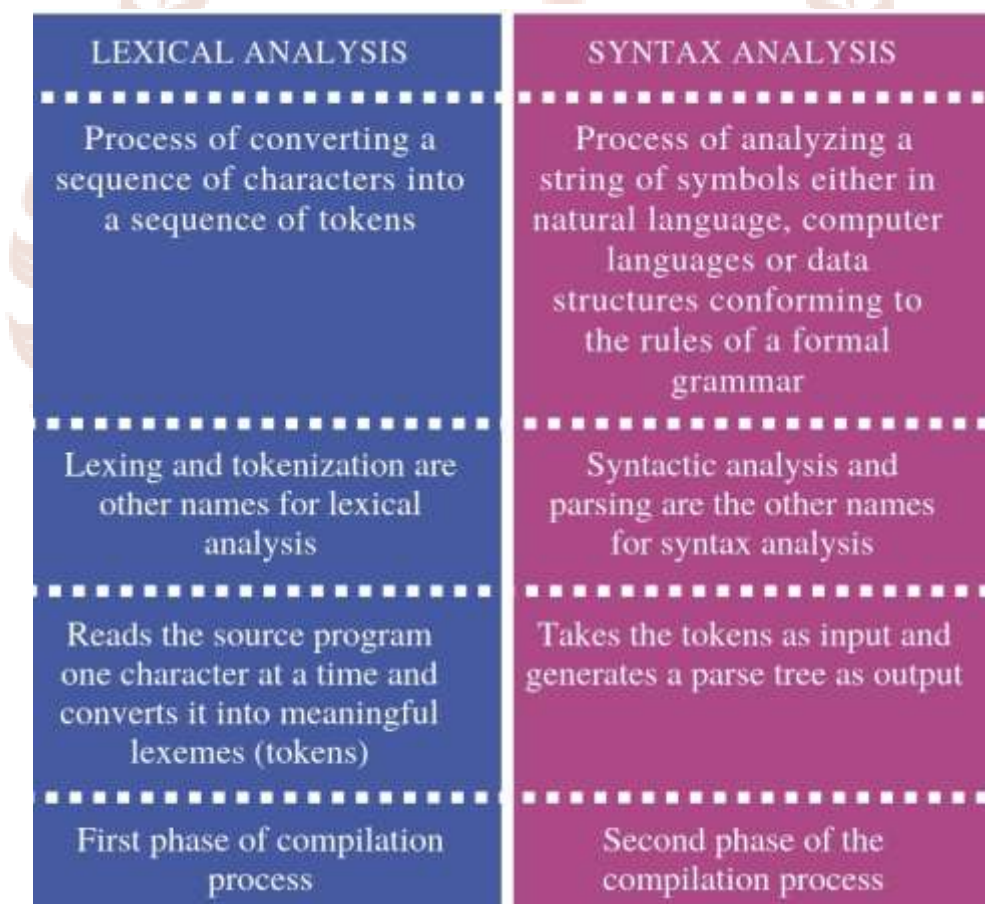
The lexical analyzer is the interface between the source program and the compiler. The lexical analyzer reads the source program one character at a time, carving the source program into a sequence of atomic units called tokens. Each token represents a sequence of characters that can be treated as a single logical entity. Identifiers, keywords, constants, operators, and punctuation symbols such as commas and parentheses are typical tokens. There are two kinds of tokens: specific strings, such as a semicolon and classes of strings, such as identifiers, constants, or labels.

The lexical analyzer and the syntax analyzer are often grouped into the same pass. In that pass, the lexical analyzer operates either under the control of the parser or a co-routine with the parser. The parser asks the lexical analyzer returns to the parser a code for the token that it found. If the token is an identifier or another token with a value, the value is also passed to the parser. The usual method of providing this information is for the lexical analyzer to call a bookkeeping routine which installs the actual value in the symbol table if it is not already there. The lexical analyzer then passes the two components of the token to the parser. The first is a code for the token type (identifier), and the second is the value, a pointer to the place in the symbol table reserved for the specific value found.

6. SYNTAX ANALYSIS

The parser (syntax analyzer) has two functions. It checks that the tokens appearing in its input, which is the output of the lexical analyzer, occur in patterns permitted by the specification for the source language. It also imposes on the tokens a tree-like structure that is used by the subsequent phases of the compiler called a parse tree.

The second aspect of syntax analysis is to make explicit the hierarchical structure of the incoming token stream by identifying which parts of the token stream should be grouped.



7. INTERMEDIATE CODE GENERATION

On a logical level, the output of the syntax analyzer is some representation of a parse tree. The intermediate code generation phase transforms this parse tree into an intermediate language representation of the Three-Address Code source program.

Three-Address Code

One popular intermediate language type is the “three-address code”. A typical three-address code statement is

$A = B \text{ op } C$

A, B, and C are operands and ‘op’ is a binary operator.

8. CODE OPTIMIZATION

Object programs that are frequently executed should be fast and small.

The output of the intermediate code generator is subjected to changes in a step of some compilers in an effort to create an intermediate-language version of the source program, from which a faster or smaller object-language program can finally be generated. This stage is frequently referred to as the optimization phase. A good optimizing compiler can improve the target program by perhaps a factor of two in overall speed compared to a compiler that generates code carefully but without using specialized techniques, generally referred to as code optimization. There are two types of optimizations used:

- Local Optimization
- Loop Optimization

9. CODE GENERATION

The code generation phase converts the intermediate code into a sequence of machine instructions. A simple-minded code generator might map the statement $A := B + C$ into the machine code sequence.

LOAD B

ADD C

STORE A

However, such a straightforward macro, like the expansion of intermediate code into machine code, usually produces a target program that contains many redundant loads and stores and that utilizes the resources of the target machine inefficiently.

To avoid these redundant loads and stores, a code generator might keep track of the run-time contents of registers. Knowing what quantities reside in registers, the code generator can generate loads and stores only when necessary.

Many computers have only a few high-speed registers in which computations can be performed particularly quickly. Therefore, a good code generator would attempt to utilize these registers as efficiently as possible. This aspect of code generation, called register allocation, is particularly difficult to do optimally.

1. Cross compilers

They create platform-specific executable machine code for a different platform than the one the compiler operates on.

2. Bootstrap Compilers

These compilers must be compiled since they are written in a programming language.

3. Source to source/transcompiler

These compilers convert the source code of one programming language. into the source code of another programming language.

4. Decompile

In essence, it isn't a compiler. It just functions as the compiler in reverse. The machine code is changed into high-level language.

Applications and Uses of Compilers

- Makes the code more platform independent.
- Removes any syntactic and semantic errors from the code.
- Create code files that can be executed.
- Converts the code between different languages.

Self-Assessment Questions - 1

1. Program which executes instructions written in a high-level language by translating that source code to object code is called _____.
2. Compiler is a Language Translator. (True/False).
3. The intermediate code generation phase transforms parse tree into an intermediate language representation of the source program called "_____".
4. Lexical analyzer converts the source program into a sequence of atomic units called "_____".
5. The syntax analyzer groups tokens together into syntactic structures called "_____".

10. DIFFERENCE BETWEEN COMPILER AND INTERPRETER

A Compiler and Interpreter both carry out the same purpose – convert a high-level language (like C, Java) instructions into binary form, which is understandable by computer hardware. They are the software used to execute the high-level programs and codes to perform various tasks. Specific compilers/interpreters are designed for different high-level languages. However, both compiler and interpreter have the same objective, but they differ in the way they accomplish their task i.e., convert high-level language into machine language. In this section, we will talk about the basic working of both and distinguish the basic difference between a compiler and an interpreter.

Compiler

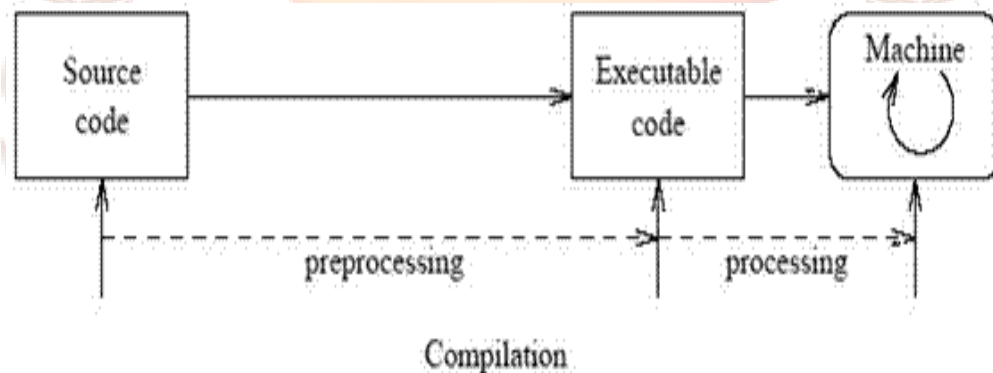


Figure 7.4: Process of compilation

Process of a Compilation

A compiler is a piece of code that translates the high-level language into machine language. When a user writes code in a high-level language such as Java and wants it to execute, a specific compiler that is designed for Java is used before it will be executed. The compiler scans the entire program first and then translates it into machine code which will be executed by the computer processor, and the corresponding tasks will be performed.

Shown in figure 7.4 is a basic outline of the compilation process; here program written in higher-level language is known as the source program, and the converted one is called an object program.

Interpreter

Interpreters are similar to compilers. They also convert the high-level language into machine-readable binary equivalents. Each time an interpreter gets a high-level language code to be executed, it converts the code into an intermediate code before converting it into the machine code. Each part of the code is interpreted and then is executed separately in a sequence, and an error is found in a part of the code. It will stop the interpretation of the code without translating the next set of codes.

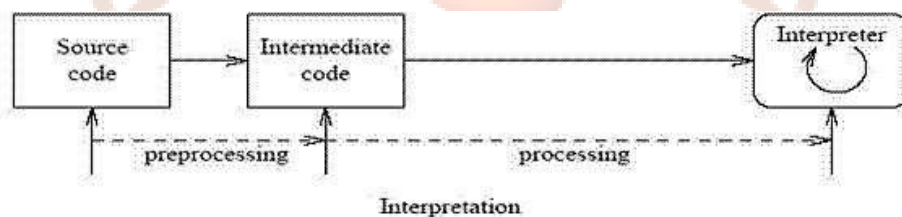


Figure 7.5: Process of Interpretation

Outlining the basic working of the interpreter above figure 7.5 represents that first, a source code is converted to an intermediate form, and then executed by the interpreter.

The main differences between compiler and interpreter are listed below:

- The interpreter takes one statement then, translates it and executes it, and then takes another statement. While the compiler translates the entire program in one go and then executes it.
- The Compiler generates the error report after the translation of the entire page, while an interpreter will stop the translation after it gets the first error.
- The Compiler takes a larger amount of time in analyzing and processing the high-level language code comparatively, interpreters take lesser time in the same process.
- Besides the processing and analyzing time, the overall execution time of a code is faster for the compiler relative to the interpreter

11. SCANNING

During scanning, the Interpreter scans the complete program and breaks the character stream into *tokens* (words). Identifiers, keywords, constants, operators, and punctuation symbols such as commas and parentheses are typical tokens. There are two kinds of tokens: specific strings, such as a semicolon and classes of strings, such as identifiers, constants, or labels. Figure 7.6 shows the structure of a typical interpreter.

If the token is an identifier or another token with a value, the value is also passed to the parser. The usual method of providing this information is for the lexical analyzer to call a bookkeeping routine which installs the actual value in the symbol table if it is not already there. The lexical analyzer then passes the two components of the token to the parser. The first is a code for the token type (identifier), and the second is the value, a pointer to the place in the symbol table reserved for the specific value found.

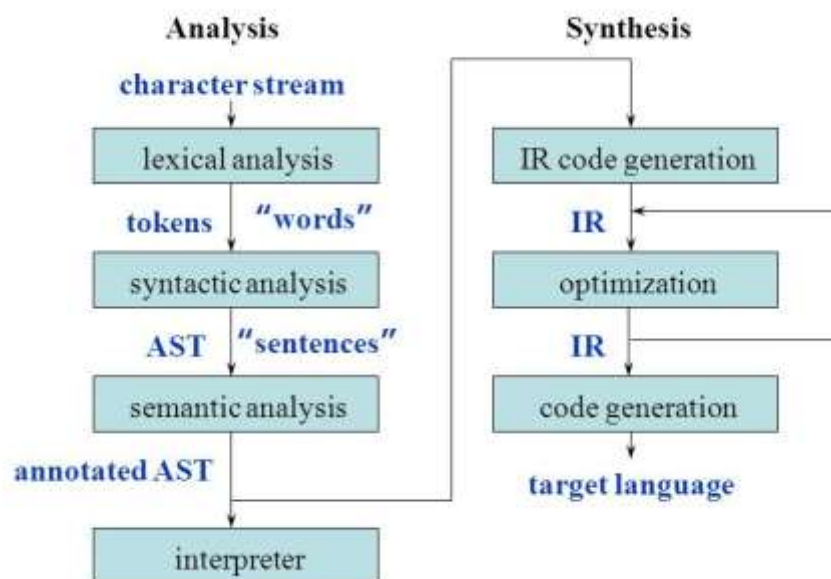


Figure 7.6: Structure of a typical Interpreter

Figure 7.7 shows the interaction between the scanner and the Parser.

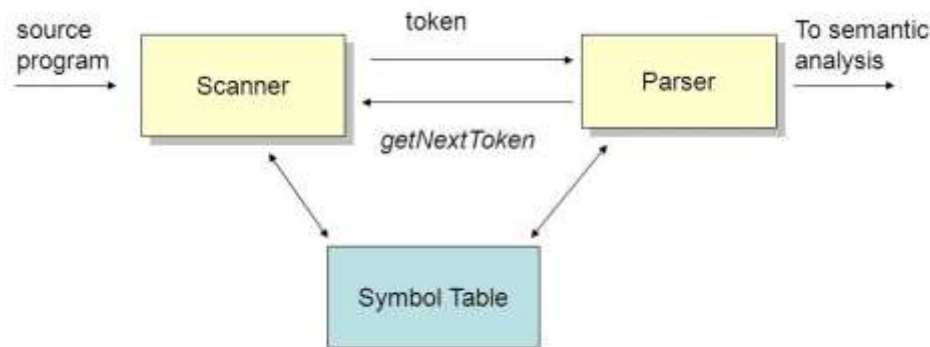


Figure 7.7: Interaction between scanning and Parsing

The token can be defined as a meaningful group of characters over the character set of the programming language, like identifiers, keywords, constants, and others.

Role of Scanner

The main task is to read the input character and procedure as an output sequence of tokens that the parser uses for syntax analysis upon receiving a "get next token" command from the parser; the lexical analysis reads the input characters until it can identify the next token. Its secondary tasks are

1. Stripping out from the source program comments and white space in the form of blank, tab, and new line characters.
2. Correlating error messages from the compiler with the source program.

Self-Assessment Questions - 2

6. A Runtime environment quickly compiles only the needed pieces of the code, which is called "_____".
7. Interpreter scans the complete program and breaks the character stream into tokens; this process is called "_____".
8. Input given to a parser is "_____".

12. SYMBOL TABLE

An essential function of an Interpreter is to record the identifier used in the source program and collect information about the various attributes of each identifier. A symbol table is a data structure containing a record for each identifier, with a field for the attribute. The data structure allows us to find the record for each identifier and to store or retrieve data from that's record quickly. When an identifier in the source program detected the lexical analyzers, the identifiers are entered in the symbol table.

A Symbol table contains all the information that must be passed between different phases of a compiler/interpreter. A symbol (or token) has at least the following attributes:

- Symbol Name
- Symbol Type (int, real, char ...)
- Symbol Class (static, automatic, cons...)

Symbol tables are typically implemented using hashing schemes because good efficiency for the lookup is needed. The classification of symbol tables as:

- Simple
- Scoped

Simple symbol tables have:

- 1) only one scope and
- 2) only "global" variables.

The complication in simple tables involves languages that permit multiple scopes. C permits, at the simplest level, two scopes: global and local (it is also possible to have nested scopes in C)

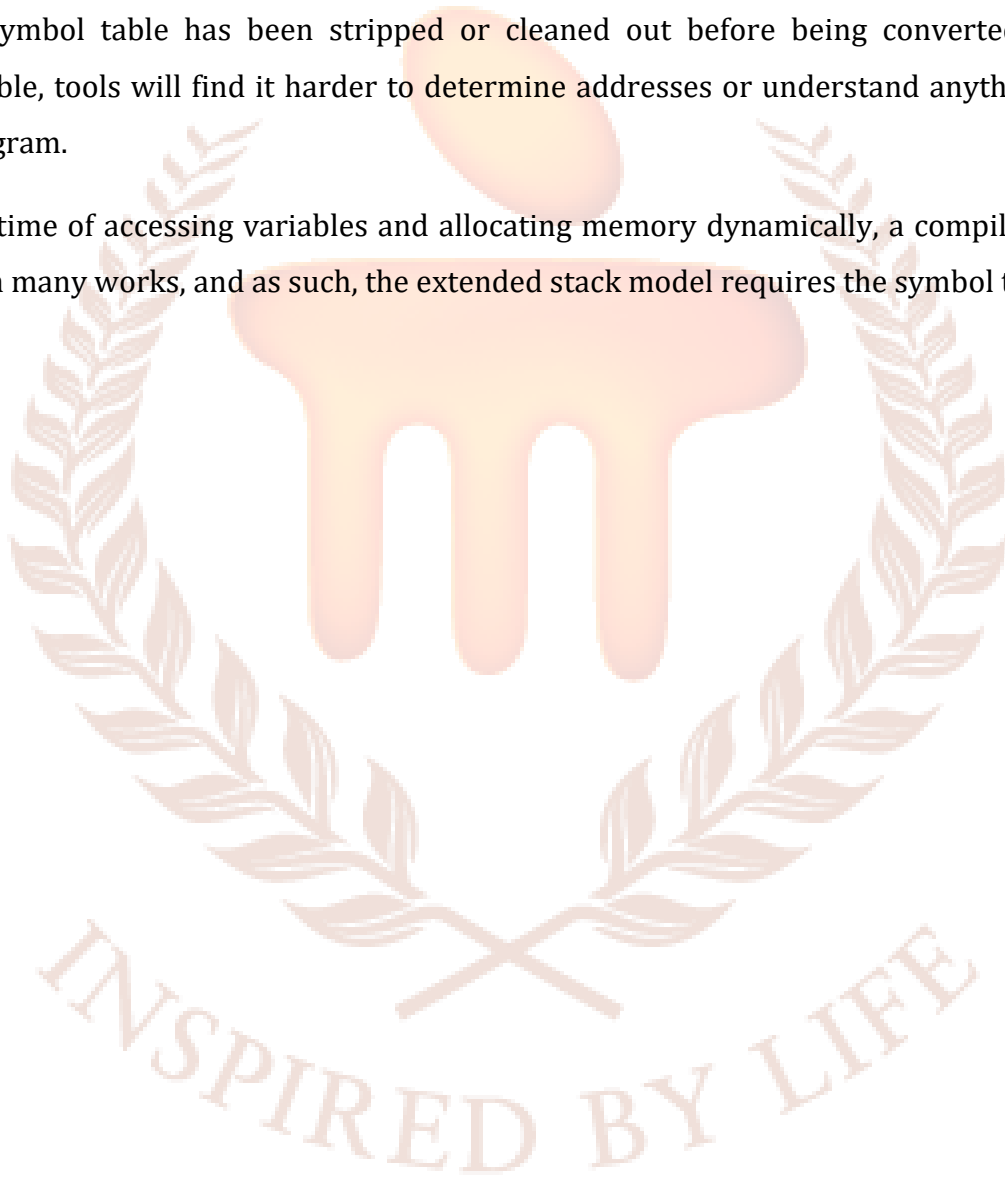
Uses of Symbol Table

An object file will contain a symbol table of the identifiers that are externally visible. When linking different object files, a linker will use these symbol tables to resolve any unresolved references.

A symbol table may only exist during the translation process, or it may be embedded in the output of that process for later exploitation, for example, during an interactive debugging session or as a resource for formatting a diagnostic report during or after the execution of a program.

If the symbol table has been stripped or cleaned out before being converted into an executable, tools will find it harder to determine addresses or understand anything about the program.

At that time of accessing variables and allocating memory dynamically, a compiler should perform many works, and as such, the extended stack model requires the symbol table.



13. PARSING EXPRESSION AND ASSIGNMENT

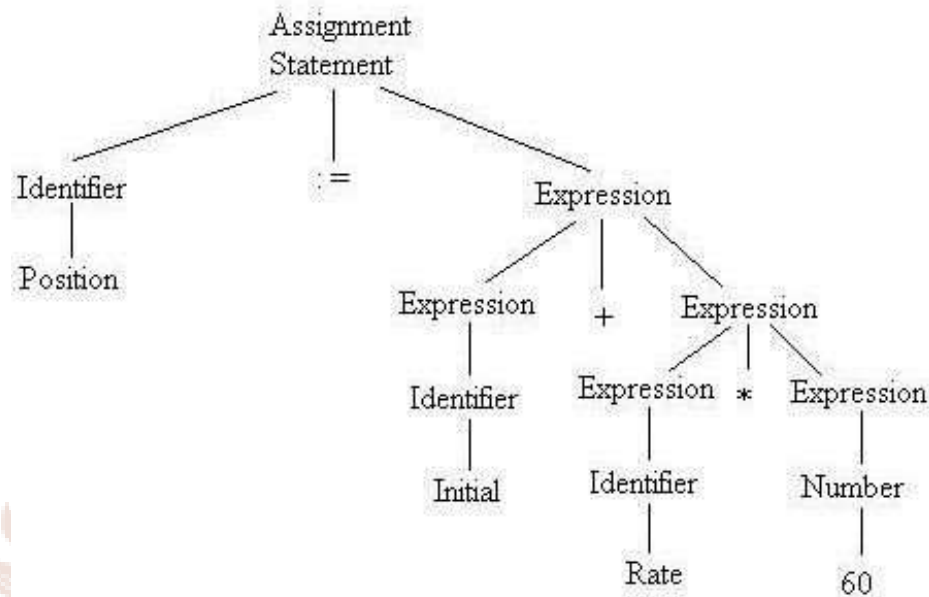
A parser (also called syntax analyzer) is a program which receives valid tokens and checks them against the grammar and produces valid parse trees; otherwise generates syntactical errors. It can be said that the parser generates valid syntactical constructs as per the grammar of the source language from the valid tokens received from the lexical analyzer and generates syntax errors otherwise.

The parser has two functions. It checks that the tokens appearing in its input, which is the output of the lexical analyzer, occur in patterns permitted by the specification for the source language. It also imposes on the tokens a tree-like structure used by the subsequent phases of the compiler. The second aspect of syntax analysis is to make explicit the hierarchical structure of the incoming token stream by identifying which parts of the token stream should be grouped.

The syntax analyzer does syntax analysis. It groups the tokens of the source program into grammatical phrases that are used by the compiler to generate output. This is represented by the parse tree (hierarchical structure). The graphical representation for derivations that filters out the choice regarding the replacement order is called the parse tree.

The figure 7.8 shows the parse tree for the assignment statement

'Position: = Initial + rate*60'.



Parse Tree for Position: = Initial + Rate *60

Figure 7.8: Parse Tree Representation

Parsers can be classified into two broad categories, namely,

1. Top-Down Parsers
2. Bottom-up Parsers.

Top-Down Parsers

These are the parsers that constructs the parse tree from the root to the leaves in preorder for the given input string. In simpler terms, the construction of the parse tree is done from top to bottom of the tree. Here the starting non-terminal is expanded to derive the given input string.

Bottom-Up Parsers

These are the parsers that constructs the parse tree from the leaves to the root for the given input string. In simpler terms, the construction of the parse tree is done from the bottom to the top of the tree. Here, the input string is reduced to starting non-terminal.

The act of checking whether a grammar “accepts” an input text as valid is called parsing. Parsing a given text means determining the exact correspondence between the text and the rules of a given grammar.

Self-Assessment Questions - 3

9. A data structure containing a record for each identifier, with a field for the attribute of identifier, is called “_____”.
10. Symbol tables having only one scope and only “global” variables are called “_____”.
11. A program which receives valid tokens and checks them against the grammar and produces valid parse trees otherwise generate syntactical errors are called _____ .
12. The graphical representation for derivations which filters out the choice regarding the replacement, order is called _____.
13. Parsers that construct the parse tree from the root to the leaves in preorder for the given input string, is called _____ .

14. CONTROL STATEMENTS

The control statements of a programming language are the collection of language features that govern the sequencing of control through a program.

The control statements of a Programming Language consist of constructs for control transfer, conditional execution, iteration control, and procedure calls.

Control transfer, conditional execution, and iterative constructs

Control transfers implemented through conditional and unconditional goto's are the most primitive control structure. When the target language of a compiler is a machine language, the compilation of control transfers is analogous to the assembly of forward or backward goto's in an assembly language program. Hence similar techniques based on the use of a label table can be used.

Control structures like *if*, *for*, or *while* cause a significant semantic gap between the Programming Language (PL) domains and the execution domain because the control transfers are implicit rather than explicit. This semantic gap is bridged in two steps. In the first step, a control structure is mapped into an equivalent program containing explicit goto. Since the destination of a go-to may not have a label in the source program; the compiler generates its labels and puts them against the appropriate statements. Figure 7.9 illustrates programs equivalent to the if and while statements the labels int1, int2 are introduced by the compiler for its purposes. In the second step, these programs are translated into assembly programs.

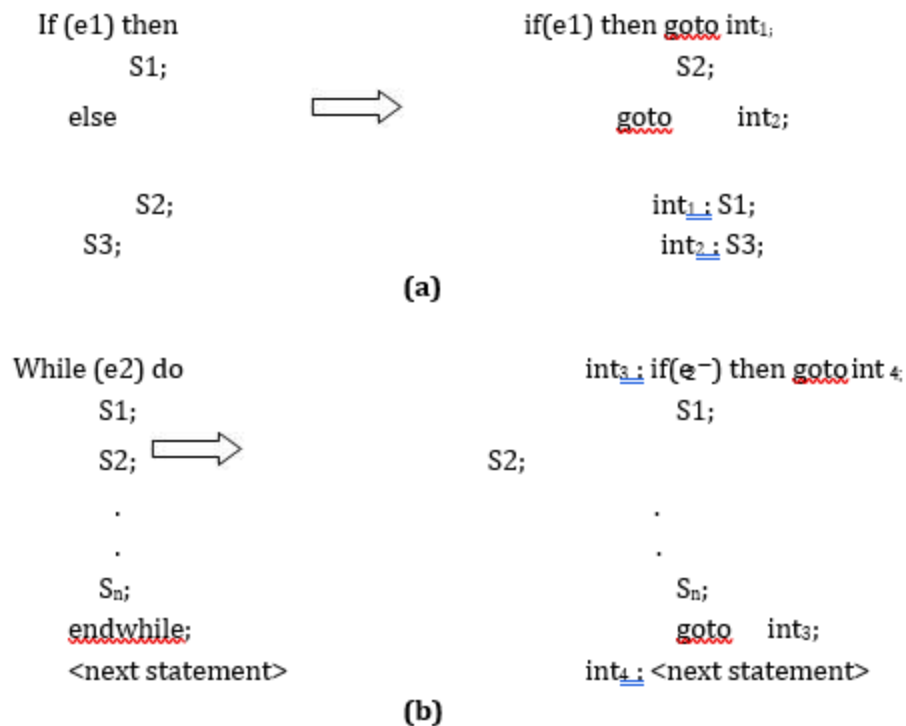


Figure 7.9: Control structure Interpretation

Function and Procedure calls

A function call, viz., the call on `fn_1` in the statement

```
X: = fn_1(y, z) + b*c;
```

Executes the body of `fn1`, and returns its value to the calling program. In addition, the function call may also result in some side effects.

A *side effect* of a function call is a change in the value of a variable that is not local to the called function.

A procedure call only achieves a side effect which, it doesn't return a value. Since all considerations except the return of a value are analogous to function and procedure calls; in the following only compilation of function calls are discussed.

While implementing a function call, the compiler must ensure the following:

1. Actual parameters are accessible in the called function
2. The called function can produce side effects according to the rules of the PL.
3. Control is transferred to and is returned from the called function.
4. The function value is returned to the calling program.
5. All other aspects of the execution of the calling programs are unaffected by the function call.

The compiler uses a set of features to implement function calls. These are described below.

1. **Parameter list:** The parameter list contains a descriptor for each actual parameter of the function call.
2. **Save area:** The called function saves the contents of CPU registers in this area before beginning its execution.
3. **Calling conventions:** These are execution time assumptions shared by the called function and its caller.

Parameter passing mechanisms

Language rules for parameter passing define the semantics of parameter usage inside a function, thereby defining the kind of side effects a function can produce on its actual parameters.

- Call by value

In this mechanism, values of actual parameters are passed to the called function. These values are assigned to the corresponding formal parameters.

- Call by value-result

This mechanism extends the capabilities of the call-by-value method by copying the values of formal parameters back into corresponding actual parameters at return. Thus, side effects are realized on return.

- Call by reference

In this mechanism, the address of an actual parameter is passed to the called function.

- Call by name

This parameter transmission mechanism has the same effect as if every occurrence of a formal parameter in the body of the called function is replaced by the name of the corresponding actual parameter.



15. SIMPLE INTERPRETER DESIGN

In this section, let us discuss the interpreter design technique. This section discusses the components involved in the interpretation process.

The interpretation can be done in two ways. In a first way, the source code is taken, and corresponding executable codes are generated. Here there is no preprocessing done. The source program is taken as such. Such a type of interpretation is permitted in interpreter base systems. In this case, the source program is kept as such. The overhead is more in these systems.

In a second way, some preprocessing is done. The source program is converted into an intermediate representation after preprocessing. This intermediate representation is used for interpretation. The overhead is comparatively less in this type of interpretation. The speed of interpretation is increased due to the use of intermediate representation.

The interpretation process has three major components. They are listed below.

1. Micro Programs
2. Symbol Table and
3. Datastore

The input and output of an interpreter are:

Input: Source program in HLL

Output: Executable machine code

Microprograms

Consider an executable statement in the source program to be interpreted. This statement is decoded to identify its actual operation of it. The interpreter systems run on microprogrammed control units. In the microprogrammed control, control memory will be present. The control memory keeps the microprograms, which correspond to the executable statements in the source program. A *microprogram* is a set of micro-instructions to carrying out a computation.

After decoding the meaning of the source program statement, the corresponding microprogram will be invoked. This microprogram will generate control signals. The control signals are responsible for execution. The control signals are software controlled. This is how the source program is interpreted.

Symbol Table

The symbol table maintains the details of the symbols present in the source program. The symbols are identifier names, labels or name of any program segments. The attribute of symbols is maintained in the symbol tables. There are a lot of data structures available for symbol tables.

Data Store

The data store keeps the necessary data items for variables, identifiers, and others. Various distinct data types are present and used during the interpretation of the source program.

Self-Assessment Questions - 4

14. The collection of language features that govern the sequencing of control through a program is called “_____”.
15. A change in the value of a variable that is not local to the called function is called “_____”.
16. The mechanism in which the address of an actual parameter is passed to the called function is called “_____”.
17. The mechanism in which values of actual parameters are passed to the called function is called “_____”.

16. SUMMARY

Let us recapitulate the important points discussed in this unit:

- An Interpreter may be a program that executes the source code directly, translates the source code into some efficient intermediate representation, and immediately executes this or explicitly executes stored precompiled code made by a compiler that is part of the interpreter system.
- Compiler is a translator. It translates the program written in a high-level language into an object language, which is a low-level language such as an assembly language or machine language.
- The interpreter scans the complete program and breaks the character stream into tokens (words).
- A symbol table is a data structure containing a record for each identifier, with a field for the attribute of the identifier.
- The control statements of a Programming Language consist of constructs for control transfer, conditional execution, iteration control, and procedure calls.
- The interpretation process has three major components: 1. Micro Programs 2. Symbol Table and 3. Data storage

17. GLOSSARY

Ambiguous grammar: a grammar that allows some sentence or string to be generated or parsed with two or more distinct parse trees.

Compiler: a program that translates from one programming language to another, typically from a high-level language such as Java to machine language

Derivation: a list of steps that shows how a sentence in a language is derived from grammar by application of grammar rules.

Grammar: a formal specification of a language consisting of a set of nonterminal symbols, a set of terminal symbols or words, and production rules that specify transformations of strings containing nonterminal into other strings.

Parsing is the process of reading a source language, determining its structure, and producing intermediate code.

Three-address code: is a linearized representation of a syntax tree or a dig in which explicit names correspond to the interior nodes of the graph.

18. TERMINAL QUESTIONS

Short Answer Questions

1. What is an Interpreter? Describe the differences between a Compiler and an Interpreter.
2. Describe Compiler. Explain different phases of compilation.
3. Describe briefly scanning and parsing.
4. Explain briefly about different control statements used in Interpretation.
5. Explain the Simple Interpreter design in detail.

19. ANSWERS

A. Self-Assessment Questions

1. Language translator
2. True
3. Three Address Code
4. Tokens
5. Parse Tree
6. Just-in-time compilation
7. Lexical analysis (scanning)
8. Tokens
9. Symbol Table
10. Simple Symbol Table
11. Parser
12. Parse Tree
13. Top-down Parser
14. Control Statement
15. Side effect
16. Call by reference
17. Call by value

B. Short Answer Questions

1. An Interpreter is a program that executes other programs. An Interpreter may be a program which executes the source code directly, translates source code into some efficient intermediate representation and immediately executes this or explicitly executes stored precompiled code made by a compiler which is part of the interpreter system. Refer to sections 2 and 10 for detail.
2. Compiler is a translator. It translates the program written in a high-level language into an object language, which is a low-level language such as an assembly language or machine language. Refer to section 4 to 9 for detail.

3. During scanning, the Interpreter scans the complete program and breaks the character stream into tokens (words). A parser (also called syntax analyzer) is a program which receives valid tokens and checks them against the grammar and produces valid parse trees; otherwise generates syntactical errors. Refer to sections 11 and 13 for detail.
4. The control statements of a programming language are the collection of language features that govern the sequencing of control through a program. Refer to section 14 for detail.
5. The interpretation can be done in two ways. In a first way, the source code is taken, and corresponding executable codes are generated. In a second way, some preprocessing is done. The source program is converted into an intermediate representation after preprocessing. Refer section 15 for detail.

20. SUGGESTED BOOKS

- Dhamdhare (2002). Systems programming and operating systems Tata McGraw-Hill.
- M.Joseph (2007). System software, Firewall Media.



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4

DCA2203
SYSTEM SOFTWARE

Unit 8

Text Editor

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
1.1	Learning Objectives	-	-	
2	Text Editor: An Introduction	1, 2, 3, 4	1, 2, 3	4 - 19
2.1	Overview of the Editing Process	-	-	
2.2	Types of Editors	-	-	
2.3	User interface	-	-	
2.4	Editor Structure	-	-	
3	Interactive Debugging System	-	4	20 - 29
3.1	Debugging Functions and Capabilities	-	-	
3.2	Relationship with Other Parts of the System	-	-	
3.3	User interface Criteria	-	-	
4	Summary	-	-	30
5	Glossary	-	-	31
6	Terminal Questions	-	-	32
7	Answers	-	-	32 - 33
8	Suggested Books and E-References	-	-	34

1. INTRODUCTION

In the previous unit, the discussion was the Interpreter, overview of the compiler, difference between compiler and interpreter, scanning, symbol table, parsing expression and assignment, control statements, and Simple interpreter design.

In this unit, will study the text editor. An overview of the editing process, user interface, and editor structure. Will also discuss the interactive debugging system, debugging functions and capabilities, its relationship with other parts of the system, and the user interface criteria.

1.1 Learning Objectives:

After studying this unit, the learner would be able to:

- ❖ *List the various types of editors*
- ❖ *List the various types of user interface*
- ❖ *Explain about the editor structure*
- ❖ *Describe the interactive debugging system*
- ❖ *Describe the debugging functions and capabilities*
- ❖ *Discuss the relationship with other parts of the system*
- ❖ *discuss the user interface criteria*

2. TEXT EDITOR: AN INTRODUCTION

Let's talk generally about an editor before we go into any discussions about text editors. The name itself implies that the major function of an editor is editing. Apart from editing, the editor does other supporting services editing. There are various functions which are supposed to be performed by an editor. The major functions of an editor are editing, traveling, viewing, and displaying. The function of the editor is also to create files. A file is a group of characters defined by its creator. The creator may define a file as a program file or a database file or a document file. The term document includes objects such as computer programs, text, equations, table linkers, loaders and Operating Systems (OS), diagrams, line art, and photographs or anything one can get on a printed form. It is only when we create a file that the above-said operations can be performed. The editor must be capable of allowing the user to create a file of his interest.

The term editing means modifying, updating, deleting, and formatting of the text. Traveling means the ability to travel the text in the file. This allows the user to move the cursor over the text. The viewing option allows the user to create a view of the file. This involves the selection of font, style, colors, and the like. This view created can be called an abstract view.

Displaying and printing text is another function of the editor. The editor should be able to generate the printable view of text objects and display it on the screen. That is editor should be able to convert the abstract view into the physical view and should also be able to print it. The editors should be able to provide facilities for all the functions mentioned above. Many other advanced features like spell checking and grammar checking as the document is created. The editors can have the facilities for editing the image and pictures too.

If a file is created in an editor, ASCII codes for all the letters, numbers, and characters are stored in the main memory by the editor. The file is created in the main memory (RAM) only. When the creation of the file is completed, the file is stored in the secondary memory either on the floppy disk or on hard disk with the corresponding extension.

There are various types of Editors. Depending upon the types of characters to be edited, editors can be broadly classified as:

- i) Text Editors and
- ii) Graphical Editors.

Depending upon how the editor edits, the editor can also be classified into

- i) Stream Editor,
- ii) Line Editor,
- iii) Screen Editor and
- iv) Structure Editor.

In this unit, will discuss mainly text editors. A text editor is a computer program that lets users enter, change, store, and usually print text. These texts are characters and numbers. The texts are encoded by the computer and input/output devices which are again arranged so that users or other programs can understand them. In a text editor, the primary elements being edited are character strings of the target text.

Typically, a text editor provides an "empty" display screen (or "scrollable page") with a fixed-line length and visible line numbers. The lines can then be filled in line by line with text.. A special command line lets you move to a new page, scroll forward or backward, make necessary changes in the document, save the document and perform other actions. A text editor allows the user to create the source program in the form of text in the main memory. Generally, the user prepares High-Level Language program as a source program. The creation, editing, modification, deletion, and updating of documents or files can be done with the help of the text editor. The scope of the editing is limited to the text only. Text editors can enter program language source statements or create documents such as technical manuals. It can also be used to create a file containing C programs, C++ programs, assembly language programs and similar programs. They support the creation and modification of documents and allow modifying or updating of files. For example, editing the text when writing a document, send emails, make a Web page, program, and many more purposes.

Text editors are often provided with operating systems or software development packages and can be used to change configuration files and

programming language source code. E.g., Notepad is provided in Windows OS whereas gedit or kedit editor is found in Linux OS.

Text editors can be considered the primary interface to the computer for all types of users who manipulate computer-based information.

Text editors are mostly used by (but not limited to):

- Programmers
- Authors and secretaries of authors
- Knowledge workers (who compose, organize, study and manipulate computer-based information)

The editors can be developed using programming tools like C, C++, VB, and VC++. An example of a popular text editor in IBM's large or mainframe computers is XEDIT. The two most commonly used text editors in UNIX systems are Emacs and vi.

Some of the popular text editors available are:

ae: ae, is a simple, easy-to-use text editor. It has modes to emulate the behavior of other text editors.

Cooledit: Cooledit is a popular, fast text editor for use in **X window systems**. Its features include anti-aliased fonts, Unicode support, and extensibility via the Python programming language

Dedit: DEdit is a simple editor for use in **X window system** with GNOME installed. It can read compressed files and display Japanese characters.

Ee: It is an editor that novices can begin to use immediately; the Easy Editor features pop-up menus.

Elvis: Elvis is a modern implementation of VI editor that comes with many new features and extensions.

Emacs: Emacs is one of the most popular text editors.

Jed: John E. Davis's jed editor is specifically for programmers whose features include drop-down menus that work in the console. jed loads quickly and makes editing files at a shell prompt easy and fast.

Joe: Joe's Editor or joe in short is a full-screen editor with a look and feel reminiscent of old DOS text editors like EDIT.

Nano: Nano is a free software editor.

Ted: Ted is a WYSIWYG text editor for use in X, which reads and writes `.rtf` files in Microsoft's "Rich Text Format."

The: The Hessling Editor is a configurable editor that uses the Rexx macro language. It was inspired by the XEDIT editor for VM/CMS and the Kedit editor for DOS.

Vi: Vi (pronounced "vye," or sometimes "vee-eye") is a visual, or full-screen, editor. Vi is a most popular editor on Linux and Unix-based systems.

Vim: Vim ("Vi improved") is a modern implementation of Vi whose features include syntax coloring, scrollbars and menus, mouse support, and built-in help.

Wily: Wily is a mouse-centric editor whose commands consist of various combinations of the three mouse buttons, called chords, which can be tricky to master.

Xedit: Xedit is a simple text editor that comes with and works in, X. It lets you insert, delete, copy and paste text and open and save files.

Xemacs: XEmacs is a version of Emacs with advanced capabilities for use in Xwindow system, including the ability to display images.

Punch cards:

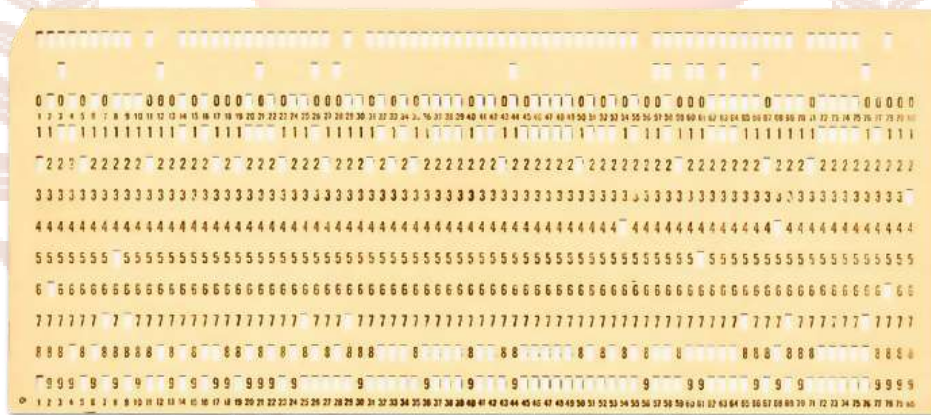
Punch cards with holes punched into them to represent computer data and instructions are known as punch cards (or "punched cards"), Hollerith cards, or IBM cards. They were a

common way for people to enter data into early computers. The cards were put into a card reader that was attached to a computer, which translated the holes' order into digital data.

For instance, an early computer programmer might write a program by hand, then use a punch card machine to convert it to multiple punched cards. The programmer would then bring the stack of cards to a computer and input the software there using a card reader. An illustration of a woman using a punch card machine to make a punch card is shown.

By punching holes in each column, which stand for one character, using a punch card machine like the one in the image above, data can be entered into the card. Here is an illustration of a punch card.

Example of a punch card



When a card is finished or the Return key is pushed, the information is technically "stored" on the card. If you develop a programme using punch cards (one card for each line of code), it requires a stack of cards because each card can only carry so much data.

Each card is placed in a punch card reader to read the data from the card and load the software into the computer. The punch card reader begins reading the card as soon as it is inserted, moving vertically from top to bottom. The card reader advances to the following column after reading one column. The information would be written to a computer's memory as the reader read it. The computer would be told to run the code when all cards had been loaded into memory. Punch cards would be produced as information was output (printed).

Self-Assessment Questions - 1

1. The “_____” Should generate a printable view.
2. If a file is created in an editor, then it is created in the “_____” Only.

2.1 Overview Of The Editing Process

Now let's have a look at the editor's editing process in general. An Interactive Editor is a computer program that allows users to create and revise a target document. The term document means an object such as computer programs, text, equations, tables, diagrams, photographs, etc. The document-editing process is an interactive user-computer dialogue designed to accomplish four tasks. They are:

1. Select the part of the target document to be viewed and manipulated.
2. Determine how to format this view online and how to display it.
3. Specify and execute operations that modify the target document.
4. Update the view appropriately.

Whenever the need to select the part of the document for viewing and editing, then “travelling” is done through the document to find that part of interest. Travelling implies the movement of the editing context to a new position within the text. Filtering controls the selection of what is to be viewed and manipulated. Filtering extracts the relevant subset of the target document at the point of interest, such as the next screen full of text or the next statement. Next is do “Formatting” to determine a visible representation on a display screen or other devices.

Editing means the way a document is created or altered with a set of operations such as insertion, deletion, replace, move and copy. To build a document editor one should think about its internal structure, formatting, Graphical User Interface (GUI), and spell check. The contents of this document are (i) Computer programs (ii) Text information (iii) Equations (iv) Tables

(v) Line art and (vi) Photographs.

The editing functions are provided to operate on elements necessary for the type of editor used. For example, a manuscript-oriented editor is provided to operate on elements such as single characters, words, lines, sentences, and paragraphs. A program-oriented editor is used to operate on elements such as identifiers, keywords, and statements. The actual editing phase has a set of operations such as Insert, Delete, Replace, Move and Copy.

2.2 Types Of Editors

It is already mentioned earlier that the editors may be broadly classified into

(i) Textual editors (ii) Graphical editors. Editors can also be classified as Stream Editor, Line Editor, Screen Editor, and Structure Editor. (See figure 8.1)

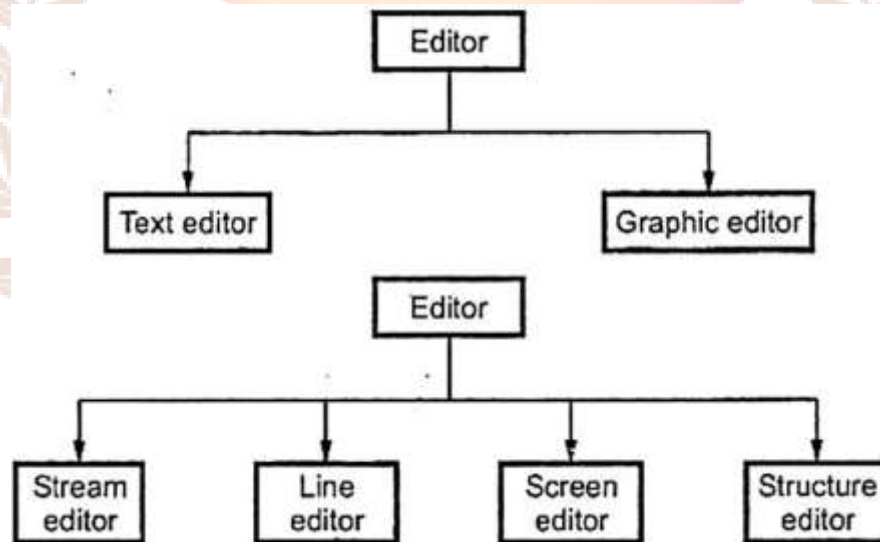


Figure 8.1: Different types of editors

- **Textual editors:** As already explained, they are the primary interface to the computer for all types of users who manipulate computer-based information.
- **Graphical editor:** A graphical editor is a program which allows the user to create the source program in Graphical User Interface (GUI) based editors. These are much user friendly.
- **Line editors:** Line editor is one of the simplest and most primitive form of editor, which uses a buffer to store information. It requires you to specify a specific line of text before making changes to it. It operates in command mode. The user gives the command to the

editor for any operation and the editor will respond to that command. (Buffer is in the main memory. That is set aside to store the information entered from the keyboard.)

- **Screen editors:** It is also called screenful-screen editors. A screen editor displays a screen full of text at a time. The user can move the cursor over the screen, position it at the point where the user desires to perform some editing, and proceed with the editing directly. The user has full control over the entire terminal. For example, the user can bring the cursor over a character to be deleted and press a delete key. It is possible to see the effect of an edit operation on the screen.
- **Stream editor:** A stream editor views the entire text as a stream of characters. These permit edit operations to cross line boundaries. Stream editor typically supports character line and context orient commands. In the stream editor, the current editing context is indicated by the position of the text pointer. This pointer can be manipulated using positioning. The stream editor maintains multiple representations.
- **Structure Editor:** The structure editor is a graphical editor that you can use to model organizational structures and relationships.

2.3 User Interface

The user interface (UI) is an editor for a user to interact with an application, as shown in figure 8.2. The user interface editors can be categorized into

(i) Interactive Editors (ii) Line Editors (iii) Screen Editors.

UI consists of a Dialog manager, presentation manager, and graphics package. The dialog manager manages the conversation between the user and the application. The Presentation manager displays the data produced by the application in an appropriate manner on the user display. A application called the graphics package gives you graphic data.

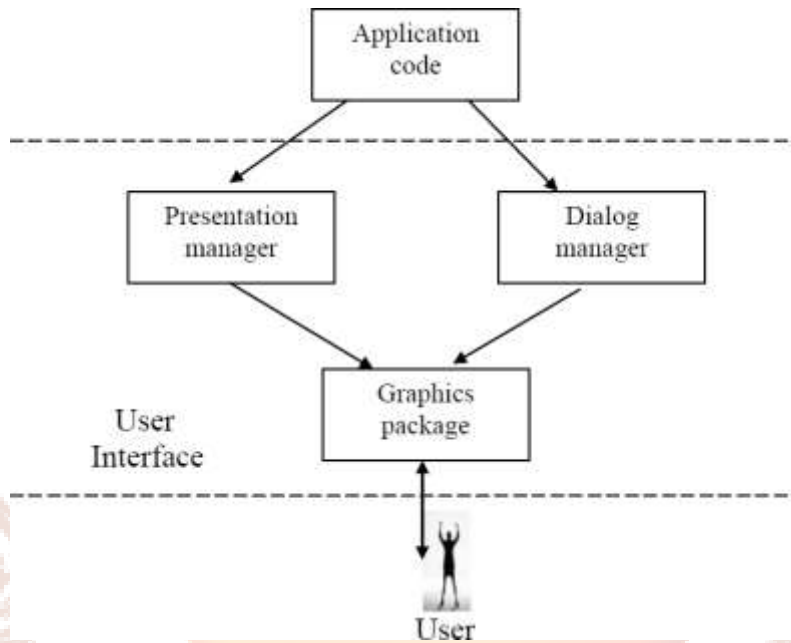


Figure 8.2: User interface

The dialog manager, is also responsible for error messages and online help function and for organizing changes in the visual context of the user. User Interface Management System (UIMS) automates the generation of user Interface. UIMS takes the specification of the presentation and dialog semantics to produce the presentation and dialog managers of the UI.

The user of an interactive editor is provided with a conceptual model of the editing system. . With a set of rules outlining how operations affect these aspects, this model offers a readily understood abstraction of the target document and its constituent parts.. In other words, this model is an abstract framework on which the editor and the world on which it operates are based.

The conceptual model essentially offers a readily understandable abstraction of the target document and its constituent parts; Besides the conceptual mode, the user interface is concerned with the input devices, the output devices, and the interaction language of the system.

Input devices are used to enter elements of the text to be edited, to enter commands, and to define editable elements. These input devices, as used with editors, can be divided into three categories:

- Text devices
- Button Devices
- Locator devices

Text or string devices are typically typewriter-like keyboards on which a user presses and releases keys, sending a unique code for each key.

Button or choice devices: These devices generate an interrupt or set a system flag. It usually causes the invocation of an associated application-program action. Such devices include a set of special function keys on an alphanumeric keyboard or on display itself.

Locator devices: These types of devices are two-dimensional analog to digital converters that are used to position a cursor symbol on the screen by observing the user's movements of the device. Such devices include joysticks, touch screen panels, data tablets, and mouse.

A locator device combined with a button device allows the user to specify either a particular point on the screen at which text should be inserted or deleted, or the start and end points of a string of characters to be operated upon.

Text devices with arrow (cursor) keys are often used to simulate locator devices. Each of these keys shown is an arrow that points up, down, left, or right. An advanced input device, namely a voice input device, translates spoken words into texts.

An output device is used to view the edited elements and results of the editing operations. Output devices like advanced CRT terminals use hardware assistance for moving the cursor, inserting and deleting characters and lines, and scrolling lines and pages.

Interaction language

The interaction language of a text editor is generally one of several common types. The categories of user interface interaction language are:

- i) Typing-Oriented or Text Command oriented [For e.g.: Unix editors]
- ii) Function-Key editor [For e.g.: 'C' editor]
- iii) Menu-oriented user interface or Icons [For e.g.: Windows editors].

i) The typing oriented or text command-oriented: In this type of editing interface, the user communicates with the editor by typing text strings both for command names and for operands. These strings are sent to the editor and are usually displayed on the output device.

Example: Unix editors

ii) The function key interface: In this type of interface, each command has associated with it a marked key on the user's keyboard. Function-key command specification is typically coupled with cursor-key movement for specifying operands, which eliminates much typing. For the common commands in a function-key editor, usually, only a single key need to be pressed.

Example: 'C' editor

iii) The menu-oriented user interface: It is a multiple-choice set of text strings or icons, which are graphic symbols that represent objects or operations. The editor prompts the user with a menu of only those actions that can be taken at the system's current state.

Example: Windows editors

Typing-oriented systems require familiarity with the system and language, as well as some expertise in typing. Function-key oriented systems often.

have either too few keys, requiring multiple keystroke commands, or have too many unique keys. The menu-oriented user interface has a menu and provides a multiple-choice set of text strings or icons, which are graphic symbols representing objects or operations. The user can perform actions by selecting items from the menu.

Self-Assessment Questions - 2

3. Which editor allows a user to enter, change, store, and usually print text?
 - a) text
 - b) graphic
 - c) stream
 - d) line
4. Which editor allows a user to create and revise a target document?
 - a) text
 - b) interactive
 - c) stream
 - d) line
5. The “_____” Process is an interactive user-computer dialogue.
6. A graphical editor is a program which allows the user to create the source program in “_____”based editors.
7. The use of an interactive editor is presented with a “_____” Model of the editing system.

2.4 Editor Structure

As already said, the fundamental functions in editing are traveling, editing, viewing, and displaying. The components of the editing structure are:

- Editing Component
- Traveling Component
- Viewing Component
- Display Component
- Filter Component

Most text editors have a structure similar to the one shown in figure 8.3.

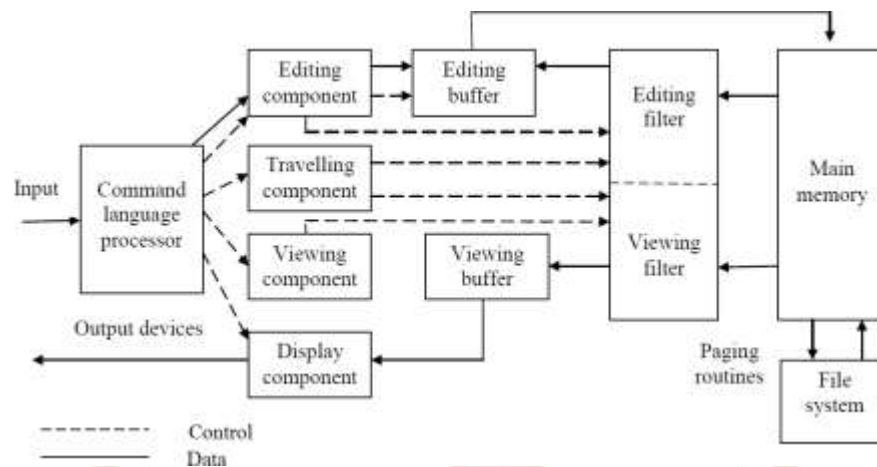


Figure 8.3: Typical Editor structure

As shown in figure 8.3, the command language processor accepts input from the user's input devices and analyses the tokens and syntactic structure of the commands. In this sense, the command language processor functions much like the lexical and syntactic phases of a compiler; just as in a compiler, the command language processor may invoke semantic routines directly. In a text editor, these semantic routines perform functions such as editing and viewing. Alternatively, the command language processor may produce an intermediate representation of the desired editing operations. An interpreter that invokes the appropriate semantic routines then decodes this intermediate representation. The use of an intermediate representation allows the editor to provide a variety of user-interaction languages with a single set of semantic routines driven from a common intermediate representation.

The semantic routines involve traveling, editing, viewing, and displaying functions. In editing a document, the start of the area to be edited is determined by the current editing pointer maintained by the editing module. The editing module is a collection of modules dealing with editing tasks. The user can set or reset the current editing pointer explicitly with traveling commands, such as the next paragraph and next screen. Or editing pointer can be set or reset implicitly by the system. The traveling module of the editor performs the setting of the current editing and viewing pointers.

The editing module invokes the editing filter when the user issues an editing command. The editing filter then filters the document to generate a new editing buffer based on the current editing pointer and the editing filter parameters. These parameters are specified by the user and the system as well. These parameters are information about the range of text that can be affected by an operation. Filtering consists of the selection of contiguous characters that begins at the current point. The semantic routines of the editing component then operate on the editing buffer, essentially a filtered subset of the document data structure.

Similarly, viewing a document determines the start of the area to be viewed by the current viewing pointer. This pointer is maintained by the viewing module of the editor, which is a collection of modules responsible for determining the next view. The current viewing pointer can be set or reset explicitly by the user with a traveling command or implicitly by the system the previous editing operation. The viewing component invokes the viewing module when the display needs to be updated. This component filters the document to generate a new viewing buffer based on the current viewing pointer and the viewing filter parameters.

In line editors, the viewing buffer can contain the current line. In screen editors, this buffer may contain a rectangular cutout of the quarter-plane of text. This viewing buffer is then passed to the display module of the editor, which produces a display by mapping the buffer to a rectangular subset of the screen, called a window or viewport.

While independent, the editing and viewing buffers can be related in many ways. In the simplest case, editing and viewing buffers are identical when the user edits the material directly on the screen (see figure 8.4).

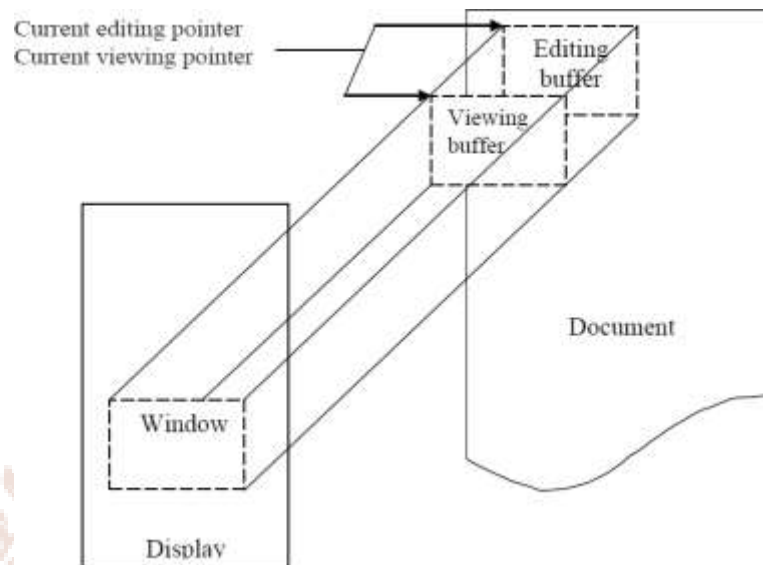


Figure 8.4: Simple relationship between editing and viewing buffers

The editing and viewing buffers can also partially overlap, or one may be completely contained in the other. Windows typically cover either the entire screen or a rectangular portion of it. Mapping viewing buffers to windows that cover only part of the screen is especially useful for editors on modern graphics-based workstations. Such systems can support multiple windows, simultaneously showing different portions of the same file or portions of different files. This approach allows the user to perform inter-file editing operations much more effectively than with a system having a single window.

The mapping of the viewing buffer to a window is accomplished by two system components. First, the viewing component formulates an ideal view, often expressed in a device-independent intermediate representation. This view may be very simple, consisting of a window's worth of text arranged so that lines are not broken in the middle of words. At the other extreme, the idealized view may be a facsimile of a page of fully formatted and typeset text with equations, tables, and figures. Second, the display module takes this idealized view from the viewing component and maps it to a physical output device in the most efficient manner possible.

Self-Assessment Questions - 3

8. The “_____” module of the editor performs the setting of the current editing and viewing pointers.
9. In viewing a document, the start of the area to be viewed is determined by the current viewing “_____”.



3. INTERACTIVE DEBUGGING SYSTEM

Let us now study what an Interactive debugging system is. An interactive debugging system is a system that provides programmers with facilities to aid in the testing and debugging of programs. Although the desirability of such systems has been recognized for some time, there are relatively few actual debugging systems in practical use.

The functions of the Interactive debugging system are:

- Debugging functions and capabilities
- Relationship with other parts of the system.
- User Interface criteria.

3.1 Debugging Functions And Capabilities

The programmer can specify the most obvious requirement for a set of unit test functions. One of the functions deals with execution sequencing, which is the observation and control of the flow of program execution. After execution is suspended, other debugging commands can be used to analyze the program's progress and diagnose the error detected; then, execution of the program can be resumed. Given a good graphic representation of program progress, enabling the program to run at various speeds called gaits may be useful .

There are other functions, such as tracing and tracing back which should be provided by a debugging system. Tracing tracks the flow of execution logic and data modifications. At various levels of detail, such as a module, subroutine, branch instruction, and so forth, the control flow can be followed. Traceback shows the path of the current statement. For a given variable or parameter, trace back can show which statements modified it. Such information should be displayed symbolically.

A debugging system should also have good program display capabilities. It must display the program being debugged, complete with statement numbers. The system should be able to save all the debugging.

specifications for a recompilation so the programmer does not need to reissue all these debugging commands. A debugging system should consider the language in which the program being debugged is written. Debugger commands that initiate actions and collect data about a program's execution should be common across languages. However, a debugging system must be sensitive to the language being debugged so that procedural, arithmetic, and conditional logic can be coded in the syntax of that language.

However, to get these requirements, a debugger has several consequences for the other software. When the debugger receives control, the execution of the debugged program is suspended temporarily. The debugger must then be able to determine the language in which the program is written and set its context accordingly.

The notation used to specify certain debugging functions varies according to the program's language. The debugger must have access to information gathered by the language translator. The internal symbol dictionary formats vary widely between different language translators. Future compilers and assemblers should aim toward a consistent interface with the debugging system. One approach is that the language translators can produce the needed information in a standard external form for the debugger regardless of the internal form used in the translator. Another possibility would be for the language translator to provide debugger interface modules that can respond to requests for information in a standard way, regardless of the language being debugged.

It is also important that a debugging system be able to deal with optimized code. Many optimizations involve the rearrangement of segments of code in the program. Blocks of code may be rearranged to eliminate unnecessary branch instructions, which provides more efficient execution. The user of a debugging system deals with the source program in its original form before optimizations are performed. The debugging of optimized code requires a substantial amount of cooperation from the optimizing compiler. In particular, the compiler must retain information about any transportation it performs on the program. Such information can be made available to the debugger and the programmer.

These are some of the advantages of debugging:

- ✓ However, some reported errors may be false alarms and hence a wastage of time.

- ✓ Error reported: Reports a problem state right away.
- ✓ Provides information: Gives helpful details about data structures and makes interpretation simple.
- ✓ Reduces useless data : Aids the developer in eliminating unnecessary and confusing information.
- ✓ Save time, energy: Cuts down on complex one-use testing code, saving time and resources for software development.

Unit Test functions:

- Execution Sequence:
- Break points
- Conditional expressions
- Gaits

The importance of the execution sequence should be investigated in order to grasp its operational principles:

- ☐ It comprises keeping an eye on and managing how program are run.
- ☐ After a certain number of instructions have been executed, It can be stopped.
- ☐ There isn't a start to the curriculum again.
- ☐ Debugging functions such as Traceback and Tracing are important :
- ☐ Under Tracing:
- ☐ Data changes and execution logic flow are monitored.
- ☐ It is only complete after several adjustments.

Under Traceback:

- ☐ demonstrates the approach utilized to begin the sentence's present execution.
- ☐ When tracing back, only the current statement's path is taken.

3.2 Relationship With Other Parts Of The System

An interactive debugger must be related to other parts of the system in many different ways. It must appear to be a part of the run-time environment and an integral part of the system. The single most important requirement for any interactive debugger is that it always be available. When an error is discovered, immediate debugging must be possible because it may be different or impossible to reproduce the program failed in another environment or at another time. Therefore, the debugger must communicate and cooperate with other operating system components, such as interactive subsystems.

Debugging is even more important at production time than it is at application-development time. The debugger must also exist in a way consistent with the system's security and integrity components. Use of the debugger must be subject to the normal authorization mechanisms and leave the usual audit trails. The debugger must coordinate its activities with those of existing and future language compilers and interpreters.

3.3 User Interface Criteria

Let us now study the different criteria's that a user interface should bear. The interactive debugging system should:

- Be simple in its organization and familiar in its language
- Have facilities organized into a few basic categories of function
- Have function that closely neglect common user tasks
- Be able to contribute to ease of training and ease of use.

Other than the above-mentioned criteria, the user interaction should take advantage of full-screen terminal devices when they are available so that more information can be displayed and changed easily and quickly. Menus should reflect a task that a user can use. Menus should have titles that identify the task. Directions should precede any choices available to the user. Techniques such as indentation should help separate portions of the menu.

Interactive users should be supported when a screen terminal device is not present. Every action a user can take at a full-screen terminal should have an equivalent action in a linear

debugging language. The command language should have a clear, logical, simple syntax. Commands should be simple rather than compound.

There should be consistent use of parameter names across the commands. Parameters should be automatically checked for errors in such attributes as type and range of values. Command formats should be as flexible as possible. The command language should minimize the use of such punctuation as parentheses, slashes, quotation marks, and other special characters. Any good interactive system should have an online HELP facility. HELP should be accessible from any state of the debugging session.

The user interface can be visualized as having two parts.

- Components of UI
 - Dialogue Manager:

Manages communication between the user and the application. It solicits a command from the user and sends it to the server.

- Presentation manager

Displays the data from the program in a usable manner.

Conversations between the customer and the program are managed by the dialogue manager.

The presentation manager shows the details that the program generates.

An application's command dialogue is used to issue commands.

The below are few examples of how order dialogue can be implemented:

- Command languages
 - Resemble operating system command languages in certain ways.
 - Syntax as follows: <action> <parameters>
- Command menus
 - Ensure the app's casual users receive clear benefits.
 - The menus represent the most basic functions

- Direct manipulation
 - The entire device universe is available to the customer.
 - Displays the most important things in the application.

Let us see the Principles of Command Dialogue Design:

These are the principles that psychologists and engineers use to ensure that command dialogues are successful.

Engineers and psychologists have created the following features to guarantee the effectiveness of command dialogues, which include:

- The command system must be consistent
- User instructions receive immediate input
- Undo facility
- Shortcuts for experienced users
- Ease of use
- Error Handling
- To stop having to write down command information, use online assistance

The term "user interface," or "UI," refers to how a person interacts with a machine. Even while a light switch, for example, might legally be referred to as an instrument of UI, most contemporary allusions concern computers and other electronic devices.

The interaction between users and machines is made possible by UI. This essential method of communication would be impossible without it.

There are four prevalent types of user interface and each has a range of advantages and disadvantages:

- Command Line Interface
- Menu-driven Interface
- Graphical User Interface
- Touchscreen Graphical User Interface

Command Line Interface:

Although it is no longer frequently used as a fundamental user interface in ordinary consumer devices, the command line interface is still in use in some situations. Users using the command line interface must enter the proper instructions on the command line. The directive instructs the computer to open the necessary file or directory first. From there, a wide range of commands, such as those for retrieving files and running programmes, become available.

Advantages:

- Simple architecture
- very little memory utilization
- Excellent for memory-constrained or slowly operating computers
- A skilled CLI user can issue commands and complete tasks considerably more quickly than when using another type of UI.

Dis-advantages:

- learning a command language is challenging
- difficult for new users
- minimal information in error messages

Menu-Driven Interface

You can access a variety of commands or options through the menu-driven user interface in the form of a list or menu that is displayed in full-screen, pop-up, pull-down, or drop-down modes. An example of a menu-driven interface is an ATM.

Advantages:

- There is no need to keep track of a lengthy number of manual commands.
- For beginners, a simple interface
- Menu options with clear instructions

Dis-advantages:

- slower for knowledgeable users
- a menu with few choices
- To execute basic operations, you frequently need to open many menu screens.

Graphical User Interface

- The form of interface that most people are most accustomed to is the graphical user interface, or GUI. By pointing and clicking on images or icons with a mouse, track pad, or other device, you can interact with these interfaces.

Advantages:

- Self-explanatory
- Simple to use
- There is no need to commit command lists to memory.
- enables the simultaneous use of several applications, programs, and tasks.
- solid infrastructure
- The same structure of several programs creates a sense of familiarity.

Disadvantages:

- Uses a lot of memory, yet as computers become more powerful, this becomes less of an issue.

Touchscreen Graphical User Interface

The touchscreen GUI is quite similar to the standard GUI, with the exception that you choose icons and carry out actions with your fingers or a stylus rather than a mouse or trackpad. Tablets, cellphones, and medical devices like the t:slim insulin pump all have touchscreen user interfaces. The touchscreen GUI offers a more personal way of interacting while having the same advantages and downsides as regular GUIs. Touchscreen GUIs are particularly practical because there are no peripherals required.

The graphical user interface, followed by the touchscreen variant, is by far the most popular of the four different forms of the user interface. The GUI continues to be the favored standard despite the other technologies that already exist and are developing. The simplicity and usability are largely to blame for this.

The majority of end users find it simpler to understand graphical user interfaces since they don't have to memorize or input complicated commands and because the icons and menus are typically self-explanatory.



Self-Assessment Questions - 4

10. Which of the following functions of the Interactive debugging system?
 - a) Debugging functions and capabilities
 - b) Relationship with other parts of the system.
 - c) User Interface criteria.
 - d) all of the above
11. In a good graphic representation of program progress, the program runs at various speeds called “_____”.
12. A debugging system should provide functions such as “_____” and “_____”.
13. Which functions can be used to track the flow of execution logic and data modifications in debugging system?
 - a) Tracing
 - b) Trace back
 - c) gait
 - d) travelling
14. Which of the following determines the programming language and sets its context accordingly?
 - a) Tracer
 - b) gait
 - c) User Interface
 - d) debugger

4. SUMMARY

Let us recapitulate the important concepts discussed in this unit:

- The major function of an editor is editing. The editor performs other supporting services to editing like editing, traveling, viewing, and displaying.
- Traveling means the ability to travel the text in the file. The viewing option allows the user to create a view of the file. Displaying generates a printable view and displays it on the screen.
- The editors may be broadly classified into two types as (i) Textual editors (ii) Graphical editors. Editors can also be classified as Stream Editor, Line Editor, Screen Editor and Structure Editor.
- A text editor is a computer program that lets users enter, change, store, and usually print text.
- Text editors are often provided with operating systems or software development packages. They can be considered the primary interface to the computer for all types of users who manipulate computer-based information.
- Some of the popular text editors are Emacs, Jed, Joe, Nano, Vi, Vim, Xedit.
- The document-editing process is an interactive user-computer dialogue designed to accomplish different tasks.
- The user of an interactive editor is presented with a conceptual model of the editing system.
- Besides the conceptual mode, the user interface is concerned with the input devices, the output devices, and the interaction language of the system.
- An interactive debugging system provides programmers with facilities that aid in the testing and debugging programs.
- A debugging system should also provide functions such as tracing and trace back.
- An interactive debugger must be related to other parts of the system and must appear to be a part of the run-time environment and an integral part of the system.
- The criteria of User Interface are that the interactive debugging system should be simple in its organization and familiar in its language.

5. GLOSSARY

Interactive Debugging: System that provides programmers with facilities to aid in the testing and debugging of programs.

Locator device: Two-dimensional analog to digital converters are used to position a cursor symbol on the screen by observing the user's movements of the device.

Text editor: A computer program that lets a user enter, change, store, and usually prints text.

Typing oriented interaction language: In editing interfaces, the user communicates with the editor by typing text strings both for command names and for operands.

User Interface (UI): An editor for a user to interact with an application as to produce the presentation and dialog managers of the UI.

User Interface Management System (UIMS): Automates the generation of user Interface and takes the specification of the presentation and dialog semantics.

6. TERMINAL QUESTIONS

Short Answer Questions

1. Explain what a text editor is. Write down the names of at least five popular text editors.
2. What is the different tasks document-editing process accomplishing? Explain.
3. Differentiate between Line editors and Stream editors.
4. Draw the diagram of a typical editor structure and explain about it.
5. What are the functions of interactive debugging systems? Explain the debugging functions and capabilities of interactive debugging systems.

7. ANSWERS

Self-Assessment Questions

1. editor
2. main memory (RAM)
3. a) text
4. b) interactive
5. document-editing
6. Graphical User Interface (GUI)
7. conceptual
8. traveling
9. pointer
10. d) all of the above
11. gaits
12. tracing, trace back
13. a) Tracing
14. d) debugger

Short Answer Questions

1. A text editor is a computer program that lets users enter, change, store, and usually print text. (Refer to section 8.2 for more details)
2. The document editing process accomplishes four different tasks. They are: 1. Select the part of the target document to be viewed and manipulated 2. Determine how to format this view online and how to display it (Refer to section 8.2.1 for more details)
Line editors operate in command mode, where you can specify a specific line of text before making changes to it whereas a stream
3. editor views the entire text as a stream of characters. (Refer to section 8.2.2 for more details)
4. (Refer to section 8.2.4 for more details)
5. The functions of the Interactive debugging system are: (i) Debugging functions and capabilities (ii) Relationship with other parts of the system (iii) User Interface criteria. (Refer to section 8.3 for more details)

8. SUGGESTED BOOKS

- Leland L. Beck (1990). *System Software*, Addison-Wesley Publishing Company.
- M. Joseph. (2007). *System Software*. Firewall Media.
- *system programming*. (n.d.). Retrieved 09 06, 2012, from [http://www.itswtech.org/Lec/Manal\(system%20programming\)/ch7_system_software_tools.pdf](http://www.itswtech.org/Lec/Manal(system%20programming)/ch7_system_software_tools.pdf)
- *Text Editor*. (n.d.). Retrieved 09 16, 2012, from http://www.princeton.edu/~achaney/tmve/wiki100k/docs/Text_editor.html





BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4

DCA2203
SYSTEM SOFTWARE

Unit 9

Device Driver – I

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
1.1	Learning Objectives	-	-	
2	Device Driver	1	1	4 - 10
2.1	UNIX/Linux device drivers	-	-	
2.2	MS-DOS device drivers	-	-	
2.3	Windows system device drivers	-	-	
3	Role of Device Drivers	2	2	11- 16
4	Classes of Devices		3	17 - 19
5	Security issues	-	4	20 - 25
6	Design issues	-	5	26 - 29
6.1	CPU issues that influence device driver design	-	-	
6.2	Bus issues that influence device driver design	-	-	
7	Summary	-	-	30
8	Glossary	-	-	31
9	Terminal Questions	-	-	31
10	Answers	-	-	32 - 33
11	Suggested Books and E-References	-	-	33

1. INTRODUCTION

We discovered the Text Editor in the preceding unit. An editor's main job is editing, and a text editor is a computer program that enables users to enter, modify, save, and typically print text. The document-editing process is an interactive user-computer dialogue designed to accomplish different tasks. You also read that an interactive debugging system provides programmers with facilities that aid in the testing and debugging programs. In this unit, you will study the device driver that acts as a component to provide Input-output (I/O) services to interact with any hardware devices that are connected to the computer system.

. Every device part of the computer system needs a device driver; otherwise, the Operating System (OS) cannot operate or cannot manage the connection between the hardware devices. The role of a device driver is to provide a certain mechanism to allow accessing data to and from the attached devices. We'll discover how the Unix/Linux operating system categorizes devices into several groups. Device drivers are necessary parts that are provided by the company that makes the device, however, there are security concerns with them. In the end we will study the design issues related to the device drivers. The focus mainly with the device drivers related to the Unix/Linux OS.

1.1 Learning Objectives:

After studying this unit, the learners should be able to:

- ❖ *Explain about device drivers*
- ❖ *Determine the role of the device driver*
- ❖ *List the classes of devices*
- ❖ *Describe the security issues concerned with device drivers*
- ❖ *Design issues associated with the device drivers.*

2. DEVICE DRIVER

The Operating System (OS) is like a manager that manages the interaction between all the devices and the underlying process between them in a computer system. It is the job of the OS to facilitate communication between all the devices connected to the computer system. However, some devices, known as *peripheral devices*, that are connected externally to the computer system may not be able to communicate because the hardware components of external devices may have been designed in a different way for communicating data to and from other devices. Hence, peripherals connected to a computer need special communication links to interfere with the CPU. The purpose of the communication link is to resolve the differences between the central computer and each peripheral.

The major differences between the central computer and each peripheral are due to the following reasons:

- Peripherals are electromagnetic and electromechanical devices, so their operations differ from the CPU and the memory.
- Data transfer rate of peripherals is different from the transfer rate of the CPU; hence a synchronization mechanism is needed.
- Data codes and formats in peripherals differ from the word format in the CPU and memory.
- The operating modes of peripherals are different from each other, and each must be controlled so as not to disturb the operation of other peripherals connected to the CPU.

To resolve these differences, computer systems include interface units between the CPU and peripherals to supervise and synchronize all types of input and output functions. Between the CPU bus and the peripheral device are interface units. To allow the OS to recognize and access the hardware devices, certain software components in the form of software interface are necessary.

A device driver is a software interface that allows the OS to provide Input-output (I/O) services to interact with an underlying device. The device driver converts the logical requests from the user into specific commands directed to the device. Generally, the term

driver is used to mean software driver, device driver, bus driver, or other drivers related to the object.

As shown in figure 9.1, in windows OS, any application must communicate to devices through the driver.

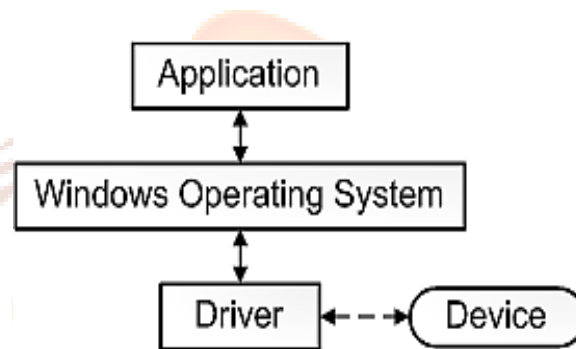


Figure 9.1: Driver between OS and devices

Although device drivers are just add-on modules, they are an integral part of the system closely integrated with the Input/ Output Control System, which deals with I/O related system calls.

Generally, device drivers are installed for every device on the computer. Some device drivers, such as disk drives, the processor and the chipset on the mother board are shipped with the operating system. Because they are necessary for the operating system to function. The device manufacturer generally provides device drivers if the operating system does not support the device. The drivers are written by the manufacturer of the device as it knows how the device hardware communicates to get the data. Drivers of any device are different for the different operating systems and their supporting versions.

Example: It's possible that the manufacturer of your visual card doesn't produce a driver for a specific operating system. Sometimes, not all drivers are written by manufacturers. Generally, a device is designed and made according to a published hardware standard. This means that the driver can be written by OS developing company (like Microsoft or MAC) or a group of people (for Linux) and hence the device designer may not provide a driver.

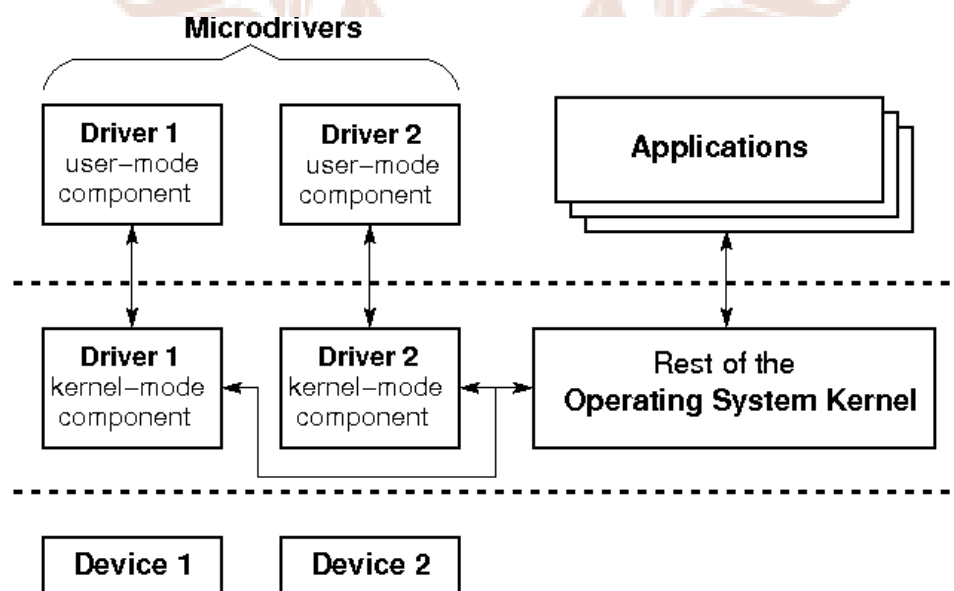
Any OS loads device drivers and calls functions in the drivers to access and carry out actions on the device. The driver functions contain the device- specific code needed to carry out actions on the device.

Example: If any application needs to read some data from a device, then the application calls a function implemented by the operating system, and the operating system calls a function implemented by the driver. After the driver gets the data from the device, it returns the data to the operating system, which returns it to the application.

Features of device drivers:

- Device responses and communications sent to the computer are managed by device drivers.
- Without system drivers, particular hardware does not function as expected or in accordance with its intended purpose.
- We can optimize the hardware thanks to it.
- It gives us the ability to speed up or slow down hardware and test its limitations.
- It helps us by informing us of the device's state.

Network Architecture for Device Driver



At the basic level, device drivers can be split into two layers, a logical layer and a physical layer.

Logical Layer:

- Logical layers process data for a class of devices.

Physical Layer:

- Specific instances of devices are communicated through physical layers.
- The physical layer will deal with the logical layer's communication needs with a specific serial port chip for them to work together.
- The device information structure and the static structure are two crucial data structures found in every device driver.
- Both the device driver installation and information sharing between entry point functions employ these structures.

Device information structure:

- The purpose of the device information structure, which is a static file supplied to the install entry point, is to send along any information needed to install a major device into the entry point, where it will be utilized to initialize the static structure.

Static structure:

- The static structure is initialized with the data stored in the information structure and is used to transmit information between the various entry points.
- The operating system uses the entry point functions to interact with the driver.

Micro drivers divide device driver functionality between a kernel-mode component and a user-mode component.

A micro driver comprises a user-mode process that handles non-essential functionality and a kernel-mode component that handles crucial functionality. At function borders, device driver functionality is divided between the kernel-mode and user-mode components.

2.1 UNIX/Linux Device Drivers

Since Linux follows the UNIX model, and in UNIX, everything is a file, users communicate with device drivers through device files. Device files are supplied by the kernel for a direct User-Driver interface. Hence in UNIX/Linux OS, device drivers are in the form of files and are usually linked to the object code of the kernel (the core of the operating system). Almost every system operation eventually maps to a physical device. Nearly all device control operations are performed by a device driver specific to the device being addressed except a few of the processor, memory, and a few other entities. Hence, in the UNIX/Linux OS, the kernel must have embedded in it a device driver for every peripheral present on a system, from the hard drive to the keyboard and the tape drive. This means that when a new device is to be used, which was not included in the original construction of the operating system, the UNIX kernel must be re-linked with the new device driver object code. This technique has the advantages of run-time efficiency and simplicity, but the disadvantage is that adding a new device requires the regeneration of the kernel. In UNIX, entry of devices is in the /dev directory associated with a device driver that manages the communication with the related device. A list of some device names is as shown below:

Device name	Description
/dev/console	system console
/dev/tty01	user terminal 1
/dev/tty02	user terminal 2
/dev/lp	line printer
/dev/dsk/f03h	1.44 MB floppy drive

2.2 MS-DOS Device Drivers

In MS-DOS, device drivers are installed and loaded dynamically, i.e., they are loaded into memory when the computer is started or re-booted and accessed by the operating system as required. This technique ensures that only those drivers which are required are loaded into the main memory. The device drivers to be loaded are defined in a special file called CONFIG.SYS, which resides in the root directory. MS-DOS reads this file at start-up of the system, and its contents acted upon. A list of some device names is shown below:

Device name	Description
con:	keyboard/screen
com1:	serial port1
com2:	serial port2
lpt1:	printer port1
A:	first disk drive
C:	hard disk drive

2.3 Windows System Device Drivers

In the Windows system, device drivers are usually library files with .sys file name extensions and are implemented as dynamic link libraries (DLLs). This technique provides advantage that DLLs contain hence contain a shareable code; hence, one copy of the code needs to be loaded into memory. Another advantage is that a software or hardware vendor can implement a new device without modifying or affecting the Windows code. You can also configure optional drivers for devices.

In the Windows system, the idea of Plug and Play device installation is required to add new devices such as a CD drive, Flash drive, etc. The objective is to make this process automatic so that the driver software is loaded when the device is attached. After that, the installation is automatic, and the settings are chosen to suit the host computer configuration.

Self-Assessment Questions - 1

1. Peripherals connected to a computer need special communication links for “_____” them with the CPU.
2. A device “_____” is a software interface that allows the OS to provide Input-output (I/O) services to interact with an underlying device.
3. Which of the following directory is associated with a device driver that manages the communication with the related device in UNIX?
 - a) /dev
 - b) /etc
 - c) /home
 - d) /lib
4. In MS-DOS, device drivers are installed and loaded “_____”.
5. In the Windows system, device drivers are usually library files with “_____” filename extensions and are implemented _____.

3. ROLE OF DEVICE DRIVERS

Before going on to discuss the role of device drivers, let us study the difference between “user space” and “kernel space.” UNIX/Linux developers use the term user space and kernel space whereas Windows developers use the term user mode and kernel mode. In UNIX/Linux, processes switch between user space and kernel space using system calls. In Windows, the processor switches between the user and kernel modes depending on what type of code runs on the processor. Applications run in user mode, and core operating system components run in kernel mode. Many drivers run in kernel mode, but some run in user mode. Explanation of these different driver forms is beyond this SLM’s scope.

- **Kernel space:** The kernel and its device drivers form an interface between the user/programmer and the hardware. Kernel space includes any routines or functions that are a kernel component (such as modules and device drivers).
- **User space:** Like the UNIX shell or other GUI-based applications, end-user programs are part of the user space. These applications interact with the system's hardware. However, in Linux, they interact through the kernel-supported functions.

Let us look at the types of Roles Performed by a device:

Console: You can install the configuration manager console on other computers and restrict access and limit what administrative users can find in the console by utilizing the configuration manager’s role-dependent administration. With the computer, it communicates. Examples include video games, system alerts, and voice instructions.

Communication: A communication device is a hardware device capable of transmitting the digital signal of the computer system over communication interfaces. Other communication devices include NIC (network interface card), Wi-Fi, and access points.

- ☐ A computer modem is a prime example of a communication device.
- ☐ With the use of a gadget, chat, and VoIP communications are possible.
- ☐ Speaking on the phone with someone else

Multimedia: The multimedia role of the device allows a person to deal with various media while eliminating the need to have a separate device for each.

- Examples include playing or recording audio content, music, and movies, narration, and live music recording.
- A multimedia device is a piece of hardware designed to play, store, record, or display multimedia files like music, films, and images.
- . People use multimedia audio devices to record and play audio, including music, sound effects, and voice.

The examples of an input device are as follows:

- The 101 keys on keyboards are necessary for communicating.
- Pointing devices include the mouse, trackball, and space ball.
- A scanner is an input device that functions similarly to a photocopier.
- Sound is captured via a microphone and then stored digitally as input.
- An input device known as a digitizer converts analog data into digital form.
- Banks utilize the MICR input device to process a significant number of cheques each day.

Working on device drivers in UNIX/Linux OS

In UNIX/Linux, a device driver communicates with the user, kernel, and hardware, as shown in figure 9.2.

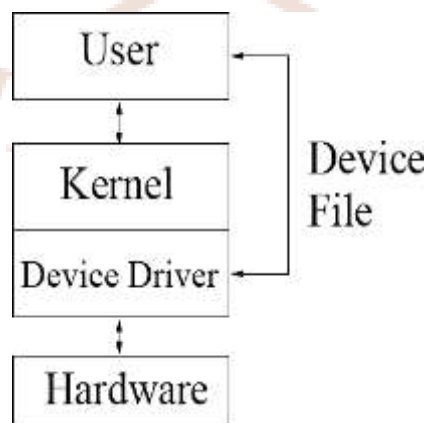


Figure 9.2: Connection of device driver with the user, Kernel, and hardware in Linux.

The kernel offers several subroutines or functions in user space, which allow the end-user application programmer to interact with the hardware. In UNIX/Linux systems, the devices are seen as files, and hence the interaction is performed through functions or subroutines to read and write files. In kernel space, UNIX/Linux offers several functions or subroutines to perform low-level interactions directly with the hardware and allow the transfer of information from the kernel to the user space. Usually, for each function that allows the use of devices or files in user space, there exists an equivalent function or subroutines in kernel space to transfer information from the kernel to the user and vice-versa.

Working on device drivers in Windows OS

Windows OS loads device drivers and calls functions in the drivers to carry out actions on the device. Each device driver exports a set of functions, and Windows calls these functions. The driver functions contain the device-specific code needed to carry out actions on the device.

Mechanism and Policy

When the one designs or programs a device driver, then you are free to choose an acceptable trade-off between the programming time required to develop a driver and the flexibility of the result. The term “flexible” here means that the role of a device driver is providing a mechanism, not a policy. The role of a device driver can be discussed in terms of *mechanism* and *policy*. As a programmer, one is free to design the driver in terms of mechanism and policy. When we talk of mechanism, it is “What capabilities are to be provided,”. In contrast, when we talk of policy, it is about “how those capabilities can be used.” Most programming problems can be split into the mechanism and the policy. If the two issues are addressed by different parts of the program, or even by different programs, it becomes much easier to develop software programs for a particular need.

For example, UNIX management of the graphic display is split between

(i) the X server and (ii) the window and session managers. The X server knows the hardware and offers a unified interface to user programs. The window and session managers implement a particular policy without knowing anything about the hardware. People can use

the same window manager on different hardware, and users can run different configurations on the same workstation. Even a different desktop environment, such as KDE and GNOME, can coexist on the same system. In the case of the layered structure of TCP/IP networking as another example, the operating system offers the socket abstraction, which has no policies in place regarding the data transfers, while the services are handled by various servers (and their associated policies).

Wherever drivers are concerned, a separation of mechanism and policy can be applied. Consider that a floppy drive is a policy free as its role is only to show the diskette as a continuous array of data blocks. Higher levels of the system provide policies, such as who may access the floppy drive, whether the drive is accessed directly or via a file system, and whether users may mount file systems on the drive. Different environments usually need to use hardware differently, so it is important to be as policy free as possible.

When a programmer is developing a device driver, pay particular attention to the fundamental concept: write kernel code to access the hardware, but do not force specific particular policies on the user, since different users have different needs. The device driver is a software layer that lies between the applications and the actual device. You can design what you can enable your driver to be, i.e., can design drivers to offer different capabilities, even for the same device and design of a driver should be balanced, considering various factors. For instance, a single device may be used concurrently by different programs, and the driver programmer can determine how to handle concurrency. Consider the trade-off between the maximum options available for users and the time taken to develop the driver, along with simplicity and error-free mechanism.

Develop a policy-free driver that can have several typical characteristics. Those characteristics can be a support for both synchronous and asynchronous operations, the ability to be opened multiple times, the ability to exploit the full capabilities of the hardware, and the lack of software layers to “simplify things” or provide policy-related operations. Drivers of this sort of work better for their end users and are easier to write and maintain. Hence making a policy-free driver is the main objective of software designers.

Device drivers can also be sent along with user applications to aid with configuration and access to the target device. With programs that can be simple utility or graphical apps, you can grant end-users independence.

Let us see the Advantage and Disadvantage of device drivers:

- A device driver aids in the generalization of programming by serving as an abstraction layer between a hardware device and the applications or operating systems that use it.
- Devices based on information may not be able to function for the purpose for which they were implemented if some incorrect information has been entered into the database.

How to use the manufacturer's website to get appropriate drivers in detail:

- You must first identify the hardware's model number and manufacturer before installing the driver.
- Then, access the support page for the manufacturer and log in. Find out where to download drivers.
- Now save the driver to your PC after downloading it.
- After that, extract the driver and adhere to the computer's instructions.

Let us see the steps to Download Driver from Third Party Driver Installer:

Download Driver Reviver first, then install it on your device to verify the directions displayed on the screen.

- Then click "Start PC Scan Now" to have your computer check for any outdated or corrupted drivers.
- The drivers are immediately updated when selecting "Update all."

Let us see an example of driver installation:

- Driver will begin the program installation process from the operating system library.
- The software driver for that specific printer model launches and manages the hardware component, such as a printer.

- Your computer should immediately identify the new printer after you connect the printer wire to your device.

Let us see how the driver works:

We need a driver since Operating System is software and the device is hardware. Both use different languages and do not understand each other's language.

The device driver's main job is translating commands from the operating system to the device.

The Operating System wants to exchange data with the devices for sending and receiving purposes.

Self-Assessment Questions - 2

6. Any subroutines or functions forming part of the kernel are considered part of "_____" whereas end-user programs, like the UNIX shell or other GUI-based applications are part of _____.
7. How does Windows OS carry out actions on the device?
 - a) Windows loads device drivers and calls functions in the drivers
 - b) Windows loads devices in the memory
 - c) Memory connects devices to the processor
 - d) Device drivers are connected to the memory and processor
8. The role of device driver design can be seen through mechanism and policy. (True/False)

4. CLASSES OF DEVICES

UNIX/Linux OS distinguishes and classifies devices attached to the computer devices into three fundamental device types. They are

(i) Character devices (ii) Block devices (iii) Network interfaces. The modules developed as device drivers are usually implemented for one of these devices. Depending upon the device types, the modules for the device driver can be classified as (i) a char module (ii) a block module (iii) a network module. Although the modules are programmed and divided into classes, it is the freedom of the programmer to either build huge modules implementing different drivers in a single chunk of code or build into the modules as classified above. If scalability and extension ability are required in the functionality of different modules, then it is better to create a different module for each new functionality.

- I. **Character devices:** A character (char) device is classified because it can be accessed as a stream of bytes (like a file). This driver usually implements at least the open, close, read, and write system calls. For example, the text console (*/dev/console*) and the serial ports (*/dev/ttyS0*) are well represented by the stream abstraction and hence are examples of char devices. Char devices are accessed using file system nodes, such as */dev/tty1* and */dev/lp0*. Char devices are just data channels which can be accessed sequentially. The difference between a char device and a regular file is that you can move back and forth in the regular file, whereas most char devices are data channels which can be accessed sequentially.
- II. **Block devices:** A block device (e.g., a disk) that can host a filesystem. Like char devices, block devices are accessed by file system nodes in the */dev* directory. In most Unix systems, a block device can only handle I/O operations that transfer one or more whole blocks, usually 512 bytes (or a larger power of two) in length. Instead, Linux allows the application to read and write a block device like a char device – it permits the transfer of any number of bytes simultaneously. As a result, block and char devices differ only in the way data is managed internally by the kernel and thus in the kernel/driver software interface. Like a char device, each block device is accessed through a file

system node, and the difference between them is transparent to the user. Block drivers have a completely different interface to the kernel than char drivers.

III. **Network interfaces:** Any network transaction is made through an interface, that is, a device that can exchange data with other hosts. Usually, an interface is a hardware device, but it is also a pure software device, like the loopback interface. A network interface is responsible for sending and receiving data packets, driven by the kernel's network subsystem, without knowing how individual transactions map to the actual packets being transmitted. Many network connections (especially those using TCP) are stream-oriented, but network devices are usually designed around the transmission and receipt of packets. A network driver knows nothing about individual connections; it only handles packets.

The Unix way to provide access to interfaces is to assign a unique name to them (such as eth0), but that name doesn't have a corresponding entry in the file system. Communication between the kernel and a network device driver is completely different from that used with char and block drivers. Instead of reading and writing, the kernel calls functions related to packet transmission.

There are other ways of classifying driver modules, also. In general, some drivers work with additional layers of kernel support functions for a given type of device. For example, every USB device is driven by a USB module that works with the USB subsystem. Still, the device itself shows up in the system as a char device (a USB serial port, say), a block device (a USB memory card reader), or a network device (a USB Ethernet interface).

Other classes of device drivers are added to the kernel nowadays, including FireWire drivers and I2O drivers.

In addition to device drivers, other functionalities related to hardware and software are modularized in the kernel. One common example is file systems. A file system type determines how information is organized on a block device to represent a tree of directories and files. Such an

The entity is not a device driver because there's no explicit device associated with how the information is laid down. The file system type is instead a software driver as it maps the low-level data structures to high-level data structures. that the file system determines how long a file name can be and what information about each file is stored in a directory entry. The file system module must implement the lowest level of the system calls that access directories and files by mapping filenames and paths (as well as other information, such as access modes) to data structures stored in data blocks. Such an interface is completely independent of the actual data transfer to and from the disk (or another medium), which a block device driver accomplishes.

In Unix/Linux platform, the ability to decode file system information stays at the lowest level of the kernel hierarchy. Unix/Linux supports the concept of a file system module, whose software interface declares the different operations that can be performed on a file system inode, directory, file, and superblock.

Self-Assessment Questions - 3

9. Which drivers are used to accessing character devices as a file?
 - a) Block drivers
 - b) char drivers
 - c) network drivers
 - d) stream driver
10. A block device is a device that can host a "_____".
11. A network driver knows nothing about individual connections; it only handles packets. (True/False)

5. SECURITY ISSUES

Security is of prime concern in modern times when any software is been developed. Any security check in the system is enforced by kernel code. There should not be security holes in the kernel due to a bad module that gets loaded into the kernel. In the official kernel distribution, only an authorized user can load modules. The system called *init _module* checks if the invoking process is authorized to load a module into the kernel. Thus, only the superuser, or an intruder who has succeeded in becoming privileged, can exploit the power of privileged code by running an official kernel.

Driver writers should avoid encoding security policy in their code as far as possible. Security is a policy issue that is often best handled at higher levels within the kernel, under the system administrator's control. However, there are always exceptions. As a device driver writer, one should be aware of situations in which some types of device access could adversely affect the system as a whole and should provide adequate controls.

Any software that uses system resources to get an illicit profit should be prohibited. You should be aware that even the slightest software flaws become obvious to everyone and might lead to misuse. For example, device operations that affect global resources (such as setting an interrupt line), which could damage the hardware (loading firmware, for example), or that could affect other users (such as setting a default block size on a tape drive), should be available to privileged users only.

The driver writers must be careful to avoid introducing security bugs. Security bugs, as an example, can be due to buffer overrun errors in which the programmer forgets to check how much data is written to a buffer. Data ends up written beyond the end of the buffer, thus overwriting unrelated data. Such errors can compromise the entire system and must be avoided.

It's important to remember that any input from user processes shouldn't be used until it can be verified. Use uninitialized memory with caution. Any memory obtained from the kernel should be zeroed or otherwise initialized before being made available to a user process or device. Otherwise, information (disclosure of data, passwords, etc.) can be leaked. If device

interprets data sent to it, be sure the user cannot send anything that can harm the system. Finally, think about the possible effect of device operations. Suppose there are specific operations (e.g., reloading the firmware on an adapter board or formatting a disk) that could affect the system. In that case, those operations should be restricted to privileged users only.

Be careful of software from third parties, especially when the kernel is concerned, because if everybody has access to the source code, everybody can break and recompile the codes. The pre-compiled kernels found in your distribution are quite reliable, one should avoid running kernels compiled by an untrusted source. For example, a maliciously modified kernel could allow anyone to load a module via the `init_` module. Note that the Linux kernel can be compiled to have no module support, so no security holes are introduced. All needed drivers must be built directly into the kernel itself to achieve this. . It is also possible, with version 2.2 and later kernels, to disable the loading of kernel modules after system boot via the capability mechanism.

Importance of Addressing Security and Design Issues:

- It is crucial to provide users of the software with adequate security since we must make sure that the data fed into the system are secure and kept private from the system itself.
- Any shortcomings in the software's design could cause it to perform incorrectly, which would be unsatisfactory to its users.

The bugs and broken authentication security issues:

- **Bugs:** The most frequent cause of software security problems are bugs. Almost all software has many types of defects.
- **Broken Authentication:** Authentication describes the procedure used to confirm that users are who they claim to be. Security problems develop when authentication-related functions behave improperly.

Let us see about Modularization Techniques:

A method of dividing a software system into numerous distinct modules is called modularization.

- Each module works independently.
- Depending on the situation, a module may be utilized multiple times. No need to use recursive writing.

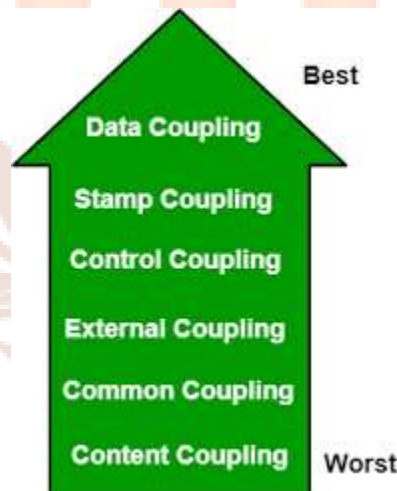
Coupling:

Dependency between the modules should be as low as possible.

The degree of interdependence between the software modules is measured by coupling.

If a software is having a low coupling rate, then it is said to be an excellent software.

Types of coupling:



Types of coupling is shown here:

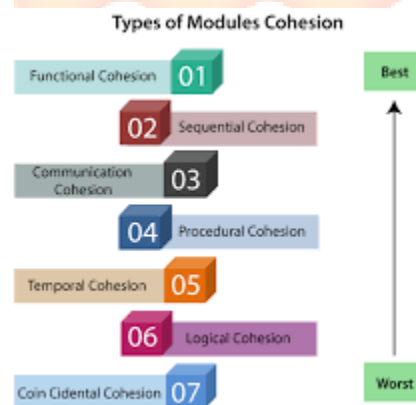
- Data Coupling: Data coupling is the transfer of data from one module to another.
- Stamp coupling: If two modules communicate via composite data items like structure, objects, etc., they are said to be stamp linked.
- Control Coupling: The manner or degree by which the execution of one software component is influenced by another software component.

- External Coupling: When one or more modules share an enforced format, interface, or communication protocol, this is known as external coupling.
- Common Coupling: Two or more functions sharing global data is known as common coupling, also known as global coupling.
- Content coupling: When one module alters or depends on the internal operations of another module, this is known as content coupling.

Next let us learn about Cohesion:

The greater the cohesion, the better is the program design.

In terms of cohesion, a module's degree of intra-dependability means that components that belong together should be kept together.



Functional Cohesion:

All the components inside the module are unconcerned with each other except for the task's execution that is linked to the issue.

Functional cohesiveness serves a singular, determined, and concentrated objective.

The module's components all carry out only the tasks that are required.

Sequential Cohesion:

The elements are involved in such a way that the data that is the outcome of one activity is used as the input data for the subsequent action.

Sequential cohesion offers a good connection and is simple to maintain.

Because the activities are useless even when employed simultaneously, it isn't easy to utilize sequential cohesion.

Communication Cohesion:

The pieces inside the module contribute to the actions that use the same input data or output data.

We cannot focus on all the activities at once; communicational cohesiveness is not adaptable in this way.

The links that produce interactions between the activities make up communication cohesion.

Communicational cohesiveness separates the functionally cohesive ones.

Procedure Cohesion:

If the module's elements are connected by sequence, then the activities are related; otherwise, they are not.

Sequential and procedural cohesion are similar concepts, but procedural cohesion's module pieces do not share any connections.

At the top of the hierarchy, like the main program's module, is procedural cohesion.

Temporal Cohesion:

The actions that are related in time include module-related components.

The startup and termination modules exhibit temporal coherence.

Because the module's components are unrelated to one another, they cannot be reused.

The optimal approach for maintaining temporal coherence is to initiate as late and end as soon as possible.

Logical Cohesion:

The module's components contribute to activities of the same kind or broad category.

Examples of tasks of the same kind or broad category that the module's components contribute to include report modules, display modules, and input-output modules.

Cohesion by Coincidence:

The module's components contribute to the nonsensical interactions between the activities.

The only difference between coincidental cohesion and logical cohesion is the types of actions involved.

Activities like rojak are a blend of coincidental coherence.

Self-Assessment Questions - 4

12. Security is an _____ issue that is often best handled at higher levels within the kernel, under the control of the _____ administrator.
13. In the official kernel distribution, the system calls _____ checks if the invoking process is authorized to load a module into the kernel.

6. DESIGN ISSUES

Let's now discuss the design considerations for creating device drivers that can function across various CPU and bus architectures. The issues that influence the device driver design can broadly be of two types:

1. CPU issues that influence device driver design
2. Bus issues that influence device driver design

6.1 CPU Issues That Influence Device Driver Design

A device driver should be designed to accommodate peripheral devices to operate on more than one CPU architecture. The following issues must be considered to make your drivers portable across CPU architectures:

- i. Control status register (CSR) access
- ii. I/O copy operation
- iii. Direct memory access (DMA) operation
- iv. Memory mapping
- v. 64-bit versus 32-bit
- vi. Memory barriers
- i. ***Control status register (CSR) access:*** Many device drivers based on the UNIX operating system access a device's control status register (CSR) addresses directly through a device register structure. Some CPU architectures do not allow access to the device CSR addresses directly. Write one device driver with the appropriate conditional compilation statements so that the device driver can operate on both types of CPU architectures.
- ii. ***I/O copy operation issues:*** I/O copy operations can differ from one device driver to another due to the differences in CPU architectures. If you use techniques other than the generic kernel interfaces that digital technology provides for performing I/O copy operations, you may need help to write one device driver to operate on more than one CPU architecture or more than one CPU type within the same architecture.

- iii. **Direct memory access (DMA) operation issues:** Direct memory access (DMA) operations can differ from one device driver to another because of the different types of buses used and its DMA hardware support features.
- iv. **Memory mapping issues:** Some CPU architectures do not support an application's use of a memory map section. You should design the device driver suitable for different types of memory map sections deployed to handle applications in different types of OS and for the one that do not use memory map sections, also
- v. **64-bit versus 32-bit:** Consider while declaring data types for 32-bit and 64-bit CPU architectures. Pay careful attention to data types to make your device drivers work on 32-bit and 64-bit systems.
- vi. **Memory barriers issues:** In certain architecture or a model of a computer system, a CPU may perform the memory operations in any order it likes, provided program causality appears to be maintained. Independent memory operations or read and write operations can be performed in random order without any problem, but this can be a problem for CPU-CPU interaction and I/O. So, some way of intervening to instruct the CPU to restrict the order is required. Memory barriers are interventions that impose a perceived partial ordering over the memory operations. Memory barrier (mb) acts as an interface that guarantees the ordering of operations. The mb interface is derived from the MB instruction. The MB instruction must be used in any system to guarantee correctly ordered access to I/O registers or memory that can be accessed via off-board DMA.

6.2 Bus Issues That Influence Device Driver Design

A device driver should be designed to accommodate peripheral devices that can operate on more than one bus architecture. Portability across bus architectures can be achieved when the bus architectures have common features and attributes. Write one device driver for a device that operates on the Industry Standard Architecture (ISA) and Extended Industry Standard Architecture (EISA) buses because their architectures have common features and attributes. It may not be feasible to write one device driver for multiple bus architectures if they exhibit dissimilar features and attributes.

Generally, the following issues must be considered to make the drivers portable across bus architectures. The following issues may apply to many bus architectures in general.

- i. Bus-specific header file issues: Each bus implemented on different architecture has a specific header file. To write portable device drivers across multiple bus architectures, you need to consider how to include bus-specific header files in the include files section of your device driver.
- ii. Bus-specific constant name issues: You must be aware of and define the bus names for the buses the driver will operate in order to develop portable device drivers that work across multiple bus architectures.
- iii. Bus-specific issues related to the `/etc/sysconfigtab` database: The “sysconfigtab” file fragment contains device special file information, bus option data information, and physically contiguous memory usage information for statically and dynamically configured drivers. You have to consider the way the sysconfigtab file fragment gets appended to the `/etc/sysconfigtab` database.
- iv. Bus-specific issues related to implementing the probe interface: The first argument associated with a driver's probe interface is bus specific. The second argument for a driver's probe interface is always a pointer to a controller structure. Controller structure pointer can be used to determine the bus to which the device controller is connected. To write portable device drivers across multiple bus architectures, you need to know the first argument associated with the probe interface for that bus.
- v. Bus-specific issues related to implementing the slave interface: The first argument for a driver's slave interface is always a pointer to a device structure. The second argument associated with a driver's slave interface is bus specific. To write portable device drivers across multiple bus architectures, y The second argument connected to the slave interface for that bus is something you need to be aware of.
- vi. Bus-specific issues for implementing the configure interface: A device driver's configure interface is called indirectly by the `cfgmgr` framework. You should know how this `cfgmgr` framework calls all single binary modules for registration and integration into the kernel.

Self-Assessment Questions - 5

14. Many device drivers based on the UNIX operating system access a device's "_____ "addresses directly through a device register structure.
15. Some CPU architectures do not support an application's use of a memory map section. (True/False)
16. The first argument associated with a driver's probe interface is "_____ " specific.



7. SUMMARY

Let us recapitulate the important concepts discussed in this unit:

- Peripherals connected to a computer need special communication links for interfacing them with the CPU. Communication links resolve the differences between the central computer and each peripheral.
- A device driver is a software interface that allows the OS to provide Input-output (I/O) services to interact with an underlying device.
- Although device drivers are in fact add-on modules, they are an integral part of the system closely integrated with the Input/ Output Control System, which deals with I/O related system calls.
- In UNIX/Linux OS, device drivers are in the form of files and are usually linked onto the object code of the kernel.
- Applications run in user mode, and core operating system components run in kernel mode.
- The role of a device driver can be discussed in terms of mechanism and policy. As a programmer, you are free to design your driver in terms of mechanism and policy. Mechanisms refer to “what capabilities are to be provided,” whereas policy refers to “how those capabilities can be used”.
- UNIX/Linux OS distinguishes and classifies devices that are attached to the computer devices into (i) Character devices (ii) Block devices (iii) Network interfaces.
- Depending upon the device types, the modules for the device driver can be classified as (i) a char module (ii) a block module (iii) a network module.
- Security is a policy issue, and there should not be security holes in the kernel due to a bad module that gets loaded into the kernel.
- The issues that influence the device driver design can be due to (i) CPU issues that influence the device driver design (ii) Bus issues that influence the device driver design

8. GLOSSARY

Buffer. A device or storage area [memory] used to store data temporarily to compensate for differences in rates of data flow, time of occurrence of events, or amounts of data that can be handled by the devices or processes involved in the transfer or use of the data.

Bus: A common pathway along which data and control signals travel between hardware devices within a computer system.

Driver: A program that links a peripheral device or internal function to the operating system and provides for activation of all device functions.

Device Driver: Device Driver is to provide certain mechanisms to allow accessing data to and from the attached devices

Interface: A peripheral device that permits two or more devices to communicate

Peripheral Device: Equipment that is directly connected to a computer. A peripheral device can be used to input data; e.g., keypad, bar code reader, transducer, laboratory test equipment; or to output data; e.g., printer, disk drive, video system, tape drive, valve controller, motor controller

9. TERMINAL QUESTIONS

Short Answer Questions

1. What is a device driver? Explain.
2. Explain the role of device drivers in terms of mechanism and policy.
3. How do UNIX/Linux OS distinguish and classify devices that are attached to the computer devices? Explain.
4. What are the issues related to CPU architecture to make drivers portable? Explain briefly.
5. What are the issues to be considered related to bus architecture to make drivers portable? Explain briefly.

10. ANSWERS

Self-Assessment Questions

1. interfacing
2. driver
3. a) /dev
4. dynamically
5. .sys, dynamic link libraries (DLLs)
6. kernel space, user space
7. a) Windows loads device drivers and calls functions in the driver
8. True
9. b) char driver
10. filesystem
11. True
12. policy, system
13. init_module
14. control status register (CSR)
15. True
16. bus

Short Answer Questions

1. A device driver is a software interface that allows the OS to provide Input-output (I/O) services to interact with an underlying device. (Refer to section 9.2 for more details)
2. The role of device drivers is to provide their capabilities and how those capabilities can be used. Mechanism refers to capabilities, and policy refers to the use of capabilities. (Refer to section 9.3 for more details)
3. UNIX/Linux OS distinguishes and classifies devices that are attached to the computer devices into (i) Character devices (ii) Block devices (iii) Network interfaces. (Refer to section 9.4 for more details)

4. The following are to be considered to make drivers portable across CPU architectures:
(i) Control status register (CSR) access issues. (ii) I/O copy operation. (iii) Direct memory access (DMA) operation etc. (Refer to section 9.6.1 for more details)
5. The issues to be considered related to bus architecture are (i) Bus- specific header file issues (ii) Bus-specific constant name issues, etc. (Refer to section 9.6.2 for more details)

11. SUGGESTED BOOKS

- Corbet, J., Kroah-Hartman, G., & Rubini, A. (2005). Linux Device Drivers. O'Reilly Media.
- *memory-barriers*. (n.d.). Retrieved September 12, 2012, from LINUX KERNEL MEMORY BARRIERS: <http://www.kernel.org/doc/Documentation/memory-barriers.txt>
- Microsoft. (n.d.). *What is a driver?* Retrieved December 18, 2012, from <http://msdn.microsoft.com:> [http://msdn.microsoft.com/en-us/library/windows/hardware/ff554678\(v=vs.85\).aspx](http://msdn.microsoft.com/en-us/library/windows/hardware/ff554678(v=vs.85).aspx)



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4

DCA2203
SYSTEM SOFTWARE

Unit 10

Device Driver – II

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
1.1	Learning Objectives	-	-	
2	PCI Bus Drivers	-	1	4 - 7
3	The Peripheral Component Interconnect (PCI) Interface	1, 2	2, 3, 4	8 - 21
3.1	Peripheral component interconnect (PCI) addressing	-	-	
3.2	Boot time	-	-	
3.3	Configuration registers and initialization	-	-	
3.4	Linux PCI initialization	-	-	
3.5	Old-style PCI probing	-	-	
4	USB Drivers	3, 4	5, 6, 7	22- 30
4.1	USB Device Basics	-	-	
4.2	USB and Sysfs	-	-	
4.3	USB Urbs	-	-	
5	Summary	-	-	31
6	Glossary	-	-	32
7	Terminal Questions	-	-	32
8	Answers	-	-	33
9	Suggested Books and E-References	-	-	34

1. INTRODUCTION

You were introduced to I/O drivers and their functions in the previous unit. The various categories of equipment that are connected to the computer system. Additionally, you read that when designing codes and device drivers, one should be mindful of security and design difficulties. In this section, you will learn about USB and PCI bus drivers as they pertain to the Linux operating system. .

Since all the peripheral devices and components attached to the computer system communicate with each other through the bus, it is very important to get familiar with the technique of how the bus drivers enable these buses communication between devices. PCI devices and their bus types are used for the interconnection of peripheral devices in most recent day's computers. USB-based devices are also used for various works that are connected through computers.

1.1 Learning Objectives

After studying this unit, learners should be able to:

- ❖ *List various PCI buses and PCI device types.*
- ❖ *Describe about the typical PCI system that is used for interconnection of PCI devices.*
- ❖ *Explain how the devices are configured when the computer boots.*
- ❖ *Discuss how the Linux kernel initializes the PCI system.*
- ❖ *Explain the working of USB devices and their configuration.*
- ❖ *Discuss about sysfs and urbs.*

2. PCI BUS DRIVERS

This unit provides an overview of the basic components that the PCI bus drivers are related to it, and also how PCI devices are initialized.

Peripheral Component Interconnect (PCI) is a standard to connect the peripheral components of a system in a structured and controlled way. This standard describes how the system components are electrically connected and how they should behave.

Let us first get introduced to the buses and PCI devices that belong to the PCI family.

PCI buses: The following buses belong to the PCI family:

- PCI: It is a 32-bit bus supporting 33 or 66 MHz.
- Mini PCI: It is found as a smaller slot in laptops.
- Card Bus: It is an external card slot in laptops.
- PIX Extended (PCIX): It is a wider slot than PCI, 64 bits, but can accept a standard PCI card.
- PCI Express (PCIe or PCIE): It is a current generation of PCI. It is Serial instead of parallel.
- PCI Express Mini Card: It replaces Mini PCI in recent laptops.
- Express Card: It replaces Card Bus in recent laptops.

All these bus technologies are compatible and handled by the same kernel drivers. The kernel can know which exact slot and bus variant is used.

PCI device types: These device types are the main types of devices found on the PCI bus. They are:

- Network cards (wired or wireless): A network card is a hardware that allows a computer to connect to a computer network.
- SCSI adapters: It is a device used to connect SCSI devices to a computer bus.
- Bus controllers (e.g., USB, PCMCIA, I2C, FireWire, IDE)

- Graphics and video cards: The Video Card, also known as a graphics card or graphics adapter, or video adapter, is an expansion card that allows the computer to send graphical information to a video display device such as a monitor or projector.
- Sound cards: It is a card that facilitates the input and output of audio signals to and from a computer.

PCI Bus Driver Installation Procedure is seen here:

- The driver/software normally comes together with the hardware. However, if you did not find one, then you may download it from the official website.
- You may require a driver to play hardware that is plugged into the PCI.
- Specifications and user manual must be checked when it is required to connect a device using a PCI bus driver.
- Same slot connection should be done as a 32-bit card in the 32-bit slot and a 64-bit card in the 64-bit slot.
- The number of PCI slots depends on how many have been provided by the manufacturer. Later it can have more.

Next is Registering PCI Bus Driver:

The driver's name must be single between all PCI drivers in the kernel. It is generally set to the same name as the module name of the driver. It displays in sysfs under `/sys/bus/PCI/drivers/` when the driver is in the kernel.

The struct PCI driver structure is the primary methods command that the entire PCI driver should execute to properly register with the kernel.

The structure includes several variables and function callbacks that define the PCI driver to the PCI core.

Const char* field names must be known by the PCI driver.

To create a proper structure, only four fields need to be initialized, as shown here:

- `static struct pci_driver pci_driver =`
`{`
`.name = "pci_skel",`
`.id_table = ids,`
`.probe = probe,`
`.remove = remove,`
`};`

Name, table, probe & remove are initialized as seen.

PCI Addressing for PCI Bus Drivers:

Windows OS automatically installs the driver that allows computers to recognize basic motherboard functions

In the 1086 process family, PCI enables independent memory and I/O ports for both 64-bits and 32-bits.

A particular structure is used rather than a binary address to interact with device drivers. .

By using `inb`, `readb`, and other conventional addressing methods, memory and I/O areas are used.

Self-Assessment Questions - 1

1. “_____” is a standard to connect the peripheral components of a system in a structured and controlled way.
2. PIX Extended (PCIX) is a wider slot than PCI, 64-bit, but can accept a standard PCI card. (True/False)



3. THE PERIPHERAL COMPONENT INTERCONNECT (PCI) INTERFACE

In the early age of computers, Industry Standard Architecture (ISA) bus, originally called the “AT bus,” was used as an expansion bus that accepted plug-in boards for sound, video display, and other peripheral connectivity. The PCI architecture was meant to replace the ISA standard with three main objectives:

- i) provide better performance while transferring data between the computer and its peripherals.
- ii) be platform independent
- iii) simplicity in adding and removing peripherals to the system.

The PCI bus performs better due to a higher clock rate than ISA. PCI is currently used extensively on IA-32, Alpha, PowerPC, SPARC64, IA-64 systems, and some other platforms as well.

PCI devices configure automatically at boot time. It means that the device driver access configuration information in the device to complete initialization during boot time.

The types of PCI Interface working:

- The PCI interface can work in synchronous or asynchronous modes.
- The Bus typically runs synchronously at the external clock frequency of the microprocessor or a submultiple of it.
- The PCI Bus speed is independent of the processor's clock when operating asynchronously.

The synchronous working of the PCI interface has been explained:

- 66-MHz Pentium could synchronously connect to a PCI Bus at half its clock frequency (33 MHz).
- In synchronous mode, the standard PCI clock can be between 20 and 33 MHz

Let us see the asynchronous working of the PCI interface:

- For the best possible performance, the asynchronous mode is used while operating at the maximum PCI Bus frequency.
- By utilizing flow-control signals that indicate when a board is prepared to send or receive data, the PCI standard additionally assists cards unable to operate at the full Bus speed (33 or 66 MHz).

The expansion devices have been shown here.

These are connected to the motherboard with the help of the expansion Bus.

- Expansion Devices are
- Additional Memory
- SCSI Controller
- ISA Interface
- Ethernet Interface
- USB Controller

This slide shows the different versions of PCI Bus drivers commonly used.

PCI Bus Specifications:

- PCI version 1.0 was developed by Intel in 1992
- PCI revision 2.0 was released in 1993, having 32-bit, 33MHz bus
- PCI revision 2.1 was released in 1995, having 32-bit, 33MHz / 64-bit, 66MHz
- PCI revision 2.2 was released in 1998, having minor enhancements
- PCI revision 2.3 was released in 2002 and removed 5v-only cards
- PCI revision 3.0 was released in 2004 and removed 5-volt interfaces altogether

Let us see the PCI Bandwidth Based on Clock Rates:

A Bus number, a device number, and a function number identify each PCI peripheral. .

132 MB/s utilizing a 64-bit data path at a 33 MHz clock rate

264 MB/s peak utilizing a 32-bit data path at 66 MHz

264 MB/s peak utilizing a 64-bit data path at 66 MHz

532 MB/s peak utilizing a 32-bit data path at 133 MHz

1064 MB/s peak utilizing a 64-bit data path at 133 MHz

The advantages of PCI Bus Drivers have been explained here:

- A maximum of five components can be connected to the PCI, and all of them can be further replaced by the built-in devices on the motherboard.
- On related machines, you'll find many PCI Buses.
- Exchange speeds will improve from 33 MHz to 133 MHz with a 1 GB/s transmission rate thanks to PCI transport.
- The PCI can support devices with a maximum voltage of 5 volts, and the pins used can switch more than one flag by one stick.

3.1 Peripheral Component Interconnect (PCI) Addressing

Before we proceed to read about PCI addressing, let us get introduced to PCI address spaces and Bridges.

PCI address spaces

The CPU and the PCI devices access the memory shared between them. Memory is used by device drivers to control the PCI devices and to pass information between them. Generally, that the shared memory contains control and status registers for the device. These registers control the devices and read their status.

Peripheral devices have their own memory spaces. The CPU can access these memory spaces. However, DMA (Direct Memory Access) channels control the way devices access the system's memory.

ISA devices have access to two address spaces: ISA I/O (Input/Output) and ISA memory, whereas PCI has access to three address spaces: PCI I/O, PCI Memory, and PCI Configuration space. CPU accesses these address spaces with the PCI I/O and PCI Memory address spaces used by the device drivers and the PCI Configuration space used by the PCI initialization code within the Linux kernel.

Some of the processors, like the Alpha AXP processors access only the system address space and not address spaces. It uses support chipsets to access other address spaces, such as PCI Configuration. A sparse address mapping scheme is used to steal part of the large virtual address space to map it to the PCI address spaces.

Please note that each PCI peripheral can be identified by:

- i) *a bus number,*
- ii) *a device number and*
- iii) *a function number.*

The PCI specification allows a single system to host up to 256 buses, but 256 buses are not sufficient for many large systems. However, Linux is supported through PCI *domains*. Each PCI domain can host up to 256 buses. Each bus hosts up to 32 devices, and each can be a multifunction board (such as an audio device with an accompanying CD-ROM drive) with a maximum of eight functions. Therefore, each function can be identified at the hardware level by a 16-bit address or key. However, device drivers written

for Linux do not need to deal with those binary addresses because they use a specific data structure `pci_dev`, to act on the devices.

Modern days computers feature at least two PCI buses. Hence, you can plug more than one bus into a single system using bridges and use special-purpose PCI peripherals to join two buses.

The overall layout of a PCI system is seen as a tree where each bus is connected to an upper-layer bus up to bus 0 at the root of the tree. The Card Bus PC-card system is also connected to the PCI system via bridges.

PCI I/O and PCI memory addresses: These two address spaces are used by the devices to communicate with their device drivers that run in the Linux kernel on the CPU. Note that only the PCI configuration code reads and writes PCI configuration addresses. The Linux device drivers only read and write PCI I/O and PCI memory addresses.

PCI-ISA bridges: These bridges support legacy ISA devices by translating PCI I/O, and PCI Memory space accesses into ISA I/O and ISA Memory accesses.

PCI-PCI bridges: PCI-PCI bridges are special PCI devices that connect the PCI buses of the system. Simple systems have a single PCI bus, but there is an electrical limit on the number of PCI devices that a single PCI bus can support. PCI-PCI bridges add more PCI buses and allow the system to support many more PCI devices.

PCI-PCI bridges (PCI I/O and PCI memory windows): PCI-PCI bridges only pass a subset of PCI I/O, and PCI memory reads and writes requests downstream.

Figure 10.1 shows a typical PCI system. The 16-bit hardware addresses associated with PCI peripherals are depicted in the struct *pci_dev* object. However, one can visualize lists of devices using the command *lspci* (part of the *pciutils* package, available with most distributions) and find the layout of information in */proc/pci* and */proc/bus/pci*.

This addressing scheme is also shown by the sysfs representation of PCI devices, with the addition of the PCI domain information. Sysfs is a feature of the Linux 2.6 kernel that allows kernel code to export information to user processes via an in-memory file system. When the hardware address is displayed, it can be shown as two values (an 8-bit bus number and an 8-bit device and function number), as three values (bus, device, and function), or as four values (domain, bus, device, and function); all the values are usually displayed in hexadecimal.

For example, `/proc/bus/pci/devices` use a single 16-bit field (to ease parsing and sorting), while `/proc/bus/pci/bus number` splits the address into three fields. The following shows how those addresses appear, showing only the beginning of the output lines:

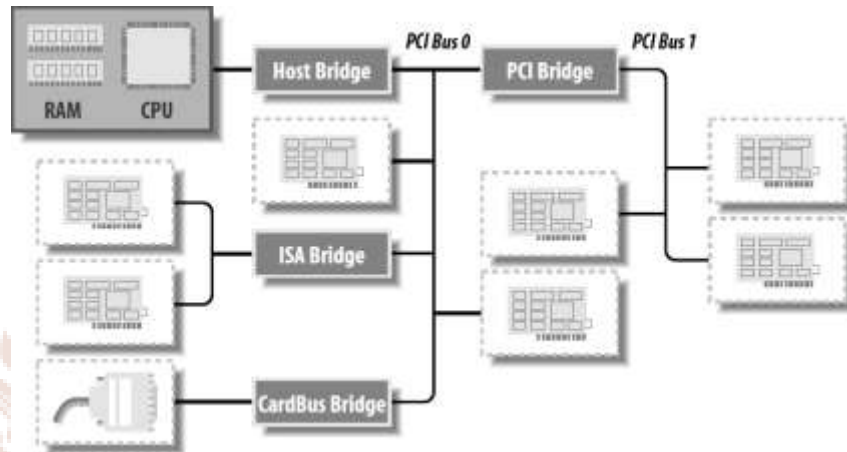


Figure 10.1: Layout of a Typical PCI System

All three lists of devices (bus, device, and function) are sorted in the same order since `lspci` uses the `/proc` files as its source of information. Taking the VGA video controller as an example, `0x00a0` means `0000:00:14.0` when split into a domain (16 bits), bus (8 bits), device (5 bits), and function (3 bits).

The hardware circuitry of each peripheral board answers queries about three address spaces:

- i) memory locations,
- ii) I/O ports, and
- iii) configuration registers.

All the devices share the first two address spaces on the same PCI bus. PCI peripherals also can work when interrupted (hardware or software). Every PCI slot has four interrupt pins, and each one of them can be used by the device functions. PCI specifies the interrupt lines to be shared; hence, even a processor with a limited number of IRQ lines, such as the x86, can host many PCI interface boards (each with four interrupt pins).

The I/O space in a PCI bus uses a 32-bit address bus (leading to 4 GB of I/O ports), while the memory space can be accessed with either 32-bit or 64-bit addresses. You can find 64-bit addresses on present-day platforms.

The PCI configuration space consists of 256 bytes for each device function (except for PCI Express devices, which have 4 KB of configuration space for each function), and the layout of the configuration registers is standardized. Four bytes of the configuration space hold a unique function ID, so the driver can identify its device by looking for the specific ID for that peripheral.

The main innovation of the PCI interface standard over ISA is the configuration address space. Therefore, in addition to the usual driver code, a PCI driver needs the ability to access the configuration space.

Self-Assessment Questions - 2

3. In Linux, each PCI domain can host up to " _____ "buses.
4. The I/O space in a PCI bus uses a " _____ "-bit address bus, while the memory space can be accessed with either " _____ "-bit " _____ bit addresses.

3.2 Boot Time

Now let's read about PCI's operation. Keep in mind that all devices are configured at system startup time.

When power is applied to a PCI device, the hardware remains inactive. That is, the device responds only to configuration transactions. When power is provided, the device has no memory, and no I/O ports are mapped in the computer's address space. All the device-specific feature, such as interrupt reporting, is also disabled.

Every PCI motherboard is equipped with PCI-aware firmware, called BIOS, NVRAM, or PROM, depending on the platform. This firmware performs configuration transactions with every PCI peripheral to allocate a safe place for each address region it offers. Memory and

I/O areas of the device are already mapped into the processor's address space by the time a device driver accesses it.

In Linux, the user can look at the PCI device list and the devices' configuration registers by reading `/proc/bus/pci/devices` and

`/proc/bus/pci/*/*`.

The former is a text file with (hexadecimal) device information, and the latter are binary files that report a snapshot of the configuration registers of each device, one file per device. The individual PCI device directories in the sysfs tree can be found in `/sys/bus/pci/devices`.

3.3 Configuration Registers And Initialization

This section, deals with the configuration registers that PCI devices contains. All PCI devices feature at least a 256-byte address space. The first 64 bytes are standardized, while the rest are device dependent. The layout of the device-independent configuration space is shown in Figure 10.2.

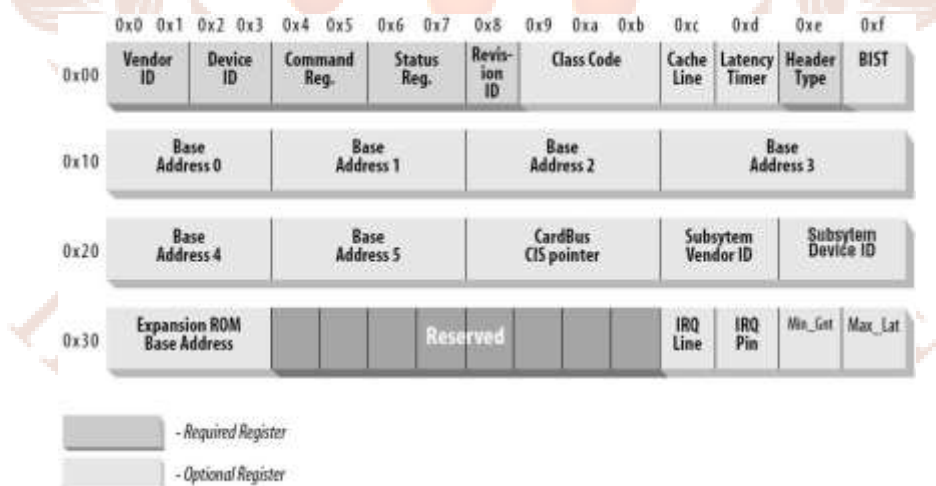


Figure 10.2: The Standardized PCI Configuration Registers

There are two registers. Required registers and Optional registers. Depending upon the value in the Header Type field, some of the PCI configuration registers are optional in the PCI configuration registers whereas some are required. Every PCI device contains some important data in the required registers, whereas the data in the optional registers depend

on the actual capabilities of the peripheral. The optional fields are not used unless the contents of the required fields indicate that they are valid. You can also see in the figure that a few registers are reserved for further use.

Every PCI device in the system has a configuration data structure in the PCI configuration address space. The PCI Configuration header helps the system to identify and control the device.

Typically, systems are designed so that every PCI slot has its PCI Configuration Header in an offset related to its slot on the board. A system-specific hardware mechanism is defined so that the PCI configuration code can attempt to examine all possible PCI Configurations.

Let us now read the description of each field in the PCI configuration registers. Note that the technical documentation released with each device usually describes the supported registers.

Vendor ID: This 16-bit register identifies the originator of the PCI device. E.g., Digital's PCI Vendor Identification is 0x1011, and Intel's is 0x8086.

Device ID: This 16-bit register is a unique number to identify the device. This ID is usually paired with the vendor ID to make a unique 32-bit identifier for a hardware device.

Command Reg: This field is used to control the device to generate and respond to PCI cycles.

Status Reg: This field gives the status of the device with the meaning of the bits of this field set by the standard.

Class Code: This identifies the type of device that this is. There are standard classes for every sort of device, video, SCSI, and so on.

Cache Line: It specifies the system cache line size in word (2-byte) units.

Latency Timer: It specifies the latency timer in units of PCI bus clocks.

Header Type: It identifies the layout of the rest of the header beginning at byte 0x10 of the header and whether the equipment has numerous functions is made clear. Where a value of 0x00 specifies a general device, a value of 0x01 specifies a PCI-to-PCI bridge, and a

value of 0x02 specifies a Card Bus bridge. If bit 7 of this register is set, the device has multiple functions; otherwise, it is a single function device.

BIST: Represents that status and allows control of a devices BIST (built-in self-test).

Base Address Registers: These registers determine and allocate the type, amount, and location of PCI I/O and PCI memory space that the device can use.

Card Bus CIS Pointer: It Points to the Card Information Structure (CSI) and is used by devices that share silicon between Card Bus and PCI.

Subsystem Vendor ID, Subsystem Device ID: The vendor sometimes sets these fields to further differentiate similar devices.

Expansion ROM base address: It consists of the base address of the expansion ROM.

Max_Lat: Max Latency is a read-only register that specifies how often the device needs access to the PCI bus.

Min_Gnt: Min Grant is a read-only register that specifies the burst period length that the device needs.

Interrupt Pin: The Interrupt Pin field describes which pins the PCI device uses. Generally, it is hardwired for a particular device.

Interrupt Line: The Interrupt Line field of the device's PCI Configuration header is used to pass an interrupt handle between the PCI initialization code, the device's driver, and the interrupt handling subsystem. It allows the interrupt handler to correctly route an interrupt from the PCI device to the correct device driver's interrupt handling code within the Linux operating system.

For the x86 architecture, this register corresponds to the PIC IRQ numbers 0-15 (and not I/O APIC IRQ numbers), and a value of 0xFF defines no connection.

Self-Assessment Questions - 3

5. When power is provided, the device has no memory, and no I/O ports are mapped in the computer's _____ space.
6. Typically, systems are designed so that every PCI slot has its PCI Configuration _____ in an offset that is related to its _____ on the board.
7. Which of the following field describes the pins of PCI device use?
 - (a) The Interrupt Pin field
 - (b) Interrupt Line field
 - (c) Class Code field
 - (d) Header Type field

3.4 Linux Pci Initialization

The PCI initialization code in Linux consists of three logical parts:

- i) **PCI device driver:** This driver searches the PCI system starting at Bus 0 and locates all PCI devices and bridges. It builds a linked list of data structures and describes the system's topology.
- ii) **PCI BIOS:** It is software that provides the services described in bib-pci-bios-specification. Even though Alpha AXP does not have BIOS services, there is equivalent code in the Linux kernel providing the same functions.

The PCI BIOS functions are a series of standard routines and are common across all platforms. For example, they are the same for both Intel and Alpha AXP based systems. They allow the CPU controlled access to all the PCI address spaces.

iii) PCI fixup

System specific fixup code ties up the system specific loose ends of PCI initialization.

Linux kernel initializes the PCI system and builds data structures reflecting the real PCI topology of the system. A `pci_dev` data structure describes each PCI device (including the PCI-PCI Bridges). Each PCI bus is described by a ***pci_bus*** data structure. The result is a tree

structure of PCI buses, each of which has several child PCI devices attached to it. As a PCI bus can only be reached using a PCI-PCI Bridge (except the primary PCI bus, bus 0), each `pci_bus` contains a pointer to the PCI device (the PCI- PCI Bridge) that it is accessed through. That PCI device is a child of the PCI Bus's parent PCI bus.

A pointer is there to all the PCI devices in the system called `pci_devices`. All PCI devices in the system have their `pci_dev` data structures. This helps the Linux kernel discover all the PCI devices in the system.

Registering a PCI driver

The main structure that all PCI drivers must create to be registered with the kernel properly is the struct ***pci_driver*** structure. This structure consists of several function callbacks and variables that describe the PCI driver to the PCI core.

The PCI device driver is not a device driver at all but a function of the operating system called system initialization time. The PCI initialization code scans every PCI bus in the system that needs to connect to all PCI devices (including PCI-PCI bridge devices).

There are various fields in this structure that a PCI driver needs to be aware of. Those fields provide:

- the name of the driver is unique among all PCI drivers in the kernel and is normally set to the same name as the module name of the driver.
- Pointer to the probe function in the PCI driver etc.
- Pointer to the function that the PCI core calls when the struct `pci_dev` is being removed from the system or when the PCI driver is being unloaded from the kernel
- Pointer to the function that the PCI core calls when the struct `pci_dev`
 - is being suspended
- Pointer to the function that the PCI core calls when the struct `pci_dev`
 - is being resumed.

To register the struct ***pci_driver*** with the PCI core, a call to ***pci_register_driver*** is made with a pointer to the struct ***pci_driver***. Note that the `pci_register_driver` function either returns a

negative error number or 0 if everything was registered successfully. It does not return the number of devices bound to the driver or an error number if no devices were bound to the driver.

When the PCI driver is to be unloaded, the struct ***pci_driver*** must be unregistered from the kernel. This is done with a call to `pci_unregister_driver`. When this call happens, any PCI devices currently bound to this driver are removed, and the remove function for this PCI driver is called before the `pci_unregister_driver` function returns.

3.5 Old-Style PCI Probing

In older kernel versions, the function `pci_register_driver`, was not always used by PCI drivers. Instead, the list of PCI devices in the system was dealt with manually, or a function would be called that searched for a specific PCI device. The ability to deal with the list of PCI devices in the system within a driver has been removed from the 2.6 kernel to prevent drivers from crashing the kernel, which occurs while modifying the PCI device lists, and a device is being removed simultaneously.

In old-style PCI probing, the ability to find a specific PCI device is provided by the following available functions:

- function to scan the list of PCI devices currently in the system.
- function to allow the subsystem vendor and subsystem device IDs to be specified when looking for the device.
- function to search the list of PCI devices in the system on the specified struct ***pci_bus*** for the specified device and function number of the PCI device.

Note that these functions cannot be called from an interrupt context. If they are, a warning is printed out to the system log.

Self-Assessment Questions - 4

8. The PCI BIOS functions are a series of standard “common _____” across all platforms.
9. To register the structpci_driver with the PCI core, a call to “_____” is made with a pointer to the structpci_driver.



4. USB DRIVERS

The full form of USB is Universal Serial Bus. The USB provides a connection between a host computer and several peripheral devices. Nowadays, USB can support almost every type of device that can be connected to a PC. The USB transfer speed can be categorized as:

- Low Speed: data transfer is up to 1.5 Mbps by the USB 1.0
- Full Speed: data transfer is up to 12 Mbps by the USB 1.1
- Hi-Speed: data transfer is up to 480 Mbps by USB 2.0

Topologically, a USB subsystem is a tree built out of several point-to-point links. The links are four-wire cables (ground, power, and two signal wires) that connect a device and a hub, just like twisted-pair Ethernet. The USB host controller asks every USB device if it has any data to send. It is because of this topology; a USB device can send data only when asked by the host controller. This configuration allows for a very easy plug-and-play type of system, whereby the host computer can automatically configure devices.

At the technological level, the USB is a single-master implementation in which the host computer polls the various peripheral devices. The USB bus has the ability for a device to request a fixed bandwidth for its data transfers to reliably support video and audio I/O. USB acts only as a communication channel between the device and the host. It does not require specific meaning or structure to the data it delivers.

The USB protocol specifications define a set of standards that any device of a specific type can follow. If a device follows that standard, then a special driver for that device is unnecessary. The different types of standards are called classes, consisting of devices like storage devices, keyboards, mice, joysticks, network devices, and modems. Those devices that do not fit into these classes require a special vendor-specific driver to be written for that specific device. Video and USB-to-serial devices are good where there is no defined standard, and a driver is needed for every device from different manufacturers.

Since a USB is built like a tree out of several point-to-point links and it has an inherent hot-plug capability of the design, a USB is handy and provides a low-cost mechanism to connect

(and disconnect) several devices to the computer without the need to shut the system down, open the cover, and swear over screws and wires.

The Linux kernel supports two main types of USB drivers:

- i) Drivers on a host system and
- ii) Drivers on a device

The USB drivers for a host system control the USB devices plugged into it from the host's point of view (a common USB host is a desktop computer.) The USB drivers in a device control how that single device looks to the host computer as a USB device. As the term "USB device drivers" is very confusing, USB developers have created the term "USB gadget drivers" to describe the drivers that control a USB device that connects to a computer (remember that Linux also runs in those tiny, embedded devices, too.) USB host drivers have a different driver for each USB control hardware and is usually available in the Board Support Package. These are architecture and platform dependent.

In this section, you will read how the USB system runs on a desktop computer, i.e., how drivers work with the devices.

In Figure 10.3 of the USB driver, you can see that the USB drivers live between the kernel subsystems (block, net, char, etc.) and the USB hardware controllers. The USB core provides an interface for USB drivers to use to access and control the USB hardware without worrying about the different types of USB hardware controllers on the system.

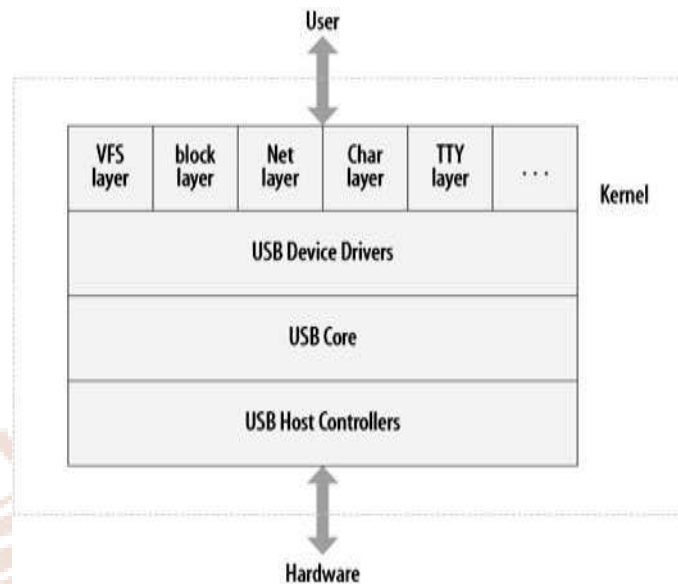


Figure 10.3: USB Driver Overview

Self-Assessment Questions - 5

10. The USB protocol specifications define a set of “_____” that any device of a specific type can follow.
11. The Linux kernel supports two main types of USB drivers: (i) Drivers on a “_____” and (ii) Drivers on a “_____”.

4.1 USB Device Basics

A USB device is described as complex in the official USB documentation. However, the Linux kernel provides a subsystem called the USB core to handle most of the complexity. In this section, you will read about the basics of a driver and the USB core. Figure 10.4 shows how USB devices consist of configurations, interfaces, and endpoints and how USB drivers bind to interfaces, not the entire USB device.

Endpoints

It is through the endpoint that the most basic form of USB communication happens. Endpoints are sources or sinks of data. Endpoints occur at the end of the communications channel at the USB function, as the bus is host-centric. A USB endpoint carries data in one

direction only. If data is carried from the host computer to the device, it is called an OUT endpoint, whereas if it is from the device to the host computer, it is called an IN endpoint. Endpoints can be thought of as unidirectional pipes.

A USB endpoint can be one of four different types that describe how the data is transmitted:

- i) **Control:** Control endpoints allow access to different parts of the USB device. They configure the device, retrieve information about it, send commands to it, or retrieve status reports about it. These endpoints are usually small in size. Every USB device consists of a control endpoint called “endpoint 0” that is used by the USB core to configure the device when it is inserted to computer. The USB protocol guarantees reserved bandwidth to access through to the device.
- ii) **Interrupt:** Interrupt endpoints transfer small data at a fixed rate every time the USB host asks the device for data. As an example, these endpoints are used for USB keyboards and mice. It is also used to send data to USB devices to control the device. The USB protocol guarantees enough reserved bandwidth to access the device.
- iii) **Bulk:** Bulk endpoints transfer large amounts of data. These endpoints are usually much larger (they can hold more characters at once) than interrupt endpoints. These endpoints are commonly used for devices that transfer data without data loss. The USB protocol does not guarantee always make it through in a specific amount of time. If the bandwidth of the bus is not enough to send the whole BULK packet, data is split up across multiple transfers to or from the device. For example, printers, storage, and network devices use this endpoint.
- iv) **Isochronous:** Isochronous endpoints also transfer large amounts of data, but the data is only sometimes guaranteed to make it through. Those devices that can handle data loss and rely more on keeping a constant stream of data flow use these endpoints. Real-time data collections, such as audio and video devices, use these endpoints.

Control and bulk endpoints are used for asynchronous data transfers whenever the driver decides to use them. Interrupt, and isochronous endpoints are periodic. This means that these endpoints are set up to transfer data at fixed times continuously, which causes their bandwidth to be reserved by the USB core.

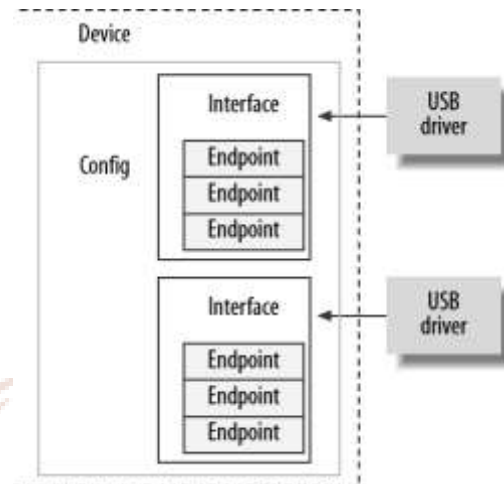


Figure 10.4: USB Device Overview

USB endpoints are described in the kernel with the structure struct *usb_host_endpoint*. This structure contains the real endpoint information in another structure called struct *usb_endpoint_descriptor*. The latter structure contains all the USB-specific data in the exact format specified by the device. The fields of this structure directly correspond to the field names in the USB specification. The fields of this structure that drivers care about are: **bEndpoint Address**, **bmAttributes**, **wMax PacketSize**, **bInterval** which provides specification about the USB devices.

Interfaces

USB endpoints are bundled up into interfaces. USB interfaces handle only one type of a USB logical connection, such as a mouse, a keyboard, or an audio stream. Some USB devices have multiple interfaces. E.g., a USB speaker might consist of two interfaces: (i) a USB keyboard for the buttons and (ii) a USB audio stream. A USB interface only represents basic functionality, so each USB driver controls an interface. For example, for the speaker, Linux needs two different drivers for one hardware device.

USB interfaces may have alternate settings for different parameters of the interface. The initial state of an interface is in the first set and is numbered zero. Alternate settings can be used to control individual endpoints for various purposes. E.g., USB bandwidth for the device can be changed. Every device with an isochronous endpoint uses alternate settings for the same interface.

USB interfaces are described in the kernel with the struct ***usb_interface*** structure. The USB core passes this structure to USB drivers, and the USB driver is responsible for controlling any functions.

Configurations

USB interfaces are also bundled up into configurations. A USB device can have multiple configurations to change the various states of the device. A single configuration can be enabled only at one point in time. Linux does not handle multiple configurations for USB devices very well, but it is a rare case.

Linux describes USB configurations with the structure struct ***usb_host_config*** and entire USB devices with the structure ***structusb_device***. You can find descriptions of them in the file `include/linux/usb.h` in the kernel source tree.

A USB device driver converts data from a given struct ***usb_interface*** structure into a struct ***usb_device*** structure that the USB core needs for various function calls. To do this, the function ***interface_to_usbdev*** is provided.

To summarize, USB devices are made up of lots of different logical units. The relationships among these units can be simply described as follows:

- Devices usually have one or more configurations.
- Configurations often have one or more interfaces.
- Interfaces usually have one or more settings.
- Interfaces have zero or more endpoints.

Self-Assessment Questions - 6

12. What type of endpoint in a USB can carry data from the host computer to the device?
- (a) in point
 - (b) out point
 - (c) mid-point
 - (d) common point
13. Isochronous endpoints also transfer large amounts of data and are always guaranteed to make it through. (True/False)
14. A USB interface represents basic functionality only, and hence each USB driver controls an

Advantages of USB Drivers

- Since USB connectors are fairly straightforward, many computing devices, including desktop and laptop computers, have them. There are USB hubs available to add more ports (if any).
- Installing a USB device is simple and only needs the OS (Operating System).
- When compared to RS232 and other parallel interfaces, USB ports are much smaller.
- External power is unnecessary.
- USB can transfer data at various distances and speeds—from 1.5 Mbps to 5 Gbps.
- In the event of faults, USB protocol alerts the transmitter to retransmit the data. To ensure error-free transmission, the USB driver is used.

4.2 USB And Sysfs

Sysfs is a feature of the Linux 2.6 kernel that allows kernel code to export information to user processes via an in-memory file system. Sysfs is used to set up and export data about devices and drivers from the kernel device model to user space.

The physical USB device is represented by a struct ***usb_device*** and the individual USB interfaces are represented by a struct ***usb_interface***. These are shown in sysfs as individual devices, as both structures contain a ***struct_device*** structure.

The USB sysfs device naming scheme is given as:

root_hub-hub_port:config.interface

E.g., The struct ***usb_device*** for the simple USB mouse, the sysfs device naming is:

/sys/devices/pci0000:00/0000:00:09.0/usb2/2-1/2-1:1.0

Sysfs does not expose all the different parts of a USB device, as it stops at the interface level. Any alternate configurations that the device may contain and the endpoints associated with the interfaces are not shown. The system's `/proc/bus/usb/` directory contains the information about it in the usbfs file system. The identical data displayed in sysfs is also shown in the file `/proc/bus/usb/devices`. It also shows alternate configuration and endpoint information for all USB devices in the system.

4.3 USB Urbs

The USB code in the Linux kernel communicates with all USB devices using urb (USB request block). A new driver is found by message passing. This message itself is called USB Request Block, or URB for short. This request block is described with the struct urb structure and is found in the `include/linux/usb.h` file.

A urb is used to send or receive data to or from a specific USB endpoint on a USB device in an asynchronous manner.

A USB device driver may allocate many urbs for a single endpoint or may reuse a single urb for many different endpoints, depending on the need of the driver. Every endpoint in a device can handle a queue of urbs, so multiple urbs can be sent to the same endpoint before the queue is empty. The typical lifecycle of an urb is as follows:

- Created by a USB device driver.
- Assigned to a specific endpoint of a specific USB device.

- Submitted to the USB core by the USB device driver.
- Submitted to the specific USB host controller driver for the specified device by the USB core.
- Processed by the USB host controller driver that transfers a USB to the device.
- When the urb is completed, the USB host controller driver notifies the USB device driver.

Urbs can also be canceled at any time by the driver that submitted the urb, or by the USB core if the device is removed from the system. urbs are dynamically created and contain an internal reference count that enables them to be automatically freed when the last user of the urb releases it.

Self-Assessment Questions - 7

15. Sysfs is a feature of the Linux 2.6 kernel that exports information about devices and drivers from the “_____” device model to user space.
16. The basic idea of the new driver is message passing, the message itself is called “_____”Request Block, or for short.

5. SUMMARY

Let us recapitulate the important concepts discussed in this unit:

- ISA devices have access to two address spaces: ISA I/O (Input/Output) and ISA memory, whereas PCI has access to three address spaces: PCI I/O, PCI Memory, and PCI Configuration space.
- All PCI devices feature at least a 256-byte address space. The first 64 bytes are standardized, while the rest are device dependent.
- As the Linux kernel initializes the PCI system, it builds data structures mirroring the real PCI topology of the system.
- The main structure that all PCI drivers must create to be registered with the kernel properly is the struct ***pci_driver*** structure.
- In older kernel versions, the list of PCI devices in the system was dealt with manually, or a function would be called that searched for a specific PCI device.
- Topologically, a USB subsystem is a tree built out of several point-to-point links.
- The USB protocol specifications define standards that any device of a specific type can follow.
- USB endpoints are bundled up into interfaces and configurations.
- Sysfs is a feature of the Linux 2.6 kernel that exports information about devices and drivers from the kernel device model to user space and is also used for configuration.
- The basic idea of the new driver is message passing; the message itself is called USB Request Block, or URB for short.

6. GLOSSARY

Interface requirement: A requirement that specifies an external item with which a system or system component must interact or sets forth constraints on formats, timing, or other factors caused by such an interaction

PCI: "Peripheral Component Interconnect "is a hardware bus used for adding internal components to a desktop computer

PCI-eXtended (PCI-X): An expansion bus and expansion card standard that enhances the 32-bit PCI Local Bus for higher bandwidth demanded by servers.

PCI Express (PCIe): An expansion bus standard designed to replace the older PCI, PCI-X, and AGP bus standards.

Universal Serial Bus (USB):

A specification to establish communication between devices and a host controller.

7. TERMINAL QUESTIONS

Short Answer Questions

1. List the various types of PCI buses and PCI device types.
2. Draw a figure of the Standardized PCI Configuration Registers and explain it in brief.
3. How is the PCI driver registered? Explain.
4. What is the endpoint in USB? Explain briefly about each of the endpoints.
5. Write short notes on (a) Sysfs (b) urbs

8. ANSWERS

Self-Assessment Questions

1. Peripheral Component Interconnect (PCI)
2. True
3. 256
4. 32,32,64
5. address
6. Header, slot
7. (a) The Interrupt Pin field
8. routines
9. pci_register_driver
10. standards
11. host system, device
12. (b) out point
13. False
14. interface
15. kernel
16. USB, URB

Short Answer Questions

1. The numerous types that make up PCI buses include: PCI, Mini PCI, Card Bus, etc. The main types of PCI devices are Network cards, SCSI adapters, etc. (Refer to section 10.2 for more details.)
2. Every PCI device in the system has a configuration data structure in the PCI configuration address space. (Refer to section 10.3.3 for more details)
3. The main structure that all PCI drivers must create to be registered with the kernel properly is the struct **pci_driver** structure. (Refer to section 10.3.4 for more details)
4. Endpoints are sources or sinks of data. (Refer to section 10.4.1 for more details)
5. Sysfs is a feature of the Linux 2.6 kernel. A urb is a message to send or receive data to or from a USB endpoint. (Refer to section 10.4.2 and 10.4.3 for more details)

8. SUGGESTED BOOKS

- Corbet J., Kroah-Hartman G., & Rubini A. (2005). Linux Device Drivers (3rd Edition ed.). California: O'Reilly Media.
- The Linux Tutorial. (N.D.). Retrieved 09 18, 2012, from Linux-Tutorial:<http://www.linuxtutorial.info/modules.php?name=MContent&pageid=303>





BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4

DCA2203
SYSTEM SOFTWARE

Unit 11

Universal Plug and Play

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4
1.1	Learning Objectives	-	-	
2	Universal Plug and Play: UPnP Introduction	1, 2	1	5 - 14
2.1	UPnP Architecture	-	-	
2.2	Basic principles of UPnP	-	-	
2.3	Uses and applications	-	-	
2.4	Benefits of UPnP technology	-	-	
2.5	Advantages/Disadvantages of UPnP	-	-	
2.6	Steps in UPnP device addressing	-	-	
3	UPnP Protocol Stack	3	2	15 - 18
4	Addressing	-	3	19 - 21
4.1	Dynamic Host Configuration Protocol (DHCP)	-	-	
4.2	Auto-IP	-	-	
4.3	Address selection	-	-	
4.4	Resolving address	-	-	
5	Ad-Hoc networks	-	-	22
6	Discovery	-	-	22
7	Simple Service Discovery Protocol (SSDP)	-	-	23
8	Service Identification	-	-	24 - 27
9	Communication Model	-	-	28
10	Discovery Requests and Presence Announcements	-	-	29

11	Network Transport		-	4	30 - 32
12	Description		-	5	33 - 34
	12.1	UPnP's description phase	-	-	
	12.2	Description document standards	-	-	
13	Control		-	6	35 - 37
14	Eventing		-	7	38 - 41
15	Presentation		-	8	42 - 43
16	Summary		-	-	44
17	Glossary		-	-	45
18	Terminal Questions		-	-	45
19	Answers		-	-	46 - 47
20	Suggested Books and E-References		-	-	47

1. INTRODUCTION

We learned about PCI drivers and USB drivers in the prior unit. You have a thorough understanding of how these gadgets function when they are linked to computers. This unit deals with UPnP technology that offers pervasive peer-to-peer network connectivity with computing devices of all forms, intelligent appliances, and wireless devices.

Everyone wants to be able to plug in their devices and use them everywhere. Many people may need to learn how to solve the intricacies of installing the device drivers. Majority wants Plug and Play types of devices without installing the device's drivers. This unit will introduce you to the technology and protocol working for running the UPnP-based devices.

1.1 Learning Objectives:

After studying this unit, the learners should be able to:

- ❖ *Explain what is UPnP*
- ❖ *Description of the working of UPnP-based devices*
- ❖ *Discuss the various UPnP protocols stack*
- ❖ *Describe how UPnP-based devices are addressed and discovered*
- ❖ *Explain about UPnP based devices*
- ❖ *Explain how the UPnP-based devices control, perform eventing and presentation*

2. UNIVERSAL PLUG AND PLAY: UPNP INTRODUCTION

UPnP stands for Universal Plug and Play. This extends an existing technology called PnP (Plug and Play). The organization behind UPnP is known as the UPnP Forum, formed on October 18, 1999, to develop the Device Control Protocol (DCP). It is a group of companies and individuals who wish to create a standard way to allow devices to configure themselves. Over 200 companies signed on at the beginning, including Intel, IBM, Sun, HP, AOL Time Warner, TI (and many others) to develop and deploy this concept related to the DCP. The main contributor to UPnP is Microsoft, which supplied the architecture that the specifications are built around. This evolved into UPnP. The Universal Plug and Play Forum define UPnP Device and Service Descriptions.

With UPnP technology, users can add devices to the home or office network without installing drivers or configuring them before using them. It is much easier to add peripherals like webcams and printers to a system that supports plug-and-play. Devices and their drivers can be set up automatically, allowing the device to relate to minimum problems. UPnP extends this functionality to the entire network, enabling other UPnP devices to be discovered and controlled. It can also make devices hot-swappable, connecting and disconnected without restarting the computer system.

2.1 UPnP architecture

The UPnP architecture is designed to offer pervasive peer-to-peer network connectivity of networked devices such as Personal Computers (PCs) of all form factors, intelligent appliances, and wireless devices. The UPnP architecture is a distributed, open networking architecture that leverages existing standards such as TCP/IP, HTTP, and XML instead of inventing new underlying mechanisms to enable seamless proximity networking, in addition, to control and data transfer among networked devices in the home, office, and everywhere in between. The architecture consists of a set of standardized protocols that each UPnP technology-enabled device implements to provide for discovery, control, and data transfer between UPnP devices (See Figure11.1).

The UPnP architecture provides:

- **Device connectivity:** The UPnP architecture defines the protocols for interacting with other devices. UPnP devices can join and leave the network transparently, advertise their services, discover other devices and services, send events, and control other devices.
- **Ad-Hoc networking:** UPnP devices can come together to form a network dynamically without the need for dedicated networking infrastructure services. These instantaneous ad hoc networks allow for device communication without requiring manual configuration.
- **Zero-configuration networks:** The UPnP architecture supports zero-configuration networking where the user is not required to configure devices before they are used on the network.

Standards-based architecture: The core of the UPnP architecture is an array of existing and proposed open standards.

Standard Internet Engineering Task Force (IETF) and World Wide Web Consortium (W3C) protocols include IP, TCP, UDP, HTTP, XML, and SOAP.

- **Platform independence:** UPnP devices can be developed on any platform – on any specific operating system, language, or hardware.
- **Media and device independence:** UPnP technology can run on any medium with an IP stack, including phone lines, power lines, Ethernet, RF, and IEEE1394.
- **Programmatic and manual device control:** The UPnP architecture enables applications to programmatically control home networking devices. In addition, users can manually control devices using the device's browser-based administrative interface.

2.2 Basic principles of UPnP

The basic principle of UPnP is to allow **invisible** networking, as referred to by the UPnP Forum. A network's IP Address, a unique identifier, is automatically assigned to the UPnP-enabled device when linked to an appropriate network. Every UPnP-enabled device can communicate with every other device via the network, so each can learn the presence of the other and their capabilities. UPnP is independent of any specific particular programming

language or operating system and is available to everyone. UPnP technology can be supported on any common operating system or hardware platform. It works with almost any type of physical networking media – wired or wireless – providing maximum user and developer choice.

The basic building blocks of a UPnP-enabled network consist of three main components. They are (i) **Devices** (ii) **Services**, and (iii) **Control Points**. Figure 11.1 shows the basic architecture of UPnP and where the UPnP Control Points, Devices, and Services are interlinked for communication in the network.

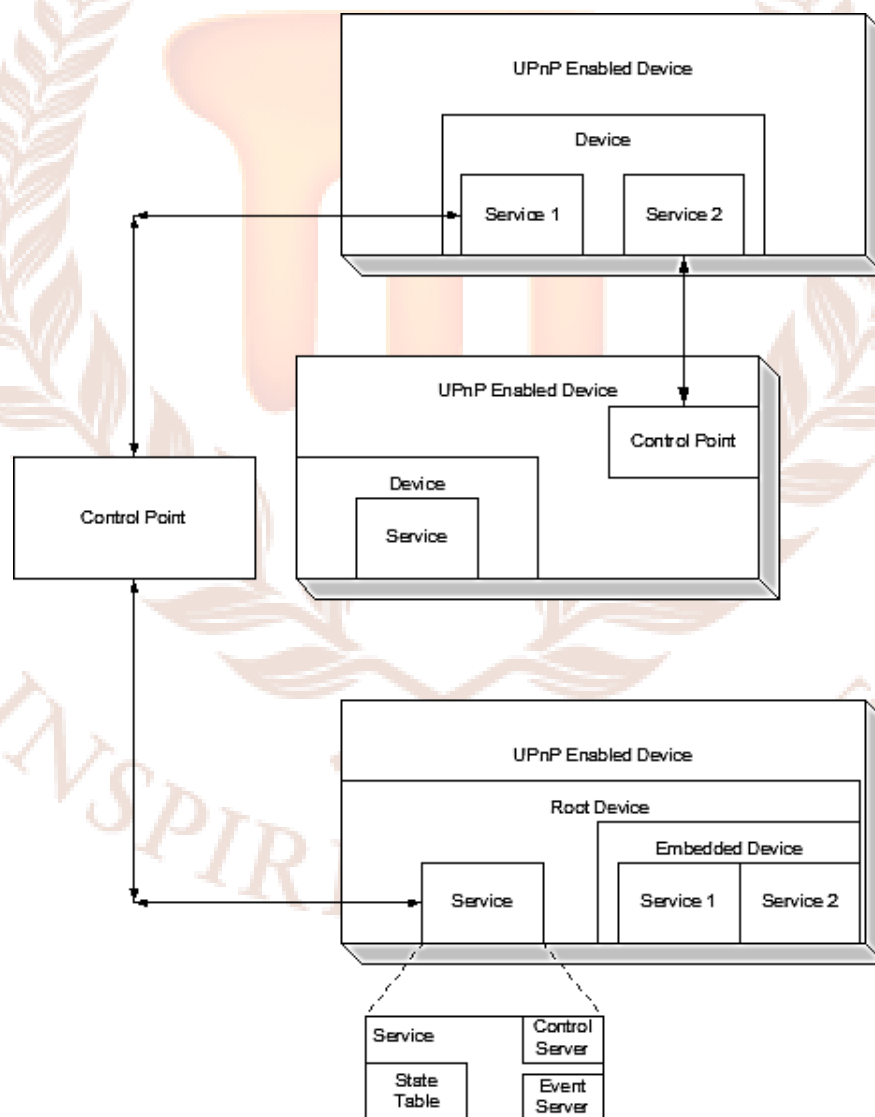


Figure 11.1: UPnP Control Points, Devices, and Services

- (i) **Devices:** A Device acts as a container that holds several services. Modern cell phones, an example of devices with UPnP capabilities. A nested device might also be present in the device. A nested device may include

Another device is built into the main device. A common example of a nested device found in phones is the digital camera.

The Device Description Document file is the brains of any UPnP device. This file is written in **XML** so any device or computer to discover the device properties can view it.

- (ii) **Services:** A Service is the set of functions that a Device supports. The available services are listed in the **Device Description Document**. Each Service on the **Service List** links to its **XML** file, which provides a detailed description of the Service. Services are supported by two main values: an **Action** and a **State**.

Action: An Action might be "receive a call" or "send a file," for instance, on a UPnP-capable mobile device.

State: A State describes the current status of a device. The state of the device changes depending on the situation. As a typical example, the device's state is **busy** if a telephone is already taking a call.

A typical UPnP device will have three functions that deal with Services:

- **State Table:** This takes the current state of the Service, e.g., Busy or Waiting, and records it. It will be updated when the State value changes.
- **Control Server:** This response to requests for action. The Service's current State value will be updated.
- **Event Server:** This is responsible for letting other devices (i.e., their services) know when the State of the Service changes, e.g., it can now accept incoming connections.

- (iii) **Control Points:** A control point discovers and then controls other devices it finds on the network. It retrieves the Device Description Document file and the Service list from each device.

2.3 Uses And Applications

UPnP is a service-based technology that relies on state changes (of the environment) to determine what happens on the network. A few of the uses and applications of UPnP-based devices are:

- Adding network devices and networked programs to a network.
- Configuration of devices by themselves in the network.
- Build networked applications.
- Connect computing hardware devices such as routers, modems, and wireless gadgets.
- Automating a house or workplace by figuring out what's going on in the neighborhood.

Let us understand the concept of components of UPnP:

- Addressing
 - UPnP uses IP addressing; so it acts as a DHCP client to assign IP itself
 - If there is no server (DHCP) found, then it assigns itself an IP; this process is called AutoIP
- Simple Service Discovery Protocol (SSDP)
 - To locate one another, gadgets employ this protocol.
 - Devices can receive messages from other devices on the same network thanks to this protocol.

The next two components include device description and service calls.

- Device Description
- Devices share data in XML forms to be aware of one other's information.
 - Name of the model, year of manufacture, etc., may be included in this data.
- Service calls
 - The device can make a service call after learning more about the other device.
 - This is accomplished with the aid of SOAP (Simple Object Access Protocol)

Now, another two components include GENA and presentation:

- General Event Notification Architecture (GENA)
 - This architecture responds to service calls
 - Control point is used to keep track of device model variables; it is informed of any changes.
- Presentation
- The gadget contains a URL for the manufacturer.
 - From the web browser, this data can be utilized to modify device settings.

2.4 Benefits Of Upnp Technology

UPnP is media and device independence. UPnP technology can run on any network using any communications media, including Radio Frequency (RF, wireless), phone line, power line, IrDA, Ethernet, and IEEE 1394. The benefits of UPnP-based devices are as follows:

- i) Platform independence: Vendors can use any operating system and programming language to build UPnP products.
- ii) Internet-based technologies: UPnP technology is built upon IP, TCP, UDP, HTTP, and XML, among others.
- iii) UI Control: UPnP architecture enables vendor control over the device user interface and browser interaction.
- iv) Programmatic control: UPnP architecture enables conventional application programmatic control.
- v) Common base protocols: Vendors agree on base protocol sets on a per-device basis.
- vi) Extendable: Each UPnP product can have value-added services layered on top of the basic device architecture by the individual manufacturers.

2.5 Advantages/Disadvantages Of Upnp

Advantages

- It allows true plug-and-play compatibility with all UPnP-enabled devices.
- It is simple for the consumer: plug in and go, so it is ideal for home care scenarios.

- Since UPnP is backed by some very large corporations, e.g., Microsoft and Intel, it can be commonplace for all developers.

Disadvantages

- UPnP will cause heavy network traffic due to communication of every other device on the network, slowing the network down if only limited bandwidth is available.
- Security can become an issue if UPnP has been used to set up a home network without expert advice. Shared files or folders could become available to external sources via the Internet, so third-party security software may be required to block this.
- UPnP seems to be a slow-moving technology. There are products that are UPnP enabled, but they are difficult to find in typical electrical outlets and are often not marketed as UPnP compatible.

2.6 Steps in UPnP device addressing

All UPnP devices must follow the same steps in acquiring an IP address. The steps, as specified by the UPnP device architecture, are shown in Figure 11.3.

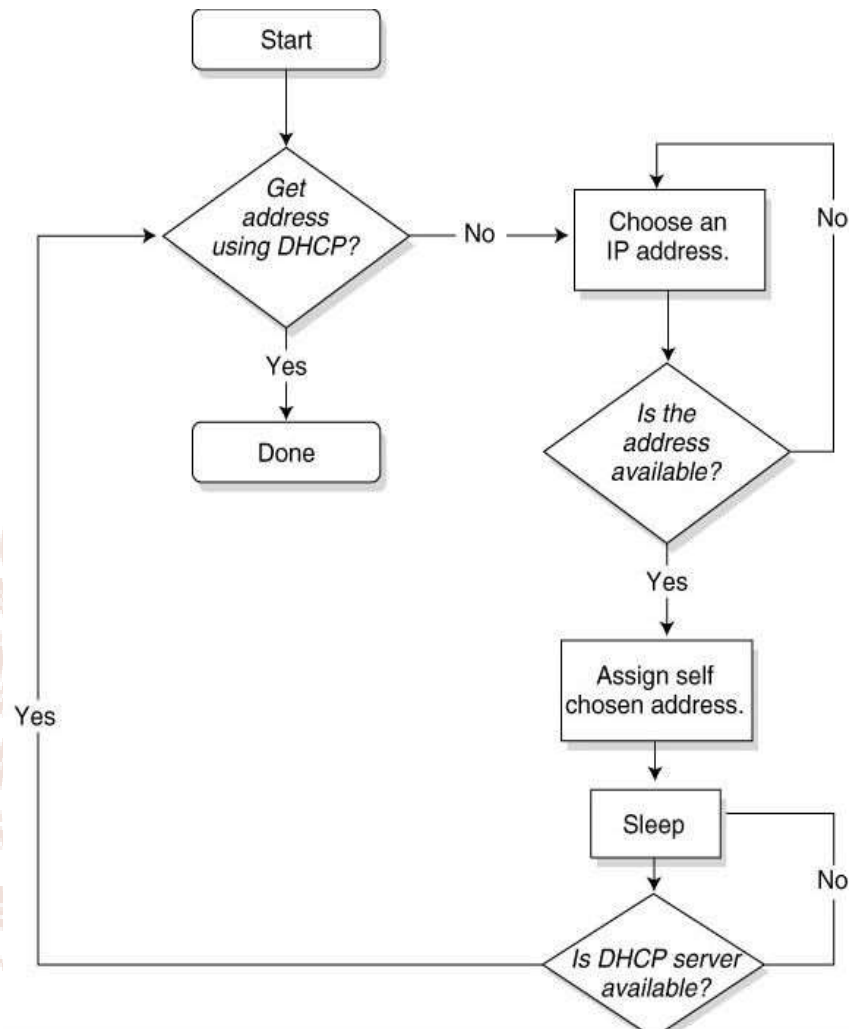


Fig 11.3: UPnP Device Addressing Flowchart

Step1: Try to obtain an address via DHCP

First, a UPnP device must try to get an address from a DHCP server. If the device successfully acquires an address, it is ready to continue with subsequent UPnP phases.

Step2: Failing DHCP, proceed with Auto-IP

If the UPnP device fails to acquire an address from a DHCP server, it begins selecting and testing an IP address.

Step 2a: Choose an IP address

To help keep clients from becoming stuck in a loop with other clients while trying to allocate an address, clients implementing Auto-IP are expected to randomize their IP address selection algorithm.

Step 2b: Test whether the address is available

When using Auto-IP to select an address, the client tests to determine whether the address is already in use. If so, the client chooses another address and tries again. A certain algorithm is used to ensure clients' auto-configuring on busy auto-configured network segments does not loop infinitely looking for an IP address.

Step 2c: Periodically check for a DHCP server

Whenever the DHCP server is available, the devices must switch to a DHCP-assigned address. UPnP devices configured using Auto-IP must, therefore, periodically test for the availability of a DHCP server. When rechecking, if the device determines that no DHCP server is available, it waits for some time and then tries again.

Step 2d: Upon finding a DHCP server, switch to a DHCP-assigned address

After receiving a response from a DHCP server, the UPnP device must respond and obtain a lease from the server. If the client successfully obtains the lease, it must drop any existing automatically configured IP addresses unless the device supports multiple addresses on the interface being configured.

The implementation defines how the device drops existing connections, but it should allow existing connections to be completed before closing them. In addition, the device should not allow new connections to the old, automatically configured address. Once all connections on the old address are closed, it can remove the address from the interface and be entirely transitioned to the new address.

Self-Assessment Questions - 1

1. The "_____ " Forum defines UPnP Devices and Service Descriptions.
2. A " " device could be another device built into the main device.
3. A control point retrieves the " " file and the Service list from each device.
4. Services in UPnP are supported by two main values: an " " and a .



3. UPNP PROTOCOL STACK

UPnP works as per a series of network protocols designed to make it easier to attach devices to computer-like devices and their networks. Devices on a UPnP network can be connected using any communications media, including Radio Frequency (RF, wireless), phone line, power line, IrDA, Ethernet, and IEEE 1394. Any medium that can be used to network devices together can enable UPnP. The only concern might be that the media being used supports the bandwidth required for the intended use.

UPnP uses open, standard protocols such as TCP/IP, HTTP, and XML. However, other technologies could be used to network devices together for many reasons, including cost, technology requirements, or legacy support. These include networking technologies like HAVi, CeBus, LonWorks, EIB, or X10. These, too, can participate in the UPnP network through a UPnP bridge or proxy.

Generally, Standard protocols like TCP/IP, HTTP, HTTPU, HTTPMU, SSDP, GENA, SOAP, and XML are used for communication in UPnP. UPnP works with a Six “step” protocol involved in UPnP networking. They are: (i) Addressing (ii) Discovery (iii) Description (iv)

Control (v) Eventing (vi) Presentation. Will study these protocols in the next sections of this unit.

Figure 11.2 shows Protocol Stack and its various layers.

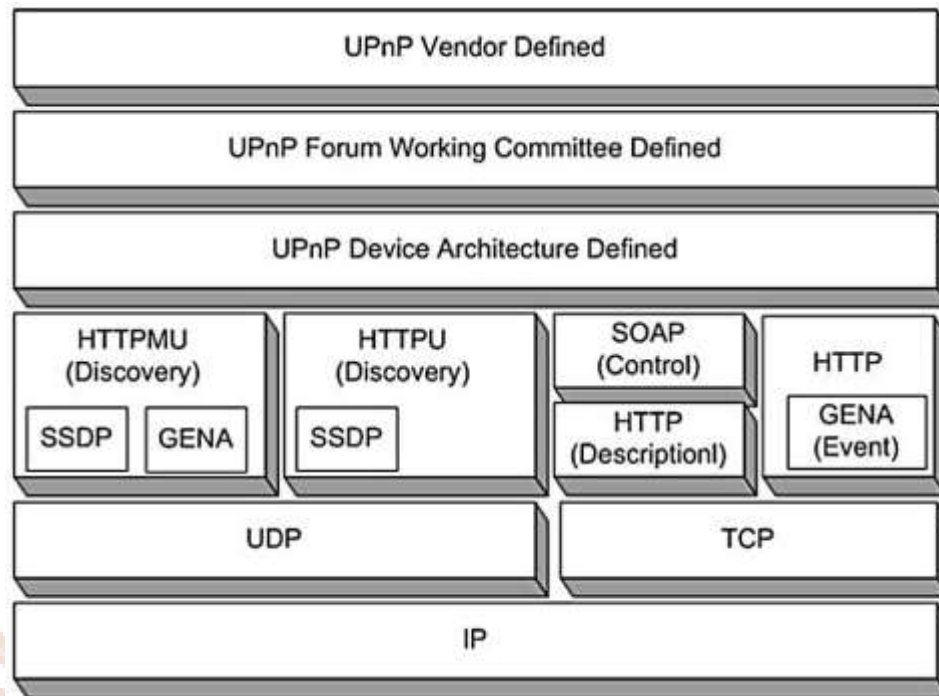


Figure 11.2: Protocol Stack

The UPnP protocol stack consists of three UPnP layers and other layers with standard protocols. Let us read about these layers.

- (1) UPnP vendor-defined: It is the top layer, and it gives vendor-specific information (e.g., serial number)
- (2) UPnP forum working committee defined: This is the second upper layer and is defined as per the UPnP forum working committee.
- (3) UPnP device architecture defined: This layer gives device-specific global information. (e.g., VCR info, DVD player info, etc.)

The other layers in the protocol stack work as per the protocols like SOAP, SSDP, and GENA. They are:

- (i) **Simple Object Access Protocol (SOAP):** It is used for Remote Procedure Calls (RPC).
- (ii) **Simple Service Discovery Protocol (SSDP):** It is used for broadcast and search. It helps with adding/removing devices from the network. SSDP is used over HTTPMU and HTTPU to provide basic information about services.

- (iii) **Generic Event Notification Architecture (GENA):** It is used to subscribe and publish messages. This mechanism handles events. A service that wants to send an event does so by either HTTP or HTTPU to registered listeners.
- (iv) **HTTP-over-Multicast (HTTPMU):** Although HTTPMU is an expired IETF draft and is not a W3C standard, it was invented to support UPnP. HTTPMU is expected to use the syntax of HTTP while changing the semantics. For HTTPMU, the contents of the datagram are HTTP requests.
- (v) **HTTP-over-UDP (HTTPU):** HTTPU sends a single datagram over UDP to a host. The recipient may reply to the host and port that sent the datagram. Again, standard HTTP requests are not expected to be sent, only that the syntax is obeyed.

The lowest three layers work as per the standard protocols like TCP, UDP, and IP. UPnP relies heavily on IP and the TCP and UDP stacks which are the Internet standards. It uses SOAP for method calls, which rely on HTTP, which in turn needs TCP. It uses HTML user interfaces, again requiring HTTP and TCP.

Self-Assessment Questions - 2

5. Which protocol is used for Remote Procedure Calls (RPC) in UPnP?
 - a) Generic Event Notification Architecture (GENA)
 - b) Simple Object Access Protocol (SOAP)
 - c) Simple Service Discovery Protocol (SSDP)
 - d) HTTP-over-Multicast (HTTPMU)
6. Which of the following is used to subscribe and publish messages in UPnP?
 - a) Generic Event Notification Architecture (GENA)
 - b) Simple Object Access Protocol (SOAP)
 - c) Simple Service Discovery Protocol (SSDP)
 - d) HTTP-over-Multicast (HTTPMU)
7. HTTP-over-Multicast (HTTPMU) is an IETF draft and is a W3C standard.
(True/False)



4. ADDRESSING

In any communication delivery system, communicating endpoints must have a unique address. If you want a UPnP to be networked, then the first logical step in networking is to obtain an address. The way UPnP devices acquire, manage and release addresses are called addressing. Addressing is the first step in UPnP networking; without an address, a device cannot proceed with subsequent UPnP phases, such as discovery where it offers its services to control points on the network.

UPnP devices are built upon the foundation of the TCP/IP protocol suite, which provides the network layer connectivity devices needed to communicate. Each endpoint on an IP network has an address that uniquely identifies it among all the endpoints on the network. Generally, every user would want their UPnP device configured automatically through some automatic addressing mechanism rather than manually configured.

There are two addressing protocols available for the automatic configuration of UPnP. They are: (i) Dynamic Host Configuration Protocol (DHCP) and (ii) Auto-IP (Without DHCP Server).

With DHCP, the device takes the IP address or domain name assigned to it. Without a DHCP server, the devices use an Auto IP addressing protocol to get an address. Auto IP is a protocol in which a system chooses its IP address from a given set of addresses.

4.1 Dynamic Host Configuration Protocol (DHCP)

Dynamic Host Configuration Protocol (DHCP) provides a framework for passing configuration information to hosts on a TCP/IP network, including the host's IP address, subnet mask, default gateway, and domain name server. A client/server protocol uses UDP as its transport. DHCP clients send messages to DHCP servers on port 67 and receive responses from servers on port 68. A DHCP server manages a pool of IP addresses, automatically assigning addresses to network hosts and reusing released addresses. Using a DHCP server to assign addresses to IP endpoints centralizes the management of IP addresses, ensuring that each.

Endpoint receives a unique address and avoids problems that arise with manual configuration.

There are three mechanisms DHCP can use to assign IP addresses to clients. They are:

- **Automatic allocation:** The DHCP server assigns a permanent IP address to a client.
- **Manual allocation:** The network administrator determines the address assignment for each host. The DHCP server conveys the address to the clients when they request an address from the DHCP server.
- **Dynamic allocation:** This is the common mode of operation where the DHCP server assigns an IP address to the client for a limited time. The client is said to have a lease on the address. Once the lease expires or is released by the client, the server may assign this address to another client.

Each UPnP device is required to have a built-in DHCP client. When a UPnP device is first connected to a network, it searches for a DHCP server to acquire an IP address. In theory, a DHCP server servicing a network of UPnP devices could use any of the three mechanisms. However, dynamic allocation provides the best match as it requires no administrative configuration per client.

4.2 Auto-Ip

Using DHCP to assign addresses to a dynamically changing set of devices has a difficulty that a DHCP server may not be running all the time without administrative support. As per the zero-administration philosophy, the designers of the UPnP architecture want UPnP devices to acquire addresses on networks without a DHCP server. Auto-IP is the solution to comply with such a philosophy.

Auto-IP is a method by which an endpoint on an IP network may automatically choose an IP address and subnet mask in the absence of a DHCP server. Auto-IP does not replace DHCP but augments it, making clients more robust by allowing them to acquire addresses without DHCP services. UPnP devices use the Auto-IP mechanism only if a DHCP server is not present or if the DHCP process fails. In addition, the UPnP device architecture specifies how a UPnP

device has configured its address using Auto-IP must. It must periodically check for the presence of a DHCP server to smoothly transition to use a DHCP-assigned address.

4.2.1 Address Selection

UPnP devices use Auto-IP to get an IP address by selecting a candidate address. The address selection must fall within a range of 169.254/16 addresses that are non-routable IP addresses. Addresses in this range will not cross gateways, so they will never make it outside an organization and onto the Internet. After selecting an address, the client configures itself with a default class B subnet mask of 255.255.0.0.

The Internet Assigned Numbers Authority (IANA) has reserved this range for private IP address, so no one can use it on the Internet. This range is known as the **LINKLOCAL net**. Also, the first and last 256 addresses are reserved for future use and must not be selected.

4.2.2 Resolving Address Conflicts

Once the UPnP device has selected an address, it must verify that it is not already being used by another device on the network. The UPnP device uses the Address Resolution Protocol (ARP) to do this .

Self-Assessment Questions - 3

8. Two addressing protocols are available for the automatic configuration of UPnP. They are "_____" and "_____".
9. DHCP clients send messages to DHCP servers on port "_____" and receive responses from servers on port _____.
10. UPnP devices use the "_____" mechanism only if a DHCP server is not present, or if the DHCP process fails.

5. AD-HOC NETWORKS

Using Auto-IP for address assignment, it is possible for hosts to come together and form an ad-hoc network without having the assistance of pre-existing network infrastructure, including DHCP and DNS servers.

6. DISCOVERY

Once a UPnP device acquires an address, it is ready to provide its services to control points on the network. Discovery is the next phase in UPnP device operation. Controllers look for any devices of interest, and devices look for a controller. Control points search for devices and services on the network and find ones that meet its search criteria.

Service discovery is the mechanism by which devices and network-based services make themselves available to clients, and clients can discover devices and services. Simple Service Discovery Protocol (SSDP) is the discovery protocol used by UPnP devices. SSDP was designed to discover HTTP-based resources based on Uniform Resource Identifiers (URI). SSDP uses a decentralized model of communication that requires no user administration. SSDP's balanced scheme of discovery requests and presence announcements minimizes network traffic. UPnP adds conventions for using SSDP, including predefined service types and the composition of URIs to search for devices and services.

7. SIMPLE SERVICE DISCOVERY PROTOCOL (SSDP)

The Simple Service Discovery Protocol (SSDP) is a simple discovery solution for HTTP-based resources on the local area network that doesn't require any configuration, management, or administration. SSDP doesn't attempt to address the problem of Internet-wide HTTP-based resource discovery. Other protocols, such as Universal Description Discovery and Integration are left to handle that issue (UDDI).



8. SERVICE IDENTIFICATION

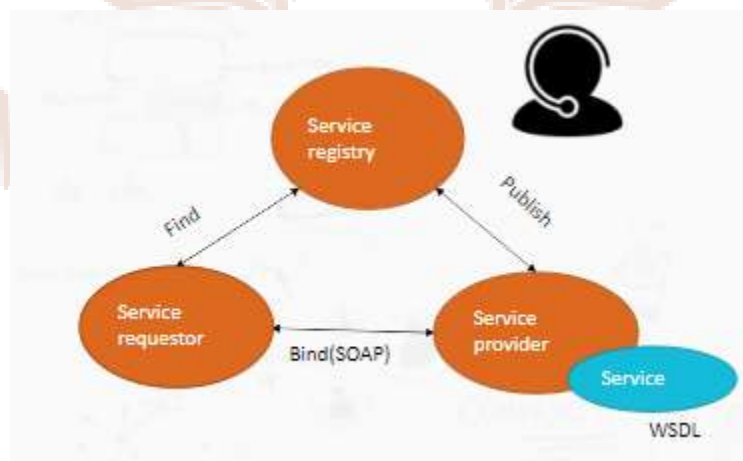
SSDP introduces two concepts related to service identification, the *service type* and the *Unique Service Name (USN)*. A service type is a URI that identifies the type or function of a particular resource – a printer service, for example. SSDP does not define service types but provides the mechanisms for discovering them. The UPnP working committees for each standard device type specify the service types for UPnP devices and services.

A Unique Service Name is a URI used to uniquely identify an instance of a particular service, allowing SSDP clients to differentiate between two services with the same service type. AUSN typically contains a Universally Unique Identifier (UUID)—a 128-bit number used to uniquely identify an object.

The benefits of service-oriented approaches are as follows:

- Benefits of a service-oriented approach
 - The binding of services may be postponed by application till they are delayed.
 - Services can be used concurrently by multiple applications because they are independent.
 - Any service provider may offer services.

Service-oriented Architecture Diagram:



The service-oriented architecture diagram is important to understand its functionalities:

Service Provider:

- It develops a web service and gives the service registry its details. Each service provider disputes a variety of hows, and whys, including which service to expose, which to prioritize—security or accessibility—and many more. Additionally, the service provider must choose what category the product should be classified under for a specific broker service and what trading partner agreements are necessary to use the product.

Service broker, service registry, or service repository:

Its primary purpose is to provide any potential requester access to the web service's information. Whoever implements the broker determines the broker's scope. Public brokers are accessible to everyone, whereas only a small portion can access private brokers. An early attempt to provide Web services discovery was UDDI, which is no longer actively supported.

Service requester/consumer:

It uses a variety of find operations to find entries in the broker registry before binding to the service provider and calling one of its web services. Any service that the service users require must be obtained from the brokers, bound to the required service, and then used. If the service offers various services, they can access them all.

Let us look at the benefits of Service-oriented Architecture:

- Local service providers or third parties can be used to deliver services.
- Services are available in any language.
- It is possible to protect investments in legacy systems.

In general, a resource may have multiple representations, and they can exist in different formats:

- Resource Operations
 - Create: Make the resource available

- Read: Obtain a replica of the resource back
- Update: Change the value of the resource
- Delete: Make the resource inaccessible

Let us now learn about resources and actions:

The resource is a data element, such as a medical record.

General resource actions:

We have CREATE(),READ(),UPDATE(),DELETE().

Web resources:

We have POST(),GET(),PUT(),DELETE().

Let us see the types of ad hoc networks:

- Mobile ad hoc networks (MANET) : A wireless, self-organizing network of portable devices is known as a mobile ad hoc network.
- Wireless sensor networks (WSN): Mobile devices called wireless sensor networks are used to collect specific environmental data like temperature, humidity, traffic volume, etc.
- Wireless mesh networks (WMN): Based on the mobility pattern, wireless mesh networks create wireless meshes.
- Vehicular ad hoc networks (VANET): The formation of a vehicular ad hoc network is communication between moving vehicles.

Ad hoc networks have the following benefits:

- Decoupling from central network administration
- Self-configuring nodes acting as routers.
- Scalability includes the addition of extra nodes;
- Self-healing through ongoing reconfiguration.
- In any scenario involving several wireless devices, mobility enables the instantaneous creation of ad hoc networks.

Understanding the issues of the ad hoc network:

- Low quality communication
- Time-varying protocol
- Scalability
- Energy conservation
- Hidden node problem
- Exposed node problem



9. COMMUNICATION MODEL

SSDP uses a decentralized approach to service discovery whereby no central store maintains information about resources, their location, and their availability. Instead, each client directly queries the network, and each resource responds directly to these requests.



10. DISCOVERY REQUESTS AND PRESENCE ANNOUNCEMENTS

There are two types of SSDP requests (i) discovery requests and (ii) presence announcements.

Discovery requests allow SSDP clients to look for SSDP resources. Presence announcements allow SSDP resources to announce their presence on the network. SSDP's balance of discovery requests and presence announcements is designed to make the protocol efficient, reducing network traffic. When a resource comes online, it announces its presence. This lets all clients know that the resource is available. From then on, the resource does not need to send out any other presence announcements, except when it is going offline. Any clients that come on-line after the resource has announced its presence send out discovery requests. If the resource supports the requested service, it responds to the client. The client does not need to repeat the discovery request because any resources that come on-line after it has issued the request will announce their presence. The result is that neither client nor server need send out steady streams of messages.

11. NETWORK TRANSPORT

SSDP uses HTTP over multicast UDP to send messages to every SSDP peer on the network. SSDP client's multicast HTTPMU discovery requests to the address 239.255.255.250:1900, which is SSDP's reserved multicast address and port that has local administrative scope, limiting the delivery of the multicast packet to an administrative domain.

SSDP services listen to the SSDP multicast channel to hear the discovery requests. If an SSDP resource receives a multicast HTTPMU discovery request that matches the service it offers, it responds by sending a response directly to the SSDP client that issued the search using the HTTPU.

The features of TCP:

- Features of TCP
 - Stream Data Transfer
 - Reliability
 - Flow Control
 - Multiplexing
 - Logical Connection
 - Full Duplex

Let us understand the features of TCP in detail:

- Stream Data Transfer: The stream of bytes is transported across the internet by the virtual circuit.
- Logical connection: The socket, sequence number, and window size together
- Full Duplex: Data can be transferred in both directions at the same time
- Multiplexing: combines two or more connections to data streams
- Flow Control: It controls the flow of data transfer
- Reliability: It adheres to the error-control mechanism and flow.

The features of UDP:

- Features of UDP
 - Source port address
 - Destination port address
 - Checksum
 - Total length

The features of UDP:

- Provides the sender's source port address in the source port field.
- Total Length: Contains information about the message's total bytes.
- Provides the receiver's destination port address in the destination port address field.
- Checksum: Offers the checksum technique to reduce error.

Let us look at the responsibilities that are provided by the transport layer:

- Transport layer services
 - End-to-End delivery
 - Addressing
 - Reliable delivery
 - Flow Control
 - Multiplexing

Self-Assessment Questions - 4

11. Which of the following is the discovery protocol used by UPnP devices?
- a) Simple Service Discovery Protocol (SSDP)
 - b) Universal Description Discovery and Integration (UDDI)
 - c) Universally Unique Identifier (UUID)
 - d) Unique Service Name (USN)
12. SSDP introduces two concepts related to service identification _____ and the "____" _____.
13. There are two types of SSDP requests: "_____" and _____.



12. DESCRIPTION

The description phase of UPnP comes after discovery. It enables control points to discover details about devices and the services they implement. After a UPnP control point discovers a device, it only contains the information in the discovery message. Those discovery messages are the device's type, its universally unique identifier, and a URL to its description document. The control point collects description documents from the device to learn additional information about the device, including the services and operations it supports.

Device and service description documents are XML documents that follow a standard schema defined by the UPnP Forum. The contents (elements) of specific devices and service descriptions are defined by various working committees of the UPnP Forum and correspond to the required information that a standard UPnP device must have.

12.1 Upnp's Description Phase

The description phase is the link between UPnP's discovery and control phases. In the discovery phase, devices advertise their presence to control points on the network, while control points search for devices. Once a control point receives these advertisements and finds a device of interest, it gets the description documents directly from the device to learn more about the device and its services. Once a control point has processed the description documents and understands the device's capabilities, it is ready to control the device.

12.2 Description Document Standards

Device and service descriptions are simply XML documents that conform to the UPnP Template Language. The UPnP Forum defines the XML syntax for creating device and service descriptions. This basic template language is used by the various working committees of the UPnP Forum to define standard devices and the services they must contain.

Self-Assessment Questions - 5

14. The "_____ "phase is the link between UPnP's discovery and control phases.
15. Device and service description documents are "_____" documents that follow a standard schema defined by the UPnP Forum.

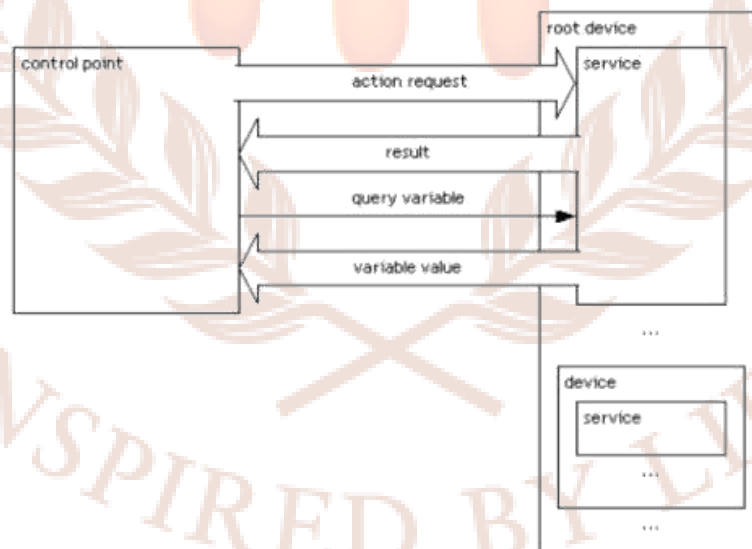


13. CONTROL

After a UPnP device acquires an IP address and advertises its presence on the network, control points can discover the device and invoke any of the actions provided by the device's services. In UPnP terminology, this invocation process is called control.

The control point sends a suitable control message to the control URL for the service provided in the device description. Control messages are also expressed in XML using the Simple Object Access Protocol (SOAP). The control protocol used between UPnP control points and devices is called Simple Object Access Protocol (SOAP). SOAP is a messaging and remote procedure call technology that can be used over a variety of transports but is primarily used over HTTP. UPnP control points use SOAP to invoke actions provided by services contained on a device. In response to the control message, the service returns any action-specific values.

In the description steps, different kinds of methods are used.



They are mentioned below. The most often used methods are device and service description, UPnP Device and Service Template.

- Device Description
- UPnP Device Template
- Service Description

- UPnP Service Template
- Non-Standard Vendor Extension
- UPnP Template Language for Devices
- UPnP Template Language for Services
- Retrieving a Description

Now let us see the protocol and action under control:

- Control messages at the top layer include vendor-specific data like parameter values.
- Vendor material is augmented with data from a UPnP Forum working group, including names for statements, actions, and variables.

Action Invoke

- For remote procedure calls, the Simple Object Access Protocol (SOAP) specifies the use of XML and HTTP.
- Control messages are sent to devices via SOAP, and results or errors are returned to control points.

Action Response

- Within 30 seconds, including estimated transmission time, the service must finish invoking the action and respond.

Now let us see the protocol and action under control:

- Control messages at the top layer include vendor-specific data like parameter values.
- Vendor material is augmented with data from a UPnP Forum working group, including names for statements, actions, and variables.
- Next, we have a query for the variable:
- Query Invoke
- To query a state variable's value, a control point must send a request in the following format.
- Query Response:

- To answer a query for the value of a state variable, the service must respond within 30 seconds, including the expected transmission time.
- If the service fails to respond within this time, whatever the control point does should be application specific. The service must send a response in the following format.

Self-Assessment Questions - 6

16. Control messages are also expressed in XML using the _____ .
17. UPnP control points use SOAP to invoke actions provided by services contained on a device. (True/False)



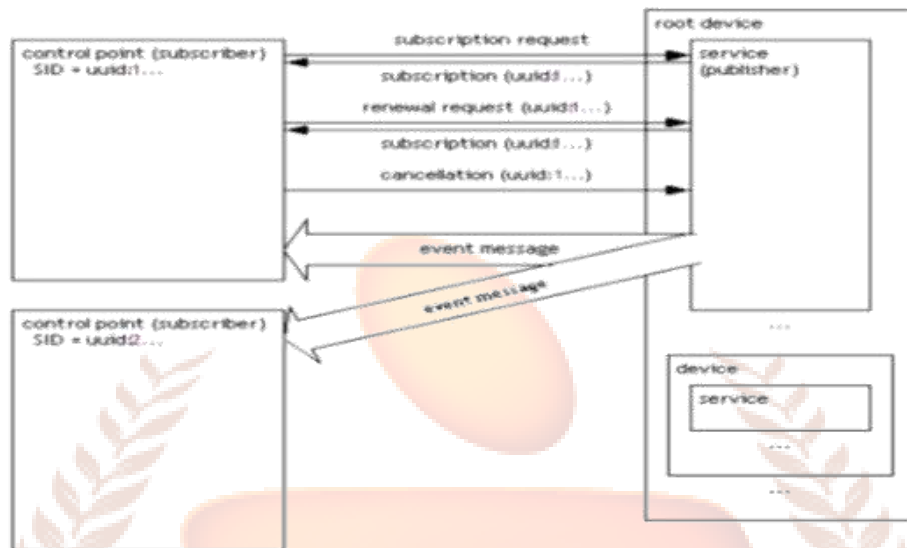
14. EVENTING

While an UPnP device is being controlled, its service changes states. This sends event messages to any subscribed controllers. A special event is generated when a controller subscribes to a device. This event causes the device to send names and values of state variables to the controller. This allows the controller to initialize its state model for that service. With the help of the Eventing protocol, a device will notify all subscribers when state changes occur. Eventing enables groups of UPnP devices and control points to be dynamic, responding automatically to state changes. Eventing allows control points to register to a device when state changes occur. Control points receive a notification and may wish to respond to that notification by invoking an action. In contrast to a synchronous remote procedure calls, eventing in a distributed system call mechanisms such as RPC. Events are expressed in XML and sent over to General Event Notification Architecture (GENA). GENA gives ability to subscribe/publish.

The UPnP architecture uses GENA for eventing, adding conventions of its own. In particular, the UPnP architecture uses GENA to publish changes to a service's evented state variables. A publisher is a service that accepts registrations from clients interested in receiving notifications. A subscriber is called a "client" and will subscribe to a publisher. If a subscriber does not wish to receive any event notifications, they may unsubscribe from the publisher. The publisher will then stop sending event messages on state changes. Subscriptions may expire over time. To keep receiving message notifications, a renewal subscription must be submitted by the client.

GENA uses HTTP as the transport layer for communication between publishers and subscribers. To achieve this GENA has introduced three new HTTP methods for subscription and notification:

- SUBSCRIBE to register for events and receive notifications.
- This is also used for renewing existing subscriptions.
- UNSUBSCRIBE to cancel subscriptions.
- NOTIFY to send events to the subscriber.



In eventing, different kinds of methods have also been used. They are as follows:

- Subscription
- Event Messages
- UPnP template language for eventing
- Augmenting the UPnP template language

After a control point, has (1) discovered a device and (2) retrieved a description of the device and its services; the control point has the essentials for eventing.

To subscribe to eventing, a subscriber sends a subscription message. If the subscription is accepted, the publisher responds with a duration for the subscription.

The different steps involved in UPnP networking are given below.

Step 1: Discovery explains how devices advertise and control points search and explains the details of the format of discovery messages.

Step 2: Description explains how devices are described and how those descriptions are retrieved by control points.

Step 3: The control section explains the description of actions, state variables, and control message format.

Step 4: The service publishes updates when these variables change, and a control point may subscribe to receive this information. The service publishes updates by sending event messages.

Step 5: The degree to which each of these can be accomplished depends on the specific capabilities of the presentation page and device.

In the description steps, different kinds of methods are used.

They are mentioned below. The most often used methods are device and service description, UPnP Device and Service Template.

- Device Description
- UPnP Device Template
- Service Description
- UPnP Service Template
- Non-Standard Vendor Extension
- UPnP Template Language for Devices
- UPnP Template Language for Services
- Retrieving a Description

Self-Assessment Questions - 7

18. Which among the following protocol is used by device to notify all subscribers when state changes occur?

- a) discovery
- b) Eventing protocol
- c) addressing
- d) control

19. The UPnP architecture uses "_____" to publish changes to a service's evented state variables.

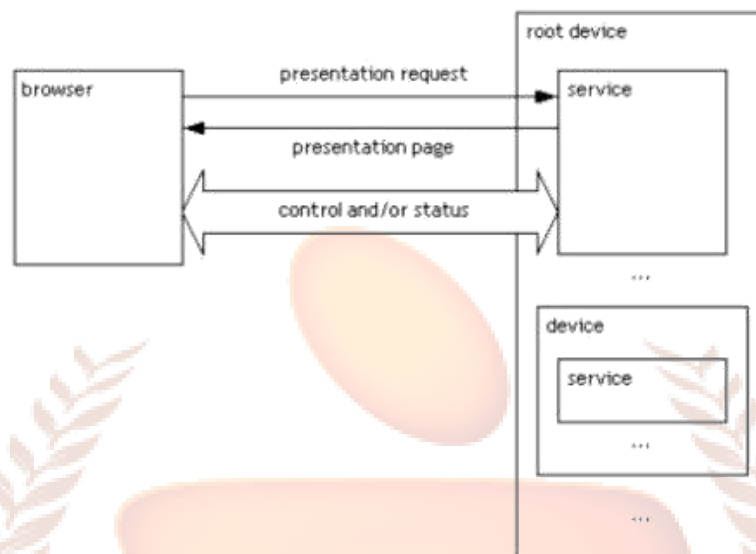


15. PRESENTATION

Presentation exposes an HTML-based user interface for controlling and/or viewing device status. Presentation is complementary to control, where control points send actions to devices and eventing where control points listen to state changes in devices.

UPnP devices have embedded web servers because most of the communications protocols, including XML-based device descriptions, SOAP-based control messages, and GENA's event-related messages, run over HTTP. Every UPnP device can provide a web interface for administrative monitoring and control through the device presentation page. The URL for the presentation is contained within the presentation URL element in the device description. The device description is delivered via a description message. Retrieving a presentation page is a simple HTTP-based process.

The interactions between control points and devices can be entirely automatic, requiring no human intervention. However, it is also possible to manually control UPnP devices using the device's presentation page, a web page provided by the device, and loaded by an administrator using a web browser. PnP technology enables network devices to present their functionality as programmable services to be manipulated by control points. Hence, depending on the device's capabilities, the presentation can be Monitored and Controlled. Although the presentation page is purely an optional administrative interface added to devices for providing a means for direct programmable control. The UPnP Forum does not dictate the required contents, appearance, or capabilities of device presentation pages, but allows these things to be completely determined by the vendor. However, if a device supplies a URL for a presentation page, the control point must be able to retrieve a page from the URL and load it into a browser. Depending upon the capabilities of the page, it must be able to allow a user to control the device and/or view device status.



Given above is the diagrammatic working of presentation:

A presentation request moves from a browser to the service in the root device.

The response is loading of the presentation page.

The entire process is managed via control and/or status request.

Similarly, multiples request and replies move in & out via the browser and the services.

Self-Assessment Questions - 8

20. Every UPnP device can provide a "_____" interface for administrative monitoring and control through the device presentation page.
21. PnP technology enables network devices to present their functionality as programmatic services to be manipulated by "_____" points.

16. SUMMARY

Let us recapitulate the important concepts discussed in this unit:

- The UPnP architecture is designed to offer pervasive peer-to-peer network connectivity of networked devices such as Personal Computers (PCs) of all form factors, intelligent appliances, and wireless devices.
- The Universal Plug and Play Forum define UPnP Device and Service Descriptions.
- The basic building blocks of a UPnP enabled the network to consist of three main components: (i) Devices (ii) Services, and (iii) Control Points.
- Generally, communication in UPnP occurs through standard protocols like TCP/IP, HTTP, HTTPU, HTTPMU, SSDP, GENA, SOAP, and XML.
- UPnP works with a Six “step” protocol in UPnP networking: (i) Addressing (ii) Discovery (iii) Description (iv) Control (v) Eventing and (vi) Presentation.
- The way UPnP devices acquire, manage, and release addresses is called addressing.
- There are two addressing protocols available for automatic configuration of UPnP. They are: (i) DHCP and (ii) Auto-IP
- Service discovery is the mechanism by which devices and network- based services make themselves available to clients, and clients can discover devices and services. SSDP is the discovery protocol used by UPnP devices.
- To find more detail about the device, including the services and actions it supports, the control point retrieves description documents from the device in description phase.
- Control points can discover the device in the network and invoke any of the actions provided by the device’s services by the invocation process called control.
- The control protocol used between UPnP control points and devices is the Simple Object Access Protocol (SOAP).
- With the help of the Eventing protocol, a device will notify all subscribers when state changes occur. Events are expressed in XML and sent over General Event Notification Architecture (GENA).
- It is possible to manually control UPnP devices using the device’s presentation page, a web page provided by the device and loaded by an administrator using a web browser.

17. GLOSSARY

- Ad-Hoc networking: Dynamic network without the need for dedicated networking infrastructure services.
- Auto-IP: DHCP to assign addresses to a dynamically changing set of devices time without administrative support.
- Home network: Home network is a computer network that facilitates communication among devices within the close vicinity of a home.
- Platform independence: Developed on any platform, no specific operating system, language, or hardware.
- Plug and play: Users can add devices to the home or office network without installing drivers or configuring the devices before using.
- Simple Service Discovery Protocol (SSDP): Simple discovery solution for HTTP-based resources on the local area network that does not require any configuration, management, or administration.
- Zero-configuration network: It is a networking where to configure devices not required to use.

18. TERMINAL QUESTIONS

Short Answer Questions

1. Explain the basic building blocks of an UPnP enabled network.
2. Draw the neat diagram of UPnP Protocol stack and explain each of its layers in brief.
3. Draw the UPnP device addressing flowchart and explain it in stepwise manner.
4. Explain how discovery is performed in UPnP device operation.
5. What is the difference between control and Presentation? Explain.

19. ANSWERS

Self-Assessment Questions

1. Universal Plug and Play
2. nested
3. Device Description Document
4. Action, State
5. b) Simple Object Access Protocol (SOAP)
6. a) Generic Event Notification Architecture (GENA)
7. False
8. DHCP, Auto-IP 9.67,68
9. Auto-IP
10. a) Simple Service Discovery Protocol (SSDP)
11. service type, Unique Service Name (USN)
12. scovery requests, presence announcements
13. XML
14. description
15. Simple Object Access Protocol (SOAP)
16. True
17. b)Eventing protocol
18. GENA
19. web
20. control

Short Answer Questions

1. The basic building blocks of an UPnP enabled network to consist of three main components: Devices, Services, and control Points. (Refer to section 2 for more details)
2. The UPnP protocol stack consists of three UPnP layers and other layers with standard protocols like TCP/IP, HTTP, HTTPU, HTTPMU, SSDP, GENA, SOAP, XML. (Refer to section 3 for more details)

3. The steps are Step 1: Try to obtain an address via DHCP, Step 2: Failing DHCP, proceed with Auto-IP (Refer to section 4 for more details)
4. Simple Service Discovery Protocol (SSDP) is the discovery protocol to discover HTTP-based resources based on Uniform Resource Identifiers (URI). (Refer to section 7 for more details)
5. Control points invoke any of the actions provided by the device's services by the invocation process called control whereas presentation exposes an HTML-based user interface for controlling and/or viewing device status. (Refer to sections 13 and 15 for more details)

20. SUGGESTED BOOKS

- Jeronimo, M., & Weast, J. (2003). UPnP Design by Example – A Software Developer's Guide to Universal Plug and Play. IntelPress.
- MATCH PROJECT Mobilizing advanced technologies for care at home. (n.d.). Retrieved 08-02, 2012, from What is UPnP?
http://www.cs.stir.ac.uk/~kjt/research/match/resources/tutorial/Home_Care_Network



**BACHELOR OF COMPUTER
APPLICATIONS
SEMESTER 4**

**DCA2203
SYSTEM SOFTWARE**

Unit 12

Android Operating System

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
1.1	Learning Objectives	-	-	
2	Android Operating System	1	1	4-6
3	Android Architecture	1	2	7-10
3.1	Linux Kernel	-	-	
4	Android Architecture Libraries	-	3	11-14
5	Android Architecture Application Framework	-	4	15-16
6	Applications	-	5	17-18
7	Security and Permission	-	6	19-23
8	Summary	-	-	24
9	Glossary	-	-	25
10	Terminal Questions	-	-	25-26
10.1	Answers	-	-	
11	References	-	-	27

1. INTRODUCTION

In the previous unit, we learned Universal Plug and Play (UPnP) and briefed that UPnP architecture offers pervasive peer-to-peer network connectivity of PCs of all form factors, intelligent appliances, and wireless devices. Additionally, the investigation was completed, and its technical capabilities and advantages. The Android operating system, its technologies, and its widely used applications for mobile devices are all covered in this topic.

Android Operating System has become the most common Operating System (OS) used mainly in the Mobile devices. It is the software to run a mobile phone without the proprietary barriers, and it offers numerous advantages. The intent of Android technology is to accelerate innovation in mobile devices and offer consumers a richer, less expensive, and better mobile experience.

1.1 Learning Objectives:

After studying this unit, the learners should be able to:

- ❖ *Explain about the android OS*
- ❖ *List and describe various layers of Android architecture*
- ❖ *Discuss about the Linux kernel used for Android*
- ❖ *Describe about the libraries of android architecture*
- ❖ *Discuss about the application framework of android architecture*
- ❖ *Determine how application file built for android OS*
- ❖ *Discuss about the security aspects of android technology*

2. ANDROID OPERATING SYSTEM

Nowadays, it's easy to find mobile devices with various types of technologies and Mobile Operating Systems (OS) running on them. The types of Mobile OS that you may find are Symbian, iPhone, RIM's BlackBerry, Windows mobile, Linux, Palm webOS, Android, etc.

Although the mobile operating system has its own merits and demerits, Android is the first truly open and comprehensive platform for mobile devices. It is the software to run a mobile phone but without proprietary obstacles.

Android is a software stack for mobile devices that includes an operating system, middleware, and key applications. The Android Software Development Kit (SDK) provides the tools and Application Programming Interfaces (APIs) necessary to begin developing applications on the Android platform using the Java programming language. Android Inc was founded in Palo Alto of, California, U.S., by Andy Rubin, Rich miner, Nick sears, and Chris White in 2003. Later, Android Inc. was acquired by Google in 2005. After the original release, there have been several updates in the original version of Android. Android is now developed by Open Handset Alliance (OHA), a group of 84 technology and mobile companies that have come together to accelerate innovation in mobile and offer consumers a richer, less expensive, and better mobile experience.

You must be wondering why we need to go for Android compared to other OSs. This is true because it offers a straightforward and effective SDK, and getting it doesn't involve paying any licensing, distribution, or development expenses. With Android, you can develop over many platforms like Linux, Mac OS, and windows. You can also get excellent documentation with the support of an active developer community. Android is built on the open Linux Kernel. Furthermore, it utilizes a custom virtual machine designed to optimize memory and hardware resources in a mobile environment. Android is open source; hence, it can be extended to incorporate new cutting-edge technologies as they emerge. The incorporate Java programming for the development of mobile device applications with the support of open-source libraries like SQLite, WebKit, OpenGL. You can import a 3rd party Java library to enhance the applications. The platform will continue to evolve as the developer community works together to build innovative mobile applications. Table 12.1 shows list of Android versions.

Name	Version number(s)
Apple pie	0.9
Banana bread	1.1
Cupcake	1.5
Donut	1.6
Eclair	2
	2.0.1
	2.1
Froyo	2.2 - 2.2.3
Gingerbread	2.3 - 2.3.2
	2.3.3 - 2.3.7
Honeycomb	3
	3.1
	3.2 - 3.2.6
Ice Cream Sandwich	4.0 - 4.0.2
	4.0.3 - 4.0.4
Jelly Bean	4.1 - 4.1.2
	4.2 - 4.2.2
	4.3 - 4.3.1
KitKat	4.4 - 4.4.4
	4.4W - 4.4W.2
Lollipop	5.0 - 5.0.2
	5.1 - 5.1.1
Marshmallow	6.0 - 6.0.1
Nougat	7
	7.1 - 7.1.2
Oreo	8
	8.1
Pie	9
Android 10	10
Android 11	11
Android 12	12

Table 12.1 List of android versions

History of Android:

It's interesting to learn about the evolution of Android. These include Aestro, Blender, Cupcake, Donut, Eclair, Froyo, Gingerbread, Honeycomb, Ice Cream Sandwich, Jelly Bean, KitKat, and Lollipop. The code names for Android now span from A to J. Let's take a chronological look at the history of the Android.

- 1) In October 2003, Andy Rubin established Android Incorporation in Palo Alto, California, in the United States.

- 2) Google acquired android Incorporation on August 17, 2005. Since that time, it has been a subsidiary of Google Inc.
- 3) Andy Rubin, Rich Miner, Chris White, and Nick Sears are some of Android Incorporation's most important workers.

Android SDK Feature

The feature of Android SDK is that one can use it as a development kit for Global System for Mobile Communications, originally Groupe Special Mobile (GSM), Enhanced Data rates for GSM Evolution (EDGE), and 3G networks, WiFi, Bluetooth technologies etc. It provides rich libraries with media, SQLite, WebKit, SSL (Secure Sockets Layer) etc. WebKit is an open-source web browser engine. SQLite is a software library that implements a self-contained, serverless, transactional SQL database engine with zero-configuration. SSL is the standard security technology for establishing an encrypted link between a web server and a browser. It also provides hardware control with an accelerometer, compass, microphone, camera, GPS, touch screen, power, etc. You can also be supported with location-based service and map (e.g., Google API).

SELF ASSESSMENT QUESTIONS – 1

1. Android utilizes a custom virtual machine that was designed to optimize “_____” and _____ resources in a mobile environment.
2. With Android, you can go for development over many platforms like Linux, Mac OS, windows. (True/False)
3. Android is built on the open “_____” Kernel.

3. ANDROID ARCHITECTURE

The Android platform is built as a stack with various layers running on top of each other, lower-level layers providing services to upper-level services. Figure 12.1 outlines the current (layered) Android Architecture.



Fig 12.1: Android Architecture

Linux Kernel is based on version 2.6 and on top of its libraries are developed in C/C++ which expose the capabilities through application framework which are introduced by the applications. The modified Linux kernel operates as the Hardware Abstraction Layer (HAL) and provides device driver, memory management, process management, as well as networking functionalities, respectively. The library layer is interfaced through Java (which deviates from the traditional Linux design). It is in this layer that the Android specific libc (Bionic) is located. The surface manager handles the user interface (UI) windows. The Android runtime layer holds the Dalvik Virtual Machine (DVM) and the core libraries. Most of the functionalities available in Android are provided via the core libraries.

Also running as processes within the Linux kernel is the application (app) runtime. Each app runs in its own instance of the Android runtime, and the core of each instance is a Dalvik Virtual Machine (VM). The Dalvik VM is a mobile-optimized virtual machine, specifically designed to run fast on the devices that Android targets. Also present at this layer, and in each app's runtime, are the Android core libraries, such as the Android class libraries, I/O, and other similar things.

One of the critical components added by Android is Android run time. Dalvik Virtual Machine is part of Android run time optimized for mobile devices. Dalvik Virtual Machine Android run time also contains a core library which provides a substantial subset of Java 5 standard edition.

In the application framework layer, you can find the code compiled for and running on Dalvik VMs that provides services to multiple applications. Running at this level are entities such as the Package Manager, responsible for managing applications on the phone, and the Activity Manager, which is responsible for loading Activities and managing the Activity stack. The application framework houses the Application Program Interface API interface. In this layer, the activity manager governs the application life cycle. The content providers enable applications to either access data from other applications or to share their own data. The resource manager provides access to non-code resources (such as graphics), while the notification manager enables applications to display custom alerts. Anyone can write code that runs within the application framework; a good example is an application that shares out information to other applications as a Content Provider.

Finally, applications (like Home, dialer, SMS-MMS etc.) run at the top layer. This includes applications that a developer writes, or Google and other Android developers do. This layer is on top of the application framework, and they are the built-in as well as the user applications. It must be pointed out that a user application can replace a built-in application, and that each Android application runs in its own process space, within its own DVM instance. Typically, applications running at this layer include one or more of four different types of components. They are activities, Broadcast Receivers, Services, and Content Providers.

3.1 Linux Kernel

At the very bottom of Android architecture is the Linux kernel. Android relies on Linux version 2.6 for core system services. This Linux kernel is responsible for most of the things that are usually delegated to the operating system kernel and includes android's memory management programs, security settings, power management software and several drivers for hardware, file system access, networking and inter-process-communication. The kernel also acts as an abstract layer between the hardware and the rest of the software stack. This is the layer where all the device-specific hardware drivers will run viz. display driver, camera driver, Bluetooth driver, shared memory driver, binder (IPC) driver, USB driver, keypad driver, Wi-Fi driver and audio drivers.

Let us take a look at the different features of Linux Kernel:

- Open source: Software is open source and available for everyone to use, copy, and modify.
- Monolithic: It handles all hardware and driver operations
- Multitasking: It permits the simultaneous execution of numerous tasks.
- Modular: Loadable kernel modules can be added and removed at runtime.

Let us see the different functions that Linux Kernel perform:

- Functions
 - Device Drivers
 - Memory Management
 - Process Management

Let us understand the concept of Device Drivers:

The operating system and hardware components can communicate with one another using drivers.

Different manufacturers' hardware components must meet Android requirements in order to function with a device.

The different types of drivers are – USB driver, Bluetooth driver, Wi-fi driver, Display driver, Audio Driver, Power Manager, Flash Memory and Binder

Let us see how Linux Kernel performs memory management:

- The Linux Kernel makes sure there are no memory space conflicts between apps that are running simultaneously.
- All apps on the device are given enough memory to work, thanks to the Linux Kernel.
- No application uses more memory than is necessary thanks to the Linux kernel, which preserves memory.

Let us see how Linux Kernel manages different processes:

- A process can be started, stopped, paused, or killed by Linux Kernel.
- The Linux kernel makes sure that no other process will clash with the space assigned for one.
- The Linux kernel supports background processes as well as multitasking processes.
- After a process is finished, the Linux kernel frees up RAM.

SELF ASSESSMENT QUESTIONS – 2

4. In Android, the modified Linux kernel operates as the _____, and provides device driver, memory management, process management, as well as networking functionalities, respectively.
5. The Android runtime layer holds the “_____” “OBJ” and the “_____”.
6. Which of the following governs the application life cycle in application framework layer?
 - a) activity manager
 - b) window manager
 - c) package manager
 - d) resource manager

4. ANDROID ARCHITECTURE LIBRARIES

On top of the kernel are the libraries. Android includes a set of C/C++ libraries written as modules of code that are compiled down to native machine code. It controls the device to handle different kinds of data efficiently. These libraries tell the device how to handle different kinds of data and are exposed to Android developers via Android application framework. It provides some of the common services that are available for applications and various components of the Android system. It exposes its capabilities through Android application framework.

Libraries in Android architecture include the Surface Manager, Media Framework, WebKit, SQLite, OpenGL/ES, FreeType, SGL, SSL, Libc. These native libraries run as processes within the underlying Linux kernel.

Libc: Libc is a standard C library tuned for embedded devices.

SSL: It is a Secure Sockets Layer cryptographic protocol for secure internet communications.

SGL: SGL is a scalable graphic library or also called clear graphic library that gives an underlined 2D graphic engine. In Android graphics platform, you can combine 3D and 2D graphics in the same application.

OpenGL/ES: OpenGL/ES is a 3D library. OpenGL supports the CD graphics which are based on OpenGL1.0, so these libraries either use hardware 3D accelerator if they are available or use the highly optimized software. They have a software implementation that is hardware accelerable if the device has a 3D chip on it.

FreeType: Freetype is used for bitmap and vector font rendering. FreeType is a free, high quality and portable font engine.

WebKit: It is the open-source browser engine, used as a core of Android's browser. It acts as the web rendering engine that powers the default browser. It is the same browser that powers Safari from Apple and it renders well on small screens and on mobile devices.

SQLite: It is the basic datastore technology for the Android platform and is a very lightweight relational database engine that manages access to display subsystem. It is used as the core of most of its data storage.

Media Framework: The Media Framework was provided by Packet Video, one of the members of the open handset alliance and that contains the entire codex that make up the core of the media experiences. So, in there one will find all the audio and video codes that one need to build a rich media experience like IMPEG4, H.264, MP3, and AAC.

Surface Manager: The surface manager is responsible for composing different drawing surfaces on to the screen or you can say that it is responsible for graphics on the device's screen. It manages access to the display subsystem and seamlessly composites 2D and 3D graphic layers from multiple applications. Surface manager seamlessly monitors the 2D and 3D graphics.

LibWebCore: LibWebCore is a modern web browser engine that gives us embeddable web view.

Media Libraries: It supports playback and recording of many popular audio and video formats, as well as static image files. Media libraries are based on PackerVideosOpenCORE. So, these libraries support playback as well as recording of many popular audio and video formats like MPEG4, H264, MP3 etc.

Android Runtime

The Android runtime layer also includes a set of core java libraries and DVM (Dalvik Virtual Machine) located in same layer. The Android runtime layer includes a set of base libraries that are required for java libraries. Every android application runs in its own process, with its own instance in the Dalvik Virtual Machine.

Core Libraries: Core Libraries are written in the Java programming language. The core library contains all the collection classes, utilities, I/O, all the utilities and tools that you use.

Dalvik Virtual Machine: Android based systems utilize their own virtual machine (VM), which is known as the Dalvik Virtual Machine (DVM). It is an extremely low memory based virtual machine, which was designed especially for Android to run on embedded systems

and work well in low power situations. It is also tuned to the CPU attributes. Applications for Android are developed in Java and executed in a virtual machine, called Dalvik VM. The DVM uses special bytecode, hence native Java bytecode cannot directly be executed on Android systems. The Dalvik VM creates a special file format (.DEX) that is created through build time post processing. Conversion between Java classes and DEX format is done by included “dx” tool. The Android community provides a tool (dx) that allows converting Java class files into Dalvik executables (dex). Dalvik executable or dex format is optimized for Android. The DVM implementation is highly optimized in order to perform as efficiently and as effectively as possible on mobile devices that are normally equipped with a rather slow (single) CPU, limited memory resources, no OS swap space, and limited battery capacity. The DVM has been implemented in a way that allows a device to execute multiple VM’s in a rather efficient manner. Every application runs in its own process if you have ten different applications, making ten different processes total. It also must be pointed out that the DVM relies on the modified Linux kernel for any potential threading and low-level memory management functionalities.

The Bionic Library

Compared to Linux, Androids incorporates its own C library known as Bionic library. The Bionic library is not compatible with Linux glibc. Compared to glibc, the Bionic library has a smaller memory footprint.

To illustrate, the Bionic library contains a special thread implementation that (i) optimizes the memory consumption of each thread and (ii) reduces the startup time of a new thread.

Android provides run-time access to kernel primitives. Hence, user-space components can dynamically alter the kernel behaviour. Only processes/ threads that do have the appropriate permissions are allowed to modify these settings.

Security is maintained by assigning a unique user ID (UID) and group ID (GID) pair to each application. As mobile devices are normally intended to be used by a single user only (compared to most Linux systems), the UNIX/Linux /etc/passwd and /etc/group settings have been removed. In addition (to boost security), /etc/services was replaced by a list of services (maintained inside the executable itself). To summarize, the Android C library is especially suited to operate under the limited CPU and memory conditions common to the

target Android platforms. Further, special security provisions were designed and implemented to ensure the integrity of the system.

SELF ASSESSMENT QUESTIONS - 3

7. In the Linux _____ layer all the device-specific hardware drivers will run.
8. Which of the following is a cryptographic protocol for secure internet communications?
 - a) Scalable Graphic Library (SGL)
 - b) Secure Sockets Layer (SSL)
 - c) FreeType
 - d) SQLite
9. Android based systems utilize their own virtual machine (VM) known as the "_____".



5. ANDROID ARCHITECTURE APPLICATION FRAMEWORK

In Android Architecture, Application framework is a set of basic tools with which a developer can build much more complex tools. It has several components and is used by all applications. Application Framework is written in Java language. It comes with mobile devices like Contacts or SMS box, or applications written by Google and any Android developer.

The developers can access all framework APIs and manage phone's basic functions like resource allocation, switching between processes or programs, telephone applications, and keeping track of the phone's physical location. The architecture is well designed to simplify the reuse of components. Application frame enables the reuse of various components. Libraries expose these capabilities to the application framework, enabling reuse of those components.

In the application framework, we have many components. Those components are Activity manager, Window manager, Content Providers, View System, Notification Manager, Package Manager, Telephony Manager, Resource Manager, Location Manager, etc.

Activity Manager: This is the component that actually manages the life cycle of the applications. It also maintains a common back stack so that applications that are running in different processes can have a smoothly integrated navigation experience.

Window Manager: The window manager manages Windows. It is mostly a java programming language abstraction on top of lower-level services provided by the surface manager.

Content providers: The work of Content provider in application framework is to enable applications access data from other applications. Since every application is running in its own classes, one application does not have access to the process area of another application but if they want to exchange data among each other, then they can achieve it through content providers. It allows applications to share their data with other applications.

View System: View System contains buttons and lists and all the building blocks of the User Interface (UI). It also handles things like event dispatching, and layout drawing.

Notification manager: Notification manager helps us to display custom alerts on status bar. This notification manager is primarily important in the case of broadcast receivers. Broadcast receiver helps us to display various times based alerts on statements.

Package manager: It is responsible for keeping track of what applications you have and what capabilities each of your applications have. Package manager essentially holds information about various applications loaded in the system. When any applications need information about other applications then package manager provides such information.

Telephony manager: It handles information about telephony. The telephony manager contains the APIs we use to build the phone application.

Resource manager: Resource manager gives us access to non-code resources such as localized strings or various others resources in the form of PNG, BMP as well as XML. Therefore, resource manager helps us to get access to those non-code resources. The resource manager stores local Strings, bitmaps, and layout file descriptions.

Location managers: It handles information about the location.

SELF ASSESSMENT QUESTIONS – 4

10. Which of the following in application framework enables applications to access data from other applications?
 - a) Activity manager
 - b) Window manager
 - c) Content provider
 - d) Resource manager
11. Which of the following handles things like event dispatching, layout drawing?
 - a) Activity manager
 - b) View System
 - c) Content provider
 - d) Resource manager
12. “_____”  gives us access to non-code resources such as localized strings or various other resources in the form of PNG, BMP as well as XML.

6. APPLICATIONS

At the top of Android Architecture is the application layer where you can find all the applications that are used by the final user. Android incorporates a set of core applications including an email client, SMS program, calendar, maps, browser, contacts, and others. By installing different applications, the user can turn his mobile phone into a unique, optimized and smart mobile phone. Underlying all applications is a set of services and systems along with a right and extensible set of views that can be used to build an application, including lists, grids, text boxes, buttons, and even an embeddable web browser. All applications are written using the Java programming language.

An application in an Android is a dot APK (.apk) file. When we compile and build our source code, then dot APK file is generated, which contains the whole application package. During the build process, Android projects are compiled and packaged into a .apk file, repository for all binary applications. It contains all the information necessary to run the applications on a device or emulator, such as compiled .dex files (.class files converted to Dalvik byte code), a binary version of the AndroidManifest.xml file, compiled resources (resources.arsc) and uncompiled resource files for the application.

The general process for a building .apk file is outlined below:

1. The Android Asset Packaging Tool (AAPT) takes your application resource files, such as the AndroidManifest.xml file and the XML files for activities, and compiles them. R.java is also produced so you can reference your resources from your Java code.
2. The aidl tool converts any .aidl interfaces that are into Java interfaces. Java code, including the R.java and .aidl files, are compiled by the Java compiler and .class files are output.
3. The dex tool converts the .class files to Dalvik byte code. Any 3rd party libraries and .class files you have included in your project are also converted into .dex files so they can be packaged into the final .apk file.
4. All non-compiled resources (such as images), compiled resources, and the .dex files are sent to the apkbuilder tool to be packaged into an .apk file.
5. Once the .apk file is built, it must be signed with either a debug or release key before it can be installed to a device.

6. Finally, if the application is being signed in release mode, you must align the .apk with the zipalign tool. Aligning the final .apk decreases memory usage when the application is running on a device.

Finally, when we get the .apk file, it is this APK file that is installed on mobile devices.

Applications of Android include a wide range of mobile applications. It includes:

- the ability for anyone to customize the Google Android platform.
- customizing a mobile phone using the Google Android platform
- features like weather details, opening screen, live RSS feeds, and even the icons on the opening screen to be customized.
- entertainment functionalities by Google Android can offer online real-time multiplayer games.

SELF ASSESSMENT QUESTIONS - 5

13. Java source code file passes through the Java compiler to get dot class files.
(True/False)
14. Android projects are compiled and packaged into an "_____ " file, The repository for all binary applications
15. Which of the following tools converts the .class files to Dalvik byte code?
 - a) dex tool
 - b) DVM
 - c) Kernel
 - d) Zipalign

7. SECURITY AND PERMISSION

Android based smartphones are nowadays used for private and business applications, such as online banking or to access corporate networks. This makes them a very valuable target to be attacked to access important and secret data or use it for one's benefit. Mobile devices contain Contact Information, Location Data, Credentials, and Private Details. One can use Trojan software to access various information stored in devices. Espionage, communication eavesdropping, blackmailing, botnet formation, collecting valid email addresses for spam mailing and recording of sensitive information such as login credentials or credit card data are just some of the classical applications of trojan software. Even if substantial or large-scale attacks have so far been unsuccessful, they are growing more complex and effective. Thus, security is of paramount importance for both private and corporate users whoever is using devices with Android OS.

File System and User/Group Permissions

Android uses Linux as its underlying kernel and much of the Android security model is like Linux security model. In Linux, security is performed as per the concept of users and groups. Each user in a Linux system is assigned a user ID (UID) to differentiate one user from another. In addition, users can be added to groups and each group has a group ID (GID) to differentiate one group from another. A user can be a member of multiple groups, and each group can have multiple members.

Permissions are assigned to each resource on a Linux system, with a resource typically being a file. Each resource has a defined owner, which is the UID of the user that has primary responsibility for the file and can alter the permissions on it. Each resource also has a defined group, which is the GID of users with a set of permissions over and above that of the world, which is the group of all users on the system.

Similar to Unix/Linux operating systems, basic access control in Android is implemented through a three-class permission model. It distinguishes between the owner of a file system resource, the owner's group, and others. For these three entities, distinct permissions can be set to read, write, or execute. This system provides a means of controlling access to files and resources.

Example: only a file's owner may write (alter) a document, while members of the owner's group may read it, and others may not even view it. However, for mobile operating systems, this is not sufficient. Finer permission distinction is needed, as an open app market is not a strongly monitored and trustworthy software source. With the traditional approach, any app executed under the device owner's user ID could access any other app's data. Hence, the Android kernel assigns each app to its own user ID on installation. This ensures that an app can only access its own files, the temporary directory and nothing else – system resources are available through API calls.

When an Android package is installed, a new user ID (one that is not currently in use on the device) is created and the new app runs under that UID. In addition, all data stored by that application is assigned the same UID, whether a file, database, or other resource.

The Linux permissions on resources for that app are set to allow full permission by the associated UID and no permissions otherwise. Note that this UID is unique to the device; there is no guarantee (or even expectation) that the same UID will be used for the same application on different devices. Linux prevents applications that have different UIDs from accessing data, or otherwise accessing the process or memory of other applications, thus providing the basis for the separation between applications on the Android platform. This concept is known as the separation of concerns. Each app is well separated from others by default. Android introduces the capability for software components to run under the same user IDs and as part of the same processes.

Android Permission Basics

The Android permission model is based on the central construct of a permission. A permission is something that is granted to applications and required by APIs (or certain functions within code) to run. A simple example is the Android INTERNET permission; all networking APIs require a calling app to have this permission to execute the requested action of allowing network communications. Opening an outgoing network connection is something that not all applications need to do, and an activity that could cause the user to incur additional charges if they pay for the amount of data their device sends or receives (plus, this is the typical way an app would send sensitive data off the device to be exploited). So, opening a network connection is restricted by INTERNET permission.

Android applications request permission for . installation. All the permissions that an app requires are specified in the AndroidManifest.xml file. As per the app's manifest file, Android permissions are sometimes called Manifest permissions. The purpose of this manifest file is to present essential information about the Android system and describe various components of the application. It declares the permissions application must have to interact with other applications, so this is another new concept of permission. If your application wants to have access to the contact database, then you must ask for this permission in AndroidManifest.xml.

While the Android system specifies a great number of system permissions that are required to access certain portions of the default APIs, there is nothing to prevent an app from using custom permissions. The app defines a new permission and then runs with that permission. It provides an API (or methods) that require a calling process to have that permission, or both. The permissions system is a powerful construct meant to be used and extended by app developers outside the core Android developers.

An app needs to specify which permissions it requires in its manifest. The question then remains: how are these permissions granted? This is all handled when the app is installed. At that time, its AndroidManifest.xml file is read, and the list of all permissions it needs to run is parsed. The installer then generates a list of these permissions and prompts the user to review and approve them. This is an all-or-nothing process; either the user can choose to accept the list of permissions that the app claims it needs, and the app will then install, or the user can choose to reject the list, and the app's installation will fail. Once an app is installed, no further interaction with the user will occur to inform them that certain permission is being exercised or to confirm that the user still wants to allow the app to execute an action requiring a permission check. To confirm each app's permissions with the user only at install time, this design choice is a key design decision for Android's permission model.

Now is a good time to think about how and where permissions are enforced. While writing an app that only calls other APIs and services, you need to determine which permissions are required by the APIs and services your app calls and specify them in your manifest. You are the consumer of such services and need to ask permission. If, however, you include methods that other applications will call or other types of inter-process communication endpoints,

you are sitting on the other side of the table as a provider of services; you are the one who needs to grant permission. As a result, you'll need to add code to your app that verifies the permissions of any other apps calling you.

Android Permission Model

The permissions formally requested by an app at installation time through its manifest file do not have to match those that made use of it. Hence, it is completely impossible for a user to determine what activities an app performs with the permissions it has requested. There are several examples of actions which can be performed without proper permissions or without any permission at all.

Some of these actions are particularly valuable for malware purposes:

- **RECEIVE_BOOT_COMPLETED:** This is permission enabling an app to start at boot time. Malicious applications like malware may start automatically and run unnoticed.
- **START_ON_INSTALL:** this action enables an app to start up automatically right after installation.

These are the two permissions that may not be formally requested and presented to the user as permission.

The two types of Dangerous permissions required by the user are:

Install time permission:

- This is the authorization that the user must grant to install an application.

Run time permission:

- When the corresponding functionality is required, the user must provide these rights. Access to location and contact details, for instance

Next is Inter-Process Communication (IPC):

Using this technique, we may apply security rules based on how an application interacts with the other end. The following classes help the IPC process.

- IPC
 - Intent
 - Binder
 - Messenger

Let us understand what Intent is and what are its types:

It is the preferred method of data transfer for interacting with participants in events or broadcast listeners. It comes in two varieties.

- Intent
 - Explicit intent
 - Implicit intent

The two types of intents:

- Explicit intent
 - Designed to be received by an explicit component
 - It ensures that only application B receives data submitted from application A.
- Implicit intent
 - It specifies an action that needs to be performed
 - Used when a task must be assigned to a third-party program because it cannot be completed by the original application.

SELF ASSESSMENT QUESTIONS – 6

16. The Android permission model is based on the central construct of a _____.
17. All the permissions that an app requires are specified in the “_____”file.
18. The permissions formally requested by an app at installation time through its manifest file do not have to match those made use of. (True/False)

8. SUMMARY

Let us recapitulate the important concepts discussed in this unit:

- Android is developed by Open Handset Alliance (OHA), a group of 84 technology and mobile companies that have come together to accelerate innovation in mobile and offer consumers a richer, less expensive, and better mobile experience.
- The feature of Android SDK is that one can use it as a development kit for GSM, EDGE, 3G networks, WiFi, Bluetooth technologies, etc.
- The Android platform is built as a stack with various layers running on top of each other, with lower-level layers providing services to upper-level services.
- Linux Kernel is based on version 2.6 and on top of its libraries are developed in C/C++ which expose the capabilities through application framework which are introduced by the applications.
- Android includes a set of C/C++ libraries written as modules of code that are compiled down to native machine code. It controls the device to handle different kinds of data efficiently.
- Every android application runs in its own process, with its instance in the Dalvik Virtual Machine.
- Compared to glibc, the Bionic library has a smaller memory footprint.
- Application framework is a set of basic tools with which a developer can build much more complex tools. It has several components and is used by all applications.
- An application in an Android is a dot APK file. When we compile and build our source code, a dot APK file is generated, containing the whole application package.
- Android uses Linux as its underlying kernel, and much of the Android security model is like as Linux security model.
- The Android permission model is based on the central construct of permission.

9. GLOSSARY

- **Applications:** Self-contained software designed for a mobile device and performing specific tasks for mobile users
- **Framework:** A platform for developing software applications, It is a foundation on which software developers can build programs for a specific platform.
- **kernel:** Is the central part of an operating system, manages the operations of the computer and the hardware, most notably memory and CPU time
- **Middleware:** Software that lies between an operating system and the applications running on it enables communication and data management for distributed applications.
- **OpenGL/ES:** OpenGL/ES is a 3D library, supports hardware 3D accelerator design.
- **Software Development Kit:** Collection of software development tools in one installable package, facilitating the creation of applications by having a compiler, debugger and perhaps a software framework.

10. TERMINAL QUESTIONS

Short Answer Questions

1. Write the features of Android SDK.
2. Draw the Android Architecture and briefly explain about each layer.
3. Briefly explain about the libraries of android architecture.
4. Briefly explain about the Application framework of android architecture.
5. How is an.apk files built? Write the steps.
6. What are manifest permissions? Explain.

10.1 Answers

Self-Assessment Questions

1. Memory, hardware
2. True
3. Linux
4. Hardware Abstraction Layer (HAL)
5. Dalvik Virtual Machine (DVM), core libraries

6. a) activity manager
7. Kernel
8. b) Secure Sockets Layer (SSL)
9. Dalvik Virtual Machine (DVM)
10. c) Content provider
11. b) View System
12. Resource manager
13. True
14. .apk
15. a) dex tool
16. permission
17. AndroidManifest.xml
18. True

Short Answer Questions

1. Features of Android SDK are that you can use it as a development kit for GSM, EDGE, and 3G networks, WiFi, Bluetooth technologies etc. (Refer to section 12.2 for more details)
2. Android architecture consists of Linux kernel, Libraries, Application Framework and Applications. (Refer to section 12.3 for more details)
3. In Android architecture, libraries are a set of C/C++ code written as modules and include the Surface Manager, Media Framework, WebKit, SQLite, OpenGL/ES etc. (Refer to section 12.5 for more details)
4. Application framework is a set of basic tools with which a developer can build much more complex tools. (Refer to section 12.6 for more details)
5. (Refer to section 12.7 for more details)
6. Android permissions, as per the app's manifest file, are sometimes called Manifest permissions. (Refer to section 12.8 for more details)

13. REFERENCES:

- Fedler, R., Banse, C., Christoph, K., & Fusenig, V. (2012). *Android OS Security: Risks and Limitations A Practical Evaluation*. Fraunhofer Research Institution AISEC.
- Six, J. (2011). *Application Security for the Android Platform*. O'Reilly.
- *framework*. (n.d.). Retrieved July 26, 2012, from developer.android:
<http://developer.android.com/guide/faq/framework.html>
- *Developers Android*. (n.d.). Retrieved 08 21, 2012, from Building and Running:
<http://developer.android.com/tools/building/index.html>

